HYU 고급표현학습 · 2026-2학기 · L01-L07

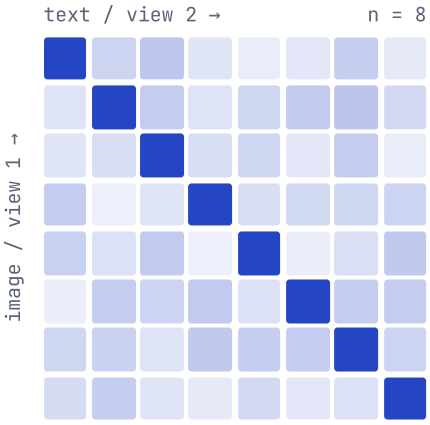
고급표현학습 한 바퀴

세미나처럼 휙휙 지나간 7강(슬라이드 508장)을 **처음 배우는 사람** 기준으로 처음부터 끝까지 다시 풀어 쓴 페이지야. 용어는 처음 나올 때 그 자리에서 풀었어. 논문 이름을 외우기 전에 **왜 다음 방법이 나왔는지**만 따라가면 돼.

이 수업의 주제는 **representation learning**이야. 사진 같은 raw data를 쓸모 있는 숫자 벡터로 바꿔 주는 네트워크(encoder)를 학습시키는 분야지. 학습엔 늘 **target**이 필요해. loss 안에서 모델 출력이 맞춰야 하는 정답 말이야. 보통은 사람이 단 정답(label)이 그 역할인데, label을 다는 건 느리고 비싸.

7강 전체를 꿰는 질문은 하나야. 사람이 label을 안 주면, 학습 target은 어디서 오나?

퀴즈 10/29(목) · 20문제 · T/F · MCQ · 빈칸 · pseudo code 9개



N×N similarity matrix. 행 = batch의 이미지 n개, 열 = 그 짝 n개(같은 사진을 자르거나 색을 바꾼 변형본(view), 또는 그 사진에 붙은 설명 문장(caption)), 칸 하나 = 두 벡터가 얼마나 비슷한지 점수야. 진한 대각선이 진짜 짝(positive), 나머지는 전부 남의 짝(negative)이고, 학습은 대각선 점수를 키우고 나머지를 줄이는 쪽이야. (뒤의 CLIP · Barlow Twins 얘기는 L04 · L05를 읽고 다시 봐도 돼.) CLIP pseudo code의 `labels = np.arange(n)` ("i번째 행의 정답은 i번째 열")이 바로 이 대각선이고 (L05 p.35), Barlow Twins도 identity(대각 1, 나머지 0)를 target으로 쓰지만 그건 샘플끼리가 아니라 **feature 차원끼리**의 상관행렬이야 (L04 p.35–38).

읽는 법

이 페이지는 representation learning을 **처음 보는 사람** 기준으로 썼어. 7강(L01–L07)을 강의 순서대로 따라가면서, 처음 나오는 용어는 그 자리에서 괄호로 풀고, 수식은 기호 하나하나를 말로 옮겼어.

처음이면 이 순서로

1. **큰 그림**(MAP)부터 봐. 7강이 "loss의 정답(target)을 어디서 구하나"라는 질문 하나로 이어진다는 것만 잡으면 돼. 표가 다 이해 안 돼도 괜찮아.
2. **용어 먼저**(ABC)는 처음엔 "기초" · "모델 부품" · "학습 · 평가 설정" 그룹만 읽어. 외우지 말고 "이런 단어가 있구나" 정도로만. Contrastive · Teacher – student · Semi-supervised 그룹은 각각 L03 · L04 · L06을 읽다가 본문의 용어 링크를 눌러 돌아와서 보면 돼.
3. **L01부터 순서대로** 읽어. 앞 강의의 한계가 다음 강의의 질문이 되는 구조라서, 건너뛰면 "이 방법이 왜 나왔지?"가 안 보여. 강의 하나가 끝날 때마다 셀프 체크 3문제로 확인해.
4. 다 읽고 나서 **공통 패턴**(PAT)과 **Pseudo code**(PC)를 봐. 여러 방법에 반복해서 나오는 부품(EMA, stop-gradient 같은 것)과 시험 대비로 골라 둔 pseudo code 9개의 핵심 줄을 모아 놨어.
5. 시험 전날엔 **마지막 체크**(D-1)로 영어 문장이 바로 떠오르는지 확인해. **Physical AI 연결**(PAI)은 동기 부여용이고 시험 범위가 아니야.

강의 하나는 이렇게 생겼어

강의마다 순서가 같아. **이 강의가 던지는 질문** → 그 질문에 답한 **방법들** → **다음 강의로 넘어가는 이유**. 방법마다 이름과 슬라이드 페이지가 붙어 있고, 설명은 작은 라벨로 나뉘어 있어. 자주 쓰는 라벨은 이래.

- `한 줄` 이 방법이 하는 일을 일상어로.
- `문제` 이전 방법의 어떤 불편함 때문에 나왔나.
- `핵심` 아이디어. `어떻게` 는 단계별 절차, `왜 돼?` 는 직관.
- `차이` 이전 방법과 다른 점. 시험에서 "A와 B의 차이"로 자주 나와.

방법 옆의 `p.N` 은 강의 PDF의 페이지 번호야. 예를 들어 `L04 p.25` 는 4강 PDF의 25쪽이야. 헷갈리면 그 장만 열어 보면 돼.

기술 용어와 방법 이름은 영어 그대로 썼어. 시험이 영어(T/F · MCQ(객관식) · 빈칸)라서, 지문에서 그 단어를 바로 알아봐야 하거든. 한국어 풀이는 처음 나올 때만 붙였어.

박스 네 종류

쉽게 말하면
본문을 일상어 · 비유 · 작은 숫자 예시로 다시 말한 거. tag는 쉽게 말하면 / 비유 / 예시로 보면 / 왜? 중 하나. 새 사실 없는 이해용이야.

RL로 번역
네 분야(RL) 개념으로 바꿔 말한 거. 슬라이드 밖이라 시험엔 안 나와. 이해용이야.

시험 함정

슬라이드 문장을 살짝 비틀어서 T/F나 빈칸으로 내기 좋은 곳. 예: 두 방법의 이름이나 두 loss의 이름을 서로 바꿔 쓴 문장.

다음으로

이 강의의 한계가 다음 강의의 질문이 되는 연결고리.

본문이 잘 안 읽히면 연한 색 박스(쉽게 말하면 · 비유 · 예시로 보면 · 왜?)부터 봐. 강의 끝마다 셀프 체크 3문제가 있어. 지문은 영어이고, 눌러야 답과 해설이 보여. 다 읽은 강의는 "다 봤음"에 체크해 둬. 목차에 ✓가 붙고, 이 브라우저에만 저장돼.

큰 그림: target은 어디서 오나

먼저, 이 수업이 하려는 것

이 수업의 주인공은 encoder야. 사진 같은 raw data x를 받아서 숫자 벡터 하나를 내놓는 네트워크(ResNet, ViT 같은 것)지. 슬라이드 표기로는 `z = f(x)`야. `f`가 encoder, 결과 벡터 `z`가 **representation**이고, embedding이나 feature도 거의 같은 뜻으로 써 (L01 p.12).

representation learning은 이런 encoder를 어떻게 학습시킬지를 다루는 분야야. 좋은 representation이 뭔지(무엇을 남기고 무엇을 버리나)는 L01에서 자세히 봐.

supervised learning에서 출발

가장 익숙한 방법은 supervised learning이야. 사진마다 사람이 정답("강아지", "고양이")을 달아 주고, 모델이 그 정답을 맞히도록 학습해. 이 사람이 단 정답이 label이야.

ImageNet(사람이 class label을 단 대규모 사진 데이터셋)으로 분류를 학습시키면, 그 과정에서 생긴 encoder의 feature가 다른 task에도 잘 통해 (L02 p.4–5). 분류가 encoder를 키우는 수단이 되는 거야.

여기서 **학습 target**이라는 말을 정해 두자. target = **loss 안에서 모델 출력이 맞춰야 하는 값**이야. 분류라면 loss는 cross-entropy(정답 class에 준 확률이 클수록 작아지는 loss)이고, target은 label이야. 이 페이지에서 "target이 어디서 오나"라고 물으면, "loss의 정답 자리에 뭘 넣었나"를 묻는 거야.

> RL로 번역
> DQN의 TD target $y = r + \gamma \cdot \max_{a'} Q(s', a')$를 떠올려 봐. 정답 Q값을 사람이 준 게 아니라, reward와 (target) 네트워크로 target을 직접 만들어서 regression하잖아. 이 수업의 대부분이 그런 "사람 대신 스스로 만든 target" 이야기야.

왜 label이 문제인가

문제는 label이 비싸다는 거야. ImageNet 14M장에 label을 다는 데 **22 human years**가 들었고, 복잡한 개념 · rare class · 새 domain은 label을 모으기가 더 어려워 (L02 p.6–10, 자세한 건 L02).

반면 label 없는 사진(unlabeled data)은 거의 공짜로 얼마든지 모여. 그래서 질문이 생겨. **사람이 label을 안 주면(또는 조금만 주면), loss의 정답 자리에 뭘 넣지?** 7강은 이 질문에 대한 답을 하나씩 늘어놓는 수업이야.

답 목록: target은 여기서 와

다음 표에서 한 행이 답 하나야. 첫 열은 loss에서 정답 역할을 하는 것, 둘째 열은 그 방식의 이름 · 대표 방법과 쉬운 풀이, 셋째 열은 나오는 강의야. 위에서 아래로 읽으면 거의 수업 순서랑 같아.

pretext task 행부터 caption 행까지는 사람이 단 class label 없이 target을 만들어. 그다음 SupCon 행은 label을 다시 써. 마지막 두 행(L06–L07)은 label이 조금 있는 상황이야.

TARGET은 여기서 옴	방식 · 대표 방법 · 쉽게 말하면	강의
사람이 단 label	Supervised pretraining (ImageNet) 출발점. 사람이 사진마다 class를 달고, 모델이 그걸 맞혀. 잘 되지만 label이 비싸.	L01–L02

TARGET은 여기서 옴	방식 · 대표 방법 · 쉽게 말하면	강의
내가 가한 변환 자체 (회전 각도, 패치 위치, 지운 픽셀, 지운 색)	Pretext task · RotNet, Jigsaw, Colorization pretext task = encoder를 훈련시키려고 만든 가짜 문제. 사진을 돌리거나, 조각을 섞거나, 일부를 지우거나, 색을 빼고 원래 상태를 맞히게 해. 내가 바꿨으니까 정답은 공짜야.	L02
같은 이미지의 다른 view = positive, 나머지 = negative	Contrastive · InfoNCE, SimCLR, MoCo view = 같은 사진을 자르거나 색을 바꿔 만든 변형본(augmentation). 같은 사진에서 나온 두 view의 벡터는 가깝게(positive), 다른 사진의 벡터와는 멀게(negative) 해. 정답은 "후보 중 어느 게 내 짝이냐"야.	L03–L05
천천히 따라오는 EMA teacher의 출력	Self-distillation · BYOL, DINO self-distillation = 따로 학습된 teacher 없이, 자기 자신(의 EMA)을 teacher로 삼아 그 출력을 따라 하는 것. 거의 같은 구조의 네트워크 두 개를 써 (BYOL은 student 쪽에만 predictor라는 작은 MLP가 하나 더 붙어). gradient로 학습하는 student, 그리고 student weight의 EMA(과거 값에 새 값을 조금씩 섞어 천천히 따라가는 평균)로만 갱신되는 teacher. student가 다른 view를 보고 teacher의 출력을 맞혀. teacher 쪽으로는 gradient가 안 흘러(stop-gradient). negative가 필요 없어. BYOL은 둘을 online / target network라고 불러.	L04
두 view의 feature 상관행렬을 identity로	Redundancy reduction · Barlow Twins batch 전체를 보고 "view 1 벡터의 차원 i와 view 2 벡터의 차원 j가 얼마나 같이 움직이냐"(cross-correlation) 행렬을 계산해. target은 대각 1(두 view가 같은 값을 내라), 나머지 0(차원끼리 같은 정보를 중복해 담지 마라)인 identity 행렬 자체야. negative 없음.	L04
cluster 번호 · prototype code	Clustering · DeepCluster, SwAV 비슷한 벡터끼리 묶어서(clustering) 그 그룹 번호를 정답처럼 써. DeepCluster는 k-means로 묶은 cluster 번호를 pseudo label(모델이 만든 가짜 label)로 삼아 분류를 학습해. SwAV는 학습되는 cluster 중심(prototype)에 배정한 결과(code)를 한 view에서 구하고, 다른 view가 그걸 맞히게 해(swapped prediction).	L05
짝지어진 caption (언어)	CLIP, X-CLR(soft target), SynCLR 사진에 딸린 글(caption)을 학습 신호로 써. CLIP은 사진과 그 caption을 짝(positive)으로 맞혀. X-CLR은 caption끼리의 유사도로 샘플 사이 target을 0/1 대신 "얼마나 비슷한지" 연속값(soft target)으로 줘. SynCLR은 같은 caption으로 생성한 이미지들을 서로 positive로 써.	L03–L05
같은 class label (다시 사람)	SupCon contrastive 틀은 그대로 두고, 사람이 단 class label로 positive를 넓혀. 같은 class면 다른 사진이어도 positive야.	L05
perturbation을 준 예측끼리 서로	Consistency · Π-model, Mean Teacher, UDA 여기부터 semi-supervised. perturbation = 입력이나 모델에 주는 작은 흔들림(augmentation, dropout, noise). label 없는 사진을 다르게 흔들어서 두 번 넣어도 예측이 같아야 한다는 생각이야. 그래서 두 예측의 차이를 loss로 써(consistency).	L06
모델 자신의 confident한 예측 → hard label	Pseudo label · Pseudo-Label, FixMatch 모델이 label 없는 사진에 대해 확신 있게(확률이 threshold를 넘게) 예측하면, 그 class를 정답으로 굳혀서 cross-entropy로 학습해. hard label = 확률을 여러 class에 나누지 않고 한 class만 1인 정답(one-hot).	L07

L01–L05 · Self-supervised

label이 **하나도 없어**. target을 data 자신에서 만들어서 학습해 (그래서 self-). 목표는 좋은 encoder를 만드는 거고, 평가는 linear probing으로 해. encoder를 얼려 두고 그 위에 linear classifier 하나만 label로 학습시켜서 정확도를 재는 거야. 분류기를 일부러 얇게 둬야 성능 차이가 encoder 덕인지 알 수 있거든.

L06–L07 · Semi-supervised

label **조금** + unlabeled **많이**를 한 번에 같이 써. 목표는 그 task를 잘 푸는 모델이야. 식은 $L = L_s + \mu(t) \cdot L_u$. L_s는 label 있는 data의 supervised loss(보통 cross-entropy), L_u는 unlabeled data로 만든 loss, $\mu(t)$는 L_u의 가중치로 학습 시간 t가 갈수록 커져 (L06 p.8, p.10). 초반엔 모델 예측을 못 믿으니까 작게 시작하는 거야.

예시로 보면

강아지 · 고양이 사진이 10,000장 있다고 해 보자 (설명용 숫자야).

- **Self-supervised**: 10,000장을 전부 label 없이 써. 예를 들어 한 사진을 두 번 다르게 잘라서 "이 둘은 같은 사진에서 나왔다"를 맞히게 해 (contrastive). 학습이 끝나면 남는 건 **encoder**야. 쓸 만한지는 나중에 label 달린 사진으로 linear classifier를 올려서 재 봐.
- **Semi-supervised**: 100장에만 "강아지/고양이" label이 있고, 9,900장은 없어. 100장은 평범한 분류 loss(L_s)로, 9,900장은 "흔들어 넣어도 같은 답"(consistency)이나 "확신 있는 예측을 정답으로"(pseudo label) 같은 L_u로 한꺼번에 학습해. 끝나면 남는 건 **강아지/고양이 분류기** 자체야.

한 줄로: self-supervised는 label 0개로 encoder를 만들고, semi-supervised는 label 조금으로 그 task의 모델을 만들어.

시험 함정

self-supervised와 semi-supervised 둘 다 "SSL"로 줄여 쓰는 경우가 많아. 지문에 labeled data 얘기가 같이 나오면 semi-supervised야. 슬라이드에서도 L02의 "Modern SSL often uses image transformations"(p.73)는 self-supervised, L06의 "Most SSL methods combine supervised and unsupervised objectives"(p.8)는 semi-supervised야.

7강을 한 줄씩

강의마다 작은 글씨가 그 강의의 질문이고, 굵은 첫 문장이 한 줄 답이야. 나머지는 풀이와 다음 강의로 넘어가는 이유.

- **L01** `좋은 representation이 뭔데?`

  쓸데없는 건 버리고(compression) 안 보이는 걸 맞히는(prediction) 표현.

  compression은 배경 · 조명 · 노이즈처럼 뜻과 상관없이 찍는 조건에 따라 바뀌는 정보(nuisance)를 지우는 것, prediction은 가려진 부분이나 미래처럼 직접 안 보이는 걸 추론하는 거야.

  학습 방법들도 압축 쪽(autoencoding · contrastive · clustering)과 예측 쪽(future prediction · imputation · pretext task)으로 나눠 봐 (p.21–23).

  다음 질문: 그럼 label 없이 이런 표현을 어떻게 학습시키지?

- **L02** `label 없이 supervision을 만들 수 있나?`

  data를 일부러 망가뜨리고 원래대로 맞히게 하는 pretext task.

  supervision은 학습 신호, 곧 정답이야. 사진을 90° 돌리거나, 조각을 섞거나, 일부를 지우거나, 색을 빼고 원래 상태를 맞히게 하면 정답은 내가 바꿨으니까 공짜야.

  진짜 목적은 그 문제를 푸는 게 아니라, 푸는 동안 encoder가 배우는 representation이야.

한계: 근데 사람이 문제를 하나하나 설계해야 하고(tedious), 배운 feature가 그 가짜 문제에 특화돼서 일반성이 약해 (p.44–49).

L03 `"같은 건 가깝게, 다른 건 멀게"를 loss로 쓰면?`

contrastive loss 계보가 InfoNCE로 모여.

문제를 하나하나 설계하는 대신, "같은 것(positive)끼리는 가깝게, 다른 것(negative)끼리는 멀게"라는 원칙 하나를 loss로 써. 이게 contrastive learning이야.

loss는 pair 단위 contrastive loss → triplet(기준 샘플 anchor 하나에 positive 하나, negative 하나) → N-pair → InfoNCE로 발전해. InfoNCE는 "후보 여럿 중에서 positive 1개를 골라내는 softmax 분류"야.

후반(p.33–72)엔 이 표준 틀의 숨은 가정을 하나씩 건드린 논문 3편을 봐: GenView(positive view를 생성 모델로 새로 그림), MCL(쉬운 feature만 배우는 feature suppression을 stage별 negative 선택으로 막음), X-CLR(0/1 정답 대신 text로 잰 soft target).

다음 질문: 실제로 잘 돌리려면 뭐가 필요하지?

L04 `실제로 잘 돌리려면? negative 없이도 될까?`

SimCLR(negative 많이) vs BYOL · Barlow Twins · DINO(negative 없이).

잘 돌리는 재료는 강한 augmentation, 큰 batch(= negative 많이), 구분하기 어려운 negative(hard negative)야.

SimCLR는 큰 batch로 negative를 많이 써.

그런데 negative는 비싸고(큰 batch가 필요), 사실 같은 종류인데 negative로 밀어내는 false negative 문제도 있어 (Debiased Contrastive Loss가 이걸 label 없이 보정해, p.7–11). 한편 negative를 아예 안 쓰는 방법들도 나와.

BYOL은 EMA teacher의 출력을 맞히고(+ student 쪽에만 붙는 작은 MLP인 predictor), Barlow Twins는 상관행렬을 identity로 만들고, DINO는 EMA teacher의 확률분포를 맞혀(+ teacher 출력에서 평균을 빼는 centering, teacher 분포를 뾰족하게 만드는 sharpening).

끝에 SynCLR(생성 이미지로 positive 만들기)와 DINOv2까지 봐.

주의: negative가 없으면 "모든 입력에 같은 벡터 내기"(collapse)가 가장 쉬운 답이 돼. 그래서 negative 없는 방법들은 이걸 막는 장치를 하나씩 갖고 있어.

L05 `negative를 싸게 많이 + 다른 supervision은?`

MoCo(queue), SwAV(cluster), CLIP(언어), SupCon(label). 두 갈래야.

① negative를 batch 크기와 상관없이 많이 확보해. MoCo는 지난 batch들에서 만든 벡터(key)를 queue에 쌓아 두고 negative로 재사용해. key를 만드는 encoder는 momentum(EMA)으로 천천히 갱신해서 queue 속 key들이 서로 일관되게 해 (비교 기준이 맞는다는 뜻이야. L06의 consistency regularization과는 다른 말).

② positive를 "같은 사진" 말고 다른 걸로 정의해. cluster(DeepCluster, SwAV), 언어(CLIP: 사진과 그 caption이 짝), label(SupCon: 같은 class면 짝).

여기까지가 self-supervised. 다음은 label이 조금 있는 상황이야.

L06 `label이 조금 있으면? ① consistency`

Π-model → Temporal Ensembling → Mean Teacher, 그다음 perturbation과 teacher 개선.

여기부터 semi-supervised야. $L = L_s + \mu(t) \cdot L_u$에서 L_u를 만드는 첫 번째 방법이 consistency regularization이야.

같은 입력을 다르게 흔들어(perturbation) 넣어도 예측이 같아야 한다는 거지.

계보는 비교 대상이 되는 예측(target)을 안정시키는 쪽으로 가.

Π-model: 한 step에서 두 번 forward해서 두 예측을 맞춰.

Temporal Ensembling: sample별 예측을 epoch마다 EMA로 평균해서 target으로 써 (prediction EMA).

Mean Teacher: 모델 weight를 EMA로 평균한 teacher의 예측을 target으로 써 (weight EMA).

그다음엔 perturbation을 어떻게 줄지(VAT, ICT, UDA)와, 최근 teacher–student 확장(PS-MT, Dual Teacher, S⁴Former).

- **L07** ② `pseudo label`

  틀린 label 문제(confirmation bias)를 줄이는 방법들 → FixMatch → threshold 개선.

  L_U를 만드는 두 번째 방법: 확신 있는 예측을 정답(pseudo label)으로 굳혀서 학습해.

  위험은 틀린 pseudo label이야. 틀린 정답으로 학습하면 모델이 그 실수를 더 확신하게 되고, 그게 다시 정답으로 쓰이면서 오류가 스스로 강화돼. 이게 confirmation bias야.

  이걸 줄이는 장치(soft label MixUp, Meta Pseudo Labels)를 보고, pseudo label을 consistency와 합친 MixMatch · ReMixMatch를 거쳐 FixMatch로 가. FixMatch는 weak augmentation(살짝 변형) 예측이 threshold τ를 넘으면 pseudo label로 만들고, 같은 사진의 strong augmentation(세게 변형) 예측이 그걸 맞히게 해.

  그 뒤엔 두 네트워크가 서로 깨끗한 데이터를 골라 주는 DivideMix도 나와.

  FixMatch 이후엔 고정 threshold와 perturbation을 다듬어: FreeMatch(threshold를 스스로 조절) → AllMatch(낮은 confidence 샘플도 활용) → IFMatch(feature-level perturbation 추가, 샘플별 강도 조절).

개념 그래프 **7강에 나온 방법들을 연도순으로 놓으면: 계보가 어떻게 이어지나**

읽는 법: 점 하나 = 방법 하나야. 가로 = 슬라이드에 적힌 발표 연도, 세로 = 방법 계열(무엇을 target으로 쓰나), 점 색 = 어느 강의에서 나왔는지. 맨 아래 lecture 줄이 색 범례이고, 속이 빈 점이 L07이야. 같은 해 점은 세로로 쌓았어(x는 연도 그대로). 2005 → 2010 사이는 축을 끊었어

(//).
계열 이름: Loss functions = contrastive 계열 loss 식 자체의 발전 · Pretext tasks = 사람이 설계한 가짜 문제(회전 맞히기 등) · Contrastive (InfoNCE family) = InfoNCE로 "같은 건 가깝게, 다른 건 멀게" 학습하는 방법 · No negatives / self-distillation = negative 없이 teacher 출력이나 상관행렬을 맞히는 방법 · Clustering = cluster 번호를 target으로 · Language / multimodal = 사진과 글(caption)을 짝지어 학습 · Semi-sup = semi-supervised(label 조금 + unlabeled 많이)의 두 갈래, consistency와 pseudo label.
줄임말: NCE = Noise-Contrastive Estimation · Rel. position = Relative Position · CDJP = jigsaw · inpainting · colorization을 한 번에 푸는 damaged jigsaw puzzle · AVTS = Audio-Visual Temporal Synchronization(소리와 영상이 시간상 맞는지 맞히기) · Inst. Disc. = Instance Discrimination · Barlow Tw. = Barlow Twins · Adv. training = Adversarial training · Temp. Ens. = Temporal Ensembling · Label Prop. = Label Propagation.
볼 것: 위에서 아래로 보면 **사람이 설계한 pretext → contrastive loss → negative 없는 방법 / clustering / 언어**로 가고, 아래 두 줄은 label이 조금 있는 semi-supervised 흐름이야. 연도가 있는 것 중 대표 방법만 골랐어. FixMatch는 슬라이드 표기 2021 · DivideMix는 슬라이드 표기 ICML 2020 기준. (L02–L07 전체)

용어 먼저 깔기

교수님이 "다들 알지?" 하고 넘어간 단어들을 모았어. 항목마다 **뜻 → 예 → 왜 필요한지** 순서로 풀었고, 용어 이름 밑 작은 글씨(Lxx p.N)는 그 말이 나오는 슬라이드야.

첫 그룹 "기초"는 딥러닝 기본기(softmax, cross-entropy, KL …)야. 이미 알면 훑기만 해도 되지만, 뒤에서 "MSE냐 KL이냐 CE냐" 같은 함정이 나오니까 한 번은 봐 둬. 보라색 RL 줄은 RL 쪽에서 이미 아는 개념과 짝지은 거야. 비유라서 완전히 같은 건 아니야.

처음엔 "기초" · "모델 부품" · "학습 · 평가 설정" 그룹만 읽어. Contrastive · Teacher – student · Semi-supervised 그룹은 아직 안 배운 방법 이야기가 많으니까, 각각 L03 · L04 · L06을 읽다가 본문 링크를 눌러 돌아와서 보면 돼.

기초: 이것부터

Label · labeled / unlabeled
L02 p.13 · L06 p.10

데이터에 붙은 정답이야. 사진 한 장에 "개"라고 적어 둔 게 label이야. label이 붙은 데이터는 labeled, 안 붙은 데이터는 unlabeled라고 해.

예 사진 10,000장 중 사람이 100장에만 "개/고양이"를 달았다면 labeled 100장 + unlabeled 9,900장.

왜 label은 사람이 하나하나 달아야 해서 비싸 ("Human provided (expensive)", L02 p.13). 이 수업 전체가 "label 없이, 혹은 조금만 갖고 어떻게 배우나"를 다뤄.

RL behavior cloning의 expert action이 label 역할이야. reward는 정답 action을 직접 알려 주지 않으니까 label이 아니야.

Supervised · self-supervised · semi-supervised
L02 p.12–13 · L06 p.4–8

학습에 쓰는 정답(label)을 어디서 얻느냐로 나눈 학습 방식이야.

Supervised: 사람이 단 label로 학습해. label이 비싸고, 배운 표현이 그 task에 특화돼 (L02 p.13).

Unsupervised: label 없이 데이터 구조만 찾아 (clustering, PCA 등).

Self-supervised: 사람 label은 없지만 데이터 자체에서 정답을 자동으로 만들어 학습해. "Obtain 'labels' from the data itself … Predict part of the data from other parts." (L02 p.12). 예: 사진을 돌려 놓고 몇 도 돌렸는지 맞히기. L02–L05의 주제야.

Semi-supervised: labeled 조금 + unlabeled 많이를 **같이** 써서 학습해. "Learn from the labeled and unlabeled samples together" (L06 p.7). L06–L07의 주제야.

왜 이 페이지의 질문 "학습 target은 어디서 오나?"에 대한 큰 분류야. target이 사람 label에서 오나, 데이터 자체에서 오나, 모델 자기 예측에서 오나.

시험 약어 **SSL**이 두 뜻이야. L02의 SSL은 self-supervised, L06–L07 슬라이드의 SSL은 semi-supervised야 (예: "Most SSL methods combine supervised and unsupervised objectives", L06 p.8). pre-train → fine-tune과의 차이는 fine-tuning 항목을 봐.

RL CURL은 RL에 self-supervised(contrastive) loss를 보조로 붙인 예야 (L05 p.20–21).

MLP (multi-layer perceptron)
L04 p.23–28 · L05 p.16

Linear layer 여러 개 사이에 ReLU 같은 비선형 함수를 끼운 작은 네트워크야. "Linear → ReLU → Linear"가 제일 흔한 2층 MLP야.

예 숫자 4개짜리 벡터 → Linear → 8개 → ReLU → Linear → 2개. 벡터를 다른 크기의 벡터로 바꿔줘.

왜 이 수업에서 encoder 뒤에 붙는 projection head와 BYOL의 predictor가 작은 MLP야. SimCLR는 head가 non-linear(MLP)일 때 linear나 head 없음보다 좋았고 (L04 p.28), MoCo v2도 SimCLR의 MLP projection head를 가져다 썼어 (더 강한 augmentation과 함께, L05 p.16).

RL policy head, value head로 쓰는 작은 MLP와 같은 물건이야.

Logit

L06 p.10 · L05 p.12, p.35

softmax에 넣기 **전**의 날것 점수야. 범위 제한이 없는 실수이고, 클수록 그 class일 가능성이 높다는 뜻이야. L06 p.10 표기: $z = f_\theta(x)$, "a vector of logits".

예 3-class 분류기의 출력 [2.0, 1.0, 0.0]이 logit이야. 음수도 될 수 있고 합이 1일 필요도 없어.

왜 contrastive 코드에선 similarity 점수 자체를 logit으로 써. MoCo의 `logits = cat([l_pos, l_neg])`, CLIP의 `logits = np.dot(I_e, T_e.T) * np.exp(t)`가 그 예야. 이걸 cross-entropy에 넣으면 "후보 중 정답 고르기" 분류가 돼.

주의 기호 z가 L01·L03에선 representation이었는데, L06 notation에선 logit이야.

RL discrete policy network가 softmax 직전에 내는 action logit과 같아.

Softmax

L06 p.10 · L03 p.20

logit 벡터를 합이 1인 확률 분포로 바꾸는 함수야. $\text{softmax}(z)_i = \exp(z_i) / \Sigma_j \exp(z_j)$.

z_i는 i번째 칸의 logit이야. exp로 모든 값을 양수로 만들고, 분모(모든 칸의 exp 합)로 나눠서 합을 1로 맞춰.

예 [2, 1, 0] → 약 [0.67, 0.24, 0.09]. logit은 1씩 차이 나는데 확률은 0.67 vs 0.24로 약 2.7배(= e) 차이가 나. exp가 차이를 곱셈으로 키우는 거야.

왜 "N개 중 하나 고르기" 확률이 필요한 곳마다 나와. 분류기 출력 $\hat{y} = \text{softmax}(z)$ (L06 p.10), InfoNCE(후보 중 positive 고르기), DINO의 student·teacher 분포가 다 softmax야. 넣기 전에 temperature로 나누면 분포의 뾰족함을 조절할 수 있어.

RL Boltzmann policy $\pi(a|s) = \text{softmax}(Q(s,\cdot)/\tau)$와 같은 함수야.

One-hot · hard / soft label

L06 p.10 · L07 p.7, p.46

정답 칸만 1이고 나머지는 0인 벡터야. "$y \in \{0,1\}^L$, a one-hot representation of the true label" (L06 p.10, L = class 수).

예 class가 [고양이, 개, 자동차]일 때 "개" = [0, 1, 0]. 이런 게 hard label이야. [0.7, 0.2, 0.1]처럼 확률이 여러 칸에 퍼진 건 soft label이야.

왜 pseudo label을 만들 때 soft 예측을 argmax로 one-hot으로 굳혀 (L07 p.7: [0.96, 0.03, 0.01] → y* = argmax). FixMatch도 weak view 예측을 one-hot pseudo label로 바꿔 (L07 p.46). 반대로 MixMatch는 sharpening한 soft label을 써. hard ↔ soft 뒤바꾸기는 시험 함정으로 내기 좋아.

RL discrete action을 one-hot으로 나타내는 것과 같아.

Cross-entropy (CE)

L03 p.20, p.23 · L06 p.18

정답 분포 y와 모델 예측 분포 p가 얼마나 다른지 재는 loss야. $\text{CE}(y, p) = -\Sigma_c\, y_c \log p_c$.

y_c는 정답 분포에서 class c의 확률이고, p_c는 모델이 c에 준 확률이야. y가 one-hot이면 정답 칸만 남아서 CE = −log(정답 칸 확률)이 돼. 그 확률이 1에 가까우면 0, 0에 가까우면 아주 커져.

예 정답 "고양이" = [1, 0, 0], 예측 [0.7, 0.2, 0.1] → CE = −log 0.7 ≈ 0.36. 예측이 [0.1, 0.8, 0.1]이면 −log 0.1 ≈ 2.3으로 커져.

왜 이 수업의 기본 loss야. labeled 데이터의 supervised loss, InfoNCE ("Cross entropy loss for a N-way softmax classifier", L03 p.20), DINO의 $H(P_t, P_s)$(target이 soft 분포인 CE), FixMatch의 pseudo label loss가 다 CE야.

RL discrete action BC loss $-\log \pi(a_{expert}|s)$가 바로 one-hot target CE야.

Entropy
L07 p.5–7

분포 **하나**가 얼마나 불확실한지 재는 값이야. $H(p) = -\Sigma_c\ p_c \log p_c$. 확률이 고르게 퍼질수록 크고, 한 칸에 몰릴수록(one-hot) 0이 돼.

예 L07 p.5 예시: Cat/Dog/Car가 거의 균등한 예측은 H = 1.09 (high entropy, uncertain), Dog에 몰린 예측은 H = 0.09 (low entropy, confident).

왜 semi-supervised에선 unlabeled 예측의 entropy를 **줄이는** 게 목표야 (entropy minimization). 그러면 decision boundary가 데이터가 드문 곳(low-density)으로 밀려나 (L07 p.6). pseudo label은 이걸 hard label로 근사한 거고 (p.7), MixMatch의 sharpening도 같은 효과를 노려.

구분 entropy는 분포 하나, cross-entropy는 분포 두 개(정답·예측)를 봐.

RL SAC의 entropy bonus는 탐험하려고 entropy를 **키워**. 여기선 확신을 높이려고 **줄여**. 방향이 반대야.

KL divergence
L06 p.10, p.28 · L04 p.18

두 확률 분포 p, q가 얼마나 다른지 재는 값이야. $KL(p\|q) = \Sigma_i\ p_i \log(p_i/q_i)$. 두 분포가 같으면 0이고, 다를수록 커져.

순서가 중요해 (KL(p‖q) ≠ KL(q‖p)). 보통 p가 target, q가 모델 예측이야.
$KL(p\|q) = CE(p, q) - H(p)$라서, target p가 고정이면 KL을 줄이는 것과 CE를 줄이는 게 같은 방향이야. target이 one-hot이면 H(p) = 0이라 KL = CE가 돼.

예 원본 사진의 예측 [0.8, 0.2]와 augment한 사진의 예측 [0.6, 0.4] 사이 KL을 줄이면 두 예측이 같아져. 이게 consistency loss야.

왜 consistency loss의 거리 D로 MSE · CE · KL이 다 쓰여 (L06 p.10). UDA는 원본과 augment 버전 예측의 KL을 줄이고 (L04 p.18), Mean Teacher는 "MSE performs better than KL divergence"라고 했어 (L06 p.28). 어느 방법이 어느 거리를 쓰는지가 함정 포인트야.

RL PPO·TRPO에서 old policy와 new policy가 너무 멀어지지 않게 거는 KL과 같은 양이야.

MSE (mean squared error)
L06 p.17–18, p.28 · L04 p.32, p.41

두 벡터의 칸별 차이를 제곱해서 평균 낸 값이야. $MSE(a, b) = (1/C)\ \Sigma_c\ (a_c - b_c)^2$ (C = 칸 수). 확률 분포가 아닌 feature 벡터끼리도 쓸 수 있어.

예 a = [0.9, 0.1], b = [0.6, 0.4] → 차이 [0.3, −0.3] → 제곱 [0.09, 0.09] → 평균 0.09.

왜 Π-model · Temporal Ensembling · Mean Teacher는 두 예측(softmax 출력)의 차이를 MSE로 줄여 (L06 p.18, p.28). BYOL은 L2-normalize한 **feature 벡터**끼리 MSE를 써: $\|\bar{q} - \bar{z}'\|^2 = 2 - 2\cdot\cos$ ($\bar{q}$, $\bar{z}'$: 길이 1로 L2-normalize한 벡터, L04 p.32). 그래서 L04 p.41 표가 BYOL은 "MSE Loss", DINO는 "Cross Entropy Loss"로 대비시켜.

RL TD error 제곱, flow matching의 velocity regression loss가 다 MSE야.

Cosine similarity
L03 p.31 · L04 p.24–25 · L05 p.35

두 벡터가 얼마나 같은 **방향**을 보는지 재는 값이야. $sim(u, v) = u^\top v / (\|u\|\ \|v\|)$. 범위는 −1(반대 방향)부터 1(같은 방향)까지야.

$u^T v$는 내적(dot product), $\|u\|$는 벡터 길이야. 길이로 나눠서 크기 영향을 없애고 방향만 비교해.

예 [1, 0]과 [3, 0] → 1 (길이는 달라도 같은 방향). [1, 0]과 [0, 1] → 0 (직각). [1, 0]과 [−1, 0] → −1.

왜 contrastive learning의 기본 similarity야 (L03 p.31, SimCLR의 $s_{i,j}$ L04 p.25). 벡터를 먼저 L2-normalize(길이 1로)하면 내적이 곧 cosine이라, CLIP 코드는 `l2_normalize` 후 내적만 해 (L05 p.35). 다만 cosine은 −1~1로 범위가 좁아서 그대로 softmax에 넣으면 너무 평평해. 그래서 보통 temperature τ로 나눠서 써.

모델 부품

Encoder f · backbone
L01 p.12, p.17

입력 x(이미지 등)를 벡터 z로 바꾸는 본체 네트워크야 (ResNet, ViT). 슬라이드 표기는 `z = f(x)`. 위에 여러 head를 얹는 몸통이라서 backbone이라고도 해.

예 224×224 컬러 사진(숫자 약 15만 개) → encoder → 숫자 몇백~몇천 개짜리 벡터.

왜 이 수업의 최종 목표는 좋은 encoder야. pretext task, contrastive loss, teacher–student는 전부 encoder를 잘 훈련시키는 수단이고, 학습이 끝나면 encoder를 여러 task에 재사용해 (L01 p.17).

RL observation encoder. pixel → latent state.

Representation · embedding · feature
L01 p.11–12

encoder가 내놓은 결과 벡터 z야. 이 수업에선 세 단어를 거의 같은 뜻으로 써. 좋은 z는 쓸모 있는 정보(무슨 물체인지)는 남기고 배경·조명·노이즈 같은 건 버린 거야 (L01 p.11).

예 같은 강아지를 밝게 찍은 사진과 어둡게 찍은 사진 → z가 가까워야 좋아. 강아지와 자동차 → z가 멀어야 좋아.

왜 representation learning이 바로 이 z를 잘 만드는 법을 배우는 분야야. 좋은 z 위에선 linear classifier 하나만 붙여도 분류가 잘 돼 (linear probing).

RL state feature $\varphi(s)$. linear value function $V(s) = w^T\varphi(s)$가 잘 되려면 φ가 좋아야 하는 것과 같은 얘기야.

Projection head g
L04 p.23–25, p.28

encoder 출력 h 뒤에 붙는 작은 MLP야. loss는 `z = g(h)`에서 계산하고, 학습이 끝나면 g는 버리고 h를 써 (SimCLR Alg.1 "return encoder network f(·), and throw away g(·)", p.25).

예 SimCLR: 사진 → f → h → g → z. 두 view의 z끼리 InfoNCE를 계산하고, downstream에는 h만 가져가.

왜 contrastive loss는 z를 augmentation에 invariant하게 만들어서, 색·방향 같은 정보가 z에서 사라져. g가 이 "버리기"를 떠맡으면 h에는 정보가 더 남아 (p.28의 "possible explanation"). 실험에서도 non-linear head > linear head > head 없음이었어 (p.28).

시험 "g를 버리고 h를 쓴다"를 "h를 버리고 g를 쓴다"로 뒤집은 지문 주의.

Predictor q
L04 p.30–32, p.41

BYOL에서 online 쪽에만 붙는 작은 MLP야. (BYOL은 네트워크 두 개를 써: gradient로 학습하는 **online**과, online 가중치의 EMA로 천천히 따라가는 **target**. 일반 이름으로는 student · teacher야. projection z는 projection head의 출력이야.) predictor는 online의 projection z를 받아서 target의 projection z′을 **예측**해: $q_\theta(z_\theta) \approx z'$.

예 online: view v → f_θ → g_θ → q_θ. target: view v′ → f_ξ → g_ξ (predictor 없음).

왜 predictor가 한쪽에만 있어서 두 branch가 비대칭이 돼. 이 비대칭이 EMA target · stop-gradient(target 쪽으로 gradient를 안 흘림)와 함께 negative 없이 collapse를 막는 데 기여해.

시험 DINO에는 predictor가 없어. L04 p.41 표에서 BYOL의 "Predictor"와 DINO의 "Softmax probability"가 같은 줄에 놓였을 뿐이야.

Prototype · code
L05 p.23–30

prototype은 학습되는 cluster 중심 벡터야 (SwAV는 K개, $C = [c_1, \ldots, c_K]$). code는 feature를 prototype들에 배정한 결과, 곧 "어느 cluster에 얼마나 속하나"를 나타낸 분포야 (SwAV).

예 prototype이 3개라면 강아지 사진 feature의 code가 [0.8, 0.1, 0.1]처럼 나와. 1번 cluster에 주로 속한다는 뜻이야.

왜 feature끼리 전부 비교하는 대신 K개 prototype하고만 비교하니까 싸 (L05 p.30: Feature ↔ Prototype). SwAV는 한 view의 code를 다른 view로 맞히게 해 (swapped prediction, p.27). 그 전의 DeepCluster는 epoch마다 dataset 전체 feature를 k-means로 다시 묶고(offline), 그 cluster 번호를 pseudo label로 써서 분류 학습을 했어 (p.23–24). SwAV는 batch가 들어올 때마다 그 자리에서 code를 계산해 (online clustering, p.26, p.30). 여기서 online은 "그때그때 바로"라는 뜻이고, BYOL의 online network와는 상관없어.

CLS token · patch token
L04 p.42, p.55–67

ViT는 이미지를 작은 조각(patch)으로 잘라 조각마다 token(벡터)을 만들어. 여기에 이미지 전체를 요약하는 특별한 token [CLS]를 하나 더 붙여. 그래서 출력도 CLS token 1개(image-level)와 patch token 여러 개(patch-level, dense)로 나와.

예 이미지를 4×4 = 16조각으로 자르면 patch token 16개 + CLS token 1개.

왜 분류처럼 이미지 전체를 판단할 땐 CLS, segmentation처럼 위치마다 판단할 땐 patch token이 필요해. DINO는 CLS 쪽만 맞추고 (p.57 "Image-level self-distillation", p.61 "Match the student and teacher CLS-token distributions"; 그 CLS의 attention이 물체 윤곽을 따라가는 게 p.42), DINOv2는 CLS(DINO loss, image-level)와 patch(iBOT loss, patch-level)를 같이 학습해 (p.61).

학습 · 평가 설정

Pretext task
L02 p.16–17, p.20

진짜 목적이 아니라 encoder를 훈련시키려고 만든 연습 문제야. "Self-supervised task used for learning representations. Often, not the 'real' task ... we care about." (L02 p.20). 정답(label)은 데이터에서 자동으로 나와.

예 사진을 90° 돌려 넣고 "몇 도 돌렸어?" 맞히기(rotation). 내가 돌렸으니 정답은 공짜야. 그 밖에 relative position, inpainting, jigsaw, colorization (L02 p.21–43).

왜 회전을 맞히려면 "개는 머리가 위에 있다" 같은 물체 이해가 필요해. 그래서 문제를 푸는 동안 encoder가 쓸모 있는 feature를 배워 (p.17). 다 배우면 pretext용 head는 버리고 encoder만 가져가 (p.16).

RL auxiliary task (본 목적은 아니지만 encoder를 키우는 보조 과제).

Downstream task
L02 p.16, p.18

진짜 하고 싶은 일이야 ("The application you care about", L02 p.16). 분류, detection, segmentation 같은 것. 보통 label이 적어.

예 rotation pretext로 encoder를 학습 → 그 encoder에 작은 head를 붙여, label 있는 사진 몇백 장으로 "개/고양이 분류"를 학습. 이 분류가 downstream이야.

왜 self-supervised 방법의 좋고 나쁨은 pretext 점수가 아니라 downstream 성능으로 재 (L02 p.18). 재는 방법은 linear probing과 fine-tuning이 대표적이야.

Linear probing · linear evaluation
L02 p.18, p.48 · L04 p.26

encoder를 얼려 놓고(freeze, 가중치 고정) 그 위 linear classifier 하나만 label로 학습해서 representation 품질을 재는 거야. "Linear classifiers on 'fixed' features" (L02 p.48).

예 encoder가 내는 z에 $\hat{y} = \text{softmax}(Wz + b)$의 W, b만 학습해. encoder 가중치는 그대로야.

왜 classifier가 단순하니까 점수 차이가 곧 representation 품질 차이야. SimCLR·BYOL·DINO 결과 표의 "Linear" 열이 이 점수야 (L04 p.26, p.33, p.43).

시험 linear probing = encoder 고정, fine-tuning = encoder까지 다시 학습. SimCLR 슬라이드 두 장이 제목은 같지만 내용이 달라: p.26 "Freeze feature encoder, train a linear classifier on top" vs p.27 "Finetune the encoder with 1% / 10% of labeled data".

RL 고정된 feature $\varphi(s)$ 위에서 linear value function만 학습하는 것과 같은 구조야.

Fine-tuning
L06 p.4 · L04 p.27

미리 학습한(pre-trained) 가중치에서 출발해서, downstream label 데이터로 encoder까지 **같이** 더 학습하는 거야. "Pre-train a powerful task-agnostic model on a large unsupervised data corpus. Then fine-tune it on the downstream task with a small set of labeled samples" (L06 p.4).

예 SimCLR로 학습한 encoder를 ImageNet label 1%(또는 10%)만으로 encoder째 더 학습 (L04 p.27).

왜 encoder도 그 task에 맞게 조금 바뀔 수 있어서 보통 점수가 더 잘 나와. 대신 "representation 자체가 얼마나 좋은지"만 따로 보기엔 linear probing보다 덜 깨끗해.

시험 pre-train → fine-tune은 2단계를 차례로(sequential), semi-supervised는 labeled + unlabeled를 한 번에(joint) 학습해 (L06 p.4 vs p.7).

RL pretrained VLA를 내 로봇 데이터로 fine-tune하는 것과 같은 말이야 (슬라이드 밖 예).

Zero-shot
L05 p.32–33, p.37

그 task용으로 따로 학습(fine-tune)하지 않고 바로 푸는 거야. CLIP은 class 이름을 문장으로 만들어 이미지와 비교해서 분류해.

예 class {dog, cat, car} → "A photo of a {object}." 틀에 넣은 prompt 3개 → text encoder로 벡터 3개 → 이미지 벡터와 cosine similarity가 가장 큰 문장의 class가 답 (L05 p.33).

왜 고정된 classifier head 없이 text가 곧 classifier 역할을 해. 그래서 학습 때 없던 class도 문장만 바꾸면 분류할 수 있어. CLIP이 특별한 이유 중 하나야 (p.37).

Augmentation · view
L03 p.32 · L04 p.13–14

같은 이미지를 자르고(crop), 색을 바꾸고(color jitter), 흐리게 하고(blur), 뒤집어서(flip) 변형하는 거야. 변형본 하나하나를 view라고 해. label이 없을 땐 같은 이미지에서 나온 두 view가 positive pair야 (L03 p.32).

예 batch에 사진 4장 → 각각 두 번 augment → view 8개. view 하나를 기준으로 짝 view 1개가 positive, 나머지 6개가 negative야 (SimCLR식: 2N개 중 2(N−1)개가 negative).

왜 좋은 augmentation은 겉모습(appearance)은 크게 바꾸되 의미(semantic)는 지켜야 해 (L04 p.13). 너무 약하면 pixel 비교 같은 꼼수(shortcut)로 맞혀 버리고, 의미까지 바꾸면 다른 걸 같다고 배워. 결국 어떤 augmentation을 쓰느냐가 "무엇을 무시할지"(invariance)를 정해.

RL pixel RL에서 쓰는 random crop과 같은 도구야. CURL은 같은 observation을 두 번 random crop해서 positive로 써 (L05 p.20–21).

Invariance · equivariance
L02 p.73–76 · L04 p.28

invariance는 입력을 변형해도 representation이 **변하지 않는** 거야: `f(t(x)) = f(x)`.

equivariance는 변형에 따라 representation이 **예측 가능하게 변하는** 거야: `f(t(x)) = φ(t, f(x))` (L02 p.75).

t는 변형(회전, 색 바꾸기 …), φ(t, ·)는 "t를 하면 representation이 이렇게 움직인다"는 규칙이야. φ가 아무것도 안 하면(identity) invariance가 돼. 그래서 invariance는 equivariance의 특수한 경우야.

예 삼각형을 돌렸을 때 invariant 표현은 그냥 "삼각형", equivariant 표현은 "돌아간 삼각형"까지 담아.

왜 contrastive learning은 두 view를 같게 만드니까 augmentation에 invariant해져. 그런데 detection은 위치, 꽃 색 분류는 색이 필요해서 crop·color invariance가 오히려 손해일 수 있어 (L02 p.73–74). 그래서 변형 정보를 보존하는 equivariant 학습(STL, p.76)이 나왔어. SimCLR projection head도 invariance 부담을 g가 떠안게 하는 장치야 (L04 p.28).

시험 invariance = "does not change", equivariance = "changes predictably". 정의를 서로 바꾼 지문 주의.

Collapse
L04 p.37, p.40, p.62–63

모든 입력이 같은(또는 거의 같은) 벡터로 뭉개지는 거야. "trivial constants (i.e. collapsed representations)" (L04 p.37). loss는 낮은데 쓸모가 없어.

예 loss가 "두 view의 출력을 같게"만 요구하면, encoder가 모든 사진에 같은 벡터를 내도 두 view는 같으니까 loss가 0이야. 하지만 개와 자동차도 같은 벡터라 정보가 하나도 없어.

왜 그래서 "당기기"만 하는 방법엔 막는 장치가 꼭 있어. contrastive는 negative를 밀어내서 막아. negative 없는 방법은 각자 다른 장치를 써: BYOL = predictor + EMA target + stop-gradient, Barlow Twins = cross-correlation을 identity로 (redundancy reduction), DINO = centering + sharpening. DINOv2는 KoLeo(feature가 한곳에 몰리지 않게)를 더하고, centering을 Sinkhorn-Knopp 방식(prototype 몇 개에만 쏠리지 않게)으로 바꿔 (p.62–63).

Contrastive

Anchor(query) · positive · negative
L03 p.31–32

contrastive learning에서 샘플이 맡는 세 역할이야.

anchor(MoCo에선 query)는 기준 샘플, **positive**는 anchor와 가까워져야 할 샘플(같은 이미지의 다른 view, label이 있으면 같은 class), **negative**는 멀어져야 할 샘플(batch 안의 다른 이미지)이야.

예 강아지 사진 A를 crop한 view = anchor, A의 색만 바꾼 view = positive, 고양이 사진 B와 자동차 사진 C의 view = negative.

왜 label 없이 "무엇이 같고 무엇이 다른지"를 정해 주는 장치야. positive를 당기기만 하면 collapse가 나니까 negative를 밀어내는 힘이 필요해. MoCo에선 비교 대상을 key라고 불러 (positive key k^+, queue 안의 negative key들).

Hard negative · false negative
L03 p.14 · L04 p.6–8

hard negative는 anchor와 embedding이 가까운데 사실 다른 class인 샘플이야. 모델이 헷갈리니까 gradient가 크고 배울 게 많아 (L03 p.14). **false negative**는 negative로 뽑혔는데 사실 anchor와 같은 class인 샘플이야 (L04 p.7).

예 anchor가 허스키 사진이면, 늑대 사진은 hard negative(닮았지만 다른 class)야. 다른 허스키 사진이 negative로 뽑혔다면 그건 false negative야.

왜 대부분의 negative는 이미 멀어서 배울 게 없어. 그래서 hard negative에 집중하면 학습 신호가 강해져 (hard negative mining). 반대로 false negative는 같은 class를 억지로 밀어내서 해로워. Debiased Contrastive Loss가 이걸 label 없이 보정해 (L04 p.8–11).

RL TD error가 큰 transition을 더 자주 뽑는 prioritized replay처럼 "어려운 샘플에 집중"하는 발상이야 (철학이 비슷할 뿐 같은 기법은 아님).

InfoNCE · NT-Xent

L03 p.20–28, p.31 · L04 p.24

"후보 N개 중 어느 것이 positive인지 맞히는" N-way softmax cross-entropy야. 사실상 이 수업의 기본 loss야.

$L = -\log \left[\exp(s^+/\tau) \,/\, \Sigma_k \exp(s_k/\tau) \right]$

s^+는 anchor와 positive의 similarity(보통 cosine), s_k는 anchor와 후보 k의 similarity야. 분모는 positive 1개 + negative 전부를 더해. 대괄호 안이 softmax로 계산한 "positive를 고를 확률"이고, 여기에 −log를 씌우면 CE야. loss를 줄이려면 분자(positive similarity)는 키우고 분모 속 negative similarity는 줄여야 해. 당기기와 밀어내기가 식 하나에 다 들어 있어.

예 positive 1 + negative 6 → 7지선다 분류 문제. 정답 칸 = positive.

왜 분류 문제로 바꾸면 보통 CE 코드로 구현할 수 있어. MoCo는 positive logit을 0번 칸에 붙여서 `labels = zeros(N)`, CLIP은 대각선이 정답이라 `labels = np.arange(n)`야. NT-Xent(Normalized Temperature-scaled Cross Entropy)는 SimCLR에서 부르는 이름이야. InfoNCE를 잘 학습하는 건 두 쪽의 mutual information(한쪽을 알면 다른 쪽에 대해 얼마나 알게 되는지 재는 정보량)을 키우는 것으로도 해석돼 (L03 p.27–28).

RL discrete action BC의 softmax CE와 같은 모양이야. 다만 class 자리에 batch의 다른 샘플들이 들어가서 후보가 batch마다 바뀌어.

Temperature τ

L03 p.29, p.31, p.68 · L04 p.40 · L05 p.35

softmax에 넣기 전에 similarity(또는 logit)를 나누는 숫자야: `softmax(s/τ)`. 분포가 얼마나 뾰족할지 정하는 손잡이야.

τ가 작으면 값 차이가 커져서 분포가 뾰족해져. 제일 큰 값이 거의 다 가져가니까, negative 중엔 anchor와 가까운 hard negative에 집중하게 돼. τ가 크면 차이가 줄어서 분포가 평평해져.

예 점수 [2, 1, 0]: τ = 1이면 약 [0.67, 0.24, 0.09], τ = 0.5면 약 [0.87, 0.12, 0.02], τ = 2면 약 [0.51, 0.31, 0.19].

왜 cosine similarity는 [−1, 1]로 좁아서 그대로 softmax하면 너무 평평해. τ로 나눠서 차이를 키워 줘 (L03 p.31). DINO는 teacher에 낮은 temperature를 써서 teacher 분포를 뾰족하게 만들어 (sharpening, L04 p.40). CLIP은 τ를 사람이 정하지 않고 학습해 (`* np.exp(t)`, L05 p.35).

RL Boltzmann(softmax) policy의 temperature와 같은 역할이야. τ → 0이면 greedy, τ → ∞면 uniform.

Memory bank · queue · dictionary

L05 p.4–14

이전 step에서 계산해 둔 embedding을 저장해 두고 negative로 재사용하는 저장소야. "Trade stale representations for efficient large-scale contrastive learning" (L05 p.4).

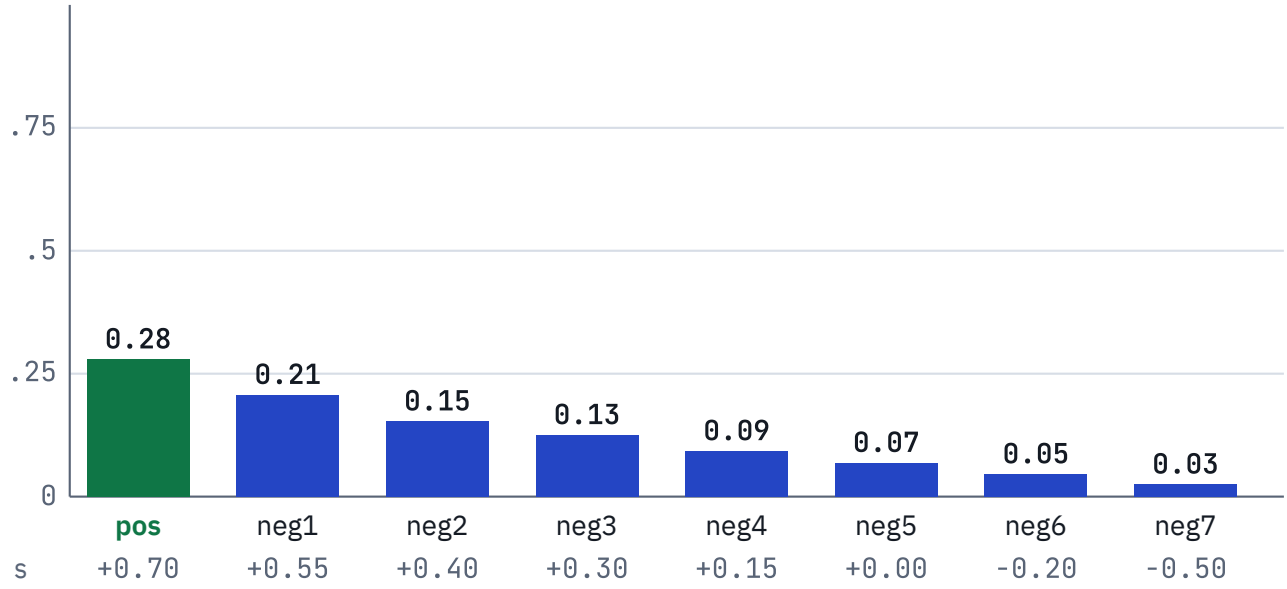
예 batch는 작아도, 저장해 둔 과거 embedding 수만 개를 negative로 꺼내 쓰면 negative 수가 batch 크기와 상관없어져.

왜 negative는 많을수록 좋은데, SimCLR처럼 batch 안에서만 구하면 큰 batch(= 큰 GPU 메모리)가 필요해. 대가는 **staleness**야: 저장된 embedding은 옛날 encoder로 만든 거라 지금 encoder와

어긋나. MoCo는 이 저장소를 dictionary라고 부르고, 크고(large) 일관된(consistent) dictionary를 원해. 그래서 bank를 FIFO **queue**로 바꾸고 key를 천천히 변하는 momentum encoder로 만들어 어긋남을 줄였어 (L05 p.10–11).

RL replay buffer처럼 "조금 오래된 데이터를 재사용"하는 trade-off야.

인터랙티브 **temperature τ가 softmax를 어떻게 바꾸나** `illustrative`



읽는 법: anchor 하나에 후보 8개가 있는 상황이야: positive 1개(pos) + negative 7개(neg1–neg7). 막대 밑 숫자는 anchor와 각 후보의 similarity s(고정값, 지어낸 값)이고, 막대 높이는 `softmax(s/τ)`, 곧 "이 후보가 positive일 확률"이야. 함께 표시되는 loss는 −log p(pos), 곧 InfoNCE loss야.

볼 것: 슬라이더로 τ를 움직여 봐. τ를 줄이면 s 차이가 커져서 **제일 큰 s 쪽으로 확률이 몰려(뾰족)**: pos는 더 확실해지고, negative 중에선 anchor와 가까운 hard negative neg1이 나머지보다 훨씬 큰 몫을 가져가. τ를 키우면 차이가 줄어서 8개가 거의 균등해져(평평). 여기선 pos의 s가 제일 커서 τ를 줄일수록 loss가 작아져. 하지만 pos보다 가까운 negative가 있으면, 작은 τ에선 그 hard negative가 분모를 거의 혼자 차지해서 loss와 gradient가 거기에 집중돼.

주의: SimCLR · MoCo · DINO · X-CLR은 `s/τ`로 나누고, **CLIP만** `logits * exp(t)`로 곱해 (learned, exp(t) = 1/τ). 개념 그림이야. (L03 p.29 Soft-NN · p.68 X-CLR · L05 p.35 CLIP)

Teacher – student

Student · teacher

= online·target, query·key · L04 p.30–31 · L06 p.24 · L07 p.15

네트워크 두 개가 짝을 이뤄서, **teacher**가 학습 target(정답 역할)을 만들고 **student**가 그걸 따라 하도록 학습하는 구조야. 방법마다 이름만 달라: BYOL은 online·target, MoCo는 query·key encoder, DINO·Mean Teacher는 student·teacher.

어떻게 MoCo · BYOL · DINO · Mean Teacher에선 둘이 같은 구조야 (BYOL은 online에 predictor만 더 있어). gradient는 student만 받고, teacher는 backprop 없이 student 가중치의 EMA로 천천히 따라가. "teacher는 학습 안 한다"가 아니라 "gradient 대신 EMA로 갱신된다"가 정확해.

예외 Noisy Student는 먼저 학습한 모델이 teacher야. 같거나 더 큰 student가 noise를 받으며 학습하고, 다음 회차의 teacher가 돼 (L07 p.15–18). DINOv2는 먼저 큰 ViT-g를 학습한 뒤(Step 1), 그 모델을 얼려서 teacher로 삼고 작은 ViT들에 distill하는 단계를 따로 둬 (Step 2, L04 p.65).

반대로 FixMatch는 별도 teacher가 없고, IFMatch의 "teacher branch"도 별도 network가 아니라 같은 모델(같은 parameter)에 weak image를 넣은 갈래야 (L07 p.97).

왜 자기 출력을 그대로 target으로 쓰면 target도 매 step 같이 흔들려서 불안정하고 collapse하기 쉬워. 천천히 변하는 teacher가 안정적인 target을 줘 (BYOL target network: "Provides a stable learning target", L04 p.31).

RL DQN의 online network(gradient로 학습) ↔ target network(천천히 갱신)와 같은 구조야.

EMA · momentum update

L04 p.30, p.40 · L05 p.11–12 · L06 p.20–28

Exponential Moving Average. teacher 가중치를 "거의 그대로 두고 student를 아주 조금만 섞는" 방식으로 갱신해: $\theta_t \leftarrow m \cdot \theta_t + (1-m) \cdot \theta_s$.

θ_t는 teacher 가중치, θ_s는 student 가중치, m은 momentum이야 (1에 가까움, 0.99–0.999 정도). m이 클수록 teacher가 천천히 움직여. 대략 최근 1/(1−m) step 동안의 student를 평균한 셈이야 (0.99 → 약 100 step, 0.999 → 약 1000 step).

예 m = 0.9, teacher 값 1.0, student 값 2.0 → 새 teacher = 0.9×1.0 + 0.1×2.0 = 1.1. 한 번에 차이의 10%만 다가가.

왜 천천히 변하는 teacher가 안정적인 target과 일관된 key를 줘. 기호는 방법마다 달라: MoCo `m`, BYOL τ ($\xi \leftarrow \tau\xi + (1-\tau)\theta$), DINO `l`(소문자 L), Mean Teacher β (초반 0.99 → 후반 0.999, L06 p.28).

시험 **weight EMA**(가중치를 평균: MoCo · BYOL · DINO · Mean Teacher) ↔ **prediction EMA**(샘플별 예측을 평균: Temporal Ensembling, $Z \leftarrow \alpha Z + (1-\alpha)z$, L06 p.20). DINO의 center C도 EMA지만 가중치가 아니라 teacher 출력의 평균이야.

RL target network의 Polyak(soft) update와 같은 식이야. SAC의 $\tau = 0.005$는 m = 0.995인 셈이야.

Stop-gradient `detach`

L04 p.30, p.40 · L05 p.12 · L03 p.66–67

한쪽(target 쪽) 출력을 상수처럼 취급해서 gradient가 그쪽 네트워크로 흐르지 않게 막는 거야. PyTorch에선 `.detach()`, 그림에선 "sg"로 표시해.

예 MoCo `k = k.detach()` (key 쪽 차단 → query encoder만 SGD), DINO `t = t.detach()` (teacher 쪽 차단), BYOL 그림의 sg(z′), X-CLR `txt_enc(captions).detach()` (고정된 text encoder 쪽 차단).

왜 양쪽이 다 gradient를 받으면, loss를 줄이는 가장 쉬운 길이 "둘 다 같은 상수로 가기"가 될 수 있어 (collapse). 또 target이 학습 중에 같이 움직이면 쫓아가기가 불안정해. 그래서 끊는 쪽은 항상 **target(teacher · key) 쪽**이야. 끊긴 쪽은 gradient 대신 EMA로 천천히 갱신되거나(MoCo · BYOL · DINO · Mean Teacher), 아예 고정돼 있어(X-CLR의 미리 학습된 text encoder, L03 p.66).

시험 SimCLR에는 stop-gradient도 EMA도 없어. 두 view가 같은 encoder를 지나고 둘 다 gradient를 받아.

RL TD target $r + \gamma Q_{target}$에 gradient를 안 흘리는 semi-gradient와 같은 이유야. 움직이는 target을 쫓다가 무너지지 않게.

Distillation

L04 p.39–41, p.65 · L07 p.18

teacher의 출력(보통 확률 분포)을 student가 따라 하게 학습하는 거야. loss는 보통 CE야: $H(P_t, P_s) = -\Sigma\, P_t \log P_s$ (P_t: teacher 분포 = target, P_s: student 분포).

예 teacher가 어떤 사진에 [개 0.7, 늑대 0.2, 고양이 0.1]을 내면, student는 one-hot "개" 대신 이 분포 전체를 따라 해. "늑대랑 좀 닮았다"는 정보까지 배울 수 있어.

왜 보통 knowledge distillation은 이미 학습된 큰 teacher의 지식을 작은 student로 옮기는 거야. DINO는 teacher를 student 자신의 EMA로 만들어 label 없이 하는 **self-distillation**이야 ("Self-Distillation with No Labels", L04 p.39). DINOv2는 큰 ViT-g teacher를 얼려서 작은 student들에 distill해 (p.65). Noisy Student는 반대로 student가 teacher와 같거나 더 커 (L07 p.18).

RL policy distillation(teacher policy의 action 분포를 student policy가 따라 하기)과 같아.

Centering · sharpening (DINO)
L04 p.40, p.63

DINO가 negative 없이 collapse를 막는 두 장치야. 둘 다 **teacher 출력에만** 걸어.

$P_t = \text{softmax}((g_t(x) - C) / \tau_t)$, $C \leftarrow m \cdot C + (1-m) \cdot \text{mean(teacher 출력)}$

$g_t(x)$는 teacher 출력(logit), C는 center(지금까지 teacher 출력의 EMA 평균), τ_t는 낮게 잡은 teacher temperature야.

Centering: C를 빼서 "항상 같은 차원만 큰" collapse를 막아. 혼자 쓰면 분포가 평평(uniform)해지는 쪽으로 가. **Sharpening**: 낮은 τ_t로 분포를 뾰족하게 해서 uniform collapse를 막아. 두 힘이 반대 방향이라 균형이 맞아. (이 역할 분담은 DINO 논문 쪽 설명이야. 슬라이드엔 pseudo code 주석 `# center + sharpen` 뿐이고, 시험 포인트는 "centering은 teacher 쪽에만".)

예 모든 사진에서 teacher 출력의 1번 칸만 크게 나오면 평균 C의 1번 칸도 커져. 빼고 나면 1번 칸의 일방적인 우위가 사라져.

시험 centering은 student가 아니라 teacher 쪽이야. 코드: `t = softmax((t - C) / tpt)`, `C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)`. DINOv2는 Sinkhorn-Knopp centering을 써 (p.63).

Semi-supervised

목표식 $L = L_s + \mu(t) \cdot L_u$
L06 p.8–10

semi-supervised 방법 대부분이 이 모양이야: $L = L_s + \mu(t) \cdot L_u$ ("supervised loss, unsupervised loss, weighting term", L06 p.8).

L_s는 labeled 데이터의 supervised loss(보통 CE), L_u는 unlabeled 데이터의 unsupervised loss(consistency 또는 pseudo label), $\mu(t)$는 L_u의 가중치야. $\mu(t)$는 학습 시간 t가 갈수록 **커져** ("increasing in time", L06 p.10). Π-model · Temporal Ensembling 코드에선 같은 걸 w(t)로 써.

예 labeled 100장 + unlabeled 1만 장. 처음엔 $\mu \approx 0$이라 사실상 labeled만으로 학습하고, 모델이 쓸 만해지면 μ를 키워서 unlabeled 쪽 신호를 점점 더 믿어.

왜 학습 초기엔 모델 예측이 엉망이라 unlabeled에 만들어 주는 target도 엉망이야. 이때 L_u를 크게 주면 틀린 target을 강화해 (confirmation bias). 그래서 0 근처에서 시작해서 키워 (ramp-up).

시험 $\mu(t)$는 시간이 갈수록 커져. 줄어든다는 지문은 F.

Consistency regularization
L06 p.15–16

입력이나 모델에 작은 perturbation(교란: augmentation, dropout 등)을 줘도 예측은 같아야 한다는 loss야. "Predictions should remain consistent under small perturbations." (L06 p.16)

$L_u = D(f(x), f(A(x)))$: x는 unlabeled 입력, A(x)는 변형한 입력, D는 두 예측 사이 거리(MSE · KL · CE)야. 식에 label y가 없어서 unlabeled에도 계산할 수 있어.

예 고양이 사진 원본의 예측 [0.8, 0.2], 좌우로 뒤집은 사진의 예측 [0.5, 0.5] → 둘이 같아지도록 학습.

왜 "가까운 입력엔 비슷한 예측"(smoothness 가정, L06 p.11)을 loss로 만든 거야. 방법들은 perturbation 종류와 target 쪽만 달라: Π-model(같은 모델 두 번 forward), Temporal Ensembling(과거 예측의 EMA), Mean Teacher(EMA teacher), VAT(adversarial noise), ICT(MixUp), UDA(strong augmentation).

주의 L05 MoCo의 "consistent dictionary"는 다른 뜻이야. queue 안 key들이 거의 같은(천천히 변하는) encoder로 만들어져서 서로 비교 기준이 맞는다는 말이지, 이 consistency loss가 아니야.

RL 같은 상황을 조금 다르게 찍은 observation 두 장에 같은 action을 내야 한다는 robustness 조건과 같아.

Pseudo label

L07 p.4-7 · L02 p.12 · L05 p.23

사람이 단 게 아니라 **모델이 자기 예측으로 만든** 가짜 정답이야. "Predict labels for unlabeled samples using the current model. Treat high-confidence predictions as pseudo labels." (L07 p.4)

예 unlabeled 사진의 예측 [0.96, 0.03, 0.01] → 확신이 높으니 argmax를 정답으로 삼아 one-hot [1, 0, 0] (L07 p.7). 이걸로 CE 학습.

왜 unlabeled 데이터를 supervised처럼 쓸 수 있게 해 줘. 예측을 confident하게 만드는 효과라서 entropy minimization의 hard-label 근사로 볼 수 있어 (p.5–7). 위험은 틀린 예측을 정답으로 굳히는 confirmation bias야. 그래서 high-confidence만 쓰고(threshold), 보통 teacher → student를 반복하는 구조로 돌려 (L07 p.15).

참고 L02 p.12의 self-supervised 정의에도 "Create Pseudo Labels"가 나와. 거기선 "데이터에서 자동으로 만든 정답"(가린 부분 등)이라는 넓은 뜻이야. DeepCluster의 cluster 번호도 pseudo label이라고 불러 (L05 p.23).

RL 자기 Q 추정치로 TD target을 만드는 bootstrapping과 같은 발상이야.

Confidence threshold τ

L07 p.42-48, p.64-75

예측의 최대 확률(confidence)이 τ 이상인 샘플만 pseudo label로 쓰고, 나머지는 loss에서 빼는 기준선이야. FixMatch는 loss 앞에 mask $1[\max(q) \geq \tau]$를 곱해 (q = weak view 예측). 조건이 맞으면 1(사용), 아니면 0(버림)이야.

예 τ = 0.9라고 하면 [0.96, 0.03, 0.01]은 통과해서 pseudo label "class 1"이 되고, [0.6, 0.3, 0.1]은 불확실해서 버려.

왜 틀렸을 가능성이 큰 예측을 정답으로 쓰면 confirmation bias가 생겨. "High confidence leads to high-quality pseudo labels." (L07 p.45). FixMatch ablation에선 threshold로 걸러내면 sharpening은 추가 이득이 거의 없었어 (p.48). FreeMatch는 τ를 고정하지 않고 학습 상태와 class에 따라 스스로 조절해 (self-adaptive, p.64–75).

시험 FixMatch의 τ는 temperature가 아니라 threshold야. 기호만 같아.

Weak · strong augmentation

L07 p.42-48

변형이 약한 augmentation과 센 augmentation이야. weak는 뒤집기와 약간의 이동 정도(flip-and-shift), strong은 RandAugment · CTAugment처럼 여러 변형을 세게 거는 거야 (AutoAugment, Cutout도 strong 쪽, L07 p.46).

예 weak: 고양이 사진을 좌우로 뒤집고 몇 pixel 옮김. strong: 색을 크게 바꾸고, 기울이고, 일부를 네모로 가림.

왜 FixMatch는 weak view에서 pseudo label을 만들고(쉬우니까 예측이 안정적), strong view가 그 label을 맞히게 학습해(어려우니까 robust해져). "Predict on an easy example, learn from a

difficult one." (L07 p.44). strong view로 label을 만들면 예측이 흔들려서 pseudo label 품질이 떨어져 (p.48).

시험 label 만드는 입력 = weak, loss 거는 입력 = strong. 둘을 바꾼 지문은 F.

Confirmation bias

L07 p.20–21, p.51

모델이 만든 틀린 pseudo label을 다시 정답으로 배워서 실수가 쌓이고 강화되는 거야. "Pseudo labels generated by an imperfect teacher can be incorrect. The student is then trained to imitate these incorrect labels." (L07 p.20)

예 여우 사진을 "개"(0.97)로 잘못 예측 → pseudo label "개" → 그걸로 학습 → 다음엔 더 확신 있게 "개" → 비슷한 여우 사진들도 "개"로 끌려가.

왜 자기 예측으로 target을 만드는 모든 방법(pseudo label, self-training)의 근본 위험이야. 슬라이드의 처방은 soft label MixUp, labeled oversampling, strong augmentation, dropout이야 (L07 p.21). μ(t) ramp-up, confidence threshold, DivideMix의 두 network 교차 판정(p.51)도 같은 문제를 겨냥해.

RL 자기 추정치로 target을 만드는 bootstrapping에서 오차가 누적되는 것(overestimation bias)과 같은 계열이야.

Sharpening

L07 p.30–35, p.48 · L04 p.40

확률 분포를 더 뾰족하게(더 확신 있게) 만드는 연산이야. MixMatch:

$\text{Sharpen}(p, T)_i = p_i^{1/T} / \Sigma_j\, p_j^{1/T}$ (L07 p.32).

p_i는 i번째 class 확률, T는 sharpening temperature야. 각 확률을 1/T 제곱한 뒤 합이 1이 되게 다시 나눠. T = 1이면 그대로, T가 작을수록 뾰족해지고, T → 0이면 one-hot이 돼. log p를 logit으로 보면 정확히 $\text{softmax}(\log p\ /\ T)$야. 곧 softmax에 temperature T를 거는 것과 같아.

예 p = [0.6, 0.4], T = 0.5 → 제곱 [0.36, 0.16] → 다시 나누면 약 [0.69, 0.31].

왜 MixMatch는 unlabeled 샘플을 K번 augment한 예측을 평균해서 label을 추측하는데(label guessing), 평균하면 분포가 평평해지기 쉬워. 그래서 sharpen으로 confident한 soft target을 만들어 (entropy minimization 효과). DINO가 teacher에 낮은 temperature를 쓰는 것도 같은 의미의 sharpening이야 (L04 p.40).

시험 MixMatch target = sharpen한 **soft** label, FixMatch target = threshold를 넘은 **hard**(one-hot) label. "Confidence thresholding is more effective than temperature sharpening." (L07 p.48)

RL Boltzmann policy의 temperature를 낮춰서 greedy에 가깝게 만드는 것과 같아.

시험 함정 · 기호 하나에 뜻 여러 개

같은 기호 τ(와 T)가 방법마다 전혀 다른 뜻이야. 문제에 τ가 보이면 "어느 방법의 식인가"부터 확인해.

- **temperature**: SimCLR · MoCo · DINO · X-CLR의 τ. similarity(또는 logit)를 τ로 **나눠서** softmax의 뾰족함을 조절해. 코드에선 이름이 제각각이야: MoCo `logits/t`, DINO `tps` · `tpt`, X-CLR `tao` · `tao_s`. DINO 코드의 `t`는 temperature가 아니라 **teacher 출력**이야 (`t1, t2 = gt(x1), gt(x2)`). **CLIP만** 코드에서 `* np.exp(t)`로 **곱해**. exp(t) = 1/τ라서 하는 일은 같지만, t를 사람이 정하지 않고 학습해 (learned). 아래첨자도 조심: X-CLR의 τ_s(`tao_s`)는 target graph(text) 쪽이고, DINO의 τ_s(`tps`)는 student 쪽, τ_t(`tpt`)가 teacher 쪽이야.
- **EMA 계수**: BYOL의 τ. target network가 얼마나 천천히 움직일지 정하는 값이야 (`ξ ← τξ + (1-τ)θ`). temperature 아님.
- **threshold 계열**: FixMatch τ(예측의 최대 확률, 곧 confidence가 넘어야 할 고정 기준, 0.95; L07 p.45, p.67), FreeMatch τ_t(여기 t는 학습 step. 학습 상태에 따라 1/C에서 올라가는 기준, p.69), DivideMix τ(confidence가 아니라 GMM이 준 clean 확률 w에 거는 기준, p.51). 전부 temperature 아님.
- DINO 코드의 `l`: 숫자 1이 아니라 소문자 L이야. teacher network의 EMA momentum이야 (`gt.params = l*gt.params + (1-l)*gs.params`).
- **T도 조심**: MixMatch Sharpen의 T(L07 p.32)와 SimCLRv2 식의 "; T"는 temperature, SimCLRv2의 아래첨자 T(θ_T, p.61)는 teacher야.

L01 Introduction 24p · W1

질문 좋은 representation이 뭔데?

수업 전체의 정의를 까는 강의야. 뒤 강의들의 함정이 여기 정의 문장에서 꽤 나와.

L02–L07은 대부분 "좋은 representation을 label 없이, 또는 적은 label로 어떻게 배우나"에 대한 서로 다른 답이야. 그 "좋은"의 기준을 여기서 정해.

AI가 배워 온 것

p.7

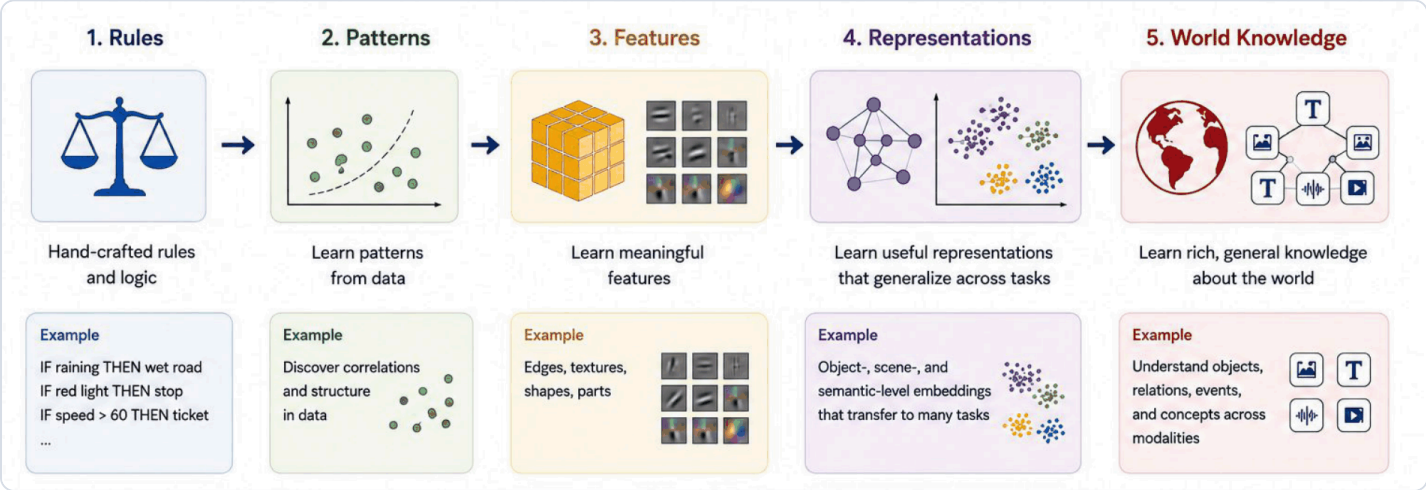
Rules → Patterns → Features → Representations → World Knowledge.

한 줄 AI가 "무엇을 배우나"가 5단계로 바뀌어 왔어 ("From rules to world knowledge").

1. **Rules**: 사람이 규칙을 직접 써. 예: IF red light THEN stop.
2. **Patterns**: data에서 상관관계(correlation)와 구조를 찾아.
3. **Features**: edge(경계선), texture(질감), shape, part 같은 특징을 배워.
4. **Representations**: "Learn useful representations that generalize across tasks". object·scene·semantic(의미) 수준의 embedding(data를 나타내는 숫자 벡터)이고, 다른 task에 옮겨 써도(transfer) 잘 통해.
5. **World Knowledge**: 여러 modality(이미지·텍스트·소리 같은 data 종류)에 걸친 objects, relations, events, concepts.

핵심 사람이 넣는 지식은 줄고, data에서 배우는 것의 추상성·일반성은 커져. 이 수업은 4단계 Representations.

시험 포인트 순서, 그리고 3단계(낮은~중간 수준 특징)와 4단계(task 간 transfer되는 의미 embedding)의 차이. 평소엔 representation·feature·embedding을 거의 같은 뜻으로 섞어 쓰지만, 이 5단계에서는 Features와 Representations가 다른 단계야.



AI가 배우는 대상이 왼쪽에서 오른쪽으로 **Rules → Patterns → Features → Representations → World Knowledge** 순으로 옮겨 왔어. 맨 왼쪽은 사람이 IF–THEN 규칙을 직접 쓰는 단계고, 오른쪽으로 갈수록 data에서 스스로 배우는 게 더 추상적이고 일반적이 돼. 아래 Example 칸에서 3번(edges, textures, shapes, parts)과 4번(여러 task로 transfer되는 embedding)의 차이를 봐. 이 수업은 4번째(Representations)가 중심이야. L01 p.7

Representation

p.9–13

문제 컴퓨터에게 이미지는 처음엔 그냥 숫자 행렬이야 ("an image is initially just a matrix of numbers", p.10). 픽셀값 하나에는 "개"라는 의미가 없고, 조명·각도만 조금 바뀌어도 값이 크게 달라져. 그래서 "Raw Pixels Are Not Understanding".

남길 것 vs 버릴 것 골든 리트리버 사진으로 보면 이래 (p.11).

- 남겨: Fur(털), Ears(귀), Tail(꼬리), Animal shape. "개"라는 걸 알려 주는 정보야.
- 버려: Background, Lighting, Noise, Camera angle, People. 찍을 때마다 바뀌는 nuisance(의미와 상관없이 data 값만 바꾸는 방해 요인)야.
- "A good representation keeps useful information and discards irrelevant information."

정의 "a transformation of raw data into a form that makes downstream tasks easier", $z = f(x)$ (p.12).

- $x \in \mathcal{X}$: raw data(가공 안 한 원본, 사진이면 픽셀값 행렬). $\mathcal{X}$는 data space.
- f: encoder. x를 z로 바꾸는 함수, 보통 학습되는 신경망 (CNN, ViT).
- $z = [z_1, \ldots, z_D] \in \mathcal{Z}$: 숫자 D개로 된 벡터 = representation. $\mathcal{Z}$는 representation space.
- 그다음 z를 downstream task(진짜로 풀고 싶은 일: Classification, Detection, Retrieval ...)에 넣어.

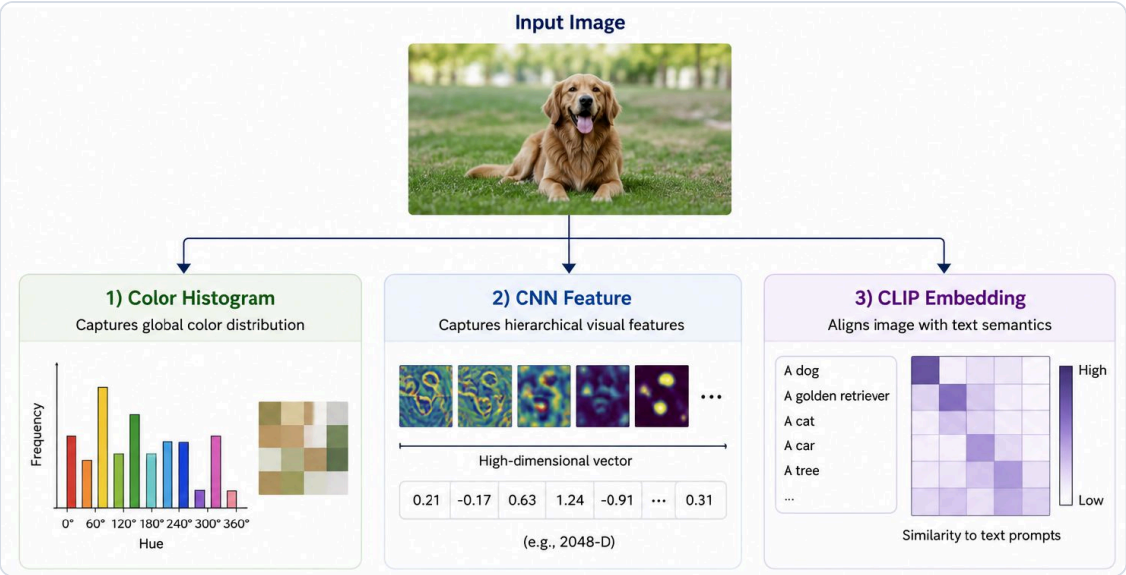
"Representation learning aims to map data points to a simpler, organized space." 비슷한 것끼리 가깝게 모인 단순한 공간으로 옮긴다는 뜻이야. 체크리스트: Learns useful structure · Captures semantics · Enables many tasks · **Often** in lower dimension ($D \ll \dim(x)$).

예시 224×224 컬러 사진은 숫자가 224×224×3 = 150,528개야. encoder는 보통 이걸 훨씬 짧은 D차원 벡터로 바꿔.

포인트 같은 사진도 여러 표현이 가능해 ("The same data can have many different representations", p.13).

- **Color Histogram** ("Captures global color distribution"): 색별 양만 세. 모양 정보는 없어.
- **CNN Feature** ("Captures hierarchical visual features"): edge부터 물체 부분까지 층층이 뽑은 벡터 (e.g., 2048-D).
- **CLIP Embedding** ("Aligns image with text semantics"): 이미지와 문장("A dog" ...)을 같은 공간에 놓아서 비슷한 정도를 잴 수 있어.

> **비유**
> representation은 지도 같은 거야. 지도는 실제 땅(raw data)에서 도로·지명처럼 쓸모 있는 것만 남겨서 길 찾기(downstream task)를 쉽게 해 줘. 같은 땅을 도로 지도·등고선 지도·위성 사진으로 그릴 수 있듯이, 같은 사진도 여러 representation이 가능해 (p.13). 이 수업이 찾는 건 한 장으로 여러 목적에 두루 쓰이는 지도야 (p.17 general-purpose representation).



같은 강아지 사진 한 장을 세 가지 representation으로 바꾼 그림이야. **Color Histogram**은 색(Hue)별로 얼마나 있는지만 세서 모양 정보가 없어. **CNN Feature**는 edge부터 물체 부분까지 계층적으로 뽑은 고차원 숫자 벡터(e.g., 2048-D)야. **CLIP Embedding**은 사진을 "A dog", "A cat" 같은 문장과 같은 공간에 놓아서, 격자 색이 진할수록 그 문장과 비슷하다는 뜻이야. 뭘 담고 뭘 버리느냐가 representation마다 다르다는 게 포인트야. `L01 p.13`

Feature engineering → feature learning

`p.14-17`

이전 Image → Hand-crafted Features(사람이 공식으로 뽑은 특징 숫자) → Classifier (e.g., SVM, Random Forest, k-NN) (p.14). "Who design the features?"의 답은 사람이야: "Feature Engineering Era: Humans manually designed features."

- **SIFT**: 눈에 띄는 점(keypoint)을 찾고 주변의 밝기 변화 방향을 숫자 묶음(descriptor)으로 기록해. 슬라이드 표기 "Keypoint + orientation + descriptor".
- **HOG**: 이미지를 작은 칸으로 나눠 칸마다 밝기 변화 방향(gradient)을 히스토그램으로 세. 슬라이드 표기 "Gradient orientation histogram".
- **SURF**: SIFT를 빠르게 근사한 버전 (슬라이드 밖 기본 설명).
- 공통점: feature 공식은 사람이 정하고, data로 학습되는 건 뒤의 classifier뿐이야.

문제 SIFT · HOG · SURF는 사람이 설계하고, task마다 새로 만들어야 해 (p.15).

1. **Human-designed**: 전문가가 손으로 만들어. domain knowledge(그 분야 지식)와 시간·노동이 많이 들어.
2. **Task-specific**: 물체 인식, 얼굴 인식, 의료 영상이 각각 다른 feature를 요구해. "New task = new feature design". (p.15 결론: "Every new task requires new features.")
3. **Limited scalability**: 복잡한 data의 모든 variation(변형)을 커버 못 해서 성능 상한(performance ceiling)이 있어.

핵심 CNN이 edge → texture → object part를 스스로 배우고, 한 번 배운 backbone(여러 task가 같이 쓰는 feature 추출 신경망 본체)을 detection·segmentation에 재사용. "Learn once, use everywhere."

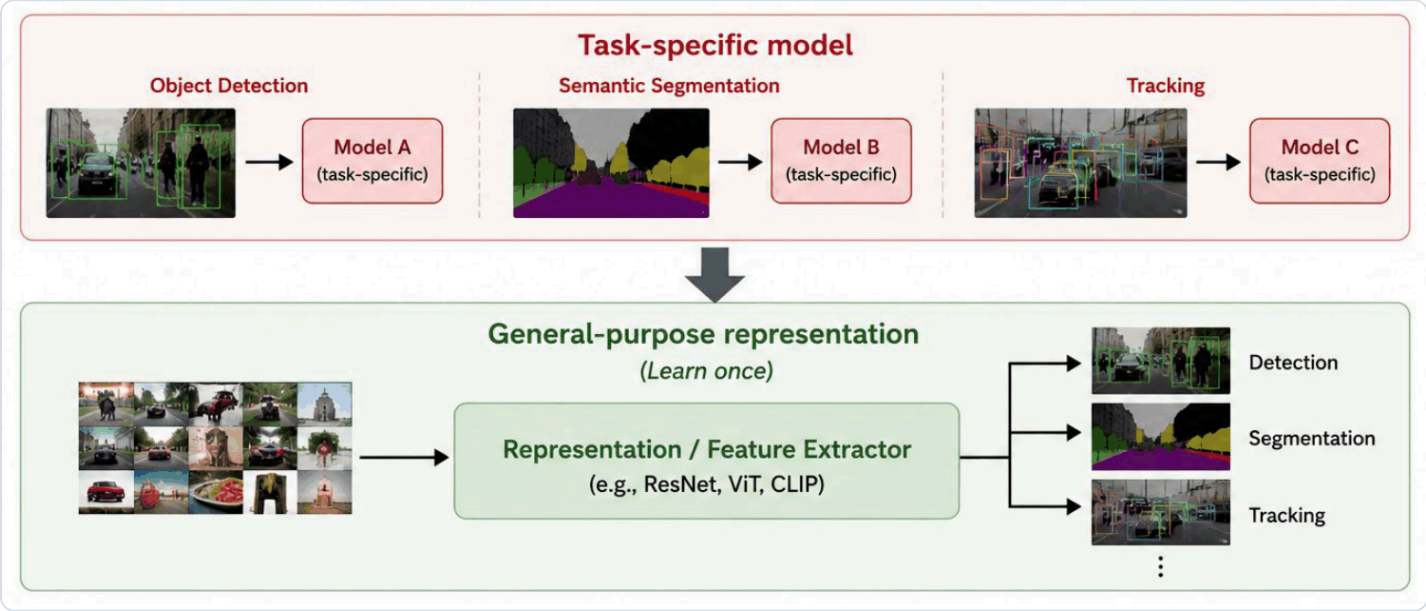
- "What if the machine could learn its own features?" → "Deep learning learns representations automatically from data." (p.16)
- AlexNet feature를 시각화하면 Layer 1은 Edges, Layer 3은 Textures, Layer 5는 Object Parts처럼 보여. 아무도 짜 넣지 않았는데 학습만으로 이 계층이 생겨. "Representations become increasingly semantic" (low-level → high-level).

재사용 ImageNet(label 달린 대규모 이미지 데이터셋)으로 pretraining(큰 data로 미리 학습)한 backbone 하나를 여러 task에 그대로 가져다 써 (p.16 "One representation, many applications."). p.17은 이걸 task마다 Model A/B/C를 따로 만드는 task-specific model과 대비해: "A good representation can be reused across many tasks."

- Detection: 물체 위치를 box로 찾기 (예: YOLO)
- Segmentation: 픽셀마다 무엇인지 칠하기 (예: Mask R-CNN)
- Tracking: 여러 frame에서 같은 물체 따라가기 (예: DeepSORT)

남은 숙제 p.16 pretraining 칸의 "Large-scale supervision" = label이 대량으로 필요해. 이게 L02로 넘어가는 고리야.

RL로 번역
tile coding처럼 사람이 state feature를 설계해 넣던 게 feature engineering, DQN이 pixel에서 CNN으로 state 표현을 알아서 배운 게 feature learning이야.



위쪽 칸은 task마다 모델(Model A/B/C)을 처음부터 따로 만드는 방식(task-specific model)이야. 아래쪽 칸은 여러 이미지로 **representation을 한 번 학습(learn once)**하고 detection(물체 위치를 box로 찾기)·segmentation(픽셀마다 무엇인지 칠하기)·tracking(frame마다 같은 물체 따라가기)에 재사용하는 방식(general-purpose representation)이야. 좋은 representation = 여러 task에 재사용 가능. `L01 p.17`

Compression

`p.18-19`

먼저 p.18의 좋은 representation 조건 3개: **Compress information** · **Support prediction** · **Generalize across tasks**. 앞 두 개가 이 블록과 다음 블록이고, 세 번째는 p.21 교집합에 다시 나와.

한 줄 "Captures just the essential characteristics of the data." (p.19)

핵심 essential characteristics만 남겨. 이점은 Less Memory, Invariance(nuisance 제거), Occam's Razor.

- **Less Memory**: "Smaller representations are easier to store."
- **Invariance**: "Remove nuisance factors that do not affect semantics." 조명·배경·각도가 달라도 같은 개면 z가 비슷해져.
- **Occam's Razor**: "Among equally useful representations, prefer the simpler one."

문장 "Compression is not about losing information. It is about removing irrelevant information."

> 비유
> 시험 요약 노트를 떠올려. 교과서보다 훨씬 짧지만 시험에 필요한 내용은 하나도 안 빠지고, 잡담과 장식만 빠져. 여기서 말하는 compression도 그래.

Prediction

p.20

한 줄 "Why do we need representations?" → "To predict what we cannot directly observe." (p.20)

핵심 관측 못 한 걸 추론해. Future (V-JEPA), Past, Missing data (MAE, inpainting).

1. **Future**: 지금 frame으로 다음 frame 맞히기. 예시: Video prediction, V-JEPA.
2. **Past**: 오늘 장면으로 어제 상태 추정. 예시: State estimation, Temporal reasoning.
3. **Missing Data**: 가려진 부분 채우기(inpainting). 예시: MAE, Image completion.

MAE(이미지 일부를 가리고 복원)와 V-JEPA(영상의 안 보이는 부분을 예측)는 여기선 예시 이름으로만 나와 (풀이는 슬라이드 밖).

문장 "Prediction means inferring unobserved properties of the world."

왜 돼? 가려진 부분을 맞히려면 "여기 뭐가 있고 어떻게 생겼나"를 알아야 해. 그래서 예측을 시키면 encoder가 data의 구조를 배워. 정답(가리기 전 원본)이 data 안에 있으니 사람이 단 label도 필요 없어. 이게 L02 self-supervised learning의 논리야.

좋은 representation = 교집합

p.21–23

한 줄 좋은 representation은 Compression과 Prediction을 둘 다 갖춘 것 (p.21 제목 "Good Representations = Compression + Prediction").

교집합 = Better Representations: Generalize better · Transfer across tasks · Support decision making · More sample efficient (적은 data로도 잘 배움).

> 왜?
> 슬라이드는 "둘 다 갖추면 Better Representations"라는 결론만 줘 (p.21). 이유를 풀면 이래 (해석, 슬라이드 밖). 버리기(compression)만 목표로 두면 "뭐가 irrelevant인지" 정해 줄 기준이 없어. 극단적으로는 모든 입력에 같은 z를 내도 "변환에 안 흔들린다"는 조건을 만족해 버려. 이게 collapse(붕괴)고, 막는 장치는 L04에서 나와. 반대로 맞히기 (prediction)만 두면, 특히 pixel을 그대로 맞히는 예측이면 배경 질감·노이즈 같은 irrelevant한 디테일까지 담으려 해. 그래서 prediction은 "뭘 남길지", compression은 "뭘 버릴지"를 서로 잡아 줘. 헷갈리지 말 것: p.19의 compression은 irrelevant 정보만 지우는 거라 정보를 "잃는" 게 아니야. 여기서 말한 위험은 기준 없이 버리기만 할 때 얘기야.

6종 분류 p.23 "Types of Representation Learners". 괄호 안이 슬라이드 원문 한 줄 요약이야.

Compression Autoencoding · **Contrastive** · Clustering

- **Autoencoding** ("Remove redundant information"): 입력을 작은 z로 줄였다가 복원하게 학습해. 작은 z에 다 못 담으니 중복부터 버려.

- **Contrastive** ("Achieve invariance to viewing transformations"): 같은 사진의 변형본(augmentation: 자르기, 색 바꾸기)끼리는 z를 가깝게, 다른 사진과는 멀게. 보는 방식이 바뀌어도 z가 안 변하게 만들어. L03–L05의 주인공.
- **Clustering** ("Quantize continuous data into discrete categories"): 연속값을 몇 개 그룹(category) 중 하나로 딱 떨어지게 묶어.

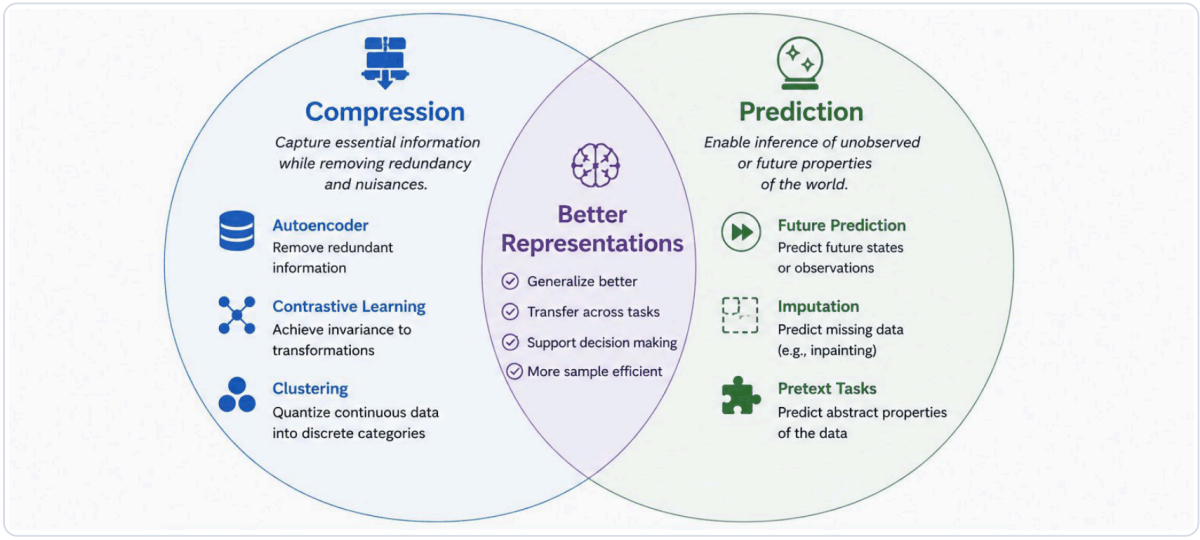
`Prediction` Future prediction · Imputation · Pretext tasks

- **Future prediction** ("Predict the future"): 다음 장면·상태 맞히기.
- **Imputation** ("Predict missing data"): 빠지거나 가려진 부분 채우기 (inpainting).
- **Pretext tasks** ("Predict abstract properties of your data"): encoder를 훈련시키려고 만든 연습 문제(pretext task). 예: 사진 회전 각도 맞히기 (L02).

`덤` Disentangled · Interpretable · Actionable · Transferable. 직접 목표가 아니라 결과로 따라오는 성질이야 (p.22 "Good representations often exhibit additional desirable properties").

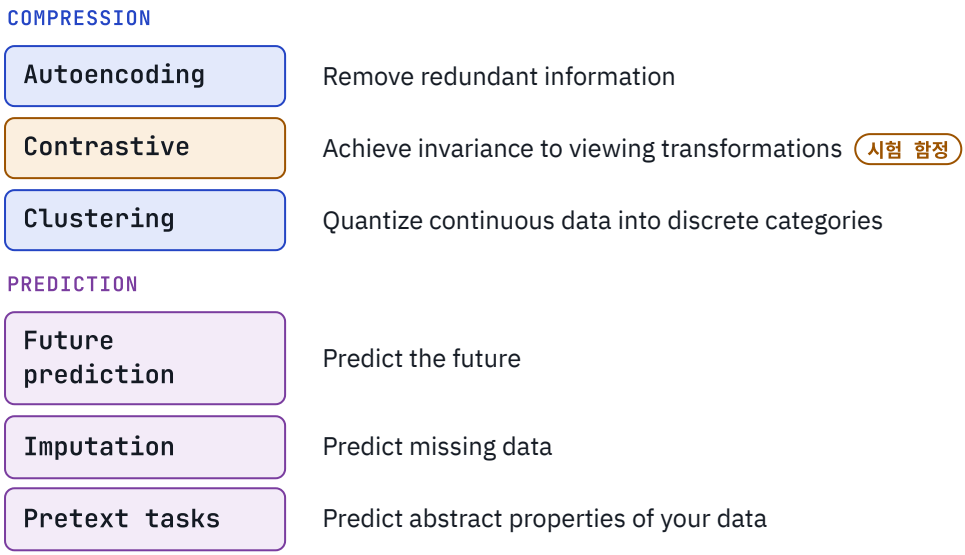
- **Disentangled**: 변하는 요인(Identity, Pose, Lighting, Background)이 z의 서로 다른 부분에 따로 담겨. 그림 예시는 VAE · β-VAE(autoencoder 계열 생성 모델).
- **Interpretable**: z의 neuron(차원) 하나하나가 사람이 알아볼 개념과 짝지어져 (Neuron 1 ↔ 털, Neuron 2 ↔ 눈).
- **Actionable**: Representation → Decision → Action으로 바로 이어져 (로봇 팔, 자율주행).
- **Transferable**: One Representation 하나로 여러 task를 풀어 (p.17 "learn once").

"These properties are often consequences of learning representations that are both" ← 원문이 "both"에서 끊겨 있어. compression과 prediction을 둘 다 갖췄다는 뜻으로 읽으면 돼.



왼쪽 원은 **Compression**(필요 없는 정보 버리기), 오른쪽 원은 **Prediction**(안 보이는 것 맞히기)이고, 가운데 겹친 부분이 좋은 representation(Better Representations)이야. 원 안에 적힌 건 그 원리에 속하는 학습 방법이야: 왼쪽 Autoencoder·Contrastive·Clustering, 오른쪽 Future prediction·Imputation·Pretext tasks. 교집합에 적힌 Generalize better, Transfer across tasks, Support decision making, More sample efficient가 둘 다 갖췄을 때 얻는 이점이야. `L01 p.21`

개념 그림 **Types of Representation Learners: 6개를 원리 2개로 나누기**



p.23 표를 학습 원리별로 묶은 거야. 오른쪽 영어 문장은 슬라이드 원문 한 줄 요약이야. 위 세 개(**Compression**)는 쓸모없는 정보를 버리면서 배우고, 아래 세 개(**Prediction**)는 안 보이는 걸 맞히면서 배워. 헷갈리기 쉬운 건 **Contrastive = Compression**이야 (변형 때문에 생긴 차이를 nuisance로 보고 버리니까). (L01 p.21–23)

RL로 번역

네가 매일 쓰는 pixel → latent state encoder가 정확히 여기서 말하는 f야 (VLA 앞단의 vision encoder도 같은 자리). p.22의 Actionable과 p.21의 More sample efficient가 RL 쪽에서 representation을 신경 쓰는 이유야: z가 좋으면 policy가 적은 경험으로도 배워. Prediction의 Future 쪽은 world model(다음 state를 예측하는 모델)이랑 같은 줄기야. 슬라이드 밖 연결이라 시험 범위는 아니야.

시험 함정

- Contrastive는 Prediction이 아니라 **Compression** 쪽 (p.23). Pretext tasks는 Prediction 쪽.
- "Compression = losing information"은 False.
- 나머지 분류도 같이: Autoencoding · Clustering은 Compression, Future prediction · Imputation은 Prediction (p.23). 3개씩이야.
- 차원은 "**Often** in lower dimension"이지 always가 아니야 (p.12).
- Disentangled · Interpretable · Actionable · Transferable은 "**often consequences**". 직접 목표나 보장된 성질이 아니야 (p.22).

다음으로

좋은 representation을 배우고 싶은데 label은 비싸. 그럼 label 없이 배울 수 있나? → L02 (p.24 "Next Week: Self-Supervised Learning and Pretext Tasks")

셀프 체크

T/F Compression in representation learning means losing information to save memory.

False. 정보를 잃는 게 아니라 irrelevant 정보를 지우는 것 (p.19). 배경·조명 같은 nuisance만 버리고 task에 필요한 정보는 남겨. Less Memory는 이점 중 하나일 뿐이야.

MCQ On p.23, contrastive learning is grouped under: (A) Prediction (B) Compression (C) Imputation (D) Pretext tasks

(B) Compression. invariance to viewing transformations를 얻는 쪽 (p.23). 보는 방식의 변화(자르기·색 바꾸기)를 nuisance로 보고 버리니까 Compression이야. (C)(D)는 원리가 아니라 Prediction 계열의 방법 이름이야.

빈칸 A representation is "a transformation of raw data into a form that makes _____ tasks easier."

downstream (p.12). representation을 받아서 진짜로 풀고 싶은 일 (classification, detection ...).

L02 Self-Supervised Learning 80p

질문 **label 없이 supervision을 만들 수 있나?**

supervision은 "이게 정답이야"라고 알려 주는 학습 신호야. 보통은 사람이 붙인 label(예: "이 사진은 고양이")이 그 역할을 해.

답은 "data를 일부러 망가뜨리고, 원래대로 맞히게 하자"야. 망가뜨린 건 우리니까 정답은 공짜로 알아. 회전 각도를 맞히려면 물체가 뭔지 알아야 하니까 encoder(이미지를 feature 벡터로 바꾸는 network, 예: CNN)가 똑똑해진다는 논리.

> **예시로 보면**
> 고양이 사진을 90° 돌려 넣고 "몇 도 돌렸게?"라고 물어. 정답은 우리가 돌렸으니 이미 알아. 그런데 맞히려면 "고양이는 귀가 위, 발이 아래"를 알아야 해. 이 지식이 encoder에 쌓이고, 나중에 진짜 하고 싶은 일(분류, detection 등)에 그 encoder를 재활용해.

Why SSL

p.3–10

배경 ImageNet으로 supervised(사람이 단 정답으로 학습) pre-train한 CNN의 중간 feature는 detection·segmentation 같은 다른 task에도 잘 통해 (p.4–5). 문제는 그 label을 모으는 단계야.

문제 label이 비싸 (ImageNet 14M장 = 22 human years, 사람 한 명이 22년 일할 양).

- hashtag·GPS 같은 semi-automatic label은 싸지만 noisy해 (틀린 게 섞여).
- video 속 행동 같은 복잡한 concept은 label 정의부터 어려워. "Labelling cannot scale to the size of the data we generate."
- long tail: 10%의 class가 data의 93%라서, 나머지 class(rare concept)는 sample이 거의 없어.
- 의료 같은 새 domain은 전문가가 label을 달아야 해서 더 어려워.

정의

p.11–17, p.20

정의 "obtain labels from the data itself", "predict part of the data from other parts" (p.12). 문장에서 단어를 가리고 맞히는 Word2vec·BERT가 대표 예야. 문장 자체가 정답을 들고 있어 (p.14).

핵심 pretext task로 encoder를 학습하고 downstream task로 transfer(배운 encoder를 다른 task에 옮겨 쓰기).

- pretext task: representation(encoder가 뽑은 feature 벡터)을 배우려고 만든 연습용 과제. 슬라이드 정의는 "Self-supervised task used for learning representations. Often, not the "real" task (like image classification) we care about" (p.20). label은 자동이지만 loss는 classification·regression 같은 supervised 형태야 (p.16).
- downstream task: 진짜 풀고 싶은 일. label은 있지만 dataset이 작아 (p.16).
- 순서: label 없는 대량 data로 pretext 학습 → pretext용 head(출력 층)는 버리고 encoder만 남김 → 새 head를 붙여 적은 label로 downstream 학습.

왜 돼? "Solving the pretext tasks allow the model to learn good features." + "We can automatically generate labels for the pretext tasks." (p.17)

차이 unsupervised(clustering, PCA)와 달리 data에서 자동으로 만든 명시적 예측 target이 있어.

평가
p.18, p.46, p.48

핵심 pretext 성능이 아니라 **downstream 성능**으로 평가해. frozen feature 위에 shallow(linear) head만 학습.

어떻게 encoder를 고정(frozen, weight를 더 안 바꿈)하고, 그 위에 linear classifier 같은 얕은 head만 적은 label로 학습해서 downstream task 점수를 재. 슬라이드 p.18은 이걸 "target task"라고 불러 (p.18, p.48 "Linear classifiers on "fixed" features"). 이게 linear probing이야.

왜 shallow? head가 크면 head가 알아서 잘해 버려서 encoder 덕인지 구분이 안 돼. head를 약하게 두면 점수 차이가 곧 representation 품질 차이야.

Hand-crafted pretext 5종

사람이 손으로 설계한 과제 5개야. 틀은 같아: 이미지를 망가뜨리고(자르기, 가리기, 섞기, 색 빼기, 돌리기) 망가뜨리기 전 정보를 맞히게 해. 좋은 pretext의 조건은 두 개야.

- **풀려면 의미(semantic)를 알아야 해.** 물체가 뭔지 모르면 못 푸는 과제여야 encoder에 그 지식이 쌓여.
- **꼼수로 풀리면 안 돼.** shortcut(trivial solution)은 의미는 모른 채 low-level 단서로 맞히는 거야. 조각 경계의 선이 이어지는지, 색·밝기 통계, chromatic aberration(렌즈 때문에 사진 가장자리로 갈수록 색 채널이 살짝 어긋나는 현상, patch 위치가 들통남) 같은 단서. 이러면 pretext 정확도는 높아도 feature는 쓸모없어.

Relative Position
Doersch, ICCV 2015 · p.21-23

맞히는 것 가운데 patch(작은 정사각형 조각)와 주변 8칸 중 한 칸의 patch를 주고, 이웃이 몇 번 자리인지 맞혀 (**8-way classification**). X = (가운데, 이웃), Y = 자리 번호.

왜 돼? 귀 patch가 얼굴 위쪽이라는 걸 알려면 "이건 귀, 이건 얼굴"을 알아봐야 해. 물체 part 사이의 **local** 공간 관계를 배워. 두 patch는 weight를 공유하는 두 tower(같은 CNN을 복사해 둔 줄기, patch를 하나씩 따로 처리)에 따로 들어가니까 patch 하나만 보고도 feature를 만들어야 해 (p.22).

꼼수 patch가 딱 붙어 있으면 경계의 선이 이어져서 의미 없이도 맞혀. patch 사이에 틈(gap)을 두고 위치를 흔들어(jitter) 막아 (논문 기준).

Context Encoders (Inpainting)
Pathak, CVPR 2016 · p.24-28

맞히는 것 이미지 가운데를 지우고 주변(context)만 보고 지운 영역의 **픽셀**을 그려 내 (inpainting = 구멍 메우기). 그러려면 object parts, scene structure, semantic context를 알아야 해 (p.24).

차이 Relative Position은 정답이 번호(classification)고, 여기선 픽셀 자체야 (pixel-level reconstruction).

loss $L_{rec}(x) = \| \hat{M} \odot (x - F((1 - \hat{M}) \odot x)) \|_2^2$ (p.25). $\hat{M}$는 구멍 자리만 1인 mask, $\odot$는 같은 위치끼리 곱하기, F는 구멍 난 입력 $(1 - \hat{M}) \odot x$를 채우는 encoder-decoder야. 앞의 $\hat{M} \odot$ 때문에 오차 제곱을 **구멍 안에서만** 재. 보이는 곳까지 재면 그냥 복사하면 되거든.

문제 L2만 쓰면 그럴듯한 답들의 평균을 그려서 blurry(흐릿)해져. 꼼수(shortcut)가 아니라 L2 loss 자체의 한계야. 그래서 adversarial loss(진짜 사진과 채운 사진을 구별하는

discriminator D를 속이게 하는 GAN식 loss, L_{adv}, p.26)를 더해. 둘을 합쳐야 선명하면서 맥락에도 맞아 (p.27).

Jigsaw

Noroozi & Favaro, ECCV 2016 · p.29–33

맞히는 것 이미지를 3×3 = 9조각(tile)으로 섞고 **어떤 순서(permutation)로 섞었는지** 번호를 맞혀. 조각이 뭔지 + 전체 배치를 알아야 하니 **global** 공간 구조를 배워.

어떻게 순서는 9! = 362,880가지라 class로 쓰기엔 너무 많아. 그래서 "Select a subset of 64 permutations that maximize pairwise Hamming distance" (p.30). Hamming distance는 두 순서에서 자리가 다른 칸 수야 (123456789 vs 213456789 → 2). 서로 많이 다른 순서만 남겨야 class끼리 안 헷갈려.

꼼수 "The network may solve the puzzle using low-level cues instead of semantics." edge continuity나 색·밝기 분포로 풀 수 있어. 막는 법: multiple puzzle configurations per image, random gaps, random crop, spatial jitter, color jitter, grayscale (p.31). 이 장치를 빼면 jigsaw 정확도는 오르는데 detection은 떨어져 (p.33).

Colorization

Zhang, ECCV 2016 · p.34–39

맞히는 것 흑백에서 색을 맞혀. Lab 색공간에서 L(밝기 = 흑백 사진)을 넣고 ab(색 두 축)를 출력해 (p.35). "Color prediction requires semantic understanding." 잔디인지 알아야 초록으로 칠해.

문제 "Multiple colors can be plausible. Regression averages them." (p.37) 흑백 자동차는 빨강도 파랑도 그럴듯해. regression(L2)은 이럴 때 평균색이 오차가 가장 작아서, 탁한 회색빛으로 도망가.

해결 ab 공간을 **313 color classes**로 나눠 pixel마다 **classification**해. 확률 분포를 내니까 "빨강일 수도 파랑일 수도"를 평균으로 뭉개지 않고 표현해.

수식 $L_{cl}(\hat{Z}, Z) = -\sum_{h,w} v(Z_{h,w}) \sum_{q} Z_{h,w,q} \log \hat{Z}_{h,w,q}$ (p.37). pixel (h, w)마다 313칸 q에 대한 cross-entropy야. $\hat{Z}$는 예측 확률, Z는 정답 색의 칸 분포, $v(\cdot)$는 pixel별 weight (슬라이드엔 정의 없음).

RotNet

Gidaris, ICLR 2018 · p.40–43

맞히는 것 이미지를 0° / 90° / 180° / 270° 중 하나로 돌리고 몇 도인지 맞혀 (**4-way classification**). "No labels are needed, the applied rotation becomes the label." (p.41)

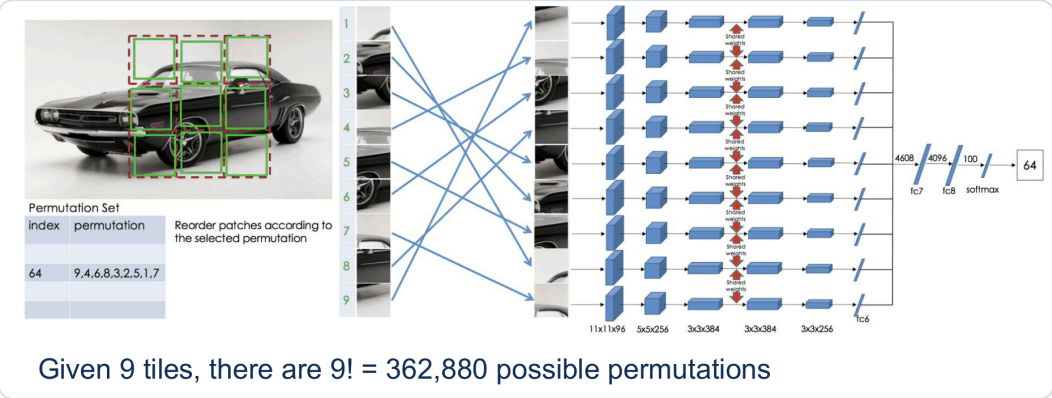
왜 돼? "To recognize rotation, the network must recognize objects." 새는 머리가 위라는 걸 알아야 각도를 맞혀. 그래서 object-centric(물체 중심) feature를 배워.

수식 원본 X를 y × 90° 돌린 $X^y = g(X, y)$마다 "Maximize prob. $F^y(X^y)$" (p.41). F^y는 network가 "y번 회전"에 준 확률이라, 4-way cross-entropy랑 같아.

장점 가장 단순해. 90° 단위라 이미지가 잘리거나 빈 모서리가 안 생겨서 경계 꼼수도 적어 (해석). 회전 개수는 4개가 2개·8개보다 좋았어 (p.42).

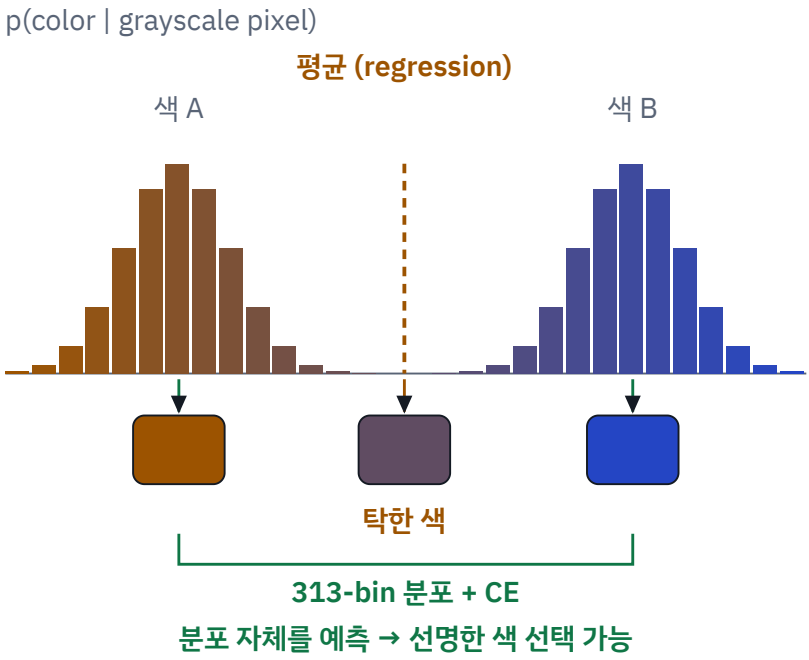
다섯 개를 한 표로 모으면 이래. "배우는 것"은 p.45 요약표의 Learned Concept야.

과제	뭘 맞히나	포인트
Relative Position Doersch, ICCV 2015 · p.21-23	가운데 patch 기준 이웃 patch의 위치 (8-way)	배우는 것: Local Spatial Context. gap·jitter로 shortcut 차단
Context Encoders (Inpainting) Pathak, CVPR 2016 · p.24-28	지운 영역의 픽셀	배우는 것: Context & Object Parts. L2만 쓰면 blurry → L2 + adversarial
Jigsaw Noroozi & Favaro, ECCV 2016 · p.29-33	3×3 조각의 순열 번호	배우는 것: Global Spatial Structure. 9! = 362,880 대신 Hamming distance가 최대인 64개 순열만 class로
Colorization Zhang, ECCV 2016 · p.34-39	흑백(L)에서 색(ab)	배우는 것: Object Semantics. regression은 색을 평균 내서 탁해짐 → 313-class classification
RotNet Gidaris, ICLR 2018 · p.40-43	0° / 90° / 180° / 270° 중 뭘 돌렸나	배우는 것: Object-Centric Understanding. 가장 단순. 돌린 각도가 곧 label



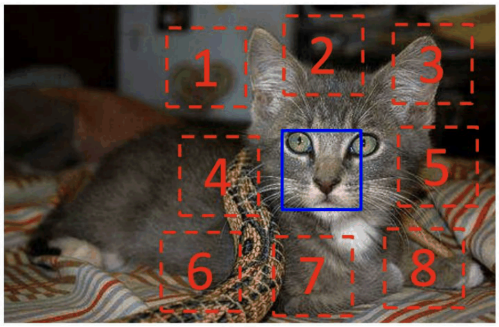
Jigsaw. 왼쪽: 자동차 사진에서 3×3 tile 9개를 뽑고, permutation set에서 고른 순서(예: index 64 = 9,4,6,8,3,2,5,1,7)대로 섞어. 오른쪽: 섞인 tile 9개가 weight를 공유하는 tower를 하나씩 따로 통과하고, 마지막에만 합쳐서 **64개 permutation 중 몇 번인지**를 classification해. 9! = 362,880개 순서를 다 쓰지 않고 Hamming distance가 큰 64개만 class로 쓰는 게 포인트. L02 p.30

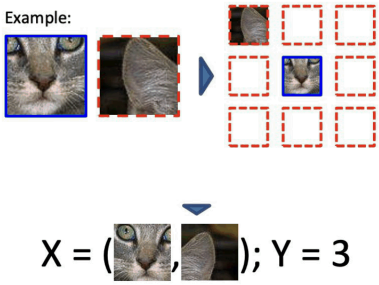
개념 그래프 **Colorization: regression은 색을 평균 내서 탁해져** `illustrative`



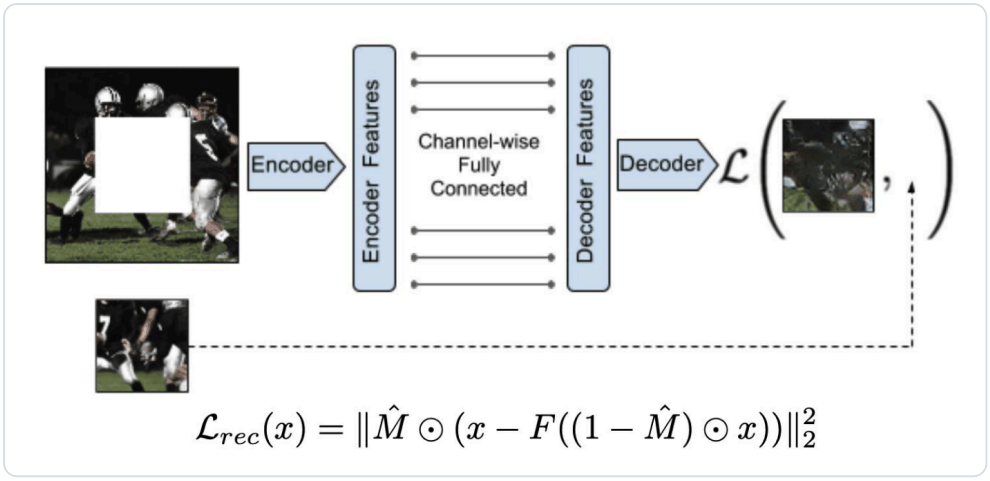
흑백 pixel 하나를 보고 "원래 무슨 색이었을까"의 확률을 그린 거야. 그럴듯한 색이 둘(색 A, 색 B)이면, 색 값 하나만 내는 MSE regression은 둘의 **평균**(가운데 점선, 탁한 색)을 내. 오차 제곱을 가장 작게 만드는 답이 평균이기 때문이야. 대신 **313-bin 분포를 예측**하고 cross-entropy로 학습하면 "A일 수도, B일 수도"를 그대로 표현해서 평균으로 뭉개지지 않고, 분포에서 선명한 색을 고를 수 있어. 분포 모양은 개념을 보여주려고 그린 것. (L02 p.36–38)

▼ 슬라이드 그림 더 보기 (3장)

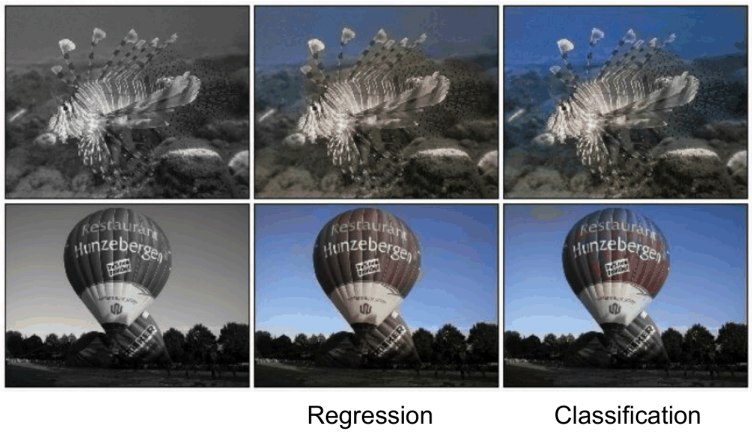




Relative Position. 파란 네모가 가운데 patch, 빨간 점선 1~8이 그 주변 8칸이야. 입력 X는 (가운데 patch, 이웃 patch 하나) 쌍이고, 정답 Y는 이웃이 몇 번 칸에서 왔는지야 (예시는 Y=3). 사람이 label을 단 게 아니라 patch를 어디서 잘랐는지에서 정답이 공짜로 나와. ← 오른쪽 예시 격자에선 귀 patch가 왼쪽 위 칸에 그려져 있어서 번호(3 = 오른쪽 위)와 안 맞아 보여 (원문 유지). `L02 p.21`



Context Encoder. 흰 네모가 지운 구멍이야. 구멍 난 이미지를 Encoder → Channel-wise FC → Decoder에 통과시켜 구멍 안을 그리고, 그 결과를 원본의 같은 영역(아래 작은 사진, 점선 화살표)과 비교해 L2 reconstruction loss를 줘. $\hat{M}$는 구멍 자리만 1인 mask라서 loss는 구멍 안에서만 계산돼. 정답이 이미지 자기 자신이라 label이 필요 없어. `L02 p.25`



Colorization. 왼쪽 흑백 입력, 가운데 regression, 오른쪽 classification 결과야. regression은 그럴듯한 여러 색을 평균 내서 탁하고 칙칙해. 313-class classification은 색이 선명해. 열기구의 붉은 줄무늬를 비교해 봐. `L02 p.38`

Pretext의 한계

`p.44-49, p.55`

문제 1 과제를 하나하나 고안하는 게 tedious해 (p.46). 어떤 과제가 좋은 feature를 줄지도 미리 몰라. pre-training 과제와 transfer 과제가 "aligned"이길 **바랄** 뿐이야 ("The Hope of Generalization", p.47).

문제 2 뒤쪽 layer가 그 과제에 특화돼서 general하지 않아 (Jigsaw로 학습하면 linear probe가 res4에서 최고, res5에서 떨어짐). layer마다 frozen feature에 linear classifier를 붙여 본 결과야. 마지막 res5(ResNet의 가장 깊은 단계)는 "퍼즐 번호 맞히기" 자체에 특화돼서 downstream엔 덜 맞아 (p.49).

CDJP · Multiple pretext

`Kim, WACV 2018 · p.50-53`

핵심 망가진 조각으로 jigsaw + inpainting + colorization을 한 네트워크에서 동시에. shared backbone + task별 head.

쉽게 Completing Damaged Jigsaw Puzzles: 조각을 섞고, 하나는 빼고(채워야 함), 색도 지워(복원해야 함). 세 과제가 encoder 몸통(shared backbone)을 같이 쓰고, 출력부(head)와 loss(L_{jig}, L_{inp}, L_{col})만 따로야 (p.52). encoder가 세 과제 모두에 쓸모 있어야 하니 한 과제로 덜 치우쳐.

한계 여전히 사람이 고른 과제 묶음에 묶여 있어 ("even multi tasks", p.55).

더 일반적인 pretext로

p.54–56

전환 "변환을 맞혀라" 대신 "변환된 이미지들은 **같은 object**니까 가깝게, 다른 이미지는 멀게". 이게 contrastive learning의 출발점.

쉽게 가린·돌린·조각난·흑백 고양이는 전부 same object, 강아지는 different object야 (p.56). (원본, 변환본)은 positive pair(feature가 가까워져야 할 쌍), (원본, 다른 이미지)는 negative pair(멀어져야 할 쌍). 변환을 맞히는 과제(task-specific)와 달리 특정 과제에 안 묶여(task-agnostic). 대신 feature가 변환에 invariant(변환해도 안 변함)해져.

Space-Time Cubic Puzzles

Kim, Cho, Kweon, AAAI 2019 · p.57–66

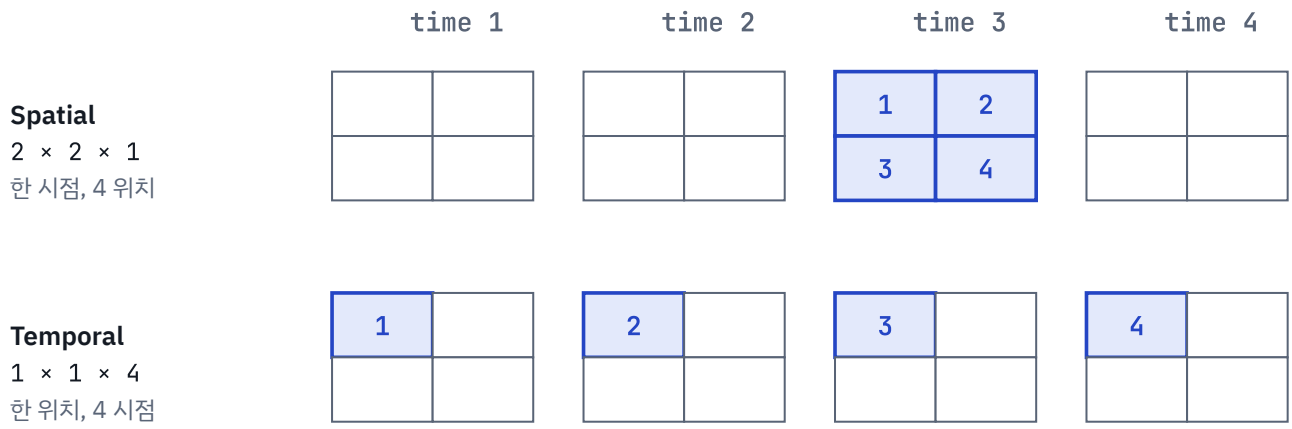
문제 video는 이미지에 없는 temporal order, motion을 줘. frame 두 장만으론 공을 던지는지 받는지 몰라 (p.58).

핵심 video를 2×2×4로 나누고, 공간(2×2×1) 또는 시간(1×1×4) 방향으로 cube 4개를 뽑아 섞은 뒤 원래 순서를 맞혀 (4! × 2 orientations = 48 class). 3D CNN tower 4개, weight 공유.

- cube = 가로×세로 영역을 몇 frame 동안 잘라 낸 짧은 동영상 덩어리.
- Spatial = 같은 시간 구간의 네 위치 ("Where was each cube located?"), Temporal = 같은 위치의 네 시간 구간 ("When did each cube occur?") (p.59–60).
- 출력은 "24 permutations × 2 orientations = 48 classes" (p.62). "2 orientations"는 바로 섰나/뒤집혔나(upside-down)야. p.63의 upside-down classification이 이거야 (notes·원 논문 기준). spatial/temporal 두 종류를 곱한 게 아니야.
- 3D CNN은 시간 축까지 convolution해서 움직임을 직접 봐. tower 4개는 맨 끝에서만 합쳐지고, "Each tower does not know whether the puzzle is spatial or temporal." 그래서 둘 다에 쓸 feature를 배워.
- 꼼수(p.63): chromatic aberration, matching cube boundaries, low-level image statistics. 막는 법: channel replication(색 채널 하나를 복사해 색 어긋남 제거), spatio-temporal jittering(큰 cube 안에서 위치를 흔들어 잘라 경계를 어긋나게), upside-down classification.

차이 image jigsaw를 시공간으로 확장. 공간·시간을 같이 배우는 게 따로 배우는 것보다 나아 (p.64).

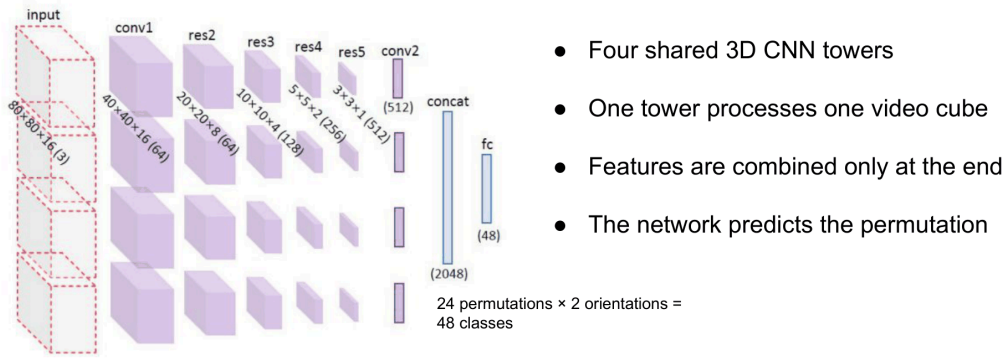
개념 그림 **Space-Time Cubic Puzzle: 2×2×4 grid에서 cube 뽑기**



→ 뽑은 4 cube를 **random permute** → 원래 배열 맞히기

4! = 24 순열 × 2 orientations = 48 class (p.60, p.62)

video를 가로 2 × 세로 2 × 시간 4 = 16칸으로 나눈 그림이야. 칸 하나가 cube(짧은 동영상 덩어리)고, 2×2 묶음 하나가 한 시간 구간이야. **Spatial**은 같은 시간 구간(time 3)의 네 위치에서, **Temporal**은 같은 위치(왼쪽 위)의 네 시간 구간에서 cube 4개를 뽑아 (번호 = 원래 순서). 이걸 섞고 원래 순서를 맞히니까 4! = 24가지 순열이 나와. 슬라이드는 여기에 "× 2 orientations"를 곱해 48 class라고 해 (= upside-down 여부, p.63. spatial/temporal을 곱한 게 아니야). 아래 p.62 그림의 입력이 이렇게 만들어져. 이미지 jigsaw는 공간만 보지만 video는 시간 순서·움직임까지 pretext에 들어와. 그림은 칸 구조를 개념적으로 그린 것. (L02 p.58–62)



cube 4개(80×80 pixel × 16 frame)를 각자 3D CNN tower에 넣어. 네 tower는 weight를 공유하고, **맨 끝에서만 concat**해서 fc(48)로 permutation을 맞혀. 중간에 섞지 않으니 tower는 cube 하나의 내용만 보고 feature를 만들어야 해. 또 tower는 이 퍼즐이 공간 방향인지 시간 방향인지 모르니까 둘 다에 쓸 feature를 배워야 해. `L02 p.62`

AVTS

Korbar, NeurIPS 2018 · p.67-70

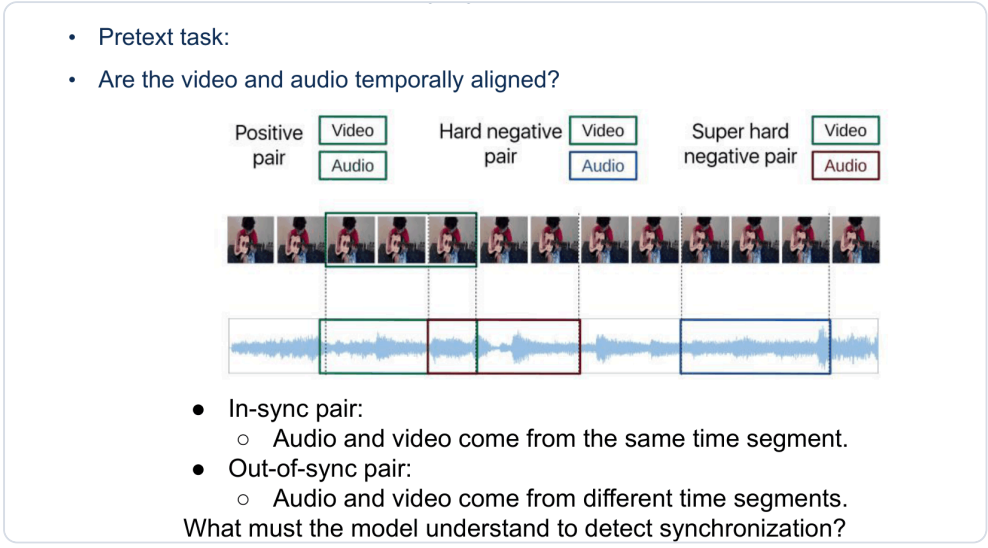
핵심 video와 audio가 시간상 맞는지(in-sync) 아닌지(out-of-sync) 판별. contrastive loss로 두 stream을 같이 학습.

쉽게 "Video and audio are naturally synchronized." 기타 줄을 튕기는 순간 소리가 나니까 그 박자가 공짜 정답이야. 같은 time segment의 audio-video가 positive, 다른 segment면 negative (p.68). contrastive loss는 positive는 가깝게, negative는 멀게 만드는 loss야. video stream은 spatio-temporal CNN, audio stream은 filterbank feature(소리를 주파수별 세기로 바꾼 것)를 받아 (p.69).

`negative` 같은 영상에서 시간이 가까울수록 소리가 비슷해서 어려워.

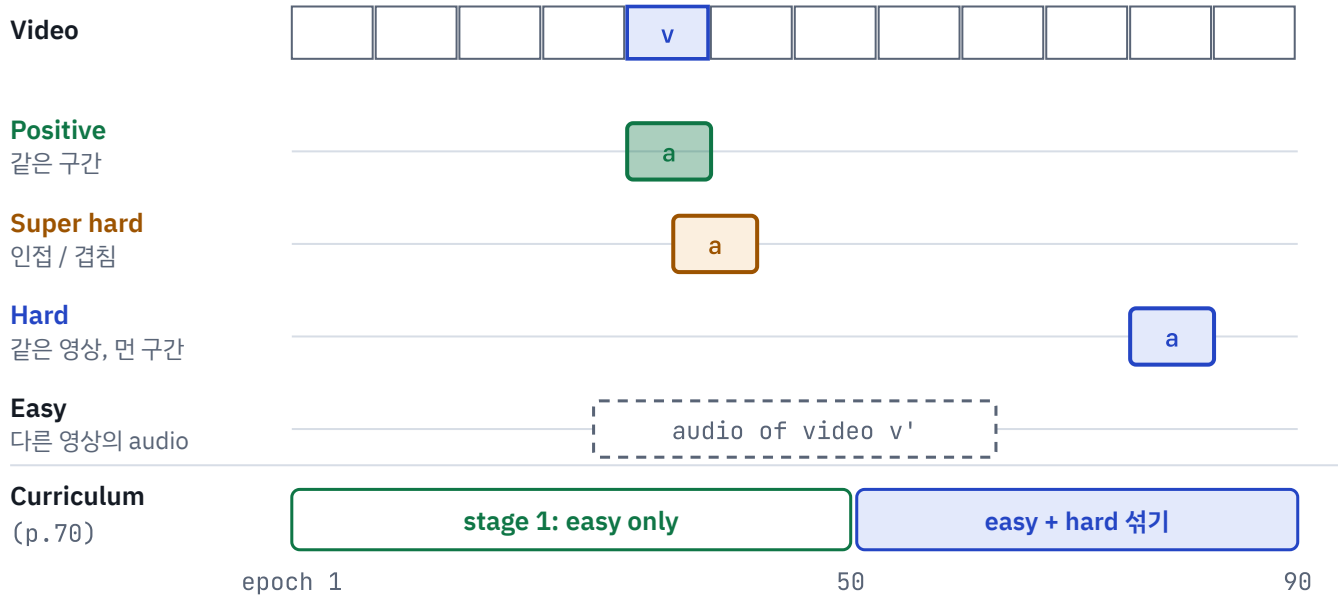
- **hard**: 같은 영상의 **멀리 떨어진** 구간 audio. 장면은 같으니 박자를 봐야 구분돼.
- **super hard**: 같은 영상의 **바로 옆, 살짝 겹치는** 구간 audio. 소리가 거의 같아서 제일 어려워.
- **easy**: 다른 영상의 audio (p.70 표엔 이름만 있고, 정의는 논문 기준).

포인트 negative 구성이 핵심이고, curriculum (easy negative 먼저 → hard negative 섞기)이 제일 좋아. 처음부터 hard를 섞으면 오히려 떨어지고, easy만으로 50 epoch 학습한 뒤 easy 75% + hard 25%로 이어 가면 최고야 (p.70). "A good pretext task depends not only on the labels, but also on how positive and negative examples are constructed."



기타 치는 영상 하나에서 짝을 만드는 법이야. 가운데 줄이 video frame, 그 아래가 audio 파형이야. 초록 = positive (같은 구간의 video와 audio). 파랑 = **hard negative** (같은 영상의 **멀리 떨어진** 구간 audio). 빨강 = **super hard negative** (초록 바로 옆에서 **살짝 겹치는** 구간 audio). 가까울수록 소리가 비슷해서 더 어려워. ← 위쪽 이름표의 위치가 아니라 Audio 상자의 **색**으로 짝지어 봐 (Hard = 파랑 상자, Super hard = 빨강 상자). `L02 p.68`

개념 그림 **AVTS: negative를 어떻게 만드느냐 + curriculum**



가로가 영상의 시간축이야. 맨 위 Video 줄의 강조된 칸 `v`가 지금 고른 video 구간이고, 그 아래 줄들은 `v`와 짝지을 audio `a`를 어디서 가져오느냐야. 같은 구간 audio가 **positive**, 같은 영상의 멀리 떨어진 구간 audio가 **hard**, 바로 인접(겹치는) 구간 audio가 **super hard** negative야. **easy**는 아예 다른 영상 v'의 audio라 장면부터 달라서 쉬워 (easy = 다른 영상 audio는 논문 기준 설명). 맨 아래 줄: 처음부터 hard를 섞으면 오히려 성능이 떨어져서, **1–50 epoch은 easy만, 51–90 epoch은 easy와 hard를 섞는 curriculum**이 가장 좋았어. (L02 p.68–70)

Invariance vs Equivariance

p.71–75

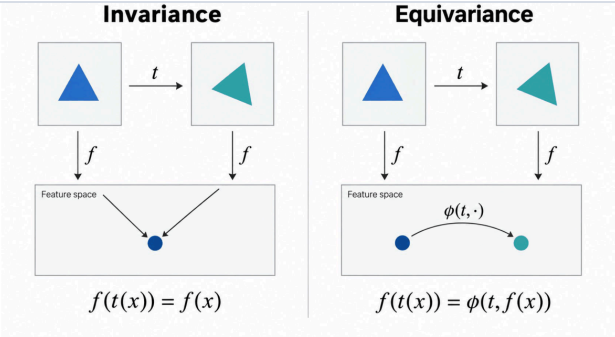
문제 "A pretext task determines what information is preserved in the representation." (p.72) 요즘 SSL은 crop·color jitter(색·밝기를 무작위로 흔드는 변환)·flip·blur 같은 augmentation(같은 사진의 변형본)을 쓰고 "trained to ignore these transformations". 그런데 늘 무시하는 게 좋을까? (p.73)

`Invariance` `f(t(x)) = f(x)`. 변환해도 표현이 안 변해. (`x` 입력, `t` 변환, `f` encoder) 무슨 변환을 했는지는 feature에서 지워져.

`Equivariance` `f(t(x)) = φ(t, f(x))`. 표현이 예측 가능하게 변해. `φ(t, ·)`는 feature 공간에서 "t에 해당하는 이동 규칙"이라, `f(x)`와 `t`만 알면 변환본의 feature가 나와. 변환 정보가 정리된 채로 남는 거야. invariance는 φ가 아무것도 안 바꾸는(identity) 특수한 경우야.

> 비유
> 사진을 90° 돌리면 invariant feature는 "고양이"라고만 말하고 각도는 몰라. equivariant feature는 "고양이, 90° 돌아감"을 같이 담아서, 더 돌리면 feature도 정해진 규칙대로 따라 움직여.

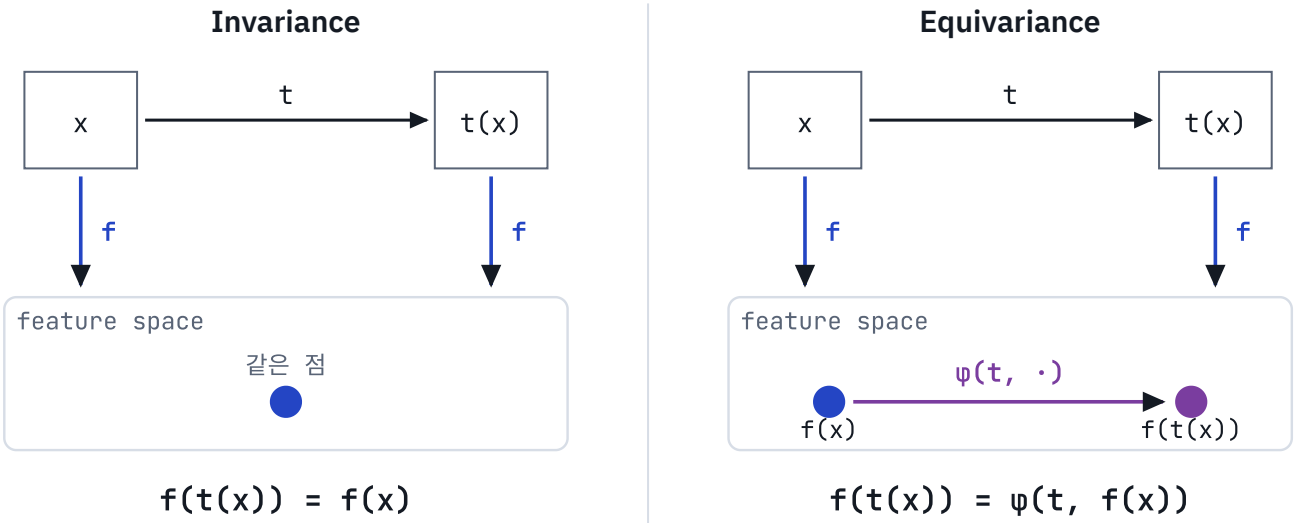
포인트 분류엔 invariance가 좋지만 detection은 위치 정보, 꽃 분류는 색 정보가 필요해. pretext가 어떤 정보를 남길지를 정해 (p.74). crop·flip에 invariant하면 "Where is the object?"에 필요한 위치가, color jitter에 invariant하면 "What color is the flower?"에 필요한 색이 사라져.



Invariance: representation does not change after the input transformation.
Equivariance: representation changes predictably according to the input transformation.

삼각형을 돌리는 변환 t를 걸고 둘 다 encoder f로 feature를 뽑은 그림이야. Invariance는 변환 전후 feature가 **같은 점**에 떨어져서 "얼마나 돌렸나"가 지워져. Equivariance는 다른 점에 떨어지지만 **φ(t,·) 화살표대로 예측 가능하게 이동**해서 변환 정보가 남아. 둘의 차이는 변환 정보를 지우느냐 남기느냐야. `L02 p.75`

구조 비교 **Invariance vs Equivariance**



양쪽 모두 위의 가로 화살표가 입력에 변환 `t`를 거는 거고 (x → t(x)), 세로 화살표 `f`는 encoder로 feature를 뽑는 거야. **Invariance**(왼쪽)는 x와 t(x)의 feature가 **같은 점**이라 "무슨 변환을 했나"가 지워져. **Equivariance**(오른쪽)는 두 feature가 다른 점이지만, f(x)에서 `φ(t,·)` 화살표를 따라가면 f(t(x))에 닿아. 즉 입력이 변한 만큼 feature도 **예측 가능하게** 변하고, 변환 정보가 남아. contrastive는 augmentation에 invariant해지면서 변환 정보를 버려. invariance는 `φ = id`(아무것도 안 바꾸는 map)인 equivariance의 특수한 경우야. (L02 p.75)

STL

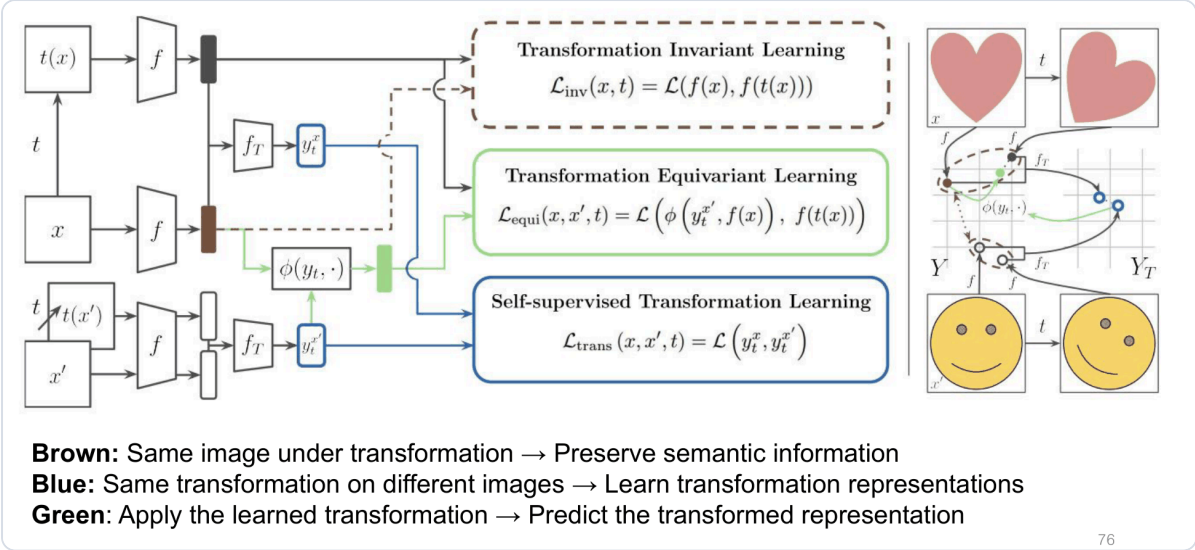
Yu, NeurIPS 2024 · p.76-79

핵심 loss 3개. 같은 이미지의 변환 전후 (invariant), **다른 이미지·같은 변환** (transformation representation 학습), 학습한 변환을 적용해서 예측 (equivariant). 아래 식의 $L(\cdot,\cdot)$는 두 표현을 가깝게 만드는 SSL loss야.

- **Brown** $L_{inv}(x, t) = L(f(x), f(t(x)))$: "Same image under transformation → Preserve semantic information". 변환 전후 feature를 가깝게 해서 내용("고양이")을 지켜.
- **Blue** $L_{trans}(x, x', t) = L(y_t^x, y_t^{x'})$: "Same transformation on different images → Learn transformation representations". 고양이 x와 자동차 x′에 같은 변환 t를 걸고, 각자에서 뽑은 변환 표현 y_t(transformation head f_T, 즉 변환 정보만 뽑는 작은 network가 뽑음)를 가깝게 해. 공통점이 "같은 변환"뿐이라 y_t엔 변환 정보만 남아.

- **Green** $L_{equi}(x, x', t) = L(\psi(y_t^{x'}, f(x)), f(t(x)))$: "Apply the learned transformation → Predict the transformed representation". x′에서 뽑은 변환 표현으로 f(x)를 옮겨 f(t(x))를 맞혀. 이게 equivariance야.

차이 변환 label(parameter) 없이 변환 표현을 배워. 그래도 crop과 color 변환이 갈라지고 비슷한 color 변환끼리 모이는 구조가 생겨 (p.77–78). "Preserving transformation information improves localization and generalization." (p.79)



STL의 세 loss를 색으로 구분해서 봐. 갈색 = 같은 이미지의 변환 전후 feature를 가깝게 (invariant, 내용 보존). 파랑 = **다른 이미지 x, x′에 같은 변환**을 걸고, 각각에서 f_T가 뽑은 변환 표현 y_t끼리 가깝게 (변환 표현 학습). 초록 = x′에서 뽑은 변환 표현으로 f(x)를 옮겨 f(t(x))를 맞히기 (equivariant). 오른쪽 하트와 스마일은 서로 다른 이미지에 같은 변환 t를 건 예시야. `L02 p.76`

RL로 번역

- pretext task는 RL의 auxiliary task랑 같은 발상이야 (본 목적은 아니지만 encoder를 키우는 보조 과제).
- 엣지 연속성이나 색 통계로 jigsaw를 풀어 버리는 shortcut은 reward hacking(reward의 허점으로 점수만 올리기)이랑 비슷한 냄새가 나.
- AVTS의 easy → hard curriculum은 RL에서 쉬운 환경부터 학습시키는 curriculum과 같은 발상이야.
- invariance vs equivariance는 로봇에서 특히 중요해. grasp하려면 물체의 위치·방향이 필요한데, crop·flip에 무조건 invariant하게 배우면 그 정보가 날아가. (이 박스는 슬라이드 밖 연결, 시험 범위 아님)

시험 함정

- Colorization은 regression이 아니라 **classification** (313 bins).
- Jigsaw는 9! 전부가 아니라 **64개** 순열 (pairwise Hamming distance 최대).
- Inpainting에서 L2만 쓰면 blurry, adversarial만 쓰면 선명하지만 context와 안 맞아.
- SSL 평가는 pretext 정확도가 아니라 downstream 성능.
- AVTS: **hard** = 같은 영상의 **먼** 구간, **super hard** = 바로 **인접**(겹치는) 구간. 가까울수록 어려워.
- Equivariance는 "안 변함"이 아니라 "**예측 가능하게** 변함". 안 변하는 건 invariance (φ = identity).
- STL의 Blue는 **다른 이미지·같은 변환**. 같은 이미지의 변환 전후는 Brown.
- Cubic Puzzle 48 class = 4! = 24 순열 × **2 orientations**(upside-down 여부, p.62–63). × 2를 spatial/temporal로 읽으면 틀려. 각 tower는 퍼즐이 spatial인지 temporal인지 몰라 (p.62).

다음으로

pretext는 사람이 일일이 설계해야 하고 마지막 layer가 그 과제에 특화돼. 그래서 "같은 건 가깝게, 다른 건 멀게"라는 일반 원리로 가. → L03 (AVTS의 "negative를 어떻게 고르나"도 거기서 다시 나와)

셀프 체크

T/F SSL methods are mainly judged by their accuracy on the pretext task.

False. pretext는 연습 문제일 뿐이야. encoder를 고정(frozen)하고 shallow(linear) head만 학습해서 본 downstream 성능으로 평가해 (p.18, p.46).

빈칸 Colorization predicts the ab channels with _____ over 313 color bins, because regression averages plausible colors.

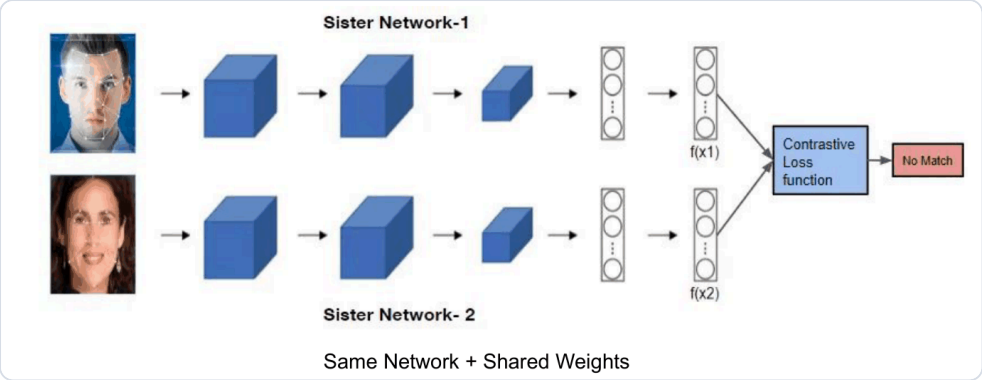
classification. pixel마다 313칸 중 어느 색인지의 확률 분포를 맞혀. regression은 그럴듯한 색들의 평균이라 탁해져 (p.37–38).

빈칸 Equivariance: f(t(x)) = _____.

φ(t, f(x)). 변환 t를 걸면 feature도 φ(t, ·) 규칙대로 예측 가능하게 움직여. invariance면 f(x), 즉 안 변해 (p.75).

L03 Contrastive Learning I 74p

질문 **"같은 건 가깝게, 다른 건 멀게"를 loss로 어떻게 쓰나?**



Siamese network (p.8). 위아래 두 줄(Sister Network-1, -2)은 사실 **같은 네트워크, 같은 weight**야 ("Same Network + Shared Weights"). 두 얼굴 사진이 각각 통과해서 벡터 f(x1), f(x2)가 되고, 같은 좌표계에 놓이니까 둘 사이 거리를 비교할 수 있어. Contrastive Loss는 같은 사람이면 거리를 줄이고 다른 사람이면 늘려. 오른쪽 "No Match"는 두 사람이 다르다는 판정이야. `L03 p.8`

L02의 pretext task(회전 맞히기, jigsaw(섞인 퍼즐 조각 순서 맞히기)처럼 encoder를 훈련시키려고 사람이 만든 가짜 과제)는 전부 사람이 손으로 설계한 objective였어 ("They are all handcrafted objectives", p.4). 그래서 이번 강의는 이렇게 물어: "Can we learn representations without designing a task?"

답은 관점을 바꾸는 거야. 슬라이드 문장 그대로 **"Representation Learning = Learning Similarity"** (p.5). 좋은 representation이라면 의미가 비슷한 샘플끼리 가까이 있어야 해. 그러니까 과제를 따로 설계하지 말고, **"이 둘이 비슷한가"라는 similarity 자체를 supervision(학습 신호)으로 쓰자**는 거야 (p.6).

기본 그림: anchor · positive · negative

encoder f(이미지를 벡터로 바꾸는 네트워크, 예: ResNet, ViT)가 입력 x를 벡터 $z = f(x)$로 바꿔. 이 벡터가 embedding(= representation)이고, 벡터들이 놓이는 공간이 embedding space야. contrastive learning은 이 공간에서 "누가 누구 옆에 있어야 하나"를 가르치는 방법이야. 등장인물은 셋이야.

- **anchor** x (reference, query라고도 해): 기준 샘플. "지금 이 샘플 입장에서 보자"의 그 샘플.
- **positive** x^+: anchor와 의미가 같아서 **가까워져야** 할 샘플. 예: 같은 고양이 사진을 잘라내거나 색을 바꾼 버전.
- **negative** x^-: anchor와 다른 샘플이라 **멀어져야** 할 샘플. 예: 강아지 사진.

슬라이드는 이걸 **metric learning**이라고 불러: "Pull Similar Samples Together, Push Dissimilar Samples Apart" (p.7). 가깝다·멀다를 재는 자는 두 종류야. 방향이 반대라서 헷갈리기 쉬워.

- **거리(distance)** d: **작을수록** 가까워. 보통 Euclidean 거리 $\|f(x_i) - f(x_j)\|_2$ (두 벡터를 빼서 길이를 잰 것).
- **유사도(similarity, score)** s: **클수록** 가까워. 내적(dot product)이나 cosine similarity(두 벡터의 방향만 비교한 값, −1~1)를 써.

원하는 건 부등식 하나야 (p.9): $\text{score}(f(x), f(x^+)) \gg \text{score}(f(x), f(x^-))$. "anchor–positive 점수가 anchor–negative 점수보다 훨씬 커라". score 함수는 미리 정해 두고, 학습하는 건 encoder f뿐이야. 이 부등식을 미분 가능한 loss로 바꾸는 방법이 여러 개라서, 그게 다음 절의 loss 계보가 돼.

Siamese network (p.8). 두 입력을 비교하려면 둘이 같은 좌표계에 있어야 해. 그래서 **weight를 공유하는 똑같은 네트워크**에 두 입력을 각각 통과시켜: $z_1 = f(x_1)$, $z_2 = f(x_2)$. 슬라이드 표현은 "Same Network + Shared Weights", "Both images are mapped into the same embedding space". 같은 사람 얼굴이면 $d(z_1, z_2)$를 줄이고, 다른 사람이면 늘려. weight가 다르면 두 벡터가 서로 다른 좌표계에 놓여서 거리를 재는 의미가 없어.

label이 있든 없든 쓸 수 있어 ("Can be applied to both supervised and unsupervised settings", p.9). 달라지는 건 positive를 정하는 법뿐이야. label(사람이 단 정답)이 있으면 같은 class가 positive야. label이 없으면 같은 이미지를 augmentation(자르기, 색 바꾸기 같은 변형)한 버전끼리 positive야 (p.32).

> **예시로 보면**
> 학습 전엔 고양이, 잘라낸 고양이, 강아지의 embedding이 아무렇게나 흩어져 있어. 학습은 encoder를 조금씩 고쳐서 "고양이–잘라낸 고양이"는 붙이고 "고양이–강아지"는 떼어 놓는 거야. 이걸 수많은 사진에 반복하면 비슷한 것끼리 모인 embedding space가 생겨.

"가깝게·멀게"를 loss로 쓰는 법. loss는 학습이 **줄이는** 값이야. 그러니까

- positive와의 거리는 loss에 **더해**. loss를 줄이면 거리가 줄어 → 당겨져.
- negative와의 거리는 "아직 충분히 안 멀면 벌점"으로 넣거나, 점수 쪽으로 바꿔서 "negative 점수가 크면 손해"로 넣어. loss를 줄이면 거리가 늘어 → 밀려나.

Loss 계보: pair에서 InfoNCE까지

계보에서 바뀌는 건 두 가지야. 한 번에 비교하는 개수(pair → triplet → batch 전체 → N개), 그리고 벌점의 모양이야. 앞의 셋(Contrastive, Triplet, Lifted)은 **거리 + margin**으로 벌점을 주고, 뒤의 넷(N-pair, NCE, InfoNCE, Soft-NN)은 **분류 문제**로 바꿔. 단 분류 방식이 둘로 갈려. N-pair·InfoNCE(그리고 positive를 여러 개 허용한 Soft-NN)는 **"후보 N개 중 positive 고르기"**(softmax, categorical)이고, NCE만 **"샘플 하나씩 data냐 noise냐"**(sigmoid, binary)야. 이후 강의의 기본 loss는 InfoNCE야.

- **Contrastive loss** `Chopra 2005 · p.11`
  pair 단위: 샘플 두 개를 한 쌍씩 보고 판정해. positive는 거리 0으로, negative는 margin ε 밖으로.
  수식 $L_{cont} = 1[y_i=y_j]\cdot d^2 + 1[y_i\neq y_j]\cdot \max(0,\ \varepsilon - d)^2$
  기호 $d = \|f_\theta(x_i) - f_\theta(x_j)\|_2$는 두 embedding 사이 거리, y는 class label이야. $1[\cdot]$은 괄호 안이 참이면 1, 거짓이면 0인 스위치(indicator)라서 두 항 중 하나만 켜져.
  읽는 법 같은 class면 d^2를 줄여서 한 점으로 당겨. 다른 class면 hinge(0 밑으로는 안 내려가는 꺾인 함수) $\max(0,\ \varepsilon - d)^2$를 써. d가 ε보다 작으면 벌점이 생겨 밀어내고, $d \geq \varepsilon$면 벌점 0이라 더는 안 밀어.
  margin ε "다른 class끼리 최소한 이만큼은 떨어져라"라는 거리야 ("minimum distance required between different classes").
  한계 (p.11): ① "considers only pairs": 한 번에 상대 하나만 봐. ② "Many negatives become inactive": 이미 margin 밖인 negative는 loss·gradient가 0이야. ③ "Does not explicitly enforce relative ordering": positive는 0으로, negative는 ε 이상으로라는 **따로따로의 절대 목표**만 있고, "positive가 negative보다 가까워야 한다"는 상대 순서는 강제 안 해.
- **Triplet loss** `FaceNet 2015 · p.12–13`
  anchor·positive·negative를 상대 비교. "positive가 negative보다 margin만큼 가까워라". 세 개를 한 묶음(triplet)으로 봐. FaceNet이 같은 사람의 다른 pose·각도 얼굴을 알아보려고 썼어 (p.12 "Enforces relative similarity", "Focuses on ranking relationships").

수식 $L_{triplet} = \Sigma \max(0, d_{ap} - d_{an} + \varepsilon)$

기호 $d_{ap} = \|f(x) - f(x^+)\|_2^2$는 anchor–positive 거리 제곱, $d_{an} = \|f(x) - f(x^-)\|_2^2$는 anchor–negative 거리 제곱이야. margin ε는 두 거리 사이에 필요한 **최소 격차**야 ("minimum offset", p.13).

읽는 법 $d_{an} - d_{ap} \geq \varepsilon$면 괄호 안이 0 이하라 loss 0, gradient 0 (이미 만족). 위반하면 d_{ap}는 줄이고 d_{an}은 키우는 쪽으로 gradient가 나와. 상대 격차만 보니까 contrastive loss처럼 positive를 꼭 한 점에 모을 필요가 없어.

toy $d_{ap} = 0.5$, $\varepsilon = 1$일 때 $d_{an} = 0.9$면 loss = 0.6 (위반), $d_{an} = 1.6$이면 loss = 0 (만족).

한계 (p.13–14): ① Triplet explosion: 샘플 N개면 triplet이 N^3개라 다 못 써. ② Most triplets are useless: 대부분 이미 만족한 easy triplet(loss 0)이라 계산 낭비야. 그래서 hard negative(embedding은 anchor와 가까운데 class는 다른 샘플)를 골라 쓰는 hard negative mining이 필요해. 모델이 헷갈리는 샘플이라 학습 신호가 강해 (p.14 "Hard negatives provide stronger learning signals").

- **Lifted Structured** Song · p.15–16

  batch 안의 모든 pair 사용. hardest negative를 고르는 max를 log-sum-exp로 부드럽게. mini-batch를 돌리면 샘플끼리 거리가 어차피 다 계산돼. 몇 개만 쓰지 말고 전부(all pairwise edges) 쓰자는 거야 (p.15).

  수식 positive pair (i, j)마다 $L^{(ij)} = D_{ij} + \max(\max_{(i,k)\in N} \varepsilon - D_{ik}, \max_{(j,l)\in N} \varepsilon - D_{jl})$, 전체 loss는 $1/(2|P|) \cdot \Sigma_{(i,j)\in P} \max(0, L^{(ij)})^2$.

  기호 D는 embedding 사이 거리, P는 positive pair 집합, N은 negative pair 집합이야. positive 거리 D_{ij}는 줄이고, i나 j 옆 negative 중 **제일 가까운 것(hardest negative)**만 max로 골라서 ε 밖으로 밀어.

  relax max는 매끄럽지 않아서("not smooth") log-sum-exp(LSE)로 바꾼 식은 $L^{(ij)} = D_{ij} + \log(\Sigma_{(i,k)\in N} \exp(\varepsilon - D_{ik}) + \Sigma_{(j,l)\in N} \exp(\varepsilon - D_{jl}))$.

  p.16 "Why Replace Max?": negative 여러 개의 점수 s_i(클수록 anchor에 가까운 hard negative)를 숫자 하나로 묶는 세 방법을, "각 negative가 gradient를 얼마나 받나"로 비교해.

  Max $\max_i s_i$: hardest 하나만 gradient를 받고 나머지는 0 → sparse.

  Sum(Average) $(1/N)\Sigma_i s_i$: 모두 1/N씩 똑같이 받아 → hard에 집중을 못 해 ("Not focused on hard negatives").

  LSE $\log \Sigma_i e^{s_i}$: s_j로 미분하면 $e^{s_j} / \Sigma_i e^{s_i}$, 즉 softmax 가중치야 → 모두 받되(dense) 어려울수록 지수적으로 더 받아(focused).

  Max = sparse gradient, Sum = uniform, LSE = dense & focused (soft hard negative mining). takeaway: "log-sum-exp is a smooth approximation of max".

- **N-pair loss** Sohn, NeurIPS 2016 · p.17–20

  positive 1개 vs negative **N−1개**를 동시에. = N-way softmax cross-entropy.

  왜 나왔나 (p.17) Lifted도 결국 pair 기반이라 1 positive ↔ 1 negative 비교고, 비교 수는 $O(N^2)$로 늘어. 그래서 "Can we compare one positive against all negatives simultaneously?" → triplet을 negative 여러 개로 일반화했어 (p.18). 입력은 anchor 1 + positive 1 + negative N−1 = (N+1)-tuplet이야.

  수식 $L = -E_X[\log(\exp(s^+) / (\exp(s^+) + \Sigma_{j=1}^{N-1} \exp(s_j^-)))]$ (p.20)

  기호 $s^+ = s(f(x), f(x^+))$는 positive pair 점수("score for the positive pair"), $s_j^- = s(f(x), f(x_j^-))$는 j번째 negative pair 점수, E_X는 여러 anchor에 대해 평균낸다는 뜻이야. loss를 줄이려면 s^+는 키우고 s_j^-는 줄여야 해.

  같은 식 p.18의 원래 꼴 $\log(1 + \Sigma_i \exp(f(x)^T f(x_i^-) - f(x)^T f(x^+)))$도 정리하면 같은 식이야 (여기선 점수 = 내적). 분수의 분자·분모를 $\exp(s^+)$로 나눠 보면 바로 보여.

  핵심 "This seems familiar... Cross entropy loss for a N-way softmax classifier! I.e., learn to find the positive sample from the N samples" (p.20). 점수들을 logit(softmax에 들어가기 전 점수)으로 보고 softmax로 확률을 만든 뒤, 정답 = positive인 cross-entropy를 거는 거야.

"1 positive ↔ many negatives → More information → No need hard negative mining" (p.17). softmax 분모 안에서 hard negative는 exp 때문에 자동으로 큰 가중을 받으니까, 따로 골라내는 부담이 줄어.

- **NCE** `Gutmann 2010 · p.21–22`

  positive vs negative를 data vs noise로 보고 **이진** 분류 (sigmoid, binary CE).

  발상 (p.21) "negative를 그냥 noise로 볼 수 있나? 확률적으로 해석할 수 있나?" → Yes. 샘플을 **하나씩** 보고 "진짜 data에서 왔어, noise에서 왔어?"를 맞히는 logistic regression이야.

  수식 logit $\ell_\theta(u) = \log p_\theta(u) - \log q(u)$, 확률 $\sigma(\ell) = 1/(1 + e^{-\ell})$, loss $-(1/N)\ \Sigma_i[\log \sigma(\ell_\theta(x_i)) + \log(1 - \sigma(\ell_\theta(\tilde{x}_i)))]$ (p.22)

  기호 ℓ은 "u가 noise보다 data에서 왔을 가능성이 몇 배인가"의 log(log-odds)야. p_θ는 학습 중인 모델 분포, q는 noise 분포, σ는 sigmoid(아무 실수나 0~1 확률로 눌러 주는 함수)야. 진짜 샘플 x_i는 σ → 1로, noise 샘플 $\tilde{x}_i$는 σ → 0으로 보내. 이게 binary cross-entropy야.

  원래 정체는 "a method for estimating parameters of a statistical model" (p.21). 다음 InfoNCE와 이름이 비슷해서 바꿔치기 단골이야.

- **InfoNCE** `Oord 2018, CPC · p.23–28`

  CPC(Contrastive Predictive Coding, p.23 출처 논문)가 낸 loss야. N개 중 positive 하나 고르기 (**categorical** CE). 정의 원문: "The InfoNCE loss uses categorical cross-entropy loss to identify the positive sample amongst a set of unrelated noise samples." (p.23)

  수식 $L = -E[\log(f(x, c) / \Sigma_{x' \in X} f(x', c))]$

  기호 여기 f는 encoder가 아니라 "x와 c가 얼마나 짝 같은가"를 재는 점수 함수야 (CPC 논문 표기 그대로). c는 context로, anchor 역할이야 ("Anchor (x) can be regarded as context (c)", p.26). x는 c와 짝인 positive, X는 positive 1개 + noise(negative) N−1개를 모은 후보 집합, f(x, c)는 양수 점수고 보통 exp(유사도)를 써. 분수 = "N개 후보 중 positive를 고를 확률"이고, −log를 씌우면 N-way categorical CE. N-pair와 같은 모양이야.

  확률 해석 positive는 c에 딸린 분포 p(x|c)에서, negative N−1개는 c와 무관한 p(x)에서 뽑았다고 놓고, "N개 중 몇 번째가 positive인가"의 확률을 Bayes rule로 쓰면 각 후보의 몫이 p(x|c)/p(x)에 비례해. 그래서 (p.23–24) 최적의 f는 $f(x, c) \propto p(x|c) / p(x)$ (density ratio)야. "c를 알 때 x가 나올 확률이, c를 모를 때보다 몇 배인가"라는 뜻이야.

  MI 이 비율의 log를 평균내면 mutual information(두 변수가 공유하는 정보량) $I(x; c) = \Sigma\ p(x, c) \log[p(x|c) / p(x)]$이 돼 (p.27). 흐름은 "Learning a good f(x,c) → Estimating a good density ratio → Increasing Mutual Information" (p.28). 즉 InfoNCE는 positive 맞히기를 넘어서 x와 c가 공유하는 정보를 키우는 쪽으로 학습돼.

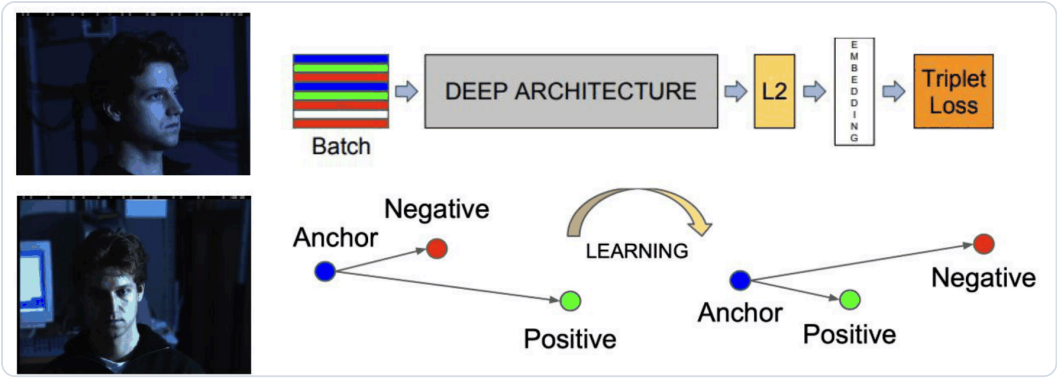
  NCE와 차이: NCE는 샘플 하나씩 **binary**(sigmoid), InfoNCE는 N개를 한 번에 보고 **categorical**(softmax). N-pair와 관계: metric learning 쪽(N-pair)과 확률·정보 쪽(InfoNCE)이 같은 식에 도착한 거야.

- **Soft-NN loss** `p.29`

  positive를 여러 개로 (같은 class 이웃 전부). label이 필요해. "extends it to include multiple positive samples" (p.29).

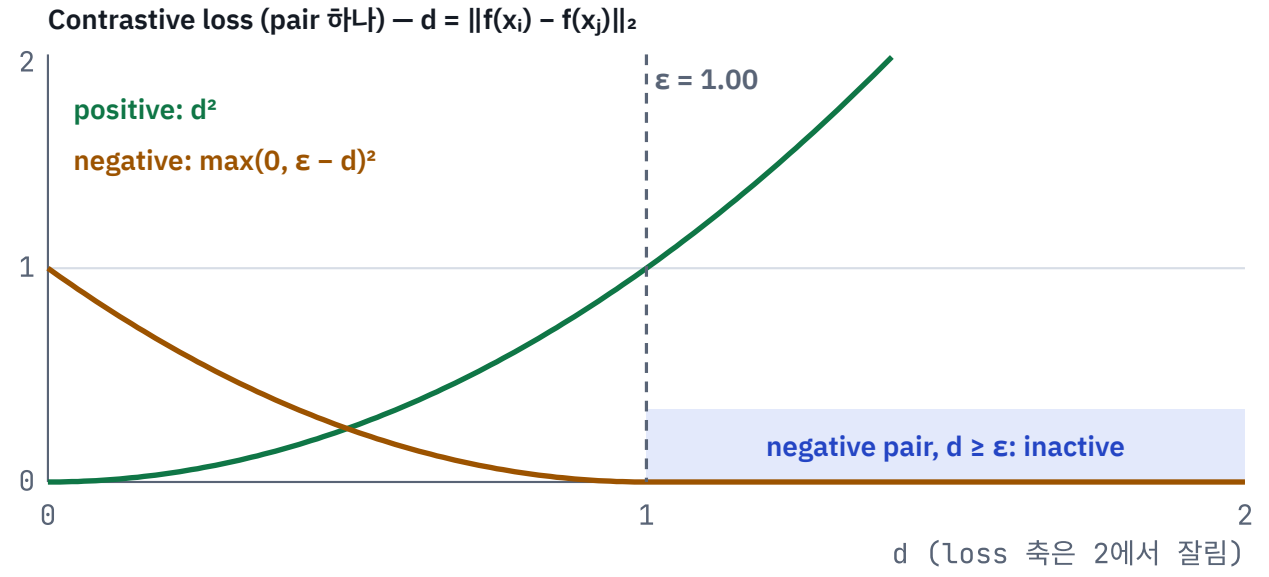
  구조 분자에는 anchor와 **같은 class인 이웃 전부**("Neighbors of the same class"), 분모에는 자기 자신 빼고 **모든 이웃**("All neighbors")이 들어가. 그래서 "이웃을 하나 soft하게 뽑았을 때 같은 class가 나올 확률"을 높이는 loss야.

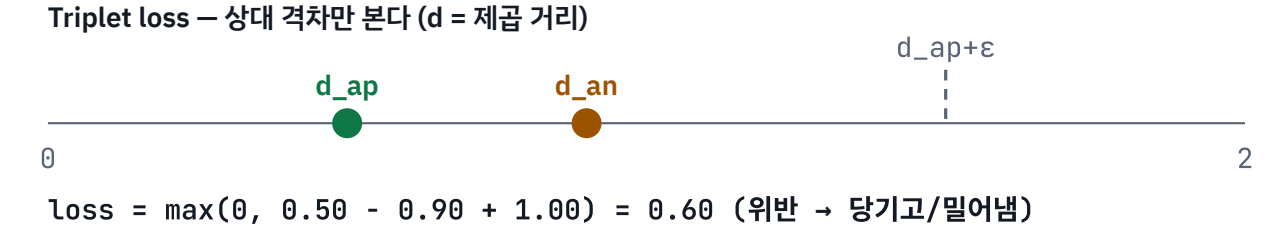
  temperature τ(similarity를 τ로 나눈 뒤 softmax에 넣는 숫자. 작을수록 분포가 뾰족해져. 자세한 건 뒤의 Modern setup)는 "how concentrated the features are in the representation space"를 조절해 (p.29). 누가 같은 class인지 알아야 하니까 label이 있어야 해.



Triplet loss (FaceNet, p.12). 위 줄은 파이프라인이야: 얼굴 batch → CNN(Deep Architecture) → L2 normalize(벡터 길이를 1로) → embedding → triplet loss. 아래 줄은 embedding 공간의 세 점이야. 파랑 = Anchor, 초록 = Positive(같은 사람), 빨강 = Negative(다른 사람). 학습 전(왼쪽)엔 Negative가 Anchor에 더 가깝고, LEARNING 후(오른쪽)엔 **Positive가 Negative보다 가까워져**. 거리를 특정 값에 맞추는 게 아니라 **상대적 순서(ranking)**를 맞춘다는 게 포인트야. `L03 p.12`

인터랙티브 **Contrastive vs Triplet: 절대 목표 vs 상대 격차** `illustrative`

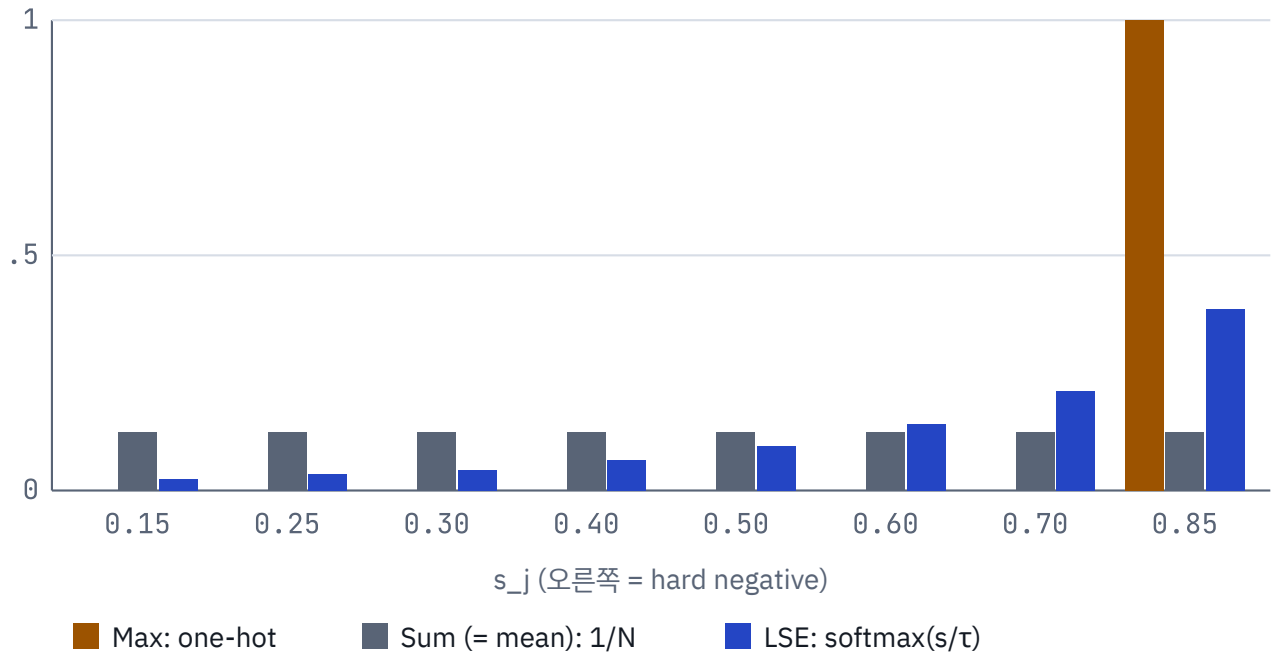




margin ε 1.00 d_an (triplet) 0.90

계보의 Contrastive loss (p.11)와 Triplet loss (p.12–13)를 비교하는 그림. **위 그래프**: 가로축은 두 embedding 사이 거리 d, 세로축은 loss야. "positive: d²" 곡선은 d가 0이 될 때까지 계속 당겨. "negative: max(0, ε − d)²" 곡선은 d < ε일 때만 밀고, 음영 구간(negative pair, d ≥ ε)에서는 loss = gradient = 0이라 inactive야. margin ε 슬라이더로 경계를 옮겨 봐. **아래 줄**: triplet도 positive는 당기고 negative는 밀지만(p.13), 절대 목표(positive → 0, negative → ε)를 없애고 상대 조건 `d_an - d_ap ≥ ε`만 요구해. d_an 슬라이더로 점을 점선(d_ap + ε)보다 오른쪽에 두면 loss 0(만족), 왼쪽이면 loss > 0(위반). 그래서 positive를 한 점에 모을 필요가 없어 (d_ap는 0.5로 고정한 개념 그림).

인터랙티브 **Max vs Sum vs Log-Sum-Exp: negative가 받는 gradient 가중치** illustrative



temperature τ 0.25

계보의 Lifted Structured (p.15–16)에서 max를 log-sum-exp로 바꾸는 이유를 보여주는 그림 (L03 p.16). 가로축은 negative 8개의 점수 s_j(오른쪽일수록 anchor에 가까운 hard negative), 막대 높이는 각 negative가 받는 gradient 가중치 ∂/∂s_j야. Max는 hardest 하나만 1이고 나머지 0(sparse). Sum(= mean)은 전부 1/N로 똑같아서 hard 신호가 묻혀. LSE는 전부 받되 hard일수록 커(dense & focused). τ 슬라이더: τ → 0이면 LSE 막대가 Max의 one-hot으로, τ → ∞면 Sum처럼 평평해져 (τ는 p.16에는 없는 보충 개념, s 값은 개념 그림).

Modern setup

p.31–32

한 줄 요즘 contrastive learning(L04의 SimCLR 등)이 공통으로 쓰는 틀이야.

anchor, positive = 같은 이미지의 다른 augmentation, negative = batch 안의 다른 이미지. L2-normalize + cosine similarity, temperature τ를 넣은 InfoNCE (NT-Xent).

부품 6개 (p.31 "Key Components")

1. **Anchor (query)**: 데이터셋의 샘플 x.
2. **Positive(s)**: 같은 x를 다르게 augmentation한 view(변형본 하나하나를 view라고 해).
3. **Negatives**: 같은 batch 안의 다른 샘플들.
4. **Encoder f(·)**: 입력을 representation으로 바꿔. 보통 L2-normalize(벡터 길이를 1로 맞춤)해서 방향만 남겨. 모든 가지가 같은 encoder를 써.
5. **Similarity**: 보통 cosine $s(a, b) = z_a^T z_b / (\|z_a\|\|z_b\|)$, 값은 −1~1.
6. **Contrastive loss**: positive는 당기고 negative는 밀어.

수식 $L = -\log[\exp(s(z, z^+)/\tau) / (\exp(s(z, z^+)/\tau) + \Sigma_{i=1}^{N} \exp(s(z, z_i^-)/\tau))]$.

z는 anchor embedding, z^+는 positive, z_i^-는 i번째 negative야. N-pair 식에 "÷τ"만 더한 모양이야. 여기선 negative를 N개로 셌고, N-pair·InfoNCE는 후보 전체를 N개(negative N−1개)로 셌어. 세는 방식만 다르고 구조는 같아. 이 loss를 부르는 이름이 NT-Xent(Normalized Temperature-scaled cross-entropy: L2-normalize한 cosine + τ + cross-entropy)야. L04 SimCLR에서 다시 나와.

예시로 보면

batch에 사진 4장이 있어. 사진마다 augmentation을 두 번 하면 view가 8개야. view 하나를 anchor로 잡으면, 같은 사진에서 나온 짝 view 1개가 positive, 다른 사진 3장에서 나온 view 6개가 negative야. InfoNCE는 "후보 7개 중에 내 짝이 누구야?"를 맞히는 7지선다 분류가 돼.

cosine similarity가 positive 0.8, 제일 헷갈리는 negative 0.7, 나머지 0.4, 0.3, 0.1, −0.1, −0.3이라고 해 보자 (설명용 숫자). 계산 순서는 similarity ÷ τ = logit → softmax로 7개 확률 → loss = −log(positive 확률).

- τ = 1: logit이 그대로라 확률이 positive 0.23, hard negative 0.20으로 거의 비슷해. loss ≈ 1.49.
- τ = 0.1: logit이 10배(8, 7, 4, ...)가 돼서 positive 0.72, hard negative 0.26, 나머지는 거의 0이야. loss ≈ 0.33.

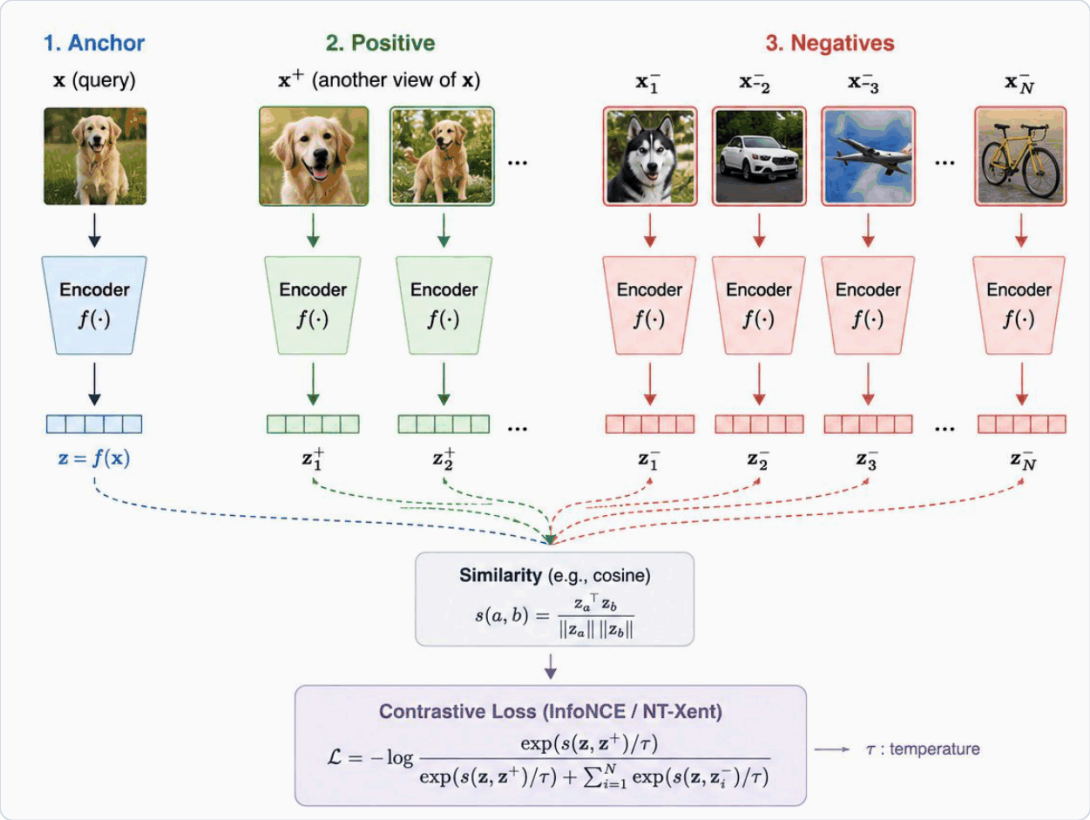
`τ (temperature)` temperature는 similarity를 τ로 나눈 뒤 softmax에 넣어. cosine은 −1~1이라 그대로 넣으면 확률이 너무 평평해. τ가 작을수록 점수 차이가 확대돼서 분포가 뾰족해져. 이 τ가 하는 일은 두 가지로 나눠 봐야 해.

- **loss 값**: positive가 이미 1등 점수일 때는 τ가 작을수록 loss 값 자체가 **작아져** (예시에서 1.49 → 0.33). "τ를 줄이면 loss가 커진다"가 아니야.
- **gradient 배분**: negative j가 받는 gradient는 그 softmax 확률 p_j에 비례해. τ가 작으면 negative들 사이에서 hard negative 몫이 커져 (예시에서 negative 확률 합 중 hard negative 비중: τ = 1이면 0.26, τ = 0.1이면 0.93). 그래서 작은 τ = hard negative에 집중. LSE의 "dense & focused"와 같은 논리야. τ가 크면 몫이 고르게 퍼져서 Sum(평균)처럼 굴어.

`label 있으면` 같은 class가 positive. 그래서 anchor 하나에 positive가 여러 개일 수 있어 (Soft-NN, 나중 L05의 SupCon(supervised contrastive: label로 같은 class를 positive로 쓰는 방법)). label이 없을 땐 어떤 augmentation을 쓰느냐가 "무엇이 바뀌어도 같은 걸로 볼지"를 정해 (p.32).

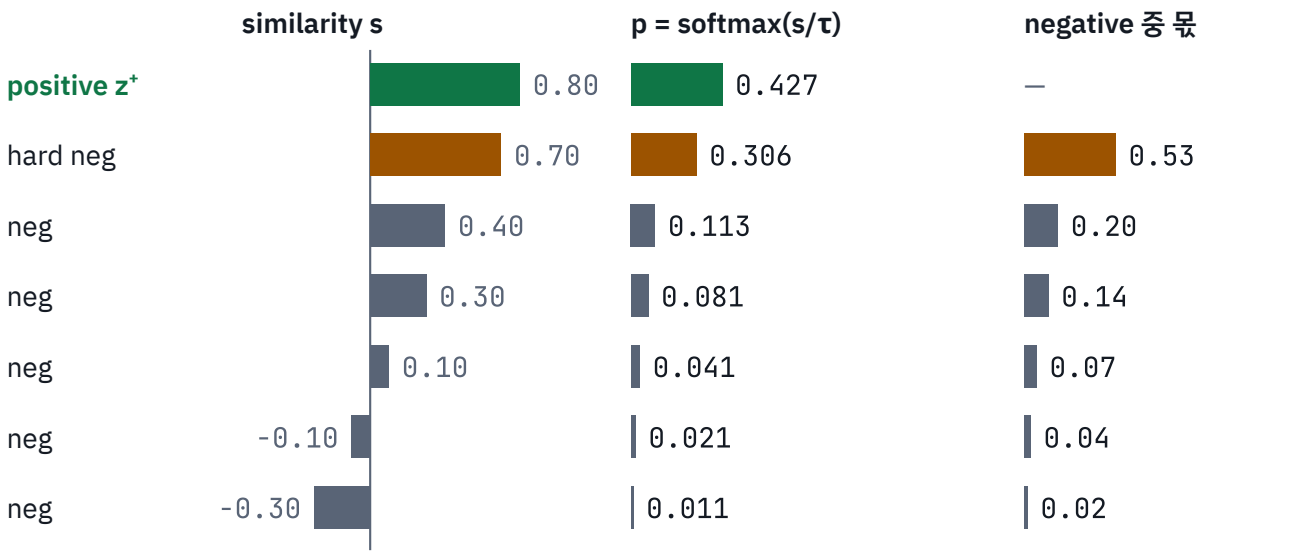
`p.32 조건` positive pair 분포 p_{pos}는 둘을 만족해. **Symmetry** $p_{pos}(x, x^+) = p_{pos}(x^+, x)$: 누가 anchor고 누가 positive인지 순서가 상관없어. **Matching marginal** $\int p_{pos}(x, x^+)\, dx^+ = p_{data}(x)$: positive를 뽑는 방식이 원래 데이터 분포를 비틀지 않아.

`조심` "batch의 다른 이미지 = negative" 규칙이라, 같은 class의 다른 사진(다른 고양이)도 negative로 밀려나. 이게 false negative야. 뒤의 X-CLR과 L04에서 다시 나와.



현대 contrastive learning의 공통 틀 (p.31). 파랑 = Anchor x(query), 초록 = Positive x^+(같은 x의 다른 augmentation view), 빨강 = Negatives(batch 안의 다른 샘플). 모두 **같은 encoder f**를 지나 벡터 z가 되고, anchor z와 나머지 각각의 cosine similarity s(a, b)를 구해서 InfoNCE / NT-Xent(Normalized Temperature-scaled cross-entropy) loss에 넣어. 오른쪽 τ는 temperature야. L03 뒷부분의 GenView·MCL·X-CLR은 이 틀에서 positive view, negative 고르기, target을 각각 바꾼 변주야. `L03 p.31`

인터랙티브 **InfoNCE = N개 후보(1 positive + N−1 negative) 중 positive 고르는 softmax 분류** illustrative



loss = -log p(positive) = 0.851

hard negative의 몫 = 0.53 (τ를 낮춰 봐) (N = 7)

temperature τ 0.30

InfoNCE와 temperature τ(p.23–32)를 직접 만져 보는 그림. 본문 예시처럼 후보가 7개(positive 1 + negative 6)인 7지선다 softmax 분류야. 왼쪽 열은 anchor와 각 후보의 similarity s, 가운데 열은 s/τ를 softmax한 확률 p, 오른쪽 열 "negative 중 몫"은 negative들끼리 gradient를 나눠 갖는 비율 `p_j / (1 - p_pos)` 야. 아래쪽 loss 줄이 −log p(positive)이고, 그 밑 줄은 hard negative 몫을 숫자로 보여줘. τ 슬라이더를 움직여 봐. 이 예시는 positive가 이미 1등(argmax)이라 τ가 작을수록 loss가 **작아져** (τ=1 → 1.49, τ=0.05 → 0.13). 작은 τ의 진짜 효과는 오른쪽 열이야: hard negative(0.70)의 몫이 커져 (τ=1 ≈ 0.26, τ=0.1 ≈ 0.93, τ=0.05 ≈ 1.00). τ가 크면 몫이 고르게 퍼져. (s 값은 설명용 숫자)

최근 논문 3편: views · features · relationships (p.73)

여기까지가 표준 틀이야. 이제 강의는 "Are positive and negative pairs always constructed correctly?" (p.33)라고 물어. 표준 InfoNCE에는 말 안 한 가정이 숨어 있고, 논문 세 편이 하나씩 건드려 (p.73 표).

- **views** "Are positive views good enough?": augmentation으로 만든 positive가 항상 좋은 짝인가? → GenView
- **features** "Are useful features suppressed?": positive와 negative를 구분만 하면 쓸모 있는 feature를 다 배우나? → MCL
- **relationships** "Are sample relations binary?": negative는 모두 똑같이 "남"인가? → X-CLR

GenView

Li, ECCV 2024 · p.33–45

한 줄 생성 모델로 positive view를 새로 그려 주고, 품질 좋은 pair에 loss 가중치를 더 주는 방법.

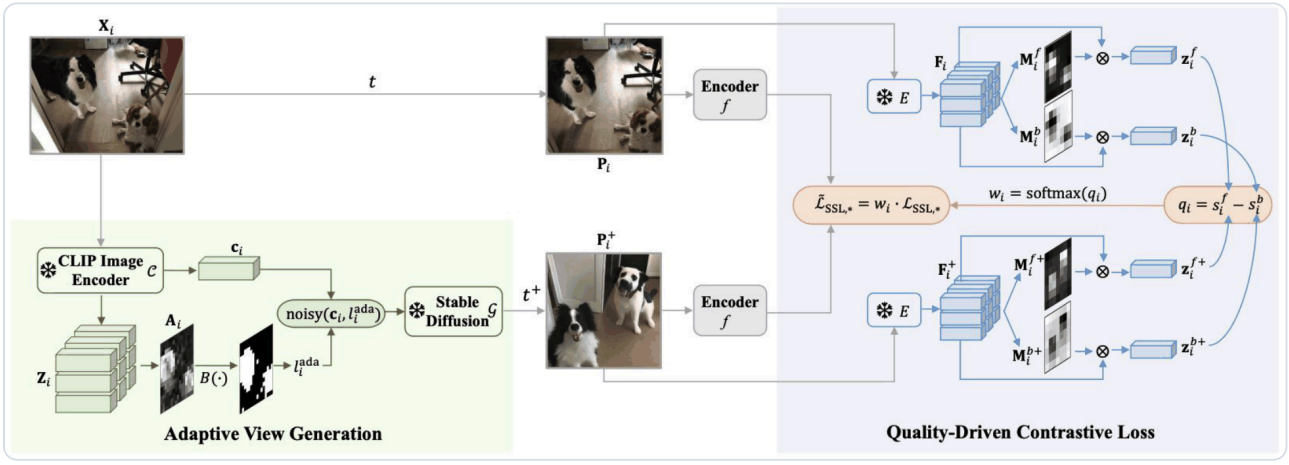
문제 손으로 만든 view의 한계. 세게 crop하면 물체가 사라지고(false positive pair), 약하게 하면 두 view가 거의 똑같아(limited diversity). 풀어 보면 (p.36): crop(사진 일부만 잘라내기)을 세게 하면 개는 없고 벽돌 배경만 남은 view가 나와. 그런데 loss는 이걸 개 사진과 "같은 것"으로 당겨. 세게 vs 약하게의 trade-off야. 결론: "A good positive pair should preserve semantics while introducing diversity".

핵심 생성 모델(Stable unCLIP)로 positive view를 만들고, foreground 비율에 따라 noise 양을 조절. 좋은 pair(foreground 비슷, background 다름)에 loss 가중치를 더 줘. 목표는 pose·viewpoint·background는 바꾸고 물체 정체(object identity)는 지키는 거야.

어떻게

1. 원본을 frozen(학습 안 하고 그대로 쓰는) CLIP(이미지와 text를 같은 공간에 놓도록 학습된 모델, L05) image encoder에 넣어 이미지 embedding을 얻어.
2. foreground(주인공 물체)가 화면에서 차지하는 비율로 noise 양을 정해 (p.40). **작은 foreground → noise 적게**: 많이 주면 물체가 쉽게 사라져. **큰 foreground → noise 많이**: 변화를 더 줘도 안전해.
3. noise를 섞은 embedding을 조건으로 frozen Stable unCLIP(diffusion 생성 모델)이 새 이미지 X_i^+를 그려. 여기에 augmentation t^+를 한 번 더 걸면 생성 view P_i^+야.
4. 평소 augmentation view P_i와 생성 view P_i^+를 positive pair로 SSL(self-supervised learning) loss를 걸어. loss 종류는 상관없어 ("various loss functions can be used", p.41).
5. 별도의 frozen encoder가 pair 품질을 매겨 (p.42): $q_i = s_i^f - s_i^b$, $w_i = \mathrm{softmax}(q_i)$, 그리고 loss에 w_i를 곱해. s^f는 두 view의 foreground끼리 유사도, s^b는 background끼리 유사도야.

왜 돼? 좋은 pair 조건이 "Similar foreground, Different background"야 (p.42). foreground가 비슷하면 같은 물체라 의미가 지켜졌고, background가 다르면 배울 변화가 있어. 그래서 q가 크면 좋은 pair야. 생성하다 물체가 바뀌면 s^f가 떨어지고, 배경이 그대로면 s^b가 높아서, 둘 다 q가 낮아져 가중치가 줄어.



GenView 전체 구조 (p.38). **왼쪽 아래 초록 영역 (Adaptive View Generation)**: 원본 X_i를 CLIP image encoder에 넣어 embedding c_i를 얻고, foreground 비율로 정한 양 l_i^{ada}만큼 noise를 섞어(noisy) Stable Diffusion(본문의 Stable unCLIP)이 새 이미지를 그리고, 그걸 augmentation t^+에 통과시킨 게 positive view P_i^+야. 위쪽 P_i는 보통 augmentation t로 만든 view야. 가운데: 두 view를 학습 중인 encoder f에 넣어 SSL loss를 계산해. **오른쪽 파랑 영역 (Quality-Driven Contrastive Loss)**: 별도 encoder E가 두 view를 foreground(M^f)와 background(M^b)로 나눠 유사도를 재고, **$q_i = s_i^f - s_i^b$**로 품질을 매긴 뒤 **$w_i = \mathrm{softmax}(q_i)$**를 loss에 곱해. 눈송이 아이콘이 붙은 모듈(CLIP, Stable Diffusion, E)은 학습 안 하는 frozen 모델이야. `L03 p.38`

MCL

Zhang, ECCV 2024 · p.46-61

한 줄 학습을 여러 stage로 나눠서, 앞 stage가 이미 써먹은 쉬운 단서로는 구분이 안 되게 negative를 골라 주는 방법. 그러면 묻혀 있던 feature를 차례로 배워.

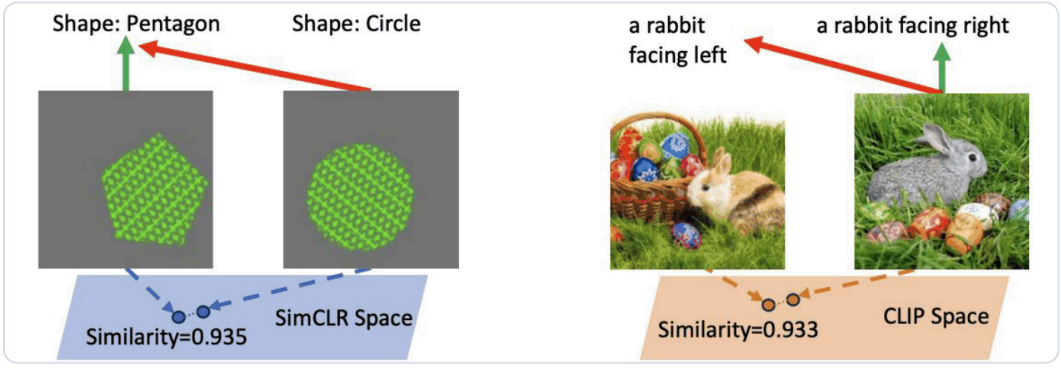
문제 feature suppression. 가장 쉬운 구별 feature(예: 색)만 배우면 loss가 이미 만족돼서 shape 같은 정보를 버려. 정의 (p.49): "useful information present in the input is lost in the representation". 예 (p.48): SimCLR는 색·texture가 같고 모양이 다른 두 그림을 similarity 0.935로 거의 같다고 봐. CLIP도 방향만 다른 두 토끼를 0.933으로 봐.

왜 생겨? (p.50) anchor와 positive는 빨간 줄무늬 원, negative는 파란 사각형이라고 해 봐. 색만 봐도 구분돼서 objective가 이미 만족돼. 그러니 texture나 shape를 더 배울 이유가 없어: "The model learns the easiest discriminative feature". 이런 쉬운 지름길 단서를 shortcut이라고 해.

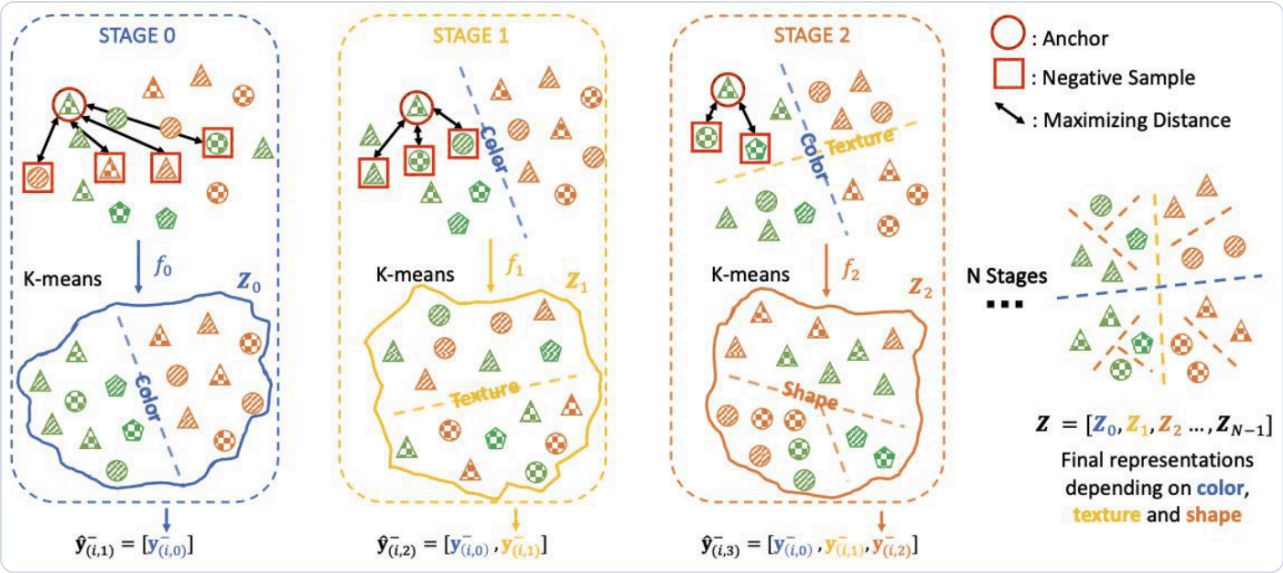
핵심 Train → K-means → 같은 cluster 안에서 negative 선택 → 반복 → feature concat. stage마다 이전 shortcut을 막아 (색 → texture → shape).

1. **Train**: stage 0은 보통 contrastive learning이야. 제일 쉬운 feature(예: 색)를 먼저 배운다고 봐.
2. **Cluster**: 배운 embedding을 K-means(가까운 것끼리 K개 그룹으로 묶는 알고리즘)로 묶어. 같은 cluster = 이번 stage가 쓴 단서(색)가 같은 샘플들이야.
3. **Select negatives within clusters**: 다음 stage는 anchor와 **같은 cluster 안에서만** negative를 뽑아. 다 같은 색이라 색으로는 구분이 안 돼. 그래서 새 encoder가 texture 같은 다른 단서를 찾아야 해.
4. **Repeat**: stage 2는 색도 같고 texture도 같은 그룹에서 negative를 뽑아 → 남은 단서는 shape.
5. **Concatenate**: stage별 representation을 이어 붙여 $z = [z_0, z_1, \ldots, z_{N-1}]$. "Each stage removes the shortcut used in previous stages", 그리고 concat이 모든 stage에서 배운 feature를 보존해.

차이 hard negative mining과 달리 "지금 가까운 것"이 아니라 "이전 stage의 같은 cluster"에서 negative를 뽑아. 목적도 달라 (p.56). hard negative mining은 어려운 샘플 찾기고, MCL은 "prevent the reuse of previously learned features"(이미 배운 feature를 또 쓰지 못하게 막기)야.



Feature suppression 예시 (p.48). 왼쪽: 무늬·색은 같고 모양만 다른 오각형과 원을 SimCLR는 **similarity 0.935**로 거의 같다고 봐 (shape 정보를 버림). 오른쪽: 왼쪽을 보는 토끼와 오른쪽을 보는 토끼를 CLIP이 **0.933**으로 봐 (방향 정보를 버림). 초록 화살표는 각 그림의 실제 설명, 빨간 화살표는 표현이 거의 같아서 생기는 혼동을 가리켜. 즉 **similarity가 높다고 같은 semantics는 아니야**. 이게 MCL의 출발 문제야. L03 p.48



MCL 파이프라인 (p.51). 빨간 원 = Anchor, 빨간 사각형 = 고른 Negative, 양방향 화살표 = 거리 벌리기. **Stage 0**: 보통 contrastive로 학습하면 공간 Z_0가 색(Color)으로 갈라져. 여기에 K-means를 돌려 cluster를 만들어. **Stage 1**: anchor와 같은 색 cluster 안에서만 negative를 골라. 색으로는 구분이 안 되니 새 encoder f_1이 texture를 배워. **Stage 2**: 색·texture가 모두 같은 그룹에서 negative를 골라 shape를 배워. 패널 아래 $\hat{y}^-$ 식은 stage마다 cluster 정보가 쌓여 가는 모습이야 (기호 정의는 슬라이드에 없어). 마지막엔 stage별 representation을 **concatenate**해서 $Z = [Z_0, Z_1, \ldots, Z_{N-1}]$로 써. `L03 p.51`

X-CLR

`Sobal, ICLR 2025 · p.62-72 · code p.67`

한 줄 "positive면 1, 아니면 0" 정답 대신, text로 잰 "얼마나 비슷한가" 연속값을 정답으로 주는 contrastive learning.

문제 target이 binary라서 다른 개, 고양이, 피아노가 똑같이 negative 취급돼. 슬라이드 질문: "Should another dog, a cat, and a piano be treated as equally negative?" (p.64). InfoNCE의 정답은 positive 칸만 1인 one-hot이라, 의미상 가까운 다른 개도 피아노와 똑같이 target 0, 즉 "완전히 남"으로 취급돼 (p.64 "Treated equally"). 게다가 softmax에서는 가까운 negative일수록 gradient 몫이 크니까, 다른 개의 embedding이 피아노보다 가까우면 오히려 더 세게 밀려나.

핵심 frozen text encoder로 caption끼리의 similarity를 구해서 그걸 soft target graph로. image similarity graph를 거기에 CE로 맞춰 (p.67 "Match the learned graph to the target graph").

여기서 graph는 batch 샘플끼리의 similarity 행렬이야. (i, j) 칸 = "샘플 j가 샘플 i에게 얼마나 positive인가". caption(사진에 붙은 설명 문장)이나 class 이름 문장("a photo of a Maltese dog")을 고정된(fixed) text encoder에 넣고, 그 similarity로 target graph를 만들어 (p.66).

수식 $L_{X\text{-}CLR} = (1/2N_b)\ \Sigma_i\ H(s_i,\ p_i)$ (p.67)

- $s_{i,j} = \mathrm{softmax}_j(G_{i,j}\ /\ \tau_s)$, $G_{i,j} = \mathrm{sim}(f_{text}(c_i),\ f_{text}(c_j))$: caption c끼리의 text similarity를 softmax한 **target** 분포.
- $p_{i,j} = \exp(\mathrm{sim}(z_i,\ z_j)/\tau)\ /\ \Sigma_{k\neq i}\ \exp(\mathrm{sim}(z_i,\ z_k)/\tau)$: image embedding z끼리의 similarity를 softmax한 **예측** 분포 (자기 자신은 분모에서 빼). InfoNCE의 softmax와 같아. ← 슬라이드 코드엔 이 mask가 없어 (의사코드 단순화).
- $H(s,\ p) = -\Sigma_j\ s_j\ \log\ p_j$: cross-entropy. N_b는 batch 크기고, view가 2개라 샘플은 $2N_b$개야.

즉 InfoNCE의 one-hot 정답을 soft label로 바꾼 거야. s_i가 one-hot이면 그냥 InfoNCE(SimCLR)로 돌아가.

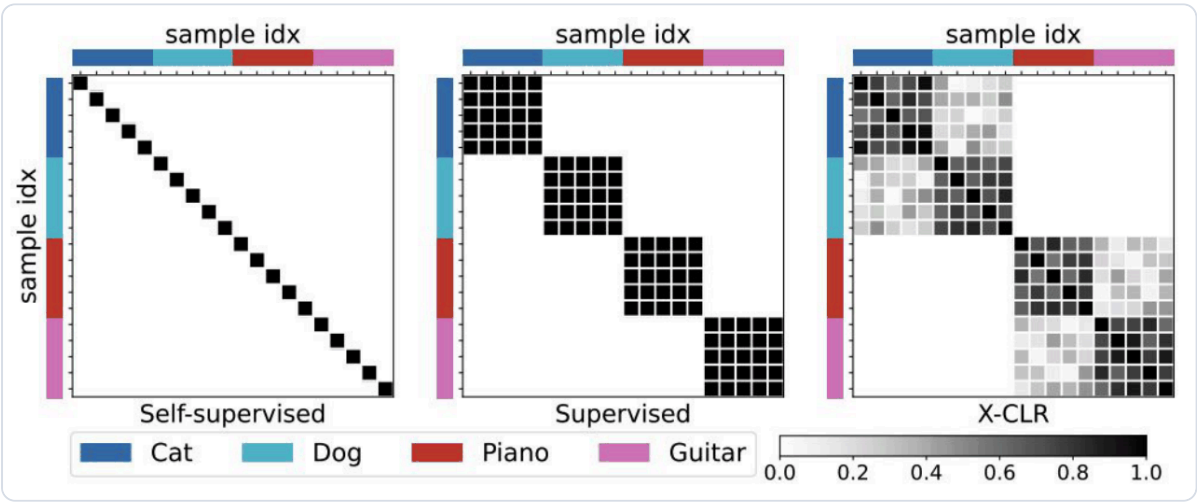
코드 (p.67) 줄마다 수식과 1:1로 맞아. 코드 철자 `tao` = τ, `tao_s` = τ_s야.

1. `txt_emb = txt_enc(captions).detach()` : caption을 frozen text encoder에 넣고 detach(stop-gradient, gradient 끊기)해. text 쪽은 상수 target이 돼.
2. `img_emb = img_enc(images)` : 학습하는 image encoder. 학습되는 건 이것뿐이야.
3. `tgt_sims = softmax(txt_emb @ txt_emb.T / tao_s, dim=1)` : text끼리 similarity를 τ_s로 softmax → target graph s_i.
4. `img_sims = softmax(img_emb @ img_emb.T / tao, dim=1)` : image끼리 similarity를 τ로 softmax → learned graph p_i.
5. `return CE(tgt_sims, img_sims, dim=1).mean()` : 둘의 cross-entropy $H(s_i, p_i)$. 인자 순서는 (target, prediction)이야.

두 τ (p.68) τ는 image 쪽의 보통 contrastive temperature야. τ_s는 target graph를 얼마나 soft하게 만들지 정해. τ_s가 낮으면 가장 강한 positive에 집중해서 표준 contrastive learning과 비슷해. 높으면 관련 샘플에 weight가 더 가서 graph가 softer해져. 그래도 "The graph should preserve the true positive while including soft positives".

주의 X-CLR의 τ_s(코드 `tao_s`)는 **target graph(text) 쪽** temperature야. L04 DINO의 τ_s(`tps`)는 **student 쪽**이고 teacher 쪽은 τ_t(`tpt`)라서, 아래첨자 s의 뜻이 반대야.

차이 SSL graph = 두 view만 연결, SupCon = 같은 class(binary), X-CLR = 정도가 다른 similarity (p.65).



세 가지 target graph (p.65). 가로·세로 모두 batch 샘플 번호고, 색 띠는 class(Cat·Dog·Piano·Guitar)야. 칸이 검을수록 target 값이 커 (오른쪽 아래 막대 0~1). **Self-supervised**: 대각선만 1 (같은 이미지의 두 view만 positive). **Supervised**: 같은 class 블록 전체가 1이고 나머지는 0 (binary). **X-CLR**: 회색 농도의 soft similarity라서, 예를 들어 Cat–Dog 블록은 회색으로 연결되고 Cat–Piano 블록은 거의 흰색이야. `L03 p.65`

인터랙티브 **Target graph: X-CLR (soft) vs SupCon vs SSL** illustrative

X-CLR (soft)

s = softmax(G/τ_s), 행 합 = 1

	dogA	dogB	cat	piano
dogA	.49	.35	.13	.03
dogB	.35	.49	.13	.03
cat	.17	.17	.63	.04
piano	.06	.06	.06	.83

SupCon

0/1: 같은 class

	dogA	dogB	cat	piano
dogA	1	1	0	0
dogB	1	1	0	0
cat	0	0	1	0
piano	0	0	0	1

Self-supervised

0/1: 같은 이미지 view

	dogA	dogB	cat	piano
dogA	1	0	0	0
dogB	0	1	0	0
cat	0	0	1	0
piano	0	0	0	1

진할수록 target 값이 큼

target temperature τ_s 0.30

X-CLR의 target graph를 SupCon·SSL과 비교하는 그림 (p.62–68). 행 i = anchor, 열 j = 후보, 칸 값 = "j가 i에게 얼마나 positive인가"의 target이고, 진할수록 값이 커. **위 X-CLR 패널만 행 정규화**된 값이야: `s_ij = softmax_j(G_ij/τ_s)`, 한 행을 더하면 1 (CE의 target 분포, p.67 코드처럼 대각 포함). 아래 SSL(대각 1 = 같은 이미지의 다른 view만 positive)과 SupCon(같은 class면 1)은 0/1 graph야. τ_s 슬라이더: 낮추면 X-CLR이 대각만 남는 one-hot(≈ 표준 contrastive)으로 가고, 높이면 평평해져서 dog–piano 같은 먼 짝도 값을 꽤 받아. 그래서 p.68은 "preserve the true positive"를 강조해. G는 지어낸 text 유사도 예시(dogA–dogB 0.9, dog–cat 0.6, dog–piano 0.2, cat–piano 0.2).

RL로 번역

- LSE는 "부드러운 max"라서 soft Q-learning·SAC의 soft value `V = α·log Σ exp(Q/α)`랑 같은 모양이야. α → 0이면 그냥 max Q가 돼. max 대신 쓰면 gradient가 모든 항에 퍼지되 큰 항에 더 몰려.
- InfoNCE의 τ는 Boltzmann(softmax) policy의 temperature랑 같아. 작으면 argmax 쪽(greedy)으로 몰리고, 크면 uniform에 가까워져.
- InfoNCE 식 자체는 discrete action N개짜리 behavior cloning loss `-log π(a*|s)`와 같은 모양이야. action 자리에 후보 샘플이, 정답 action 자리에 positive가, logit 자리에 similarity/τ가 들어가.
- InfoNCE를 처음 낸 CPC 논문엔 RL 실험(DeepMind Lab auxiliary loss)도 있었어. 나중에 CURL(L05)까지 이어지는 loss가 바로 이거야.

(전부 슬라이드 밖 연결이라 시험 범위 아님)

시험 함정

- NCE = **binary** (sigmoid), InfoNCE = **categorical** (softmax). 바꿔치기 단골.
- N-pair = 1 positive vs N−1 negatives (1 vs 1 아님). 1 vs 1은 triplet이고, p.17은 pair 기반인 Lifted도 결국 "1 positive ↔ 1 negative"라고 정리해.
- Contrastive loss = 절대 목표, Triplet = 상대 순서. margin도 contrastive에선 "다른 class 사이 최소 거리", triplet에선 "두 거리 사이 최소 격차(offset)".
- MCL ≠ hard negative mining. X-CLR에서 τ_s(target graph용)가 크면 graph가 더 soft. τ는 image 쪽 보통 contrastive temperature. DINO의 τ_s는 student 쪽이라 아래첨자 s의 뜻이 반대야.
- temperature τ가 작을수록 softmax가 뾰족해져서, **negative들 사이에서** hard negative가 gradient 몫을 더 많이 가져가 ("작은 τ = 고르게 퍼짐"은 반대). loss 값의 방향은 상황에 따라 달라 (positive가 이미 1등이면 τ가 작을수록 loss가 작아져). p.31엔 τ 이름만 있고, 이 방향은 p.16 LSE 논리를 τ에 적용한 거야.
- GenView: **small** foreground → **less** noise, large foreground → more noise. 좋은 pair = similar foreground + different background.
- X-CLR의 text encoder는 frozen (`.detach()`). 학습되는 건 image encoder.

다음으로

loss는 정했어. 그럼 실제로 잘 돌리려면 뭐가 필요하고, negative가 꼭 있어야 할까? → L04

셀프 체크

MCQ Which loss solves a binary data-vs-noise classification? (A) InfoNCE (B) NCE (C) N-pair (D) Triplet

(B) NCE. logistic regression, sigmoid + binary CE (p.21–22). 샘플을 하나씩 보고 "data냐 noise냐" yes/no로 판정해. (A) InfoNCE는 이름이 비슷한 함정이야. N개 후보 중 하나를 고르는 categorical CE라서 binary가 아니야.

T/F N-pair loss compares one positive with one negative at a time.

False. 1 positive vs N−1 negatives (p.17). 한 번에 negative 하나와 비교하는 건 triplet(그리고 pair 기반인 Lifted)이고, N-pair는 그걸 negative 여러 개로 늘려서 N-way softmax 분류로 만든 거야.

빈칸 In X-CLR, the target similarity graph is built from a frozen _____ encoder.

text (caption similarity, p.65–67). caption끼리의 text similarity를 τ_s softmax해서 soft target을 만들어. text encoder는 frozen이라 코드에서 `.detach()`로 gradient를 끊어.

L04 Contrastive Learning II 72p

질문 **잘 돌리는 재료는 뭐고, negative 없이도 될까?**

L03 복습부터. **contrastive learning**은 "비슷한 건 가깝게, 다른 건 멀게" 놓이도록 representation을 배우는 방법이야. label이 있으면 같은 class끼리 positive로 묶을 수도 있지만(L03 p.32), L04가 다루는 건 label 없는 self-supervised 쪽이야. 사진 한 장을 두 번 다르게 변형하면 둘은 "같은 사진에서 나온 짝"이니까, embedding(encoder가 사진을 바꿔 놓은 숫자 벡터) 공간에서 가깝게 당겨. 다른 사진은 멀리 밀어내. 기준 샘플이 anchor, 당길 짝이 **positive**, 밀어낼 샘플이 **negative**야.

L04는 두 질문을 따라가. (1) 이걸 잘 돌리려면 뭐가 필요하지? (2) negative 없이도 학습이 될까? 끝에서는 positive를 생성 모델로 만드는 SynCLR, DINO를 범용 backbone으로 키운 DINOv2/v3까지 가.

재료 3개

p.4–6

한 줄 positive는 잘 만들고, negative는 많이, 그리고 좋은 걸로 줘야 해.

1. **heavy augmentation** (positive 담당, p.4): augmentation(같은 사진을 자르거나 색을 바꿔서 만든 변형본)을 세게 걸어 positive를 만들어. 단, 의미(semantic)는 바뀌면 안 돼.
2. **large batch** (negative의 양·다양성, p.5): batch 안의 다른 샘플이 전부 negative가 돼 (in-batch negative). 클수록 negative가 많고 다양해져. SimCLR·CLIP처럼 InfoNCE(negative들 사이에서 positive를 골라내는 softmax loss)를 쓰면 필수야.
3. **hard negative** (negative의 질, p.6): hard negative는 의미는 다른데 embedding이 anchor와 가까운 샘플이야. 이미 멀리 있는 easy negative보다 배울 게 많아.

문제 hard negative는 label이 있으면 쉽게 찾는데, self-supervised에선 label이 없어서 어려워. 그래서 큰 batch나 memory bank(예전 embedding을 저장해 두고 negative로 다시 쓰는 창고)로 후보를 늘려서 hard negative가 섞일 확률을 올려. 대가는 메모리야 (trade-off).

Debiased Contrastive

Chuang 2020 · p.7–11

한 줄 "negative라고 뽑았는데 사실 같은 종류"인 샘플 때문에 생기는 손해를 label 없이 보정하는 방법.

문제 contrastive learning은 label이 없으니까 negative(멀어져야 할 샘플)를 batch에서 그냥 아무거나 뽑아. 그러면 anchor가 강아지 사진일 때 negative 중에도 강아지 사진이 섞일 수 있어. 이게 **false negative**(sampling bias)야. 같은 class를 억지로 밀어내니까 성능이 크게 떨어져 (p.7–8).

핵심 어느 샘플이 false negative인지는 label 없이 몰라. 대신 "아무거나 뽑으면 η^+ 비율로 같은 class가 섞인다"는 사실을 이용해서, 섞인 분포에서 positive 몫을 빼 진짜 negative 분포를 추정해 (p.9): $p^- = (p - \eta^+ p^+) / \eta^-$.

- p: 그냥 random으로 뽑을 때의 분포 (진짜 negative + false negative 섞임)
- p^+: 같은 class(positive) 분포, η^+: 같은 class일 확률 (anchor class 확률이 uniform이라고 가정)
- $\eta^- = 1 - \eta^+$로 나눠서 다시 확률 분포로 맞춰

왜 돼? random 분포는 "같은 class 몫 + 다른 class 몫"의 섞임이야: $p = \eta^+ p^+ + \eta^- p^-$. 이걸 p^-에 대해 풀면 끝. 슬라이드 말로는 True Negative = (Random Sampling – Positive

Contribution) / η^-.

어떻게 p^-에서 직접 뽑을 순 없지만 loss엔 기댓값만 있으면 돼. random 샘플 N개($u_i \sim p$)와 positive 샘플 M개($v_i \sim p^+$)로 negative 항을 추정해서, InfoNCE 분모의 negative 합 자리에 $N \cdot g$를 끼워 (p.10):

$$g = \max\{ (1/\eta^-)\cdot[R - P], e^{-1/\tau} \}$$

- $R = (1/N) \Sigma_i e^{f(x)\cdot f(u_i)}$: random 몫, $P = (\eta^+/M) \Sigma_i e^{f(x)\cdot f(v_i)}$: positive 몫. p.9 식을 그대로 옮긴 거야 (random 몫 − positive 몫, η^-로 나눔).
- 샘플 수가 유한하니까 뺄셈 결과가 0 이하로 나올 수 있어. 그러면 log가 깨지니까 $e^{-1/\tau}$로 아래를 막아 (max). 이 값은 유사도가 가장 작을 때의 값, 즉 이론적 최솟값이야 (embedding 길이를 $1/\sqrt{\tau}$로 맞춰 두면 $f(x)\cdot f(x') = \cos/\tau \geq -1/\tau$라서 지수 안이 $-1/\tau$보다 작아질 수 없어. 이 유도는 notes 보충이고, 슬라이드엔 max 식만 있어).

예시로 보면

class가 10개로 고르게 섞여 있으면 $\eta^+ = 0.1$이야 (숫자는 설명용). negative를 아무거나 10개 뽑으면 평균 1개는 anchor와 같은 class야. Debiased loss는 그 "1개 몫"을 positive 샘플로 어림해서 빼 줘.

Augmentation 계열

positive를 augmentation으로 만드니까, 어떤 변형을 쓰느냐가 곧 모델이 무엇에 흔들리지 않을지(invariance)를 정해. 좋은 augmentation은 **겉모습(visual appearance)은 크게 바꾸고, 의미(semantic meaning)는 지켜** (p.13). 너무 약하면 색 같은 꼼수로 두 view를 맞춰 버리고, 의미가 바뀌면 "둘은 positive"라는 정답 자체가 틀려.

기본 연산은 Random Resized Crop(일부를 잘라 원래 크기로), Color Jitter(밝기·대비·채도·색조 랜덤), Gaussian Blur(흐리게), Horizontal Flip(좌우 반전), Grayscale(흑백), Resize야 (p.14). 다음 표의 앞 세 줄은 좋은 조합을 **자동으로 찾기**, UDA는 augmentation으로 **unlabeled 데이터 쓰기**, 끝 세 줄은 **두 샘플 섞기**야.

방법	한 줄	짝꿍과 차이
AutoAugment CVPR 2019 · p.15	RL controller로 validation accuracy가 최대인 augmentation policy를 탐색. policy = "어떤 연산을, 어떤 확률로, 얼마나 세게". controller가 policy를 뽑음 → child network를 학습 → validation accuracy를 reward로 controller 업데이트 → 반복	fixed policy, 탐색 비쌈 (child를 매번 새로 학습)
RandAugment 2019 · p.16	랜덤 선택 + N(이미지당 연산 개수), M(모든 연산 공통 강도) 두 개만	탐색 공간이 아주 작아짐 (AutoAugment는 연산·확률·강도를 다 탐색)
PBA ICML 2019 · p.17	population-based training(여러 모델을 같이 학습하며 잘 되는 설정을 복사·변형하는 진화식 탐색)으로 학습 중에 policy를 진화	fixed policy 대신 schedule (학습 시점마다 달라지는 policy)
UDA NeurIPS 2020 · p.18	unlabeled에 augmentation, 원본과 예측이 같도록 KL(두 확률 분포의 차이)	consistency regularization: 입력을 조금 바꿔도 예측은 같아야 해 (L06에서 다시)

방법	한 줄	짝꿍과 차이
Mixup ICLR 2018 · p.19	두 이미지 전체를 픽셀 단위 가중합: $\tilde{x} = \lambda x_i + (1-\lambda)x_j$, label도 같은 비율 $\tilde{y} = \lambda y_i + (1-\lambda)y_j$ (λ: 0~1 섞는 비율). 반투명하게 겹친 사진이 돼	global-level
CutMix ICCV 2019 · p.20	패치를 잘라 붙이고 label은 면적비 (개 60% + 고양이 패치 40%면 label도 0.6 / 0.4)	region-level, 진짜 픽셀로 채움
MoCHi NeurIPS 2020 · p.21	hardest negative끼리, query(anchor 역할 샘플)와 hard negative를 섞어 더 어려운 negative 합성	pixel이 아니라 **feature 공간**에서 섞음 (embedding 벡터를 섞음)

대표 프레임워크 4개

네 방법은 뼈대가 같아 (슬라이드 묶음 이름은 Parallel Augmentation, p.22). 사진 한 장을 두 번 augmentation해서 view 두 개를 만들고, 두 view의 출력이 서로 맞도록 학습해. 갈리는 지점은 **collapse를 어떻게 막느냐**야.

왜?

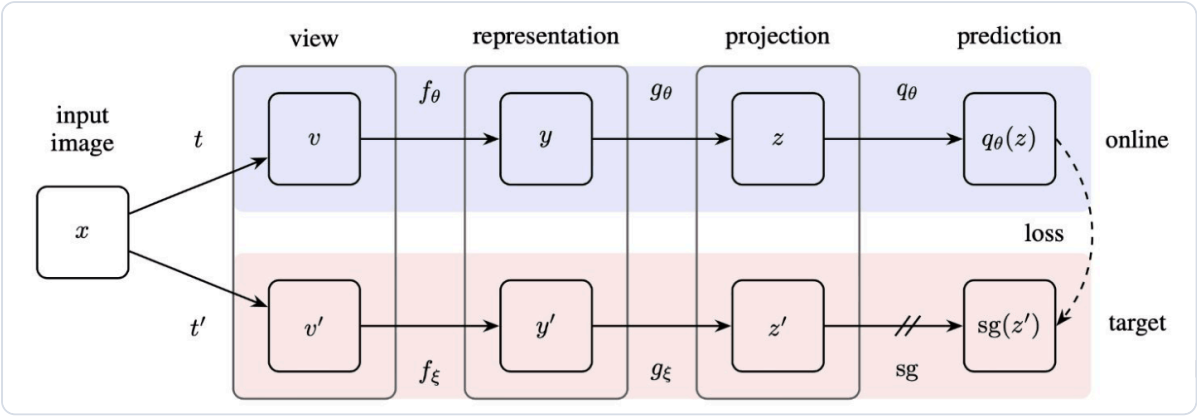
"두 view의 출력을 같게 해"만 시키면 제일 쉬운 답이 있어. 어떤 사진에도 똑같은 벡터를 내는 거야. loss는 최소인데 representation엔 정보가 하나도 없어. 이게 collapse(모든 입력이 같은 출력으로 무너지는 것)야. negative가 있으면 "다른 사진은 멀어져야 해"가 막아 주고, 없으면 다른 장치가 필요해.

- **SimCLR**: negative가 밀어내니까 한 점으로 못 모여.
- **BYOL**: 비대칭 구조. 한쪽(online)에만 predictor(출력 뒤에 하나 더 붙인 작은 MLP)가 있고, 다른 쪽(target)은 gradient를 끊고 EMA로 천천히 따라와.
- **Barlow Twins**: 모든 차원이 같은 값을 내면 서로 다른 차원끼리의 상관(비대각)이 커지는데, 그걸 0으로 밀어.
- **DINO**: teacher 출력에 centering(평균 빼기) + sharpening(낮은 temperature로 분포를 뾰족하게).

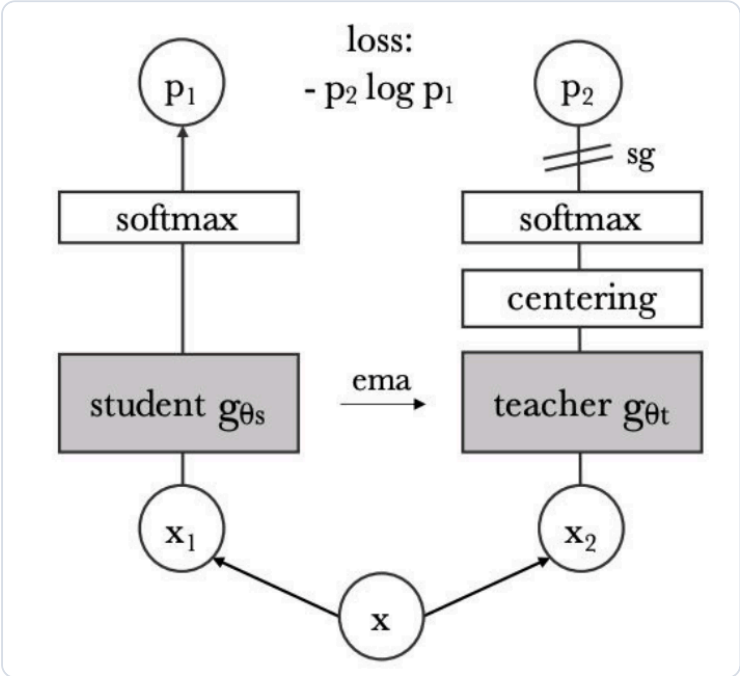
BYOL의 online, DINO의 student는 **gradient로 학습**하는 쪽이야. BYOL의 target, DINO의 teacher는 gradient를 안 받고 (stop-gradient) 학습하는 쪽 가중치의 EMA(지수 이동평균)로만 바뀌어. 멈춰 있는 게 아니라 천천히 따라오는 거야.

	SIMCLR	BYOL	BARLOW TWINS	DINO
출처	ICML 2020 · p.23–29	NeurIPS 2020 · p.30–34	ICML 2021 · p.35–38	ICCV 2021 · p.39–44
negative	있음. batch 안 2N−2개	없음	없음	없음
두 branch	같은 encoder	online(+predictor) / EMA target gradient는 online만	같은 encoder 양쪽 다 backprop	student / EMA teacher gradient는 student만
loss	InfoNCE (NT-Xent)	normalized MSE	cross-correlation → identity	softmax 분포 간 CE
collapse 방지	negative가 밀어냄	predictor + EMA + stop-gradient	off-diagonal → 0	teacher centering + sharpening

	SIMCLR	BYOL	BARLOW TWINS	DINO
한 줄	두 view 당기고 나머지 밀기, 큰 batch 필수	negative 없이 EMA target을 예측	feature 차원끼리 상관 = I	학생이 EMA 선생의 분포를 따라감



BYOL. 위 줄이 online, 아래 줄이 target이고, 왼쪽부터 view → representation(encoder) → projection(projector = projection head, SimCLR의 g와 같은 역할) → prediction 순서야. online에만 **predictor q_θ**가 있어. target 끝의 `//` 표시가 `sg`(stop-gradient)라서 target 쪽으로는 gradient가 안 흘러. 점선 loss는 $q_\theta(z)$와 $\mathrm{sg}(z')$를 비교해. target 파라미터 ξ는 학습이 아니라 θ의 **EMA**로만 갱신되고, negative는 없어. EMA 식은 `ξ ← τξ + (1-τ)θ`인데, 여기서 τ는 temperature가 아니라 **EMA 계수**야. `L04 p.30`

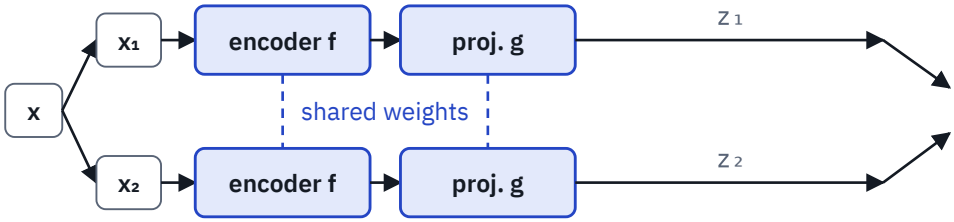


DINO. 같은 x에서 만든 두 view가 왼쪽 student, 오른쪽 teacher로 들어가. **centering은 teacher 쪽에만** 있고 (softmax는 양쪽 다), `sg`(stop-gradient, `//` 표시)도 teacher 쪽에만 붙어. 가운데 **ema** 화살표는 student → teacher 방향이야. teacher가 student 가중치를 이동평균으로 따라간다는 뜻. loss는 teacher 분포 p_2를 정답으로 둔 student 분포 p_1의 cross-entropy (`-p2 log p1`). 그림은 한 방향만 그렸고, 코드는 두 view를 엇갈려 양방향으로 계산해. `L04 p.40 (왼쪽 도식)`

구조 비교 **SimCLR · BYOL · Barlow Twins · DINO를 같은 그림 문법으로: 누가 gradient를 받고 누가 EMA로 따라가나**

SimCLR · InfoNCE

shared encoder · negative 있음

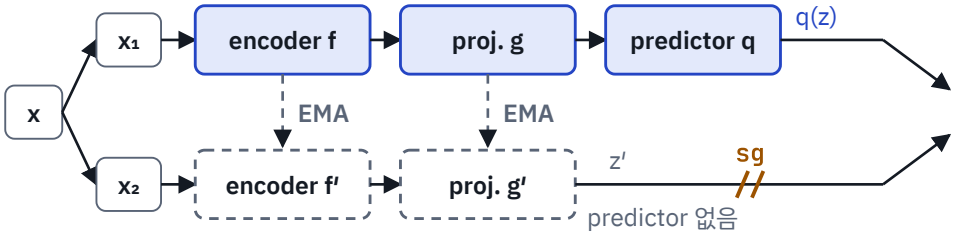


negative = 같은 batch의 다른 샘플

학습 후 g 버림, f 사용 (p.25)

BYOL · MSE

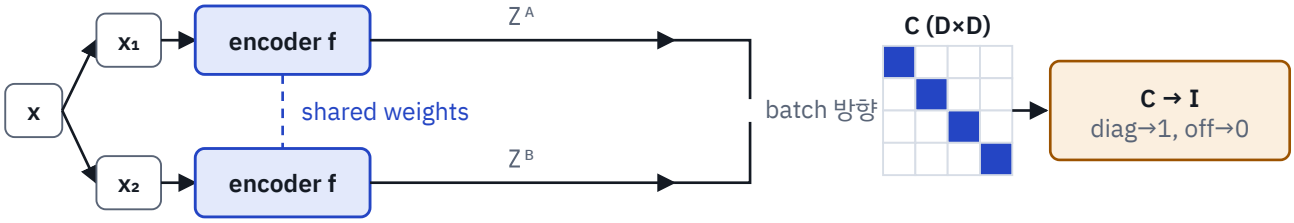
online ↔ EMA target · negative 없음



gradient는 online 쪽에만 흐름

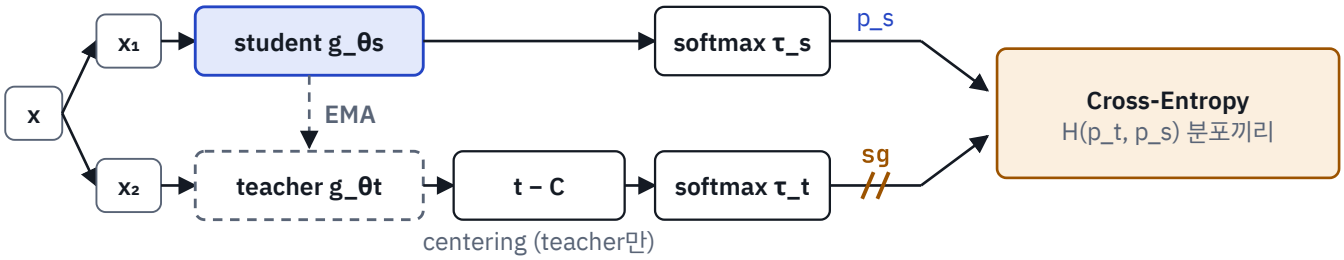
Barlow · C→I

shared encoder · negative 없음



DINO · CE

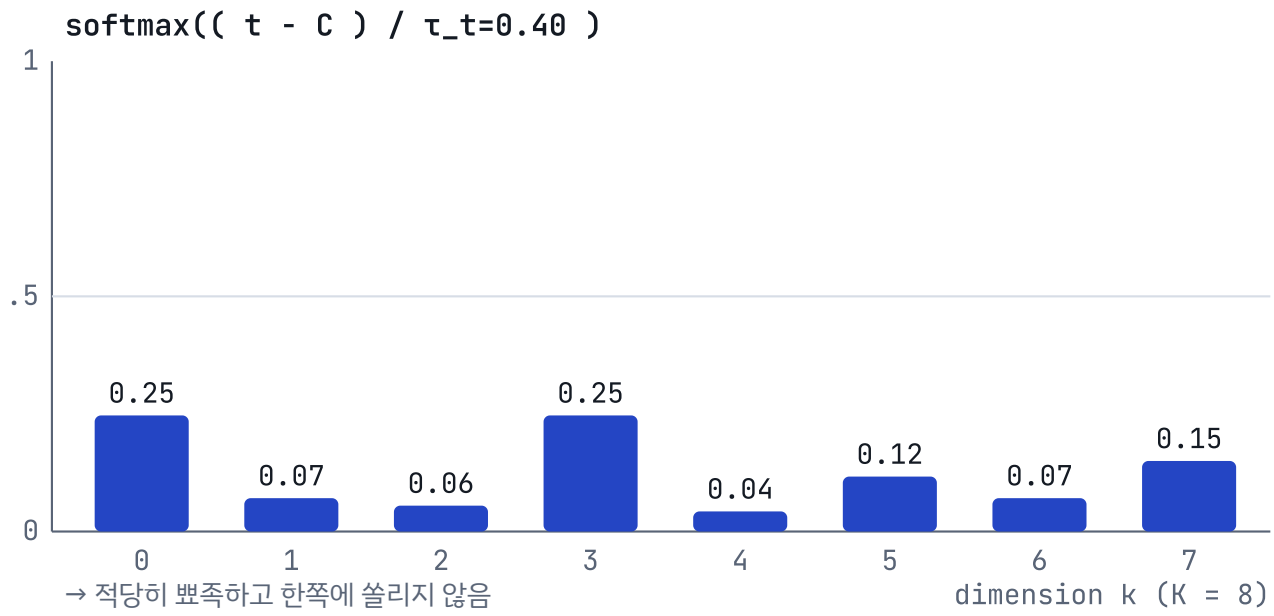
student ↔ EMA teacher · negative 없음



gradient로 학습 · EMA로만 갱신 · // sg stop-gradient · loss

네 줄을 위에서 아래로 비교해 봐. 모두 같은 사진 x에서 view x_1, x_2를 만들어 두 branch에 넣어. 상자 모양이 핵심이야: 실선으로 채워진 상자는 **gradient로 학습**, 점선 상자는 **EMA로만 갱신**(BYOL의 target, DINO의 teacher)이고, `// sg`는 stop-gradient야. SimCLR·Barlow는 두 branch가 같은 encoder(shared weights)라 점선 상자가 없어. predictor q는 BYOL 줄에만, centering(t − C)은 DINO teacher 쪽에만 있어. loss 상자 이름(InfoNCE / normalized MSE / C → I / Cross-Entropy)이 네 방법의 차이야. (L04 p.23–44)

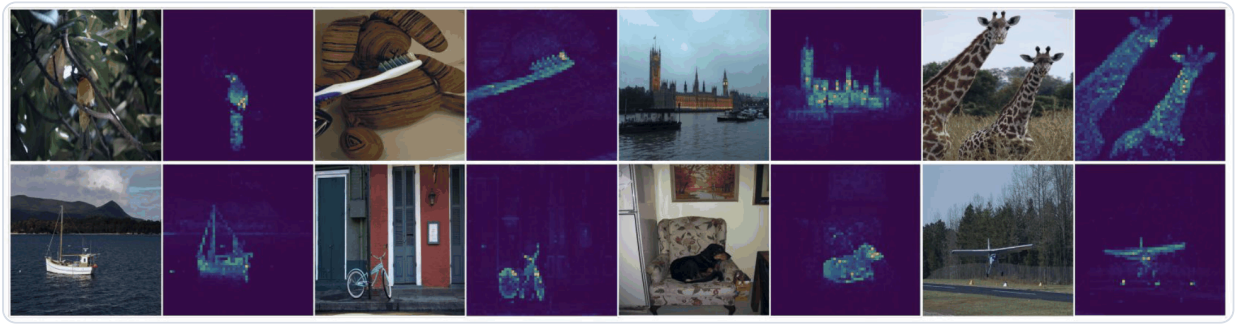
인터랙티브 **DINO teacher: centering은 한 차원 독점을, sharpening은 uniform을 막아** illustrative



teacher temp τ_t 0.40 centering on sharpening on

막대 8개는 teacher가 한 사진에 대해 내는 확률 분포 `softmax((t - C) / τ_t)`야 (L04 p.40, K = 8). 막대 위 숫자가 확률이고 합은 1이야. logit `t`와 center `C`는 개념용 값이야. 버튼으로 하나씩 꺼 봐. **centering off**면 늘 큰 한 차원(k=3)이 독점해서 one-hot 쪽으로 collapse해 (모든 사진이 같은 답). **sharpening off**(τ_t를 크게)면 분포가 평평해져서 uniform collapse야 (아무 정보 없음). 슬라이더로 τ_t를 키워도 평평해지는 걸 볼 수 있어. 둘이 서로 반대 방향을 막으니까 같이 써야 해. 이 collapse 해석(centering은 한 차원 독점, sharpening은 uniform을 막음)은 슬라이드 밖 논문 보충이고, 슬라이드엔 `# center + sharpen` 주석뿐이야. logit·τ 값은 개념용. (개념 그림)

▼ 슬라이드 그림 더 보기 (1장)



DINO의 emergent property. 사진 오른쪽 칸은 [CLS] token(이미지 전체를 요약하는 token)이 각 patch에 주는 attention을 밝기로 칠한 거야. label 없이 학습했는데 배경이 아니라 물체 모양(새, 칫솔, 기린, 배, 자전거 …)을 따라 켜져. 그래서 unsupervised object segmentation처럼 보여. L04 p.42

SimCLR 디테일

p.23–29 · code p.25

한 줄 같은 사진의 두 view는 당기고, 같은 batch의 나머지 view는 전부 밀어내는 가장 단순한 framework.

어떻게 Algorithm 1 (p.25) 순서야.

1. 사진 N장마다 augmentation t, t′을 따로 걸어 → view 2N개.
2. encoder f로 representation $h = f(\tilde{x})$.
3. projection head g(encoder 위에 얹는 작은 MLP)로 $z = g(h)$. loss는 z에 걸어.

4. 모든 view 쌍의 cosine similarity $s_{i,j}$를 구해. 같은 사진 짝이 positive, 나머지 2N−2개가 negative.
5. InfoNCE로 학습. SimCLR는 이 loss를 NT-Xent(Normalized Temperature-scaled cross-entropy: cosine으로 정규화 + τ로 나눔 + cross-entropy)라고 불러. 두 view가 번갈아 anchor가 되니까 2N개 loss를 평균해:
   $L = (1/2N)\ \Sigma_k\ [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$.

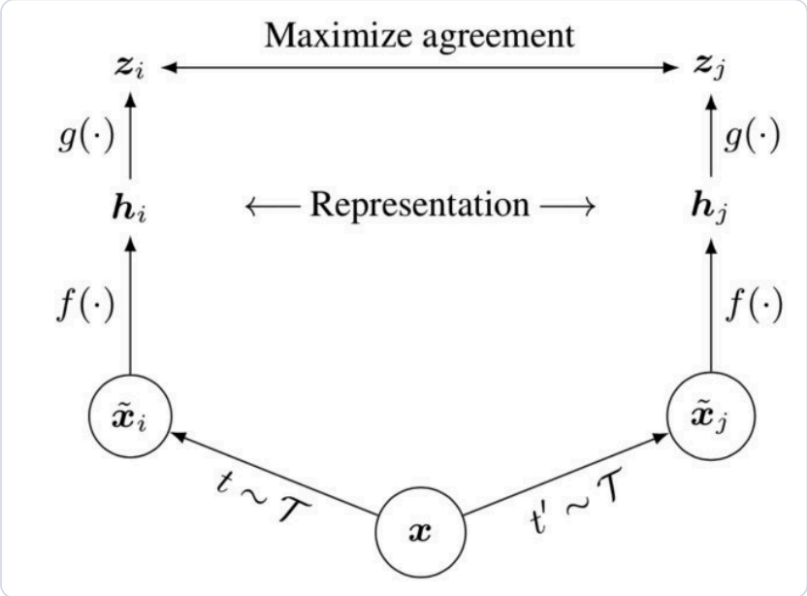
$\ell(i,j) = -\log [\ e^{s_{i,j}/\tau} / \Sigma_{k \neq i}\ e^{s_{i,k}/\tau}\ ]$

- 분자: anchor i와 positive j의 유사도. 클수록 loss가 줄어.
- 분모: 자기 자신(k = i)만 빼고 2N−1개 전부 (positive 1 + negative 2N−2). negative와 가까울수록 loss가 커져.
- τ: temperature. 작을수록 softmax가 뾰족해져서 가까운(hard) negative에 더 크게 반응해.
- 즉 "2N−1개 후보 중 진짜 짝 맞히기" 분류의 cross-entropy야.

> **예시로 보면**
> 사진 4장(N = 4) → view 8개. view 하나를 anchor로 잡으면 positive 1개, negative 6개(= 2N−2), softmax 후보는 7개(= 2N−1).

왜 g를 버려? contrastive loss는 z가 augmentation에 흔들리지 않게(invariant) 만들어. 그러면 색·방향처럼 downstream task엔 쓸모 있는 정보가 z에서 지워져. g가 그 손실을 떠맡아서 h에는 정보가 더 남아 (p.28, possible explanation). 실험도 non-linear head > linear > 없음.

왜 큰 batch? memory bank 없이 batch 안 negative만 쓰니까 batch가 커야 해. 그만큼 backprop 메모리가 커서 TPU 분산 학습이 필요했어 (p.29).



SimCLR 구조. 아래에서 위로 읽어. 같은 x에 augmentation `t, t'`을 따로 걸어 두 view → encoder `f`로 representation `h` → projection head `g`로 `z` → 맨 위에서 두 z의 agreement(유사도)를 contrastive loss로 최대화. loss는 `h`가 아니라 `z` **공간**에서 걸리고, 학습이 끝나면 g는 버리고 h를 써. `L04 p.23-24`

BYOL 디테일

p.30–34

한 줄 negative 없이, "천천히 따라오는 나 자신(target)"이 다른 view에 낸 출력을 예측해 (Bootstrap Your Own Latent).

어떻게 같은 구조의 network 두 개가 주고받아 (p.30–31).

- **online (θ)**: encoder f_θ → projector g_θ(= projection head. SimCLR의 g와 같은 역할의 작은 MLP) → predictor q_θ(online에만 있는 작은 MLP). **gradient descent로 학습**하고 target 출력을 예측해.
- **target (ξ)**: f_ξ → g_ξ, predictor 없음. 출력에 sg(stop-gradient)가 붙어 gradient가 안 흘러가고(backprop 없음), 대신 **EMA로만 갱신**돼: $\xi \leftarrow \tau\xi + (1-\tau)\theta$.
- 여기 τ는 temperature가 아니라 **EMA 계수**(momentum)야. 1에 가까울수록 target이 천천히 움직여 (예: $\tau = 0.99$면 한 step에 1%만 online 쪽으로, 숫자는 설명용).

loss view v는 online, v′는 target에 넣어. online 예측 $q_\theta(z_\theta)$와 target 출력 z'를 길이 1로 정규화한 뒤 MSE를 줄여 (p.32). 길이 1 벡터끼리 제곱 거리를 풀면 $\|\bar{q} - \bar{z}'\|^2 = 2 - 2\cdot\cos$(슬라이드엔 없는 식 정리)라 결국 방향 맞추기야. v, v′를 바꾼 loss도 더해 대칭으로 만들고 **θ만** 최적화해.

왜 돼? 슬라이드 근거는 "target이 안정적인 학습 목표를 준다"(p.31)이고, 세 장치의 역할은 직관으로 보면 이래.

1. **stop-gradient**: target 출력을 이번 step의 고정된 정답(상수)으로 둬. loss가 target 쪽까지 끌어당겨서 두 쪽이 한꺼번에 "다 같은 값"으로 가는 길을 줄여 줘.
2. **EMA target**: 천천히 변하니까 정답이 매 step 출렁이지 않아.
3. **predictor**: online에만 있어서 비대칭. online은 target을 베끼는 게 아니라 예측해야 해.

셋 중 하나만으로 collapse가 막힌다는 증명은 없어. 셋을 같이 썼을 때 실험적으로 collapse가 안 일어났다는 게 논문 쪽 설명이야. 시험용으로는 표의 "predictor + EMA + stop-gradient" 세 이름을 확실히 기억해.

차이 SimCLR은 당기고 밀기(contrast), BYOL은 다른 view를 예측하기(learn by prediction). negative가 없으니 큰 batch에 기댈 필요가 없어 (p.34).

Barlow Twins 디테일

p.35–38

한 줄 "feature 차원 i와 j가 얼마나 같이 움직이나"를 행렬로 만들고, 그걸 identity로 맞춰.

어떻게 사진 B장의 두 view를 같은 encoder(양쪽 다 backprop)에 넣어 B×D 행렬 Z^A, Z^B를 얻어 (D = feature 차원 수). 상관행렬은 **batch 방향**으로 계산한 D×D야. 차원 쌍마다 batch 전체 값끼리 cosine을 구해 (p.36):

$$C_{ij} = z^A_{:,i} \cdot z^B_{:,j} \;/\; (\|z^A_{:,i}\| \; \|z^B_{:,j}\|)$$

$z^A_{:,i}$는 view A의 i번 차원 값을 batch 전체로 모은 길이 B 벡터야. 그리고 C를 identity로 밀어 (p.35):

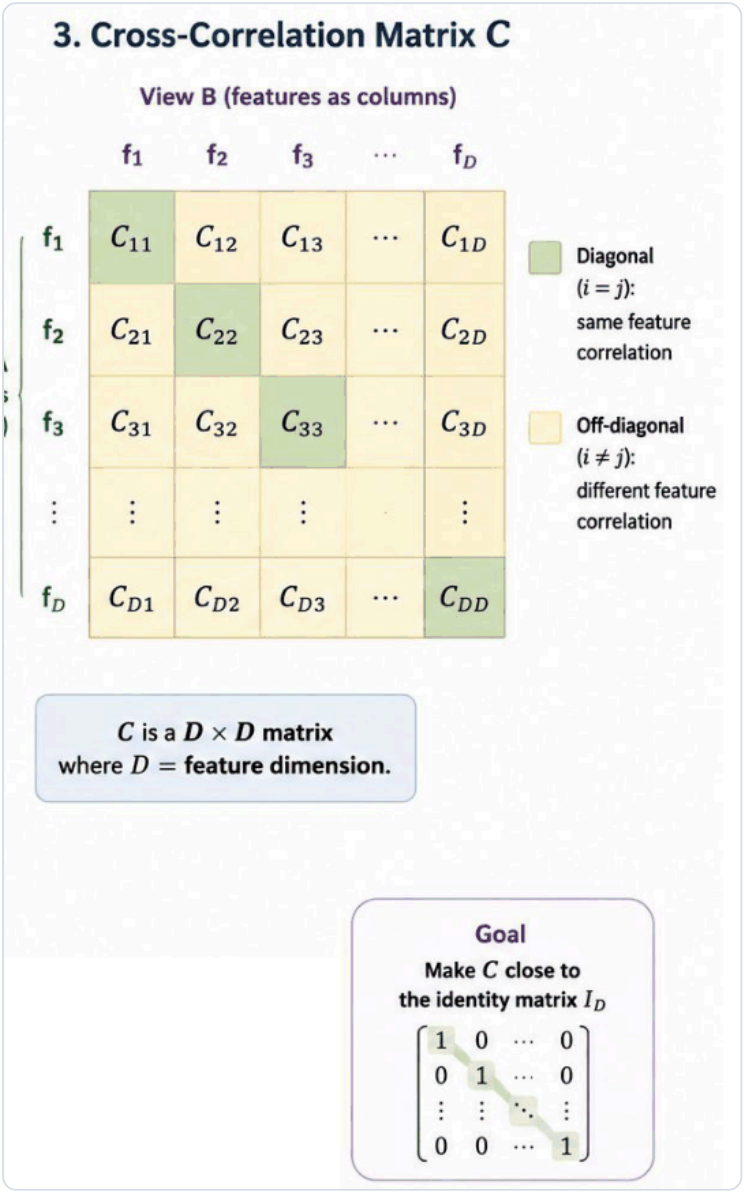
$$L_{BT} = \Sigma_i (1 - C_{ii})^2 + \lambda \, \Sigma_i \Sigma_{j \neq i} \, C_{ij}^2$$

- 대각선 → 1 (**invariance term**): 같은 차원은 augmentation이 달라도 같이 움직여야 해.
- 비대각 → 0 (**redundancy reduction term**): 다른 차원끼리는 상관이 없어야 해. 차원마다 다른 정보를 담고 중복하지 않아.
- λ: 두 항의 균형 가중치.

예시로 보면

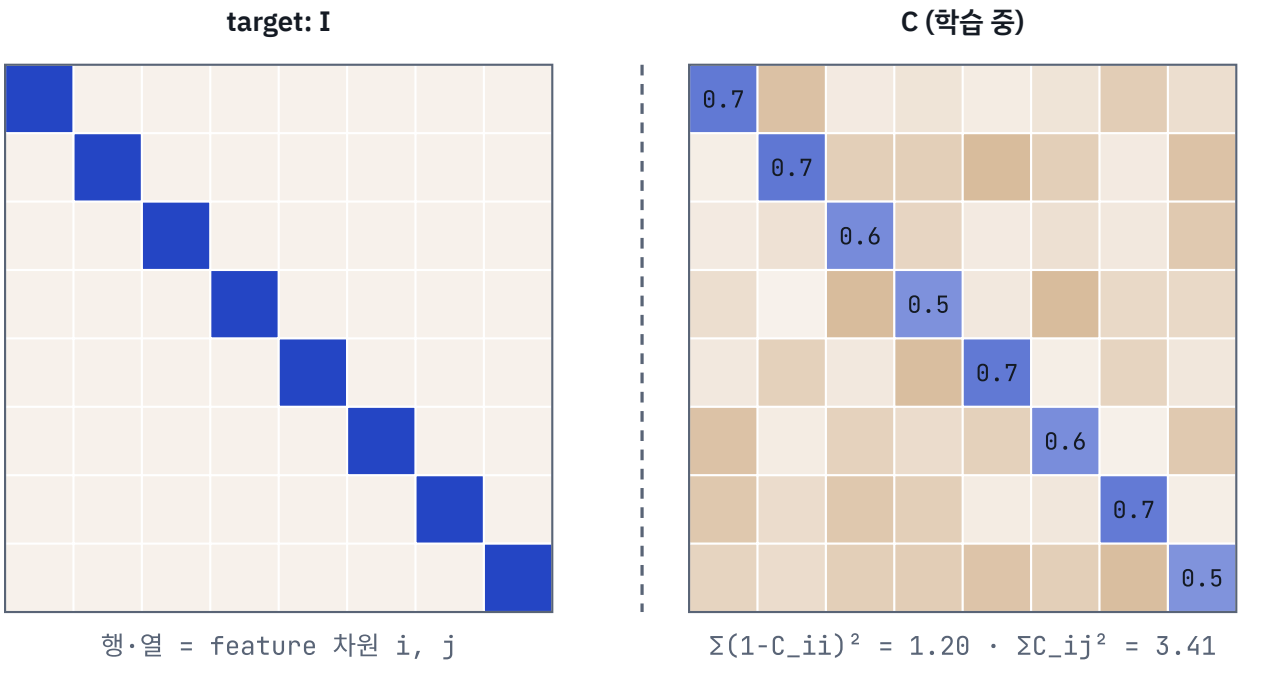
D = 3이면 C는 3×3. C_{11}은 "view A의 1번 차원"과 "view B의 1번 차원"이 batch에서 같이 오르내리는 정도야. encoder가 무너져 세 차원이 사실상 같은 값만 내면(1번이 오를 때 2·3번도 똑같이 오름), 대각선 C_{11}, C_{22}…는 1이라 첫째 항은 만점이야. 대신 C_{12}, C_{13}…도 1 가까이 커져서 둘째 항이 penalty를 줘. 그래서 negative 없이도 collapse가 막혀.

차이 SimCLR은 **샘플끼리**(B×B), Barlow Twins는 **feature 차원끼리**(D×D) 비교해. negative·predictor·EMA가 다 없고 대칭이야. batch size에 robust (p.37).



C 행렬 확대. 행은 view A의 feature 차원, 열은 view B의 feature 차원이야 (샘플×샘플이 아니라 차원×차원, D×D). 초록 대각선(same feature)은 1로, 노란 비대각선(different feature)은 0으로 만드는 게 **Goal: C ≈ I_D**야. `L04 p.36`

인터랙티브 **Barlow Twins: D×D cross-correlation이 identity가 되어 가는 과정** illustrative



training progress 0.25

왼쪽은 목표 I, 오른쪽은 학습 중인 C야. 슬라이더(training progress)를 오른쪽으로 밀면 C가 I에 가까워지는 모습을 볼 수 있어. C는 두 view의 embedding을 **batch 방향**으로 계산한 D×D cross-correlation이야 (sample×sample이 아니라 feature×feature, L04 p.35). 칸 (i, j) = "view A의 i번 차원과 view B의 j번 차원이 batch에서 같이 움직이는 정도". 대각선(진한 파란 칸)을 1로 올리는 게 **invariance term**, 비대각(갈색)을 0으로 누르는 게 **redundancy reduction term**이야. 아래 숫자 $\Sigma(1-C_{ii})^2$와 ΣC_{ij}^2가 두 항의 크기고, 둘 다 0으로 줄어. 진행도가 0일 땐 차원끼리 많이 겹쳐 있어 (정보 중복). 값은 개념용이고 실제 학습 결과가 아니야. (개념 그림, L04 p.35–38)

DINO 디테일

p.39–44 · code p.40

한 줄 student가 EMA teacher의 softmax 분포를 cross-entropy로 따라가는, label 없는 self-distillation (self-**di**stillation with **no** labels).

정의 distillation은 원래 "다 배운 teacher의 출력 분포를 student가 흉내 내며 배우기"야. DINO엔 미리 학습된 teacher가 없어. teacher를 **student 자신의 EMA**로 만들어 (p.39: "Train the student to match the teacher's output distribution").

- **student** g_{θ_s}: gradient를 받아 SGD로 학습 (`update(gs)`). 여기 g는 SimCLR의 projection head g가 아니라 student / teacher network 전체야 (슬라이드 표기).
- **teacher** g_{θ_t}: 구조는 같고, gradient는 끊고(loss 함수 `H(t, s)`의 첫 줄 `t = t.detach()`: teacher 출력 t를 상수로 취급, 그림의 sg) student의 EMA로만 갱신 (`gt.params = l*gt.params + (1-l)*gs.params`, `l`은 숫자 1이 아니라 소문자 L).

어떻게 pseudo code (p.40) 순서야.

1. 시작할 때 teacher = student 복사 (`gt.params = gs.params`). 매 batch마다 view x1, x2를 student와 teacher **둘 다**에 넣어. 출력은 K차원 logit(softmax 전 점수).
2. student: `softmax(s / τs)`. teacher: `softmax((t - C) / τt)`. teacher 쪽에서만 center C를 빼고(centering) 작은 τ_t로 나눠 뾰족하게 해(sharpening). **centering · sharpening 둘 다 teacher 쪽에만**이야 (슬라이드엔 τ 값이 없고 근거는 `# center +`

`sharpen` 주석). 기호도 조심해: s, t는 student 출력 · teacher 출력이고, $\tau_s \cdot \tau_t$는 student · teacher temperature야. L03 X-CLR의 τ_c(target 쪽)와는 반대 뜻이야.

3. loss는 view를 엇갈려: `H(t1, s2)/2 + H(t2, s1)/2`. $H(P_t, P_s) = -\Sigma P_t \log P_s$, teacher 분포가 정답인 cross-entropy야.
4. center도 EMA: `C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)`. C는 parameter가 아니라 **teacher 출력** 평균이야. 그래서 EMA가 두 개(teacher 가중치, center).

> **예시로 보면**
> negative 없이 무너지는 방향은 둘이야 (K = 4, 숫자는 설명용).
>
> - **한 차원 독점**: teacher가 어떤 사진에도 (3, 1, 1, 1) 비슷한 logit을 내면 softmax는 늘 (0.71, 0.10, 0.10, 0.10). 모든 사진이 1번으로 몰려. centering이 늘 높은 1번 차원의 평균을 빼서 이 공짜 우위를 없애.
> - **uniform**: 분포가 평평하면 정보가 없어. logit (1, 0.5, 0, 0)을 $\tau = 1$로 softmax하면 (0.43, 0.26, 0.16, 0.16), $\tau = 0.25$면 (0.85, 0.12, 0.02, 0.02). sharpening이 평평해지는 걸 막아.
>
> centering만 하면 평평해지고, sharpening만 하면 한 차원이 독점해. 그래서 같이 써. 이 역할 분담은 논문 쪽 설명이고, 슬라이드엔 주석 `# center + sharpen` 뿐이야.

차이 p.41: BYOL은 feature matching(MSE) + predictor, DINO는 distribution matching(CE) + softmax probability. 둘 다 EMA teacher, no labels. DINO엔 predictor가 없어.

결과 label 없이 학습한 ViT의 [CLS] token(이미지 전체를 요약하는 token) attention이 물체 모양을 따라가 (p.42). 시키지 않았는데 생긴 성질이라 emergent property야. 많은 현대 SSL(self-supervised learning, iBOT · DINOv2)의 기반 (p.44).

SynCLR

Tian, CVPR 2024 · p.45–54

문제 augmentation은 crop·색 바꾸기 같은 픽셀 수준 변형이라 자세·배경 같은 큰 변화를 못 줘. "positive를 augmentation 말고 생성하면?" (p.47)

핵심 Llama-2(대형 언어 모델)가 caption을 만들고 text-to-image 모델이 caption당 4장 생성. 같은 caption에서 나온 이미지들 = positive set (multi-positive loss). 진짜 이미지 없이 학습.

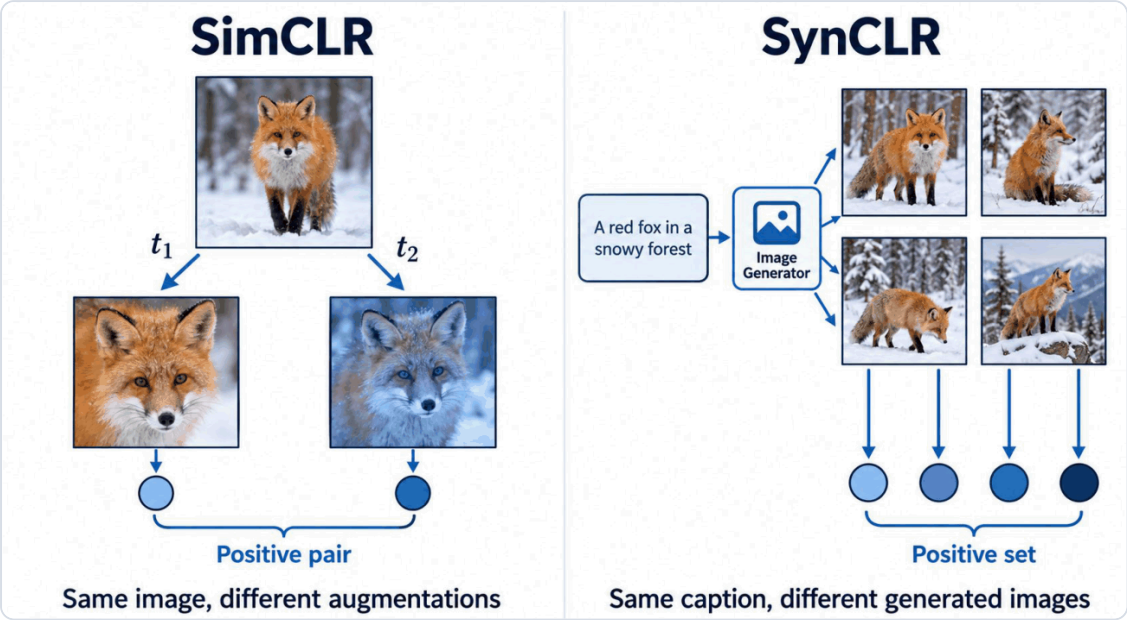
1. caption (p.49): 예시 3개를 context로 주면 새 caption이 나오고, 그걸 다시 context에 넣어 반복 → 다양성↑.
2. 이미지 (p.50): caption 하나에 서로 다른 random noise로 시작하고, CFG scale(caption을 따르는 세기)을 낮춰 다양하게. 150M caption × 4 = 600M 장.
3. 학습 (p.51): 같은 caption은 positive, 다른 caption은 negative.

$q_i = e^{a \cdot b_i/\tau} / \Sigma_j e^{a \cdot b_j/\tau}$, $p_i = 1_{match(a,b_i)} / \Sigma_j 1_{match(a,b_j)}$,

$L(a) = H(p, q) = -\Sigma_i p_i \log q_i$

- a: anchor, b_i: 후보 K개. q: 모델이 "짝은 b_i"라고 보는 softmax 확률.
- p: 정답 분포. 같은 caption이면 1인 indicator를 합 1로 나눠 (positive 3개면 각 1/3).
- positive가 1개면 p가 one-hot → 그냥 InfoNCE. 즉 InfoNCE의 multi-positive 버전.

차이 SimCLR은 image identity를 보존, SynCLR은 caption 수준 의미를 보존. pair 대신 set이고, positive가 같은 사진에서 나올 필요가 없어 (p.54). caption이 class의 granularity(얼마나 잘게 나눌지)를 정해 (p.48): SimCLR은 너무 잘게(사진마다 class), supervised는 너무 거칠게(골든 리트리버는 행동·배경 무시하고 한 class), SynCLR은 중간(물체·행동·관계 반영).



SimCLR vs SynCLR. SimCLR는 사진 한 장의 augmentation 둘이 positive pair야. SynCLR는 **같은 caption에서 생성한 이미지들**이 positive **set**이야. positive를 정하는 기준이 "같은 사진(image identity)"에서 **"같은 caption(caption-level semantics)"**으로 바뀐 거야. `L04 p.47`

DINOv2

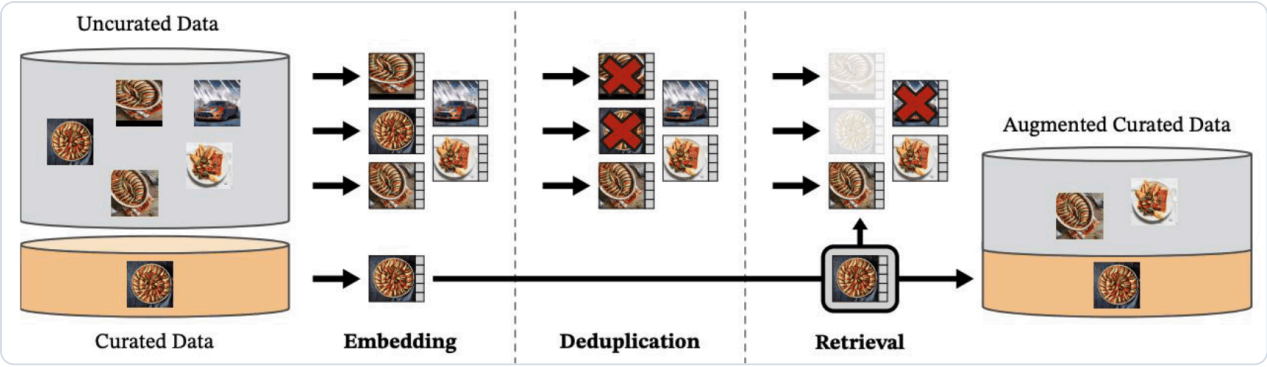
`Oquab, TMLR 2024 · p.55-67`

한 줄 DINO를 "한 번 학습해 두면 여러 task에 그대로 갖다 쓰는" visual foundation model로 키운 것. 목표는 이미지만으로 배운 frozen feature(encoder를 고정한 채 뽑은 feature)가 global·patch 수준 둘 다 좋게 (p.56).

핵심 DINO + iBOT(patch-level masked) + KoLeo(feature 퍼뜨리기) + curated data(LVD-142M) + scaling(ViT-g를 작은 모델로 distill). ViT-g는 약 1B parameter의 아주 큰 ViT야 (p.57).

- **DINO** (p.61): student·teacher의 [CLS] 분포를 CE로 → 이미지 전체 의미(global).
- **iBOT** (p.61): student 입력의 patch 일부를 가리고(mask), 가린 위치에서 teacher patch 분포를 따라가게 → 위치별 정보(local). DINO head와 iBOT head는 따로.
- **KoLeo** (p.62): 서로 다른 이미지의 image-level feature를 밀어내 공간 전체에 퍼지게 해 (crowding 방지). $L_{koleo} = -(1/n)\ \Sigma_i \log d_{n,i}$, $d_{n,i} = \min_{j \neq i} \|x_i - x_j\|$. d는 각 feature와 가장 가까운 이웃 사이 거리야. d가 커질수록(서로 멀수록) loss가 줄어.
- **Sinkhorn-Knopp centering** (p.62–63): DINOv2 공식엔 없지만 같은 slide에 나오는 장치야. teacher 출력을 재정규화해서 prototype(출력 차원이 가리키는 대표 방향) 할당이 몇 개에 몰리지 않게 해. KoLeo는 feature를 퍼뜨리고, SK는 teacher target을 고르게 만든다는 게 차이야.
- **Curated data** (p.58): 웹 이미지 1.2B → 중복 제거 → curated 데이터와 비슷한 것만 retrieval → 142M.
- **Scaling** (p.65): 큰 ViT-g teacher를 학습하고 freeze한 뒤 작은 ViT로 distill.

차이 새 loss 하나가 아니라 통합 시스템. 이미지만으로(label·caption 없이) 학습한 범용 frozen feature. 교훈: SSL objective만으론 부족하고, 데이터 품질·patch-level·안정적 학습 ·distillation이 필요해 (p.67).



DINOv2 데이터 curation. 왼쪽에서 오른쪽으로 읽어. 정리 안 된(uncurated) 웹 이미지를 embedding(벡터)으로 바꾸고 → 거의 같은 이미지를 지우고(**deduplication**, 빨간 X) → 잘 정리된(curated) 예시와 비슷한 것만 검색해 가져와(**retrieval**) LVD-142M을 만들어. 새 loss가 아니라 **데이터 품질이 scaling의 핵심**이라는 그림이야. `L04 p.58`

DINOv3 · Gram anchoring

`p.68-70`

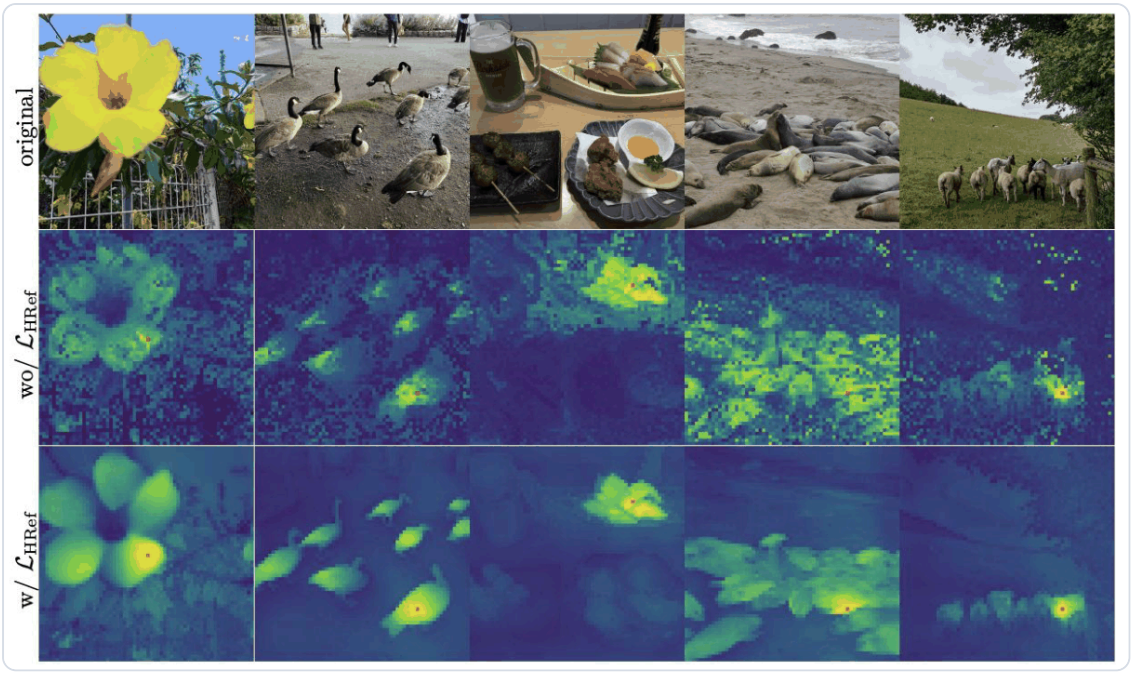
문제 오래 학습하면 global feature는 좋아지는데 dense(patch) feature가 망가져. dense feature는 ViT가 patch마다 내는 벡터로 segmentation 같은 task에 써. 분류는 계속 오르는데 patch끼리 유사도가 noisy해지고 segmentation이 떨어져 (p.68).

핵심 patch feature가 깨끗하던 예전 checkpoint의 patch similarity 구조(Gram matrix XX^T)를 유지하게 붙잡아. feature 자체가 아니라 유사도 구조를 맞춰.

$L_{Gram} = \|X_S X_S^T - X_G X_G^T\|_F^2$ (p.69)

- X: 이미지 한 장의 patch feature를 줄줄이 쌓은 행렬. XX^T의 (a, b) 칸 = patch a와 b의 유사도.
- 첨자는 슬라이드에 정의가 없어. S = Student(지금 학습 중인 모델), G = Gram teacher(patch가 깨끗하던 예전 checkpoint)로 읽으면 돼 (notes 보충).
- $\|\cdot\|_F^2$: 칸별 차이를 제곱해서 다 더한 값.

왜 Gram? feature 값을 그대로 맞추면 예전 모델에 통째로 묶여서 global이 더 좋아질 여지가 줄어. Gram은 "어느 patch끼리 비슷한가"라는 관계만 담아서, 관계는 지키고 feature는 자유롭게 둬. 이 이유는 논문 쪽 설명이고, 슬라이드는 "similarity 구조를 보존한다"(p.69)까지만 말해.



Gram anchoring 효과. 맨 위가 원본 사진이고, 아래 두 줄은 빨간 점(슬라이드 표현은 red cross, 기준 patch)과 나머지 patch의 유사도를 밝기로 칠한 map이야. 가운데 줄(w/o L_{HRef})은 유사도가 배경까지 번지고 noisy해. 아래 줄(w/ L_{HRef})은 같은 물체 부분에만 깨끗하게 모여. 오래 학습하면 dense feature가 망가지는 문제를 고친 거야. (L_{HRef}는 p.70에 설명 없이 나오는 이름이고, p.69의 Gram anchoring loss 계열로 보면 돼.) `L04 p.70`

RL로 번역

- BYOL의 "bootstrap"은 TD learning의 bootstrap과 같은 뜻이야. 정답 대신 천천히 움직이는 target network의 출력을 target으로 써. DQN에서 target을 천천히 움직여야 안정적인 것과 같은 직관이야.
- EMA 식은 DDPG·SAC의 soft target update와 같은 모양이야. 단 τ 자리가 반대야: soft update는 `θ′ ← τθ + (1-τ)θ′`(τ 작음), BYOL은 `ξ ← τξ + (1-τ)θ`(τ가 1 근처).
- stop-gradient도 semi-gradient TD랑 같은 이유야. TD target `r + γV(s′)`을 상수 취급하듯 target·teacher 출력을 상수 취급해.
- AutoAugment는 아예 RL(controller, reward = validation accuracy)로 augmentation을 찾아. SynCLR·GenView는 diffusion/text-to-image 생성 모델을 view 생성기로 써. 네 쪽 generative model이 여기선 데이터 공장이야.

시험 함정

- BYOL: negative **없음**, loss는 **MSE**. DINO: loss는 **CE**, centering은 **teacher에만**.
- SimCLR: negative 2N−2개, 학습 후 g를 버림 (f를 버리는 게 아님).
- Barlow Twins: sample끼리가 아니라 feature 차원끼리 비교.
- SynCLR positive = 같은 caption. DINOv2 = 이미지만 (caption 안 씀).
- BYOL의 τ는 **EMA 계수**, temperature 아님. predictor는 **online에만**, sg는 **target 쪽**.
- EMA 방향은 teacher/target ← student/online. teacher·target은 gradient는 안 받지만 EMA로 계속 바뀌어.
- DINO center C는 **teacher 출력** 평균의 EMA (parameter 아님). loss는 교차 view `H(t1, s2)`, `H(t2, s1)`.

다음으로

SimCLR은 negative를 batch에서만 가져와서 batch가 커야 해. batch를 안 키우고 negative를 많이 구할 순 없을까? negative 말고 cluster·언어·label로 supervision을 줄 순 없을까? → L05

셀프 체크

T/F BYOL needs many negative samples, so it requires a large batch size.

False. negative 없이 EMA target을 예측해 (p.30–34). negative를 모으려고 batch를 키우는 건 SimCLR 쪽 사정이야.

빈칸 Barlow Twins pushes the cross-correlation matrix of the two views toward the _____ matrix.

identity (p.35–38). 대각선 1(invariance), 비대각 0(redundancy reduction).

MCQ Which method trains a student to match an EMA teacher's softmax distribution with cross-entropy? (A) BYOL (B) DINO (C) Barlow Twins (D) SimCLR

(B) DINO (p.39–44). BYOL은 MSE로 feature를 맞춰. student/teacher + softmax 분포 + cross-entropy가 DINO의 키워드야.

L05 Contrastive Learning III 76p

질문 **negative를 싸게 많이 구하려면? negative 말고 다른 supervision은?**

먼저 L03–L04 말을 짧게 상기하자.

- encoder: 이미지를 벡터 하나로 바꾸는 network. 그 출력이 embedding(= representation, feature)이야.
- augmentation: 같은 사진을 자르거나(crop) 색을 바꾼 변형본. 변형본 하나하나를 **view**라고 불러.
- anchor(MoCo에선 **query**): 기준 샘플. **positive**는 anchor와 가까워질 짝, **negative**는 멀어질 나머지.
- InfoNCE: "positive 1개 + negative 여러 개 중 positive 고르기"를 softmax 분류로 보고 거는 cross-entropy. similarity를 temperature τ로 나누고, τ가 작을수록 softmax가 뾰족해져.

이 chapter는 두 줄기야. (1) negative를 **싸게 많이** 모으기: memory bank → MoCo → MoCo v2 → CURL. (2) negative 말고 **다른 신호**로 "무엇을 같은 것으로 볼지" 정하기: cluster(DeepCluster, SwAV), text(CLIP), label(SupCon), 이웃(NNCLR, All4One), 최근 확장(PAPN, SCL-RHE).

negative 창고 짓기

Memory bank

p.4

한 줄 예전에 계산한 embedding을 창고(memory)에 저장해 두고 negative로 다시 꺼내 쓰는 방법.

문제 "Large numbers of negative samples improve contrastive learning." (p.4) 그런데 negative를 늘리기가 비싸.

- **왜 많을수록 좋아?** InfoNCE 분모가 전체 데이터를 더 잘 대표하고, anchor와 헷갈릴 만큼 비슷한 hard negative가 섞일 확률도 커져.
- **왜 비싸?** negative도 매번 encoder에 통과시켜야 해 ("computationally expensive"). SimCLR(L04)처럼 같은 mini-batch 안 사진만 negative로 쓰면 negative 수가 batch 크기에 묶여.

핵심 한 번 계산한 embedding을 **저장(store)**했다가 negative로 **재사용(reuse)**해. negative 수가 batch와 상관없어지고, 다시 계산할 필요도 없어.

대가 저장된 값은 **저장 당시의 encoder**로 만든 거야. encoder는 계속 학습돼 바뀌니까 창고 값은 지금 기준으로 낡았어. 이걸 **stale**하다고 해.

Key Idea: "Trade stale representations for efficient large-scale contrastive learning." mini-batch negative는 **적지만 fresh**, memory bank negative는 **많지만 조금 stale**.

> **비유**
> 카메라 필터를 매일 조금씩 바꾸며 사진을 찍는다고 해 봐. 앨범(bank) 사진은 찍은 날마다 필터가 달라 색감이 제각각이라, 오늘 사진(query)과 비교하면 내용이 아니라 필터 때문에 달라 보일 수 있어. 이게 stale 문제야.

Instance Discrimination

Wu, CVPR 2018 · p.5–9

한 줄 "사진 한 장 = class 하나"로 놓고 각 사진이 자기 자신을 맞히게 분류를 학습하는, memory bank의 대표 방법.

정의 "Each image is considered a unique class." positive pair = 같은 이미지(실제론 그 augmentation), negative pair = 나머지 모든 이미지 (p.5).

문제 사진이 백만 장이면 백만-way 분류야. class마다 weight를 두면 softmax head가 너무 커져 ("Unfeasible to train a softmax layer with these many heads", p.6).

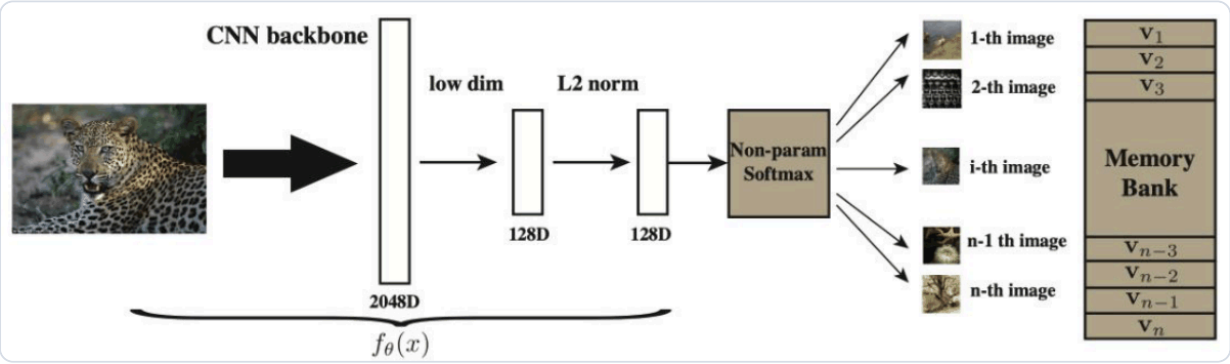
핵심 두 장치로 풀어.

1. **non-parametric softmax** (p.7): class i의 classifier weight 대신 bank에 저장된 i번 사진의 feature v_i를 그대로 써. 학습할 weight가 따로 없어서 "non-parametric".
   $P(i \mid v) = \exp(v_i^T v/\tau) \;/\; \Sigma_j \exp(v_j^T v/\tau)$
2. **NCE**(Noise Contrastive Estimation) (p.6): class 전부를 계산하는 대신 negative를 일부만 뽑아서 근사해 ("samples only some negatives"). 문제를 "이 (feature v, ID i) 쌍이 진짜 데이터에서 왔나, noise에서 왔나?"를 맞히는 문제로 바꾸는 거야 ("Given feature v and ID i, what is the probability that it originated from the actual data source?", p.8). 분모의 N개 합은 무작위 M개로 근사해 (Monte Carlo approximation, p.8).

기호: $v = f_\theta(x)$는 지금 encoder로 만든 현재 사진의 embedding(L2 norm한 128차원, p.6), $v_i^T v$는 bank의 i번 feature와의 내적(클수록 "i번 사진" 확률이 커짐), τ는 temperature.

positive 쪽 loss엔 **proximal** 항을 더해서 각 embedding이 이전 iteration 값에서 너무 벗어나지 않게 안정화해. 수렴하면 점점 사라지는 항이야 (p.8).

한계 bank feature가 옛날 encoder로 만든 거라 stale해서, 현재 query를 outdated feature와 비교하게 돼 (p.9). 장점(negative 많음, 계산 저렴)은 살리고 이 단점을 고친 게 MoCo야.



Instance Discrimination 구조. 이미지 → CNN backbone → 128D 벡터 → L2 norm(길이 1로 맞춤)까지가 encoder $f_\theta(x)$야. 오른쪽 memory bank의 각 행 $v_1 \ldots v_n$은 이미지 한 장 한 장의 저장된 feature고, **학습되는 classifier weight 대신 이 행들과 비교**해서 softmax를 계산해(non-parametric softmax). 그래서 class 수 = 이미지 수야. L05 p.6

MoCo

He, CVPR 2020 · p.10–15 · code p.12

한 줄 memory bank 대신 "최근 key만 담는 queue"와 "아주 천천히 변하는 key 전용 encoder"로 negative를 많이, 그리고 서로 **일관되게(consistent)** 만든 방법 (Momentum Contrast).

질문 "Can we keep a large dictionary while reducing feature staleness?" (p.9)

용어 MoCo는 비교 대상 모음을 **dictionary**라고 불러 ("dynamic dictionary of negative keys", p.10).

- **query** q: 지금 학습하는 샘플(anchor)의 embedding. query encoder f_q로 만들어.
- **key** k: dictionary 안의 비교 대상. key encoder f_k로 만들어. k^+는 같은 사진을 **다르게 augmentation**해서 key encoder에 넣은 positive key야 ("k+ is differently augmented query", p.11). k^+는 이번 mini-batch에서 바로 만든 key고(코드의 `l_pos`), negative는 **queue에 쌓여 있던 key K개**야 ("Keep a running queue of keys (negative samples)", p.11 · 코드의 `l_neg`).

좋은 dictionary는 **large**(key가 많음)하면서 **consistent**(key들이 서로 비슷한 encoder로 만들어짐)해야 해 (p.10). memory bank는 large지만 consistent하지 않고, **end-to-end**(SimCLR처럼 query와 key를 둘 다 지금 encoder로 계산하고 양쪽 모두 backprop하는 방식. negative = 같은 mini-batch)는 consistent하지만 key가 batch 크기만큼뿐이야.

주의 여기 "consistent"는 queue 안 key들이 거의 같은(천천히 변하는) encoder로 만들어져서 서로 비교 기준이 맞는다는 뜻이야. L06의 consistency regularization(입력을 흔들어도 예측이 같게 만드는 loss)과는 다른 말이야.

핵심 두 장치가 하나씩 맡아.

1. **FIFO queue**(First In, First Out) → "large" 담당. 매 step 현재 mini-batch의 key를 넣고(enqueue) 가장 오래된 mini-batch의 key를 빼(dequeue). queue 길이 K를 batch와 따로 정하니까 negative 수가 batch에 안 묶여 ("Decouple mini-batch size with the number of keys", p.11). 오래된 key는 계속 빠지니까 bank보다 덜 낡아.
2. **momentum encoder** → "consistent" 담당. key encoder는 gradient로 학습하지 않고 query encoder를 천천히 따라가기만 해: $\theta_k \leftarrow m\theta_k + (1-m)\theta_q$

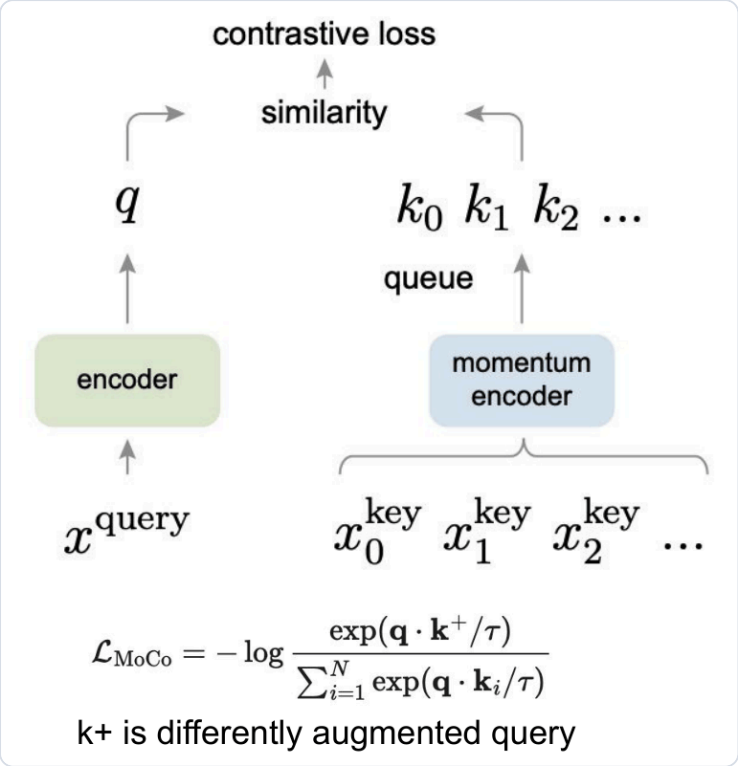
- θ_q: query encoder 파라미터(backprop으로 학습), θ_k: key encoder 파라미터.
- m: 1에 가까운 momentum 계수. key encoder는 자기 값을 m만큼 유지하고 query encoder를 (1−m)만큼만 섞어. 예: m = 0.99면 매 step 새 정보는 1%.
- m이 클수록 key encoder가 천천히 변해 ("slowly progressing", p.11). 그래서 queue 안 key들은 몇 step 차이로 만들어졌어도 거의 같은 encoder 산물이라 consistent해. 반대로 m = 0이면 key encoder가 매 step query encoder를 그대로 복사해서 query encoder만큼 빨리 바뀌어. 그러면 queue 안 key들이 step마다 다른 encoder로 만들어져 consistent하지 않게 돼.
- 이런 갱신이 EMA(exponential moving average): 과거 값에 큰 가중치, 새 값에 작은 가중치를 주는 평균이야.

`loss` InfoNCE (p.11): $L = -\log \left[\exp(q \cdot k^+/\tau) \,/\, \Sigma_i \exp(q \cdot k_i/\tau) \right]$. 분자(query와 positive key의 similarity)가 커지면 loss가 줄고(당기기), 분모의 negative similarity가 커지면 loss가 커져(밀기). 분모 Σ는 k^+ 1개 + queue의 negative K개, 합쳐서 1+K개 key 전부야 (코드 `logits = cat([l_pos, l_neg])`가 이 1+K칸). 곧 "1+K개 후보 중 0번 칸(positive) 맞히기"라서 pseudo code가 `labels = zeros(N)`이야 (p.12).

`gradient` **query encoder로만** 흘러 ("Compute gradients and update the encoder only through the queries", p.11). 코드: `k = k.detach()`로 key를 gradient에서 떼고 → `update(f_q.params)`로 f_q만 SGD → momentum 줄로 f_k 갱신 → enqueue/dequeue

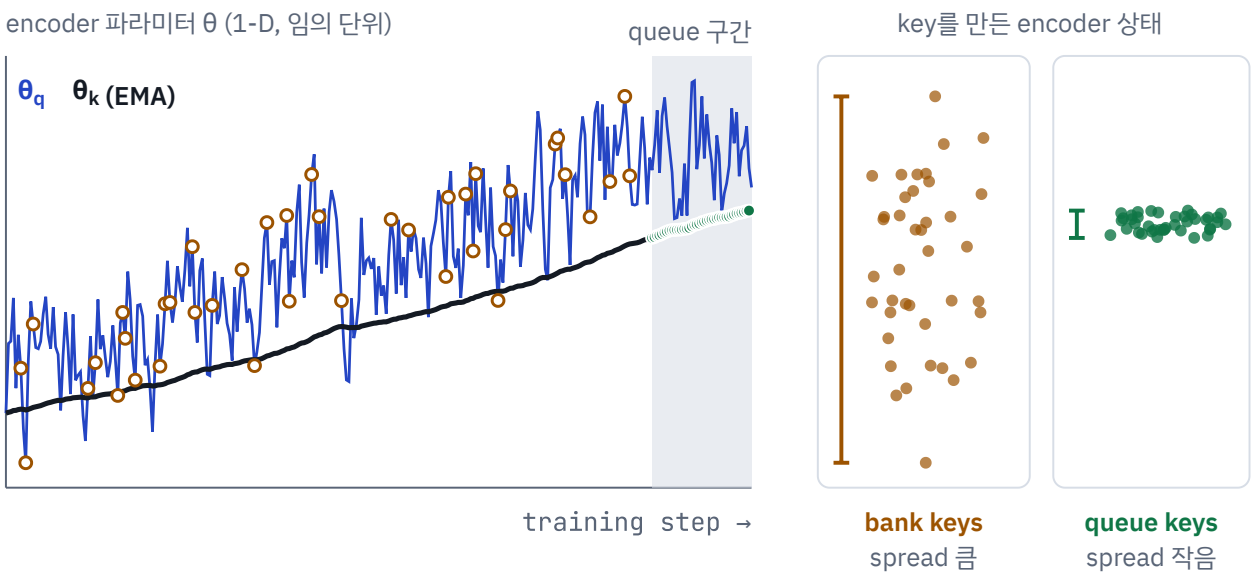
(p.12). key에 backprop을 안 하니 key가 많아도 계산·메모리 부담이 작아. 순서 주의: enqueue는 loss와 update가 끝난 **뒤**라서, 이번 batch의 key는 이번 step에선 negative가 아니고 다음 step부터 negative로 쓰여.
차이 SimCLR 대비 (p.11): running queue of keys를 negative로, gradient는 query로만, batch 크기와 key 수 분리, key encoder는 momentum으로 천천히.

> **비유**
> 필터 비유를 이어 가면: 앨범용 카메라(key encoder)는 필터를 아주 천천히만 바꾸고 (momentum), 앨범엔 최근 사진만 남겨(queue). 그러면 사진은 여전히 많은데 색감이 거의 같아서 공정하게 비교할 수 있어.



MoCo 구조. 왼쪽 encoder가 query 이미지로 q를 만들고, 오른쪽 **momentum encoder**가 key 이미지들로 k_0, k_1, k_2...를 만들어 **queue**에 쌓아. q와 key들의 similarity로 contrastive loss를 계산하는데, **gradient는 왼쪽 encoder(query 쪽)로만** 흘러. queue 길이를 batch 크기와 따로 정할 수 있어서 negative 수가 batch에 안 묶여. 아래 식은 InfoNCE야: 분자는 q와 positive key k^+, 분모는 k^+와 queue key 전부. "k+ is differently augmented query"는 k^+가 같은 사진을 다르게 augmentation해 key encoder에 넣은 것이라는 뜻이야. `L05 p.11`

인터랙티브 **momentum encoder가 key를 일관되게 만드는 이유** illustrative



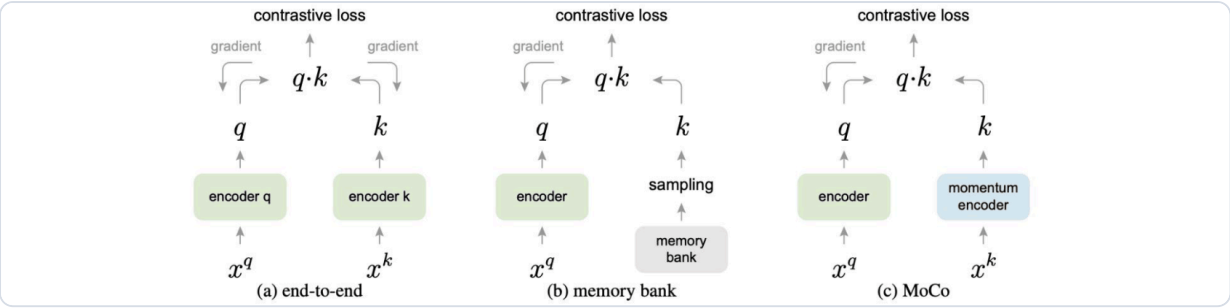

```
f_k.params = m*f_k.params + (1-m)*f_q.params
```

momentum m 0.9 **0.99** 0.999

첫 번째 그래프: 가로축은 학습 step, 세로축은 encoder 파라미터 θ를 1차원으로 단순화한 값이야. 얇은 파란 선 θ_q는 매 step gradient로 흔들리며 움직이는 query encoder, 굵은 선 θ_k는 그걸 `θ_k ← m·θ_k + (1-m)·θ_q`(EMA)로 따라가는 key encoder야. 버튼으로 m을 바꿔봐. **m이 클수록 θ_k가 느리고 매끈해져** (300 step만 그려서 0.999는 거의 안 움직여 보여). 속 빈 주황 점 = memory bank key가 만들어진 시점, 초록 점 = queue key(음영 친 최근 구간)야. 두 번째 그래프: 점 하나 = key 하나, 세로 위치 = **그 key를 만든 encoder의 상태**. memory bank key는 과거 여러 시점의 θ_q로 만든 거라 넓게 퍼지고(inconsistent), queue key는 천천히 변하는 θ_k의 최근 구간에서 나와서 좁게 모여(consistent). 데이터는 지어낸 개념 그림이야. (L05 p.11–12)

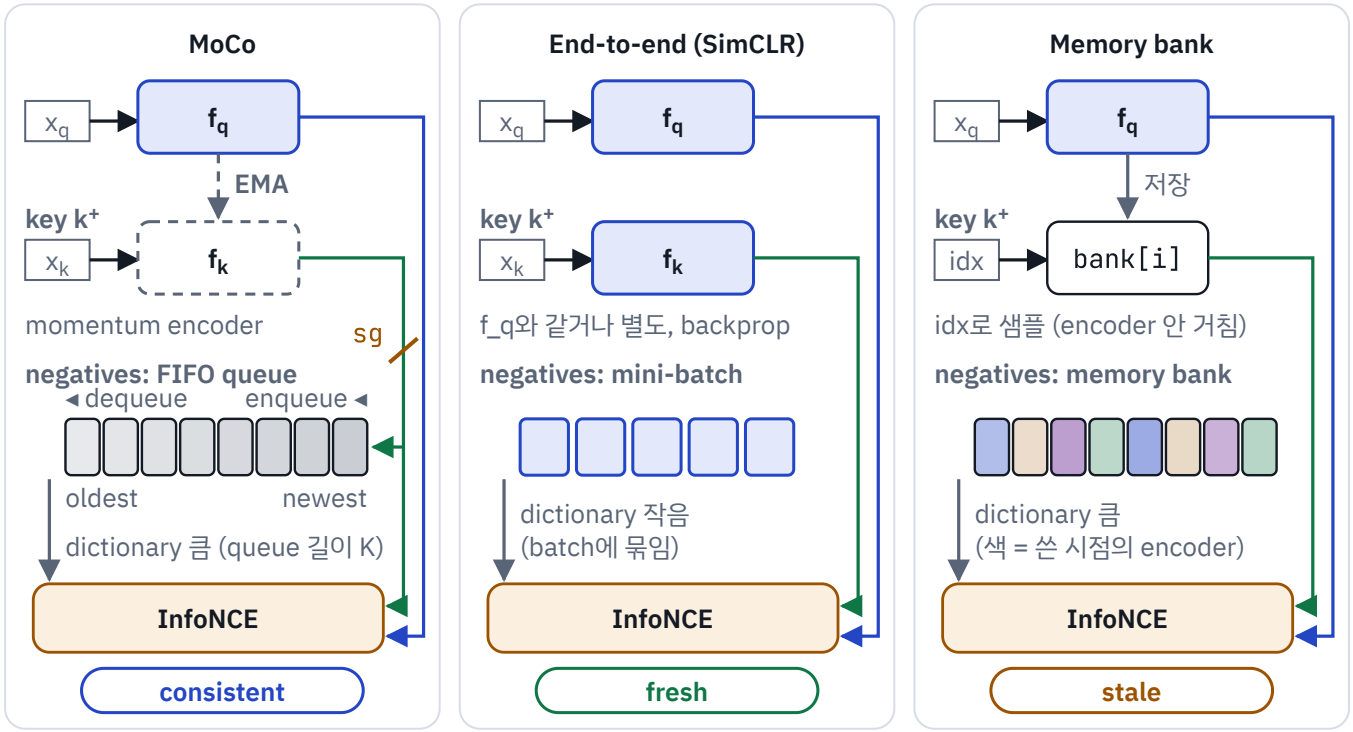
세 방식 비교 (p.14). 차이는 **key를 어디서 얻고, key encoder를 어떻게 갱신하나**야. **fresh**는 지금 encoder로 방금 계산함, **stale**은 여러 과거 encoder로 계산해 낡음, **consistent**는 최신은 아니어도 key들이 거의 같은 encoder로 만들어져 기준이 맞음(fresh와 다른 말). "MoCo is essentially a hybrid of end-to-end learning and memory banks": bank처럼 dictionary가 크고, momentum encoder로 key가 더 consistent해.

P.14 비교	END-TO-END (SIMCLR)	MEMORY BANK	MOCO
query 만드는 쪽 (Query)	현재 encoder	현재 encoder	현재 encoder
key 만드는 쪽 (Key)	현재 encoder (backprop)	저장된 feature	momentum encoder
negative 출처 (Negative Source)	mini-batch	memory bank	FIFO queue
dictionary (Dictionary Size)	작음 (batch에 묶임)	큼	큼
key 상태 (Feature Freshness)	fresh	stale	**consistent**
encoder 갱신 (Encoder Update)	backprop	backprop	momentum update (key만, query는 backprop)



MoCo 논문의 세 방식 그림. 세 칸 모두 q는 학습되는 encoder로 만들고, 차이는 **k를 어디서 얻고 key 쪽을 어떻게 갱신하나**야. 회색 **gradient** 화살표를 봐: (a) end-to-end는 encoder q와 encoder k **양쪽**으로 gradient가 가. (b) memory bank는 key encoder 없이 bank에서 저장된 feature를 sampling해 (오래된 값이라 stale). (c) MoCo는 **momentum encoder**가 key를 그때그때 만들고, gradient는 q 쪽으로만 가. 이 key들이 queue에 쌓여 negative가 돼. `L05 p.13`

구조 비교 **key와 negative는 어디서 오나: End-to-end vs Memory bank vs MoCo**



세 칸 모두 query는 학습되는 encoder `f_q`(파란 상자, 파란 선)로 만들어. 다른 건 **positive key k^+와 negative가 어디서 오나**야. 초록 선 = key가 loss로 가는 길. **첫 칸 MoCo**: key encoder `f_k`는 회색 점선 상자 = gradient 없이 `f_q`의 EMA로만 갱신돼. 빗금과 `sg`(stop-gradient) 표시는 key 쪽으로 gradient가 안 간다는 뜻이야. 만든 key는 loss에 쓰이고, queue 오른쪽 끝(newest)으로 들어가서 왼쪽 끝(oldest)부터 빠져(FIFO). 그래서 negative가 많고(large) 서로 consistent해. **둘째 칸 End-to-end**: key도 지금 encoder로 계산하고 backprop해서 fresh하지만, negative가 같은 mini-batch뿐이라 batch 크기에 묶여. **셋째 칸 Memory bank**: 저장해 둔 값을 index로 꺼내 써서(encoder를 안 거침) negative가 많지만, 상자 색이 제각각인 것처럼 과거 여러 시점의 encoder가 쓴 값이라 stale해. (p.14 표 순서와 달리 MoCo를 맨 앞에 뒀고, End-to-end의 key encoder는 슬라이드 표기가 p.13/p.14에서 달라 "f_q와 같거나 별도"로 썼어.) (L05 p.11–14)

MoCo v2

`Chen 2020 · p.16-19`

한 줄 MoCo 구조는 그대로 두고 SimCLR(L04)의 레시피 두 개를 붙인 개선판: MLP projection head(encoder 뒤에 붙여 loss를 거는 작은 비선형 network)와 더 센 data augmentation.

queue와 momentum encoder는 그대로라 ("Retains the efficient momentum encoder and queue mechanism", p.16) **새 architecture가 아니야**.

결과 batch 256으로 batch 8192 SimCLR를 이겨 (p.18). negative 수가 batch가 아니라 queue 길이로 정해지니까 ("Decoupling mini-batch size with negative sample size"). GPU 메모리도 훨씬 덜 써 ("much smaller memory footprint", p.19, 여기서 "end-to-end" = SimCLR).

Key takeaway "Non-linear projection head and strong data augmentation are crucial for contrastive learning." (p.17)

CURL

Srinivas 2020 · p.20-21

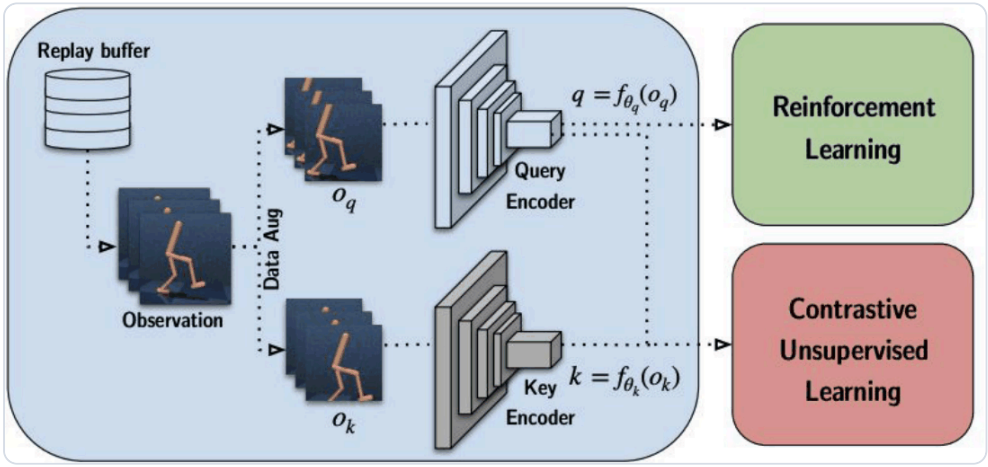
한 줄 MoCo식 contrastive learning을 RL agent의 이미지 관측(observation)에 붙여, 적은 환경 상호작용으로도 RL이 잘 되게 한 방법.

문제 pixel 관측 RL은 sample-inefficient해. reward 신호만으로 고차원 이미지에서 쓸 만한 feature를 배우려면 상호작용이 아주 많이 필요해.

어떻게 (p.21)

1. replay buffer(agent 경험 저장소)에서 observation을 꺼내 **random crop**을 두 번 → positive pair.
2. 한 view는 query encoder, 다른 view는 momentum key encoder로 (MoCo 그대로) → contrastive loss로 encoder 학습.
3. **동시에** 그 representation으로 RL policy/value network도 학습 ("Simultaneously optimize").

차이 미리 학습하고 얼려 두는 게 아니라 RL loss와 함께 최적화하는 보조(auxiliary) 목표야. downstream(배운 representation을 실제로 쓰는 목표 task)이 이미지 분류가 아니라 RL 정책 학습이라는 점도 달라.



CURL 구조. replay buffer에서 꺼낸 observation에 augmentation(random crop)을 두 번 해서 query용·key용 view를 만들어. **query encoder 출력은 RL(policy/value) 학습과 contrastive loss 양쪽에 같이 쓰이고**, key encoder(momentum) 출력은 contrastive loss에만 쓰여. RL과 representation 학습이 동시에 돌아간다는 게 핵심이야. L05 p.21

negative 말고 다른 신호

같은 사진의 변형본만 positive로 보면, 같은 종류의 다른 사진도 negative로 밀어내는 **false negative**가 생겨. 그래서 여기부턴 "무엇을 같은 것으로 볼까"를 다른 신호로 정해. DeepCluster·SwAV는 pair 대신 cluster 번호/배정(code)을 맞힐 target으로 쓰고, CLIP은 caption, SupCon은 label, NNCLR·All4One은 이웃을 positive로 써.

DeepCluster

Caron, ECCV 2018 · p.23-25

한 줄 지금 network의 feature로 사진을 비슷한 것끼리 묶고(clustering), 묶음 번호를 가짜 정답으로 분류를 학습하는 방법.

용어 **k-means**는 그룹 수 k를 정해 두고 "각 점을 가장 가까운 중심(centroid)에 배정 → 중심 재계산"을 반복하는 기본 clustering이야. pseudo label은 사람이 아니라 모델이 만든 가짜 label로, 여기선 cluster ID야.

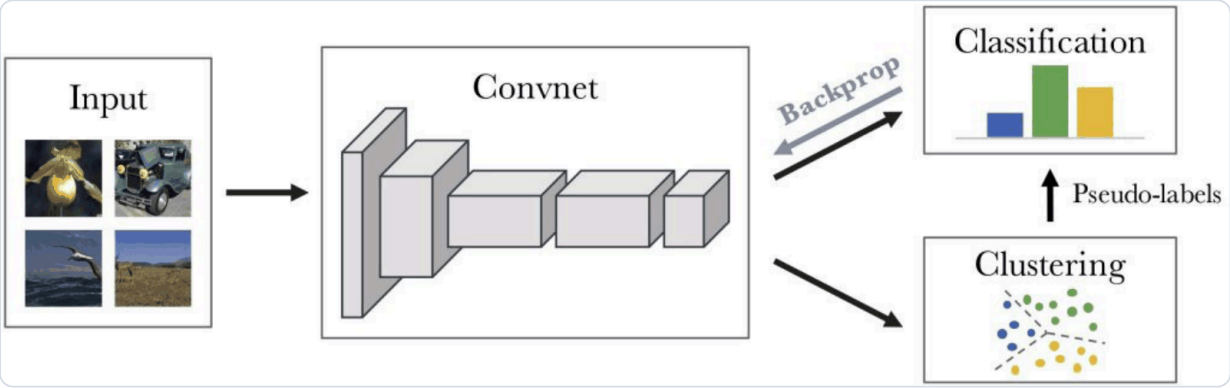
어떻게 (Process, p.24)

1. 현재 network(convnet)로 이미지 feature를 뽑아.
2. k-means로 clustering하고, cluster ID를 pseudo label로 붙여.
3. 그 label을 맞히도록 classification으로 학습해 (backprop).
4. 좋아진 feature로 다시 1번부터, 수렴할 때까지.

핵심 "Alternate between clustering and classification." (p.24) feature가 엉성해도 묶음을 맞히다 보면 feature가 나아지고, 나아진 feature로 다시 묶으면 묶음도 나아져.

차이 positive/negative pair를 아예 안 만들어 (p.23). 비슷한 사진은 같은 label을 받으니 false negative도 피해. 대신 **offline**이야: mini-batch마다가 아니라 주기적으로 dataset 전체 feature를 다시 뽑아 k-means를 돌려서 무거워.

기여 (p.25) clustering 기반 SSL(self-supervised learning, label 없이 데이터 자체로 학습) 도입, pseudo-label learning을 contrastive의 대안으로 확립, SwAV·DINO에 영향.



DeepCluster 루프. 화살표를 따라가면: convnet이 feature를 뽑고 → **k-means로 clustering** → cluster 번호를 **pseudo-label**(가짜 정답)로 붙이고 → 그걸 맞히는 classification loss로 convnet을 backprop. 이걸 반복해. 정답이 사람 label이 아니라 network 자기 feature에서 나온다는 점을 봐. L05 p.23

SwAV

Caron, NeurIPS 2020 · p.26-30

한 줄 같은 사진의 두 view에 "어느 그룹이야?"를 각각 묻고, 한 view의 답을 다른 view가 맞히게 하는 방법 (Swapping Assignments between multiple Views).

문제 contrastive는 feature끼리 일일이 비교하느라 negative가 많이 필요하고, DeepCluster는 dataset 전체를 offline으로 clustering해야 해.

용어

- prototype $C = [c_1, \ldots, c_K]$: 학습되는 벡터 K개 ("K trainable prototype vectors", p.27). k-means centroid를 network 파라미터로 만든 cluster 대표점이라고 보면 돼.
- **code** q: 한 view가 어느 prototype에 얼마나 속하는지 정한 배정(assignment). 다른 view가 맞힐 정답이야. (MoCo의 query q와는 다른 기호)
- **prediction** p: feature와 prototype들의 similarity를 softmax한 확률 분포.

어떻게 두 view의 feature z_1, z_2를 같은 encoder로 뽑고, prototype과 비교해 code q_1, q_2를 mini-batch 안에서 그때그때 계산해 (**online** clustering, p.26). 그다음 **swap**: view 1의 prediction으로 view 2의 code를, view 2의 prediction으로 view 1의 code를 맞혀.

loss (p.27)

$$L(z_t, z_s) = \ell(z_t, q_s) + \ell(z_s, q_t)$$

$$\ell(z_t, q_s) = -\Sigma_k \; q_s^{(k)} \log p_t^{(k)}$$

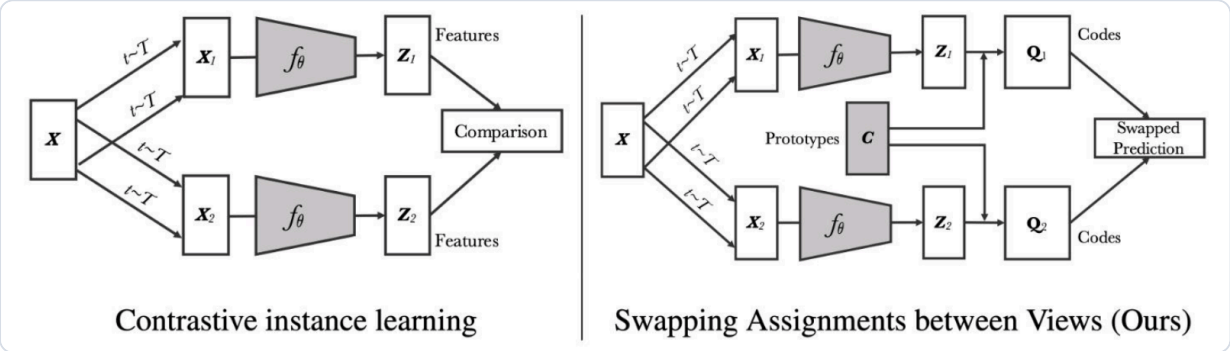
$$p_t^{(k)} = \exp(z_t^T c_k/\tau) \; / \; \Sigma_{k'} \exp(z_t^T c_{k'}/\tau)$$

t, s는 같은 사진의 두 view 번호야 (DINO의 teacher / student t, s와는 무관해. SwAV엔 teacher / student 구조가 없어). $p_t^{(k)}$는 view t가 prototype k에 속할 확률이고, $\ell(z_t, q_s)$는 view s의 code를 정답으로 둔 cross-entropy야. q_s가 가리키는 prototype에 p_t가 높은 확률을 줄수록 작아져. 두 방향을 더해서 대칭(symmetric)이야.

왜 swap? 자기 code를 자기가 맞히는 건 trivial(너무 쉬워서 배울 게 없음)해. 다른 view의 code를 맞혀야 "잘리거나 색이 달라도 같은 그룹"인 view-invariant feature를 배워.

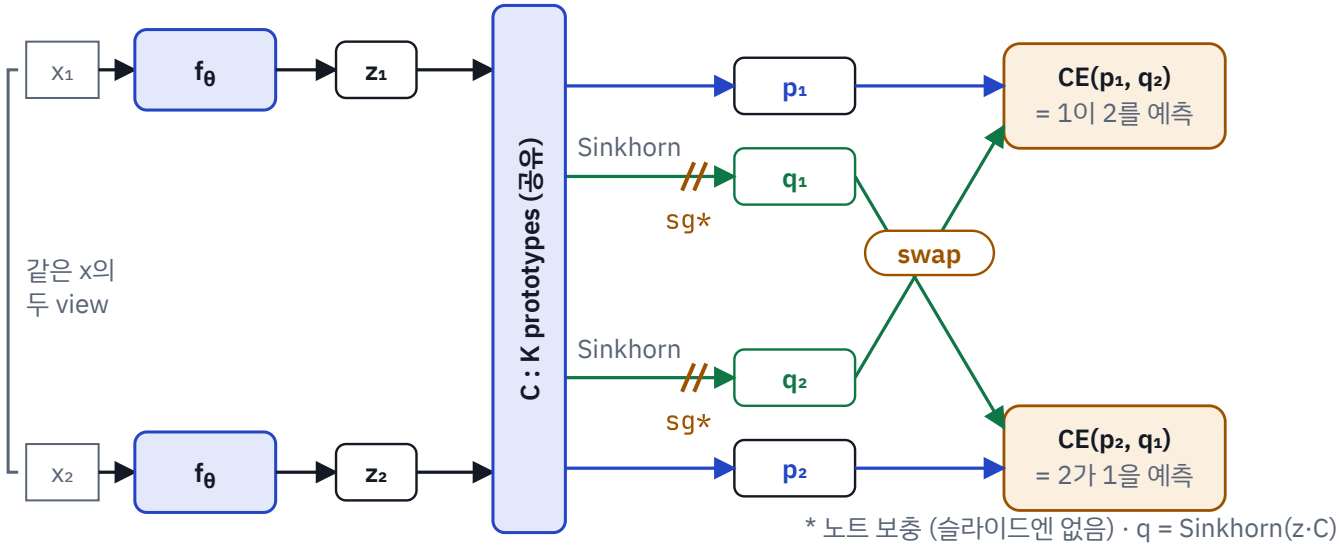
차이 contrastive는 **Feature ↔ Feature**, SwAV는 **Feature ↔ Prototype** (p.30). 샘플끼리 직접 비교하지 않으니 명시적 negative pair가 없고, offline clustering도 없어 (p.26). p.30 표의 나머지 짝: Positive/Negative pairs ↔ Prototype assignments, Similarity matching ↔ Swapped assignment prediction, Pairwise comparison ↔ Online clustering.

범위 시험은 "online code + 학습되는 prototype + swapped prediction"까지야. code를 실제로 어떻게 계산하는지는 슬라이드에 없어.



왼쪽은 두 view의 feature z_1, z_2를 직접 비교하는 일반 contrastive, 오른쪽이 SwAV야. SwAV는 feature를 학습되는 **prototype C**와 비교해서 각 view의 **code Q**(어느 prototype에 속하나)를 만들고, **한 view의 code를 다른 view로 예측**해(swapped prediction). feature끼리 비교하지 않고 prototype을 거친다는 게 포인트야. L05 p.27

구조 비교 **SwAV: 한 view의 code(어느 prototype에 속하나)를 다른 view가 맞혀**



Contrastive	Feature ↔ Feature · positive/negative pair 비교
SwAV	Feature ↔ Prototype · online code, 명시적 negative 없음

같은 사진 x에서 만든 두 view x_1, x_2가 같은 encoder f_θ를 지나 feature z_1, z_2가 되고, 둘 다 같은 prototype C(K개, gradient로 학습되는 파란 상자)와 비교돼. 여기서 두 가지가 나와: 파란 **p** = prototype별 확률로 바꾼 **예측(prediction)**, 초록 **q** = 이 view가 어느 prototype에 속하나를 정한 **code**(정답 역할). loss 상자를 봐: CE(p_1, q_2)는 view 1의 예측으로 view 2의 code를 맞히고, CE(p_2, q_1)은 반대야. 초록 화살표가 엇갈리는 게(swap) 핵심이야. 자기 code를 자기가 맞히면 너무 쉬우니까(trivial), 엇갈려 맞혀야 view가 바뀌어도 같은 feature(view-invariant)를 배워. q를 Sinkhorn으로 계산한다는 것과 q 쪽에 gradient가 안 간다는 표시(sg*)는 노트 보충이라 슬라이드엔 없어 (시험 범위 밖). 맨 밑 두 줄은 p.30 비교표 요약: contrastive는 Feature ↔ Feature, SwAV는 Feature ↔ Prototype. (L05 p.26–30)

CLIP

Radford 2021 · p.32–37 · code p.35

한 줄 사진과 그 설명 글(caption)을 짝으로 놓고, 짝끼리는 가깝게 아닌 것끼리는 멀게 해서 이미지와 텍스트를 같은 embedding 공간에 놓는 contrastive learning.

문제 지금까지 positive는 "같은 사진의 augmentation"이었어. CLIP은 인터넷의 (image, text) 쌍을 positive로 써서 자연어가 supervision이 돼 ("Uses natural language as the supervisory signal", p.37). 그래서 섹션 이름이 "Working with Supervised Datasets"야.

구성 image encoder(ResNet 또는 Vision Transformer)와 text encoder(CBOW(단어 순서를 무시하고 단어 embedding을 평균하는 간단한 text encoder) 또는 Text Transformer)를 **따로** 두고 **같이** 학습해 (p.32, p.35). 출력 차원이 달라서 학습되는 projection W_i, W_t로 같은 차원 d_e로 보내고 L2 normalize해. 이 공통 공간이 **shared embedding space**고, 길이가 1이니 내적 = cosine similarity야.

어떻게 학습 (p.33)

1. (image, caption) 쌍 n개를 각자 encode해서 **n×n similarity 행렬**을 만들어. 칸 (i, j) = 이미지 i와 텍스트 j의 cosine × scale.
2. 대각선 (i, i)는 진짜 짝 → positive, 나머지 n^2-n칸 → negative. 그래서 `labels = np.arange(n)` : "i번 이미지의 정답은 i번 텍스트".
3. 행 방향 CE("이 이미지에 맞는 텍스트는?")와 열 방향 CE("이 텍스트에 맞는 이미지는?")를 평균해. 그래서 **symmetric** contrastive loss야. 코드: `loss_i =`

`cross_entropy_loss(logits, labels, axis=0)`, `loss_t = …(axis=1)` →

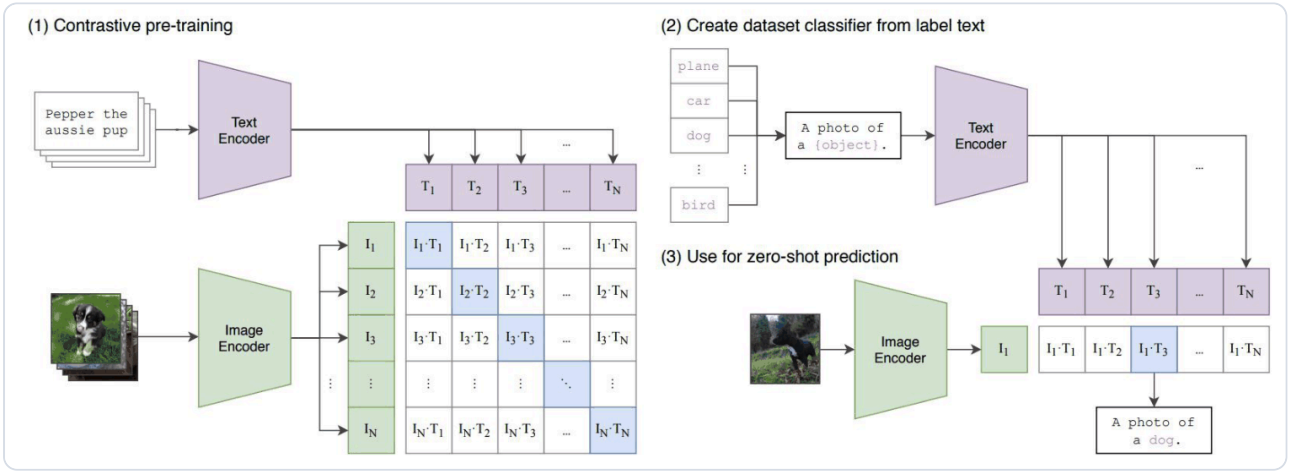
예시로 보면

n = 3: (강아지 사진, "a dog on grass"), (고양이 사진, "a sleeping cat"), (자동차 사진, "a red car"). 정답은 대각선 3칸이야. 강아지 사진 행에선 "a dog on grass" 칸이, "a sleeping cat" 열에선 고양이 사진 칸이 제일 높아야 해. 행마다, 열마다 3지선다를 푸는 셈이야.

`temperature` `logits = np.dot(I_e, T_e.T) * np.exp(t)` (p.35). cosine에 `np.exp(t)`를 **곱해**. t는 **학습되는 파라미터**("learned temperature parameter")고, τ로 나누는 것과 같은 역할(`exp(t) = 1/τ`)이야. exp라서 scale은 항상 양수. MoCo의 `logits/t`(나누기)와 헷갈리기 좋아.

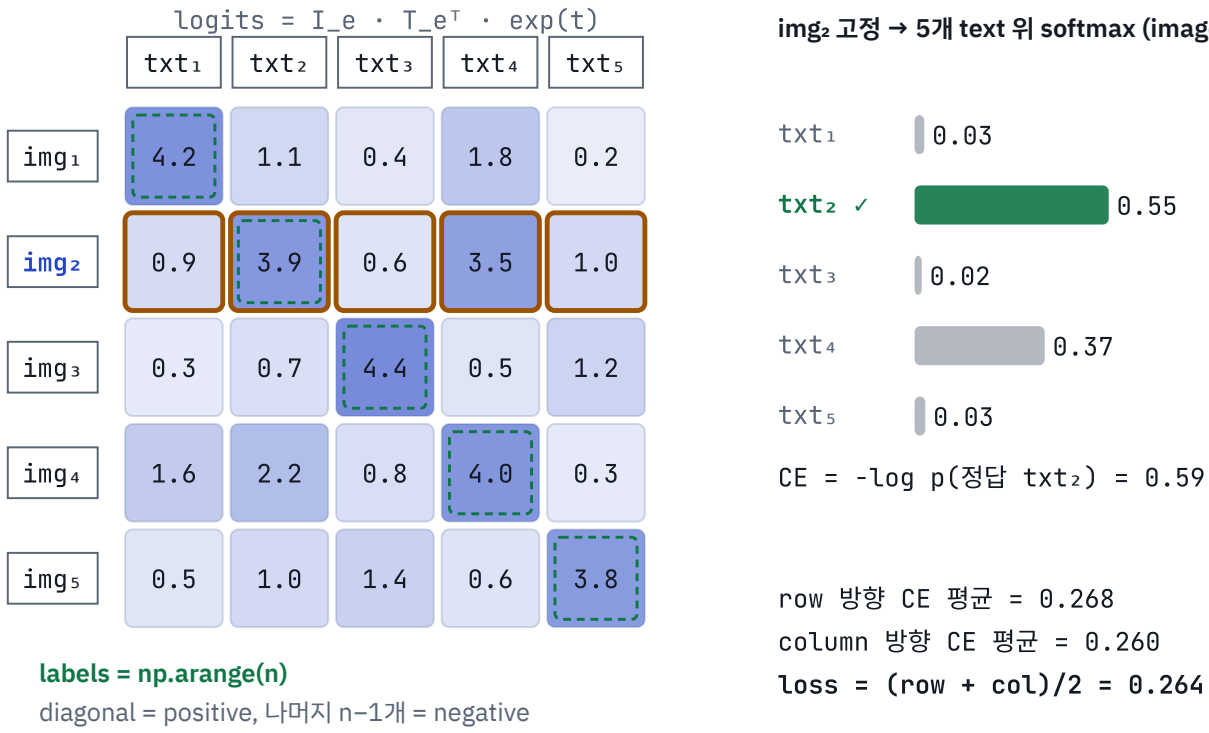
`추론` zero-shot(그 task용 labeled 데이터로 추가 학습 없이 바로 푸는 것) 분류 (p.33): class 이름을 "A photo of a {object}." prompt로 만들어 text embedding → 이미지 embedding과 similarity가 가장 높은 prompt의 class를 답으로. 학습된 classifier 대신 text embedding이 classifier야 ("Classification is performed by image-text matching instead of a trained classifier"). class 목록이 바뀌면 텍스트만 바꾸면 돼.

`포인트` "맞는 쌍 vs 틀린 쌍"만 구별하는 contrastive가 caption 예측보다 약 **4× data-efficient**하고, 단어 순서를 무시하는 Bag-of-Words text encoder가 Transformer language model보다 나았어 (p.36). "The success of CLIP comes less from sophisticated language modeling and more from contrastive alignment between images and natural language." 학습 데이터는 400 million image–text pairs (p.37).



CLIP 전체 그림. (1) 학습: text encoder와 image encoder 출력으로 **N×N similarity 행렬**(칸 = $I_i \cdot T_j$)을 만들고, **대각선(진짜 짝)은 올리고 나머지는 내리는** contrastive 학습. (2) class 이름을 `A photo of a {object}.` 문장에 넣어 text embedding으로 만들어 두고, (3) 새 이미지와 가장 비슷한 문장을 고르면 추가 학습 없이 **zero-shot 분류**가 돼. `L05 p.33`

인터랙티브 **CLIP의 n×n 행렬: row와 column을 각각 softmax하는 symmetric loss** illustrative



row 보기 (image → text) | column 보기 (text → image) | index 2

batch에 (image, caption) 짝이 5개 있는 예시야 (숫자는 지어낸 개념 그림). 행렬 칸 (i, j) = 이미지 i와 text j의 similarity 점수 (cosine × `exp(t)`). 초록 점선 칸 = 대각선 = 진짜 짝(positive), 나머지 칸 = negative야. **row 보기**: 이미지 하나를 고정하고 그 행을 softmax하면 "이 이미지의 caption은 5개 중 어느 것?"(image→text) 확률이 막대그래프로 나와. **column 보기**: text 하나를 고정하고 그 열을 softmax하면 반대 방향(text→image) 문제야. 정답 칸 확률이 클수록 CE = −log p(정답)가 작아져. 최종 loss는 행 방향 CE 평균과 열 방향 CE 평균을 다시 평균한 값(symmetric)이야. 칸을 눌러도 선택돼. 슬라이드 코드는 `loss_i`(axis=0), `loss_t`(axis=1)로 쓰는데 axis가 어느 방향인지는 정의가 없어서 해석이 갈려. 시험엔 '`labels = np.arange(n)` + 두 방향 CE 평균 = symmetric'만 확실히 기억해. (L05 p.33, p.35)

SupCon

Khosla, NeurIPS 2020 · p.38-42

한 줄 class label이 있을 때의 contrastive learning. 같은 class는 전부 positive로 당기고, 다른 class는 negative로 밀어.

문제 SimCLR·MoCo는 positive가 "같은 사진의 다른 view" 하나뿐이라 같은 class의 다른 사진도 negative로 밀어(false negative). label이 있으면 피할 수 있어.

핵심 "Treats all samples from the same class as positive pairs." (p.38) p.40 요약: SimCLR의 InfoNCE를 그대로 쓰되, 같은 class 샘플 전부를 positive로 놓고 positive마다 InfoNCE를 계산해 평균해.

loss (p.40) n개 (image, label)에 augmentation 두 번 → view 2n개.

$$L_{supcon} = -\Sigma_{i=1..2n} \; 1/(2|N_i|-1) \cdot \Sigma_{j \in N(y_i),\, j \neq i} \log p_{ij}$$

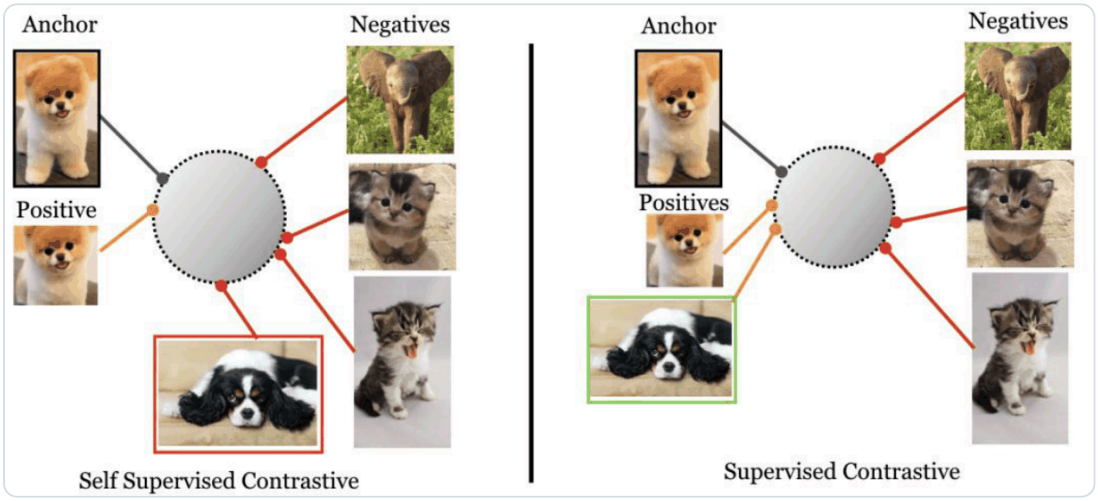
$$p_{ij} = \exp(z_i \cdot z_j/\tau) \; / \; \Sigma_{k \in I,\, k \neq i} \exp(z_i \cdot z_k/\tau)$$

- i: anchor view, z_i: 그 normalized embedding. $j \in N(y_i)$, $j \neq i$: anchor와 label이 같은 다른 view 전부 ("For each positive").

- 분자는 "Positive similarity"(클수록 loss ↓), 분모는 anchor 자신을 뺀 모든 view와의 similarity 합(다른 positive도 분모에 들어가. negative가 가까우면 loss ↑).
- $|N_i|$: batch 원본 n장 중 anchor와 같은 class인 사진 수 (anchor 자기 사진도 포함, "A set of Positives of the same class as Anchor"). 사진마다 view가 2개라 자기를 빼면 positive view는 $2|N_i|-1$개고, 이걸로 나눠 평균해.
- positive가 1개면 SimCLR의 InfoNCE와 같아져 (InfoNCE의 일반화). 슬라이드는 soft nearest-neighbor loss(L03 p.29의 Soft-NN loss: 같은 class 이웃 전부를 분자에 넣는

> **예시로 보면**
> 사진 4장(강아지 2, 고양이 2) → view 8개, anchor = 강아지1의 view 하나. SimCLR면 positive는 강아지1의 다른 view 1개, negative는 강아지2 view까지 포함한 6개야. SupCon은 $|N_i|$ = 2라 positive가 2·2−1 = 3개(강아지1 다른 view + 강아지2 view 2개), negative는 고양이 view 4개야.

결과 CE(cross-entropy로 학습하는 보통 분류)보다 정확도는 "only by a small amount" 나아(p.42). 대신 ImageNet-C(noise, blur, contrast 변화 같은 자연스러운 손상을 가한 ImageNet)에서 더 robust하고 hyperparameter 변화에 덜 민감해. "Contrastive objectives remain beneficial even with class labels." (p.41)



왼쪽(Self Supervised Contrastive)에선 anchor 강아지와 같은 '개' class인 다른 사진(빨간 테두리)이 **negative**로 밀려나. 잘못 밀어내는 false negative야. 오른쪽(Supervised Contrastive, SupCon)은 label을 써서 그 사진(초록 테두리)까지 **positive**로 당겨. positive가 1개에서 여러 개로 늘어나는 걸 봐. `L05 p.39`

NNCLR → All4One

`Estepa, ICCV 2023 · p.43-53`

질문 "Can the memory bank provide more than negatives?" (p.43) 창고에서 **positive**도 꺼내자는 거야.

NNCLR 과거 embedding을 **support set**에 쌓아 두고, 첫 view와 가장 가까운 이웃(nearest neighbor) 1개를 두 번째 view의 positive로 써 ("The nearest neighbor replaces one augmented view in the positive pair", p.47). 같은 사진 변형본보다 다양한 positive를 얻지만, 고른 이웃이 다른 class면 신호가 오염돼.

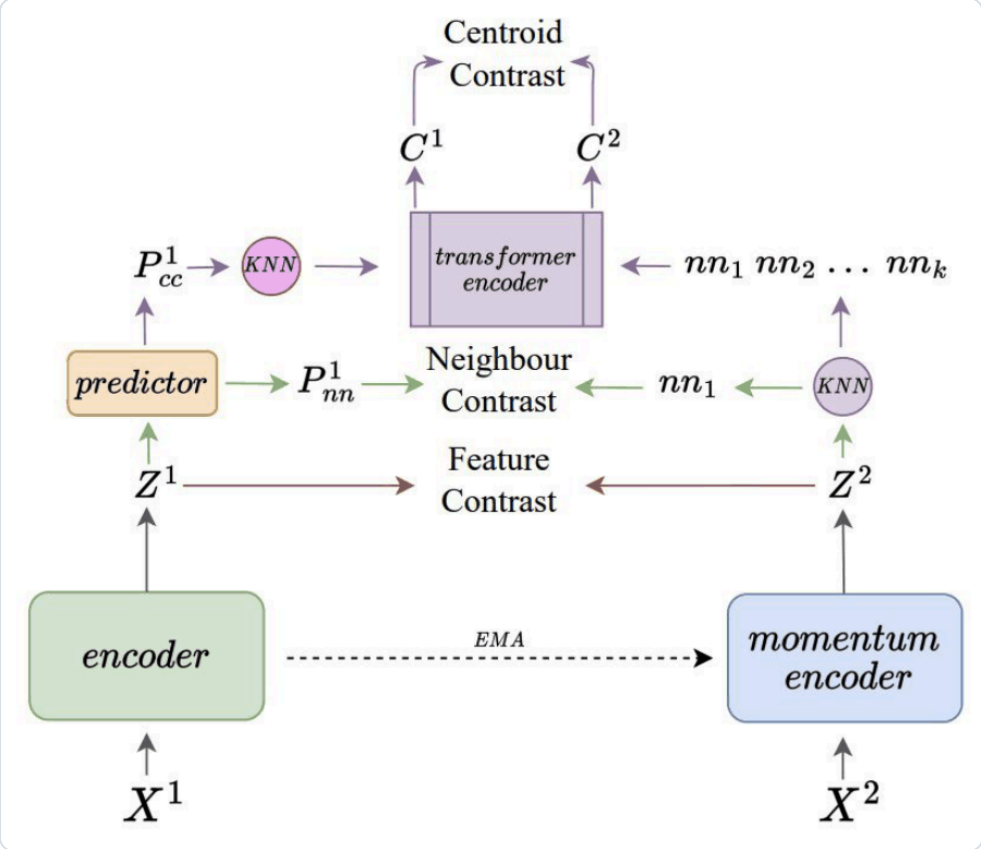
`All4One` 이웃 K개를 작은 Transformer에 넣고 **self-attention**(원소들이 서로를 보고 중요도 가중치를 학습해 섞는 연산)으로 합쳐. Transformer의 첫 출력을 centroid로 쓰고, 두 view의 centroid끼리 InfoNCE를 계산해 ("K neighbors, one contrastive comparison", p.48). "NNCLR uses one neighbor. All4One summarizes multiple neighbors into one representation." (p.45)

"The centroid is produced by self-attention. It is not an average or a cluster center." (p.48) 이웃마다 중요도를 달리 줄 수 있어서 틀린 이웃의 영향을 줄일 수 있어.

`loss 3개` $L_{All4One} = \sigma L_{NNCLR} + \kappa L_{Centroid} + \eta L_{Red}$ (σ, κ, η = 가중치). "All4One combines three losses defined on three different representations." (p.46)

- **Neighbor Contrast**: nearest neighbor 1개 ↔ 다른 view의 prediction(학습되는 encoder 뒤에 붙은 predictor라는 작은 MLP의 출력. "online prediction from the second view", p.47). direct positive alignment.
- **Centroid Contrast**: K개 이웃을 self-attention으로 요약한 centroid끼리. contextual neighbor representation.
- **Feature Contrast** (Barlow Twins식, L04): 두 view의 원래 feature Z로 cross-correlation 행렬(두 view에서 차원 i와 차원 j가 batch 전체에서 얼마나 같이 움직이는지 모은 D×D 표)을 만들어. 대각선 → 1(같은 차원은 view가 바뀌어도 일관), 비대각 → 0(차원끼리 중복 제거, redundancy reduction).

"Neighbor losses compare samples. Feature contrast compares feature dimensions." (p.49)



All4One 구조. 왼쪽 encoder(+predictor)가 gradient로 학습되고, 오른쪽 momentum encoder는 **EMA**로만 따라가(점선 화살표). 그 위에 loss 세 개가 서로 다른 표현에 걸려 있어: **Neighbour Contrast**(P_{nn} ↔ nearest neighbor 1개), **Centroid Contrast**(KNN 이웃들을 transformer로 요약한 C^1 ↔ C^2), **Feature Contrast**(Z^1 ↔ Z^2, Barlow Twins식). `L05 p.46`

PAPN

Tan, ICCV 2025 · p.54-64

질문 "Can cluster-based SSL preserve fine-grained visual details?" (p.54)

문제 fine-grained recognition은 "새인가?"가 아니라 "어떤 종의 새인가?"야. 부리·머리·날개·꼬리 무늬 같은 작은 part가 답을 갈라 (p.56). 그런데 강한 augmentation은 **얕은 stage** feature를 크게 바꿔 (p.58).

- **stage**: backbone을 몇 덩어리로 나눈 구간. 앞쪽(shallow)은 색·에지·질감, 뒤쪽(deep)은 전체 의미를 담아.
- 강한 aug면 shallow layer는 같은 새의 두 사진을 다른 개체로 보고, deep layer는 덜 흔들려. → "One augmentation strength does not fit every stage."

핵심 두 아이디어.

1. **SPA** (Stage-wise Progressive Augmentation): 얕은 stage엔 약한(mild), 깊은 stage엔 강한(strong) augmentation view로 stage마다 positive pair를 맞춰. "Positive pairs are created based on the intensity of augmentation that each stage can withstand." (p.59)
2. **PHP** (Prototype-based Holistic-Partial learning): 학습되는 prototype으로 part(부위) feature를 뽑아 whole feature(물체 전체 모양을 담은 global feature, "overall object appearance")와 이어 붙여(concat) InfoNCE에 넣어. "Do not choose between global and local features—contrast both together." (p.61)

SCL-RHE

Long, ECCV 2024 · p.65-75

질문 "What if a mini-batch contains Imprecise Class Labels?" (p.65) SupCon은 label로 pair를 정하니까, label이 틀리면 pair 관계도 틀려.

용어 (p.67, p.69) **assigned label (AL)** = 사람이 붙인 label, **latent label (LL)** = 진짜 class. **false positive** = AL 같고 LL 다름 → 잘못 당김(pull). **false negative** = AL 다르고 LL 같음 → 잘못 밀어냄(push).

문제 틀린 신호의 99% 이상이 **false positive**야 (p.73). 사람은 시각적으로 비슷한 class를 헷갈리니까 잘못 붙은 샘플이 anchor와 이미 가까워서(p.68) 쉬운 positive(easy positive)처럼 보여. confident한 pair에 가중치를 더 주는 기존 noise 대처법은 이걸 오히려 강화해 (p.71).

핵심 similarity가 **높은** positive(easy positive)는 sampling weight를 **낮게**, 낮은 positive(hard positive: 같은 class인데 지금은 멀리 있는 샘플)는 높게 (p.72).

$$q^{+}(x^{+}) := q(x^{+} \mid z(x) = z(x^{+})) \propto \exp(-\beta \cdot f(x)^{\top} f(x^{+})) \cdot p_{x}^{+}(x^{+})$$

q^{+}는 SCL-RHE가 positive를 뽑는 분포야. $z(x) = z(x^{+})$는 anchor와 positive 후보가 같은 실제 class라는 조건이야 ("share the same actual class"). 슬라이드의 $1/e^{\beta\cdots}$를 $\exp(-\beta\ldots)$로 쓴 거라 같은 식이야. p_{x}^{+}는 true positive 분포, $f(x)^{\top} f(x^{+})$는 anchor–positive similarity야. similarity가 클수록 $\exp(-\ldots)$가 작아져 weight가 낮아지고, β는 그 세기야 ($\beta = 0$이면 원래 SupCon처럼 모두 같은 weight).

왜 돼? easy positive는 이미 가까워 새로 배울 게 적고, human labeling error가 섞여 있을 수 있어 (p.72). 그래서 덜 믿고 정보가 많은 hard positive를 더 봐.

RL로 번역

- CURL은 이 수업에 직접 나오는 RL 논문이야. pixel 관측 RL에 contrastive loss를 보조 loss로 붙였어 (원 논문은 SAC 위에 얹었는데, 슬라이드 밖 디테일).
- MoCo의 momentum update(예: m = 0.999, 논문 값)는 SAC target network의 soft update와 같은 꼴이야. 둘 다 "학습되는 network를 천천히 따라가는 복사본"으로 target(여기선 key)이 출렁이지 않게 해. 계수 방향만 반대: m은 **유지** 비율이라 1에 가깝고, soft update 계수는 **섞는** 비율이라 0에 가까워 (이 계수를 τ로 쓰기도 하지만 temperature τ와는 다른 거야).
- `k.detach()` 는 TD target에 gradient를 안 흘리는 semi-gradient와 닮았어.
- queue는 off-policy replay buffer의 staleness trade-off와 닮았어: 조금 오래된 key를 재사용하지만, key encoder가 천천히 변해서 queue 안 key들은 서로 일관돼.

시험 함정

- MoCo: `k.detach()` → gradient는 **query** encoder로만. key encoder는 backprop 없이 momentum(EMA)으로만 갱신. "key encoder도 backprop으로 학습"은 False.
- MoCo `labels = zeros(N)` (positive를 0번 칸에 붙여서 정답이 전부 0) ↔ CLIP `labels = np.arange(n)` (정답 = 대각선). 바꿔치기 단골.
- temperature 코드 모양: MoCo `logits/t` (나누기, 정해 둔 t) ↔ CLIP `* np.exp(t)` (곱하기, 학습되는 t).
- p.14 표: MoCo key 상태는 fresh가 아니라 **consistent**. fresh는 end-to-end, stale은 memory bank.
- MoCo v2 = 새 architecture가 아니라 MLP head + 센 aug. queue와 momentum encoder는 그대로.
- DeepCluster = offline, SwAV = online. SwAV는 명시적 negative 없음 (Feature ↔ Prototype).
- SupCon은 CE보다 정확도가 "조금"만 나아. 장점은 robustness(ImageNet-C)와 hyperparameter 둔감함.
- SCL-RHE: similarity 높은 positive → 가중치 **낮게**.

다음으로

여기까지는 label이 아예 없거나(self-supervised) 다 있는(SupCon) 경우야. 현실은 label 조금 + unlabeled 많이고, 둘을 같이 쓰는 게 semi-supervised learning이야. → L06

셀프 체크

빈칸 In MoCo, the key encoder is updated by a _____ update, not by back-propagation.

momentum (EMA, p.11–12). key encoder는 `f_k.params = m*f_k.params+(1-m)*f_q.params` 줄로만 바뀌고, `k.detach()` 때문에 key 쪽으론 gradient가 안 흘러.

T/F In the CLIP pseudo code, labels = np.zeros(n).

False. `np.arange(n)` (대각선). i번 이미지의 정답이 i번 텍스트라 정답 번호가 0, 1, …, n−1이야. `zeros(N)` 은 MoCo: positive를 늘 0번 칸에 붙여서 정답이 전부 0 (p.35, p.12).

MCQ SwAV compares: (A) Feature ↔ Feature (B) Feature ↔ Prototype (C) Image ↔ Text (D) Query ↔ Queue

(B) (p.30). SwAV는 feature를 학습되는 prototype과 비교해 code를 만들고, 다른 view의 code를 맞혀. (A)는 일반 contrastive, (C)는 CLIP, (D)는 MoCo.

L06 Semi-Supervised Learning I 76p · consistency

질문 **label이 조금 있을 때 unlabeled를 어떻게 써먹나? ① consistency**

여기서부터 상황이 바뀌어. L02–L05에서 주로 본 self-supervised는 label이 **하나도 없는** 데이터로 쓸 만한 representation을 미리 배우는 게 목표였어. L06–L07의 **semi-supervised**(반지도 학습)는 사람이 단 정답(label)이 있는 데이터(labeled)가 **조금**, label 없는 데이터(unlabeled)가 **많이** 있는 상황이야. 목표는 그 task(예: 사진 분류) 자체를 잘 푸는 모델이야.

예를 들어 사진 10만 장을 모았는데 class를 달아 준 건 1천 장뿐이라고 해 봐. 1천 장만으로 학습하면 아까운 9만 9천 장이 놀아. 이 강의는 그 unlabeled를 학습에 끌어들이는 첫 번째 방법, **consistency regularization**을 다뤄.

label 부족 해법 4개

p.4–7

1. **pre-training + fine-tuning** (p.4): label 없는 큰 데이터로 특정 task에 묶이지 않은(*task-agnostic*) 모델을 먼저 학습해. 그다음 적은 labeled로 원하는 task(downstream task)에 맞게 조금 더 학습(fine-tune)해. L02–L05에서 주로 본 방법들이 이 앞 단계야.
2. **active learning** (p.5): 다음에 사람이 label을 달 unlabeled 샘플을 **골라 주는** 방법. 예산(limited budget)이 정해져 있으니 가장 가치 있는 샘플부터 달자는 거야. 사람 labeling은 여전히 필요해.
3. **pre-training + dataset auto-generation** (p.6): 이미 잘 학습된 모델에게 labeled 샘플을 **자동으로 잔뜩 만들게** 해. 예시 몇 개만 보고 새 task를 푸는 few-shot learning이 잘 되면서 언어 쪽에서 특히 인기야.
4. **semi-supervised learning** (p.7): labeled와 unlabeled를 **같이**(together) 학습해.

차이 (1)은 pre-train → fine-tune의 **2단계 순차**(sequential)야. (4) semi-supervised는 두 데이터를 **한 번의 학습에서 동시에**(joint) 써. (2)는 "어떤 샘플에 label을 달지 고르기"라서 사람 손이 계속 필요해.

목표식

p.8–10

한 줄 labeled에서 오는 loss와 unlabeled에서 오는 loss를 더해. 단, unlabeled 쪽 무게는 처음엔 작게, 갈수록 크게.

식 $L = L_s + \mu(t) \cdot L_u$ (semi-supervised objective). 세 조각이야 (p.8).

- L_s (supervised loss): label 있는 샘플에서 예측과 정답을 비교해. 보통 cross-entropy(CE).
- L_u (unsupervised loss): label 없는 샘플에서 계산하는 loss. 정답이 없으니 "무엇과 비교하나"를 따로 설계해야 해.
- $\mu(t)$ (weighting term): L_u에 곱하는 가중치. t는 학습 시간(epoch)이고, μ(t)는 **시간이 갈수록 커져** ("increasing in time", p.10). 작게 시작해서 점점 키우는 걸 **ramp-up**이라고 해.

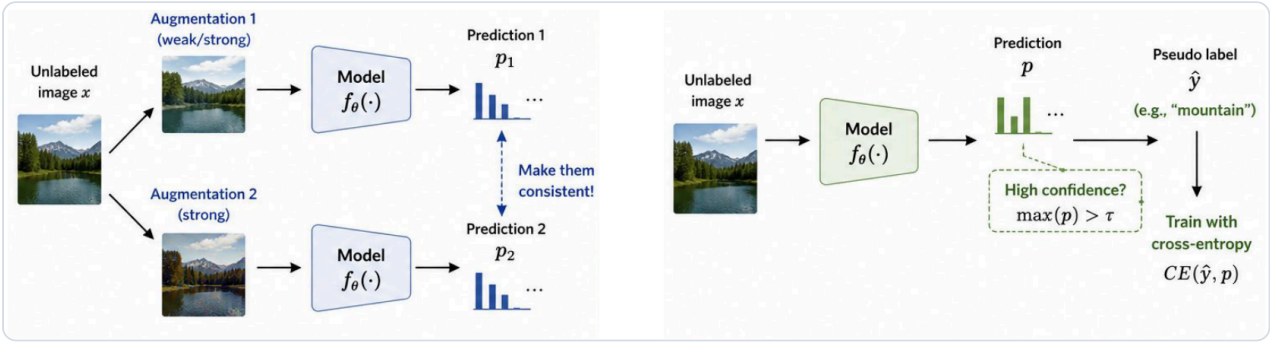
왜 키우나 L_u의 비교 대상(target)은 결국 **모델 자신의 예측**에서 나와. 학습 초반 모델은 거의 랜덤이라 그 예측도 엉망이야. 이때 L_u를 세게 주면 틀린 예측을 정답처럼 더 굳히게 돼.

그래서 처음엔 labeled(L_s)로 기본기를 잡고, 예측을 믿을 만해질수록 unlabeled 몫을 키워. Π-model 코드(p.18)에선 같은 가중치를 `w(t)`라고 써.

핵심 질문 "L_u를 어떻게 만드나?"가 semi-supervised의 핵심 질문이야 (p.9). 답은 두 갈래야.

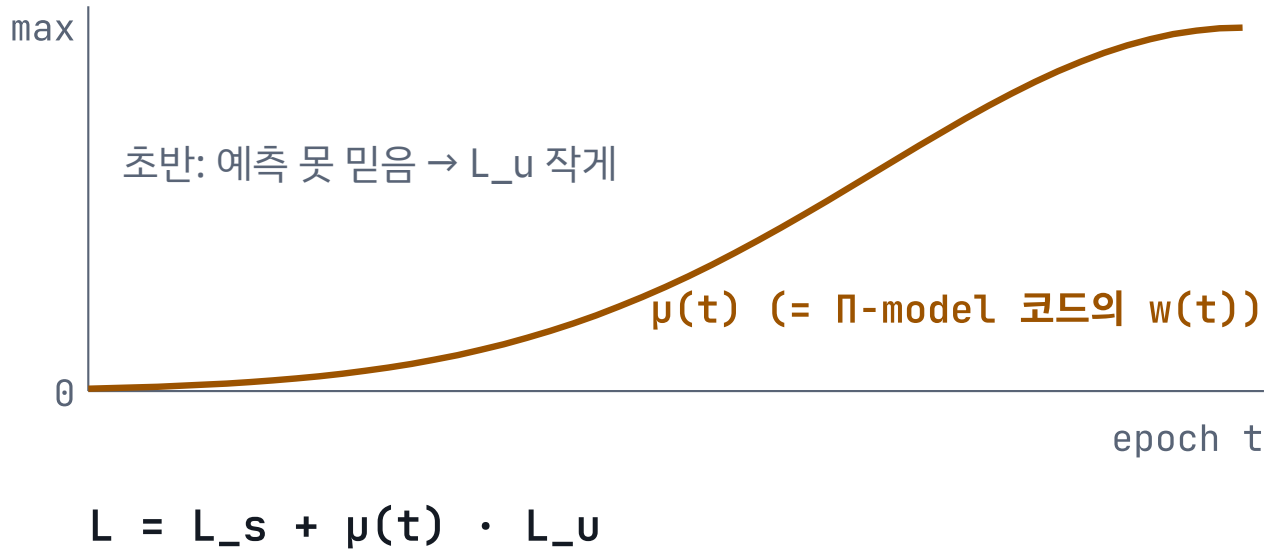
- **Consistency regularization** (이번 L06): 같은 사진을 두 가지로 변형(augmentation)해서 넣고, 두 예측 확률 p_1, p_2가 **서로 같아지게** 해. "정답이 뭔지"는 정하지 않고 "두 예측끼리 일치"만 요구해.
- **Pseudo label** (다음 L07): 예측 p가 충분히 확신에 차 있으면(`max(p) > τ`, τ는 confidence threshold) 확률이 가장 높은 class를 **가짜 정답 ŷ**로 정해 버려. 그다음 labeled처럼 `CE(ŷ, p)`로 학습해.

공통점 둘 다 사람이 단 label 없이, **모델 자신의 예측**에서 학습 신호를 만들어.



p.9 Key Question 그림. **왼쪽 consistency regularization**: label 없는 사진 x를 두 가지로 augmentation(위는 weak 또는 strong, 아래는 strong)해서 같은 모델 f_θ에 넣고, 두 예측 막대그래프 p_1, p_2를 "Make them consistent!" 서로 맞춰. 정답 class는 정하지 않아. **오른쪽 pseudo label**: 예측 p의 최댓값이 threshold τ를 넘으면(max(p) > τ) 그 class를 가짜 정답 ŷ(예: "mountain")로 굳혀서 CE(ŷ, p)로 학습해. target이 '다른 예측'이냐 '확정된 label'이냐가 갈리는 지점을 봐. `L06 p.9`

개념 그래프 **μ(t): unsupervised 항 가중치는 시간이 갈수록 커진다** illustrative



가로축은 학습 시간(epoch t), 세로축은 unlabeled loss L_u에 곱하는 가중치 μ(t)야. 슬라이드에는 "increasing in time"이라고만 나와. 곡선 모양은 **예시**(Gaussian 형태 ramp-up)일 뿐이야. 왜 이렇게 키우는지는 본문 "왜 키우나" 참고. 이 가중치를 p.10에선 μ(t), Π-model 코드(p.18)에선 w(t)라고 써. (L06 p.10, p.18)

가정 H1–H4

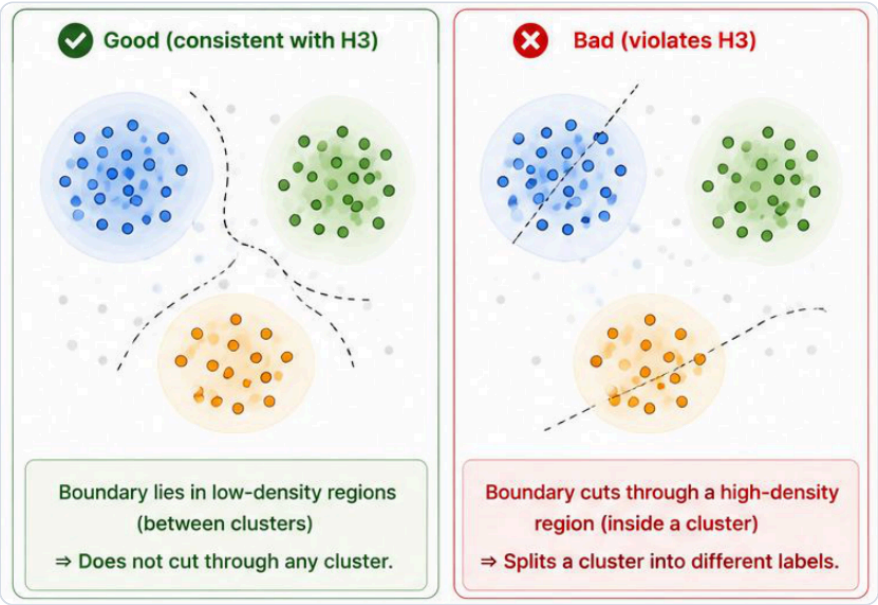
p.11-14

문제 unlabeled는 정답이 없는데 왜 도움이 될까? label이 없어도 "데이터가 어디에 어떻게 모여 있나"는 알 수 있어. 아래 가정이 맞다면 이 모양 정보가 "boundary를 어디에 그어야 하나"를 알려줘.

용어 여기서 "점"은 사진 한 장을 숫자 벡터로 바꿔 공간에 찍은 거야. **decision boundary**는 모델이 class를 가르는 경계선이야 (선 이쪽은 A, 저쪽은 B로 예측). **density**(밀도)는 그 근처에 점이 얼마나 빽빽한지야.

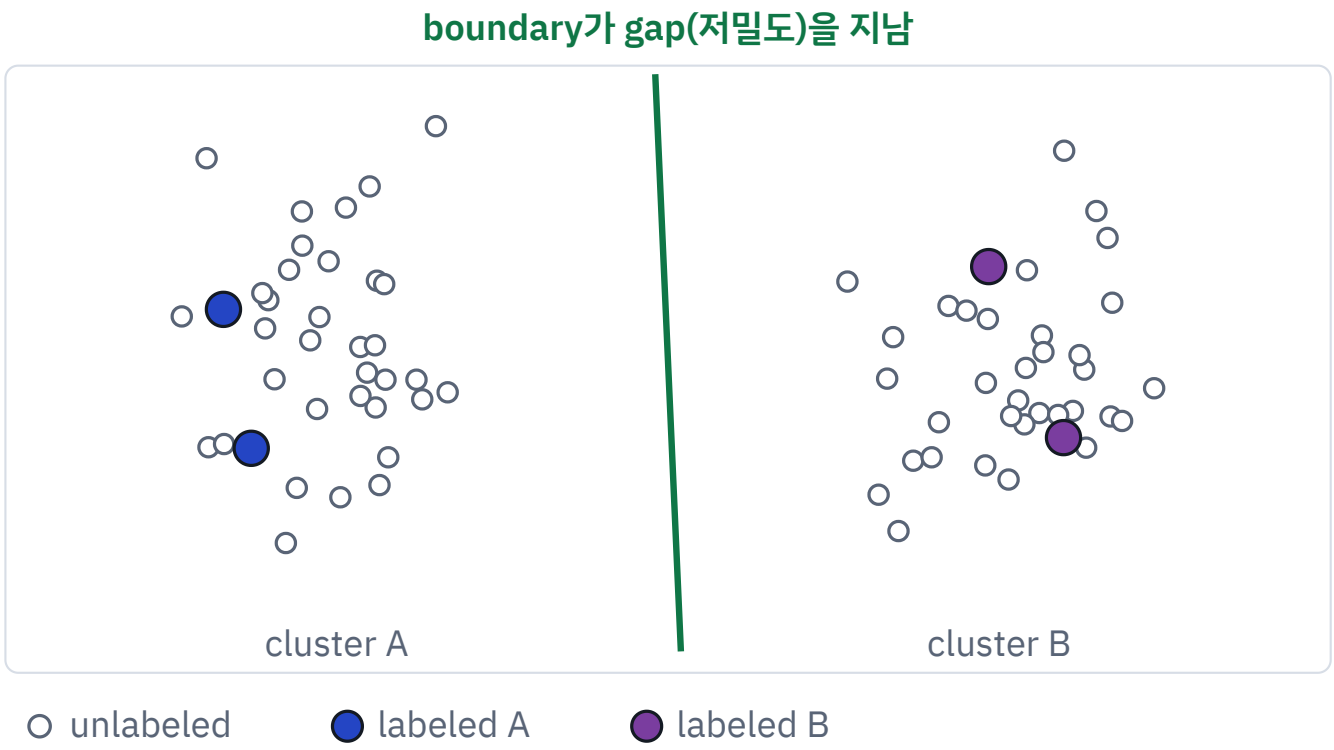
- **H1 Smoothness**: "Nearby samples should have similar predictions. Small perturbations should not change the label." 가까운 샘플은 예측도 비슷해야 하고, 살짝 흔들어서(perturbation) label이 바뀌면 안 돼. 고양이 사진을 조금 자르거나 밝기를 바꿔도 여전히 고양이야.
- **H2 Cluster**: "Samples within the same cluster are likely to share the same label. Decision boundaries should not split a cluster." cluster는 점들이 뭉친 덩어리야. 한 덩어리는 대체로 한 class라서, 덩어리 안에 labeled 점이 하나만 있어도 덩어리 전체의 label을 짐작할 수 있어.
- **H3 Low-density separation**: "Decision boundaries should lie in **low-density** regions." boundary는 점이 **드문** 곳, 즉 cluster 사이 빈 공간을 지나가야 해. cluster를 가로지르면 안 돼 ("Separate clusters rather than cutting through them").
- **H4 Manifold**: "High-dimensional data lie on a **low-dimensional** manifold." 224×224 컬러 사진은 숫자 15만 개짜리 벡터지만, 진짜 사진들은 그 거대한 공간 속 얇은 **저차원 곡면**(manifold) 위에만 모여 있어. 비슷한 샘플은 이 곡면을 따라 이어져 있어.

연결 H1이 consistency regularization의 직접 근거야. H2와 H3는 같은 직관을 두 쪽에서 본 거야 (H2는 "한 cluster = 한 label", H3는 "boundary는 빈 곳에"). 뒤에 나오는 ICT가 H3와 맞물려.



H3(low-density separation) 그림. 색깔 점 = 데이터, 동그란 무리 = cluster, 점선 = decision boundary(class를 가르는 선). 왼쪽(Good)은 점선이 cluster 사이 **빈 곳**(점이 드문 low-density)을 지나서 어떤 cluster도 안 잘라. 오른쪽(Bad)은 점선이 cluster **안**(점이 빽빽한 high-density)을 가로질러서, 한 cluster가 두 label로 쪼개져. 이게 H3 위반이야. L06 p.13

인터랙티브 **Low-density separation: boundary는 unlabeled가 비어 있는 곳으로** illustrative



gap을 지나는 boundary | cluster를 가르는 boundary

H3 만족: boundary가 점이 드문 cluster 사이를 지나니까, 각 cluster가 통째로 한 class가 돼. H2도 만족해.

작은 빈 점 = unlabeled, 색칠한 큰 점 = labeled, 굵은 선 = decision boundary야. 버튼으로 boundary를 바꿔 봐. 두 boundary 모두 **labeled 점 4개는 다 맞혀**서, label만 보면 어느 쪽이 나은지 구분이 안 돼. 하지만 unlabeled까지 보면 "cluster를 가르는 boundary"는 같은 cluster 안의 점들을 둘로 갈라서 H2(cluster)·H3(low-density)를 어겨. unlabeled의 역할은 이런 boundary를 걸러내는 거야. (개념 그림, 점은 임의 생성, L06 p.11–14)

Consistency regularization

p.15–16

한 줄 입력을 살짝 흔들어도 예측은 그대로여야 한다는 걸 loss로 만든 것. "Predictions should remain consistent under small perturbations."

식 $L_u = D(f(x), f(A(x)))$

- x: label 없는 입력 (사진 한 장)
- $A(x)$: x를 변형한 버전 (A = augmentation. 예: crop, 색 바꾸기)
- $f(\cdot)$: 모델. 출력은 class별 확률 같은 예측
- D: 두 예측이 얼마나 다른지 재는 거리. MSE, cross-entropy, KL divergence 등을 써 (p.10). 작을수록 두 예측이 같아.

perturbation 출처 (p.16) ① **data augmentation**: 입력 자체를 바꿔. ② **model stochasticity**: 모델 안의 랜덤성. 대표가 dropout이야. dropout은 학습 중 neuron 일부를 랜덤으로 끄는 거라, 같은 입력도 forward할 때마다 출력이 조금씩 달라져.

왜 돼? 식에 정답 label y가 **없어**. 그래서 unlabeled에도 계산할 수 있어. 모델은 "흔들어도 안 바뀌는" 예측을 하도록 밀려나. 이게 H1(smoothness)을 그대로 loss로 옮긴 거야.

regularization은 "모델이 엉뚱한 쪽으로 가지 않게 거는 추가 제약"이라는 뜻이야.

예시로 보면

label 없는 강아지 사진 x를 한 번은 그대로, 한 번은 좌우 반전 + 밝게 해서 넣어. 모델이 첫 번째엔 [강아지 0.6, 고양이 0.4], 두 번째엔 [강아지 0.3, 고양이 0.7]을 냈다면 D가 커. 학습은 이 차이를 줄이는 쪽으로 가. 정답이 강아지인지는 몰라도 "변형에 흔들리지 마"라는 신호는 줄 수 있어.

차이 L03–L05 contrastive는 두 view의 embedding(사진을 바꾼 벡터)을 가깝게 했어. 여기선 두 view의 **class 예측**을 같게 해.

뒤에서 바뀌는 것 이후 방법들은 세 가지 선택만 달라. ① perturbation을 어떻게 주나, ② 비교 대상(target) 예측을 어디서 가져오나, ③ D로 뭘 쓰나.

target을 어디서 얻나: Π → TE → Mean Teacher

consistency loss는 예측 두 개를 비교해. 하나는 지금 학습하는 모델의 출력 z고, 다른 하나는 "이거랑 같아져라"는 비교 대상(target) $\tilde{z}$야. 세 방법은 **$\tilde{z}$를 어디서 가져오나**만 달라.

Π-model

p.17–18 · Alg.1 p.18

한 줄 같은 입력을 noise만 바꿔 **같은 network에 두 번** 넣고, 두 출력이 같아지게 해.

어떻게 (Algorithm 1, p.18) minibatch B마다 이렇게 해.

1. $z_i \leftarrow f_\theta(g(x_i))$: 입력을 랜덤 augmentation g에 통과시키고, dropout이 켜진 network f_θ로 forward해.
2. $\tilde{z}_i \leftarrow f_\theta(g(x_i))$: 똑같은 줄을 한 번 더 ("again, with different dropout and augmentation"). 글자는 같지만 g와 dropout이 랜덤이라 값이 달라.
3. loss = supervised CE + $w(t)$ × 두 출력의 squared difference.
4. θ를 업데이트해 (예: Adam).

식 $L_U^{\Pi} = \Sigma_{x \in D}\ \mathrm{MSE}(f_\theta(x),\ f'_\theta(x))$ (p.17). f_θ와 f'_θ는 **같은 파라미터 θ**에 다른 랜덤 noise를 준 거야. 파라미터가 다른 두 모델이 아니야. 여기 D는 거리가 아니라 dataset 전체(labeled + unlabeled)야. 슬라이드에선 필기체 D로 구분해.

코드의 loss 두 항을 풀면 이래.

- $-(1/|B|)\ \Sigma_{i \in B_nL}\ \log z_i[y_i]$: supervised CE야. B_nL은 이번 minibatch 중 label 있는 샘플만. $z_i[y_i]$는 정답 class에 준 확률이라, 1에 가까울수록 loss가 작아.
- $w(t) \cdot (1/(C|B|))\ \Sigma_{i \in B}\ \|z_i - \tilde{z}_i\|^2$: consistency 항이야. $i \in B$라서 minibatch **전부**(labeled + unlabeled)에 걸려. C는 class 수라서 결국 원소별 평균 제곱 오차(MSE)야.

Temporal Ensembling

Laine, ICLR 2017 · p.19–22 · Alg.2 p.20

한 줄 두 번째 forward 대신, 그 샘플에 대해 **예전 epoch들에서 냈던 예측의 평균**을 target으로 써.

문제 Π-model은 입력마다 forward를 2번 해서 계산 비용이 커 (p.19).

핵심 "use the historical predictions of the same sample as the consistency target". 샘플마다 예측의 EMA(exponential moving average)를 저장해 두고 그걸 target으로 써.

EMA는 "기존 평균은 많이, 새 값은 조금" 섞어서 천천히 따라가는 평균이야.

어떻게 (Algorithm 2, p.20)

1. 처음에 $Z \leftarrow 0_{[N\times C]}$, $\tilde{z} \leftarrow 0_{[N\times C]}$로 표를 만들어. N은 전체 학습 샘플 수, C는 class 수야. 샘플 하나마다 예측 벡터 한 줄씩이야.
2. minibatch마다 forward는 **1번**: $z_i \leftarrow f_\theta(g(x_i, t))$. loss 식은 Π-model과 똑같아. 다만 $\tilde{z}_i$를 network가 아니라 **저장해 둔 표에서 읽어**. 그래서 $\tilde{z}$는 gradient가 안 흐르는 상수야.
3. epoch이 끝나면 $Z \leftarrow \alpha Z + (1-\alpha)z$: 이번 epoch에 모은 예측 z를 누적 평균 Z에 섞어. α는 ensembling momentum이야 ($0 \le \alpha < 1$).
4. 이어서 $\tilde{z} \leftarrow Z / (1 - \alpha^t)$: **bias correction**으로 다음 epoch의 target을 만들어. t는 epoch 번호야.

bias correction Z는 0에서 시작하니까 초반엔 값이 0 쪽으로 쏠려(bias). 예측이 매 epoch 똑같이 z라고 하면 $Z_t = (1-\alpha^t)\cdot z$가 돼. 그래서 $1-\alpha^t$로 나누면 정확히 z로 돌아와. t가 커지면 $\alpha^t \to 0$이라 보정은 저절로 사라져.

> **예시로 보면**
> α = 0.9이고 한 샘플의 예측이 계속 0.8이라고 하자. epoch 1 끝: Z = 0.9×0 + 0.1×0.8 = 0.08. 진짜 값 0.8의 1/10밖에 안 돼. $1 - 0.9^1 = 0.1$로 나누면 0.8이야. epoch 2 끝: Z = 0.9×0.08 + 0.1×0.8 = 0.152. $1 - 0.9^2 = 0.19$로 나누면 다시 0.8이야.

왜 돼? epoch마다 예측은 서로 다른 augmentation, dropout, weight에서 나왔어. 이걸 평균 내면 여러 모델을 합친 ensemble 같은 효과가 나서, 한 번 뽑은 noisy한 예측보다 안정적이야 ("More stable target than Π-model", p.22).

장단점 (p.22) 장점: forward 1번, Π보다 안정적인 target, 낮은 계산 비용. 한계: 샘플마다 EMA를 들고 있어서 N×C 메모리가 들고, target이 **epoch당 한 번만** 갱신돼서 큰 dataset으로 키우기 어려워.

Mean Teacher

Tarvainen · p.23–29

한 줄 예측 대신 **모델 weight**를 평균 내서 "teacher 모델"을 만들고, 그 teacher의 예측을 target으로 써.

문제 바로 위 TE의 한계 두 개(N×C 메모리, epoch당 1번 갱신)를 풀려고 나왔어 (p.23).

핵심 "Replace the EMA of predictions with an EMA of model weights." 모델이 두 개 있어 (student / teacher).

- **student** (weight θ): 실제로 gradient descent로 학습하는 모델이야. supervised loss + consistency loss로 학습해.
- **teacher** (weight θ′): target 예측을 만들어 주는 모델이야. 따로 학습한 큰 모델이 아니라 **student weight의 EMA**야. backprop은 안 받고 EMA로만 갱신돼 ("No backpropagation", p.24).

식 $\theta' \leftarrow \beta\theta' + (1-\beta)\theta$ (p.23). β는 EMA decay야. β = 0.99면 매 step teacher가 자기 값을 99% 유지하고 student 쪽으로 1%만 다가가. 이걸 반복하면 teacher는 최근 여러 step student weight의 평균이 돼.

어떻게 (p.24) iteration 하나는 이렇게 돌아.

1. 같은 입력에 서로 다른 perturbation(η, η′)을 줘서 student와 teacher에 각각 넣어.
2. student 예측 vs label → classification cost (label 있는 샘플).
3. student 예측 vs teacher 예측 → consistency cost ("the distance between predictions by the student and teacher").
4. **student만** backprop으로 업데이트해.
5. teacher는 `θ′ ← βθ′ + (1-β)θ`로 갱신해. **매 iteration**마다.

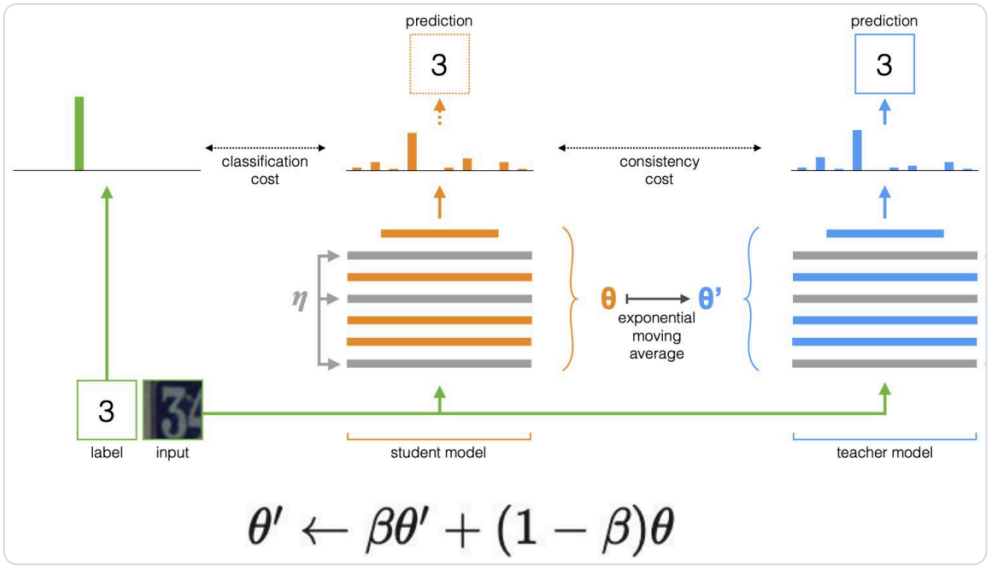
왜 돼? student weight는 step마다 흔들려. 여러 step을 평균 낸 teacher는 덜 흔들리고, 실험에서도 student보다 성능이 좋았어 (p.26). student가 좋아지면 teacher도 따라 좋아져.

장점 (p.25) stable teacher predictions, no per-sample prediction memory, teacher updated every iteration, scales well to large datasets.

문장 "Learn from a stable teacher whose parameters are the EMA of the student." (p.25) L04 BYOL의 target network, DINO의 teacher, L05 MoCo의 momentum encoder도 전부 같은 모양의 EMA 식이었어. 차이는 teacher가 내는 target이 무엇이냐야 (Mean Teacher는 class 예측 분포, DINO는 softmax 분포, BYOL은 projection 벡터, MoCo는 key).

세 방법을 표로 한 번에 놓고 보면 이래.

	Π-MODEL P.17–18 · CODE P.18	TEMPORAL ENSEMBLING LAINE, ICLR 2017 · P.19–22 · CODE P.20	MEAN TEACHER TARVAINEN · P.23–29
target	같은 net에 다른 noise로 한 번 더 forward	그 샘플의 과거 예측 EMA	EMA teacher의 예측
EMA 대상	없음	**predictions** (N×C 저장)	**weights**
갱신 시점	매 step	epoch 끝 + bias correction	매 iteration
forward	2회	1회	student 1 + teacher 1 (teacher는 gradient 없음)
약점	비용 2배	메모리, 느린 갱신, scale 어려움	teacher가 student를 너무 닮아감 (→ Dual Teacher)

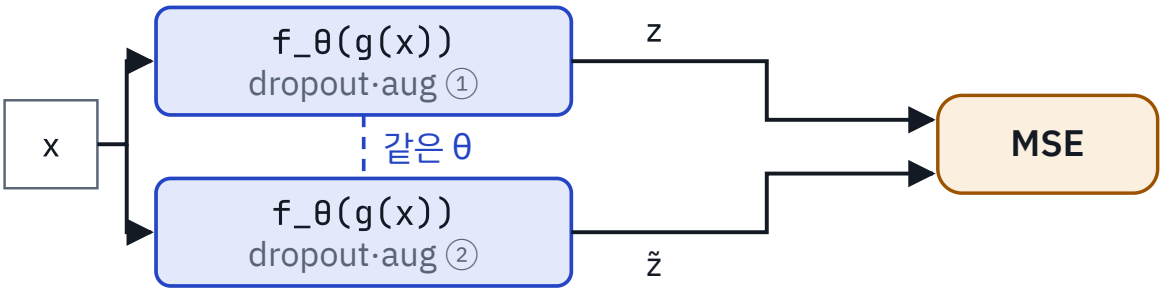


Mean Teacher. 같은 입력(숫자 3 사진)이 student(주황)와 teacher(파랑)에 들어가. student 예측은 label(초록)과 **classification cost**로, teacher 예측과 **consistency cost**로 비교돼. 가운데 θ → θ′ 화살표가 핵심이야. teacher weight는 gradient가 아니라 **student weight의 EMA**(θ′ ← βθ′ + (1−β)θ)로만 갱신돼. `L06 p.24`

구조 비교 **Π-model vs Temporal Ensembling vs Mean Teacher: target z̃는 어디서 오나**

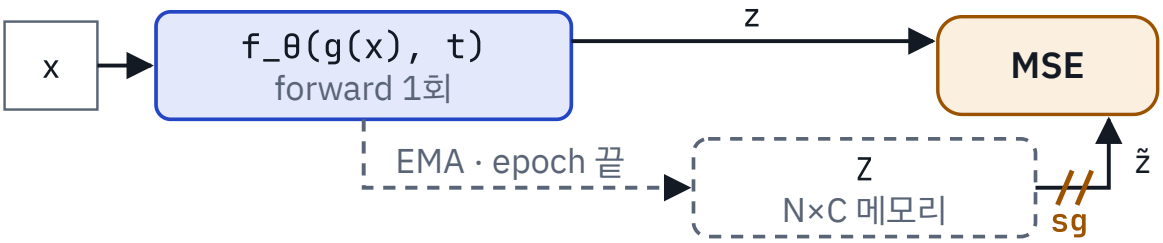
Π-model

`forward 2회 · target z̃ = 같은 θ의 2번째 forward`



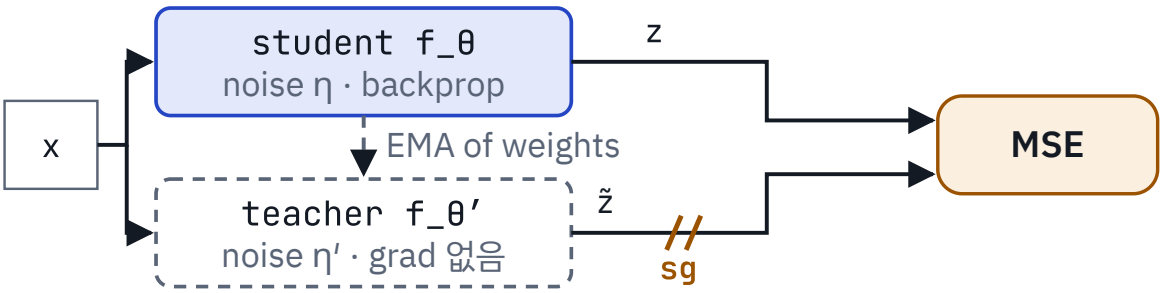
Temporal Ensembling

`forward 1회 · Z ← αZ+(1-α)z , z̃ = Z/(1-α^t)`



Mean Teacher

`forward 2회 · θ′ ← βθ′+(1-β)θ (매 iteration)`

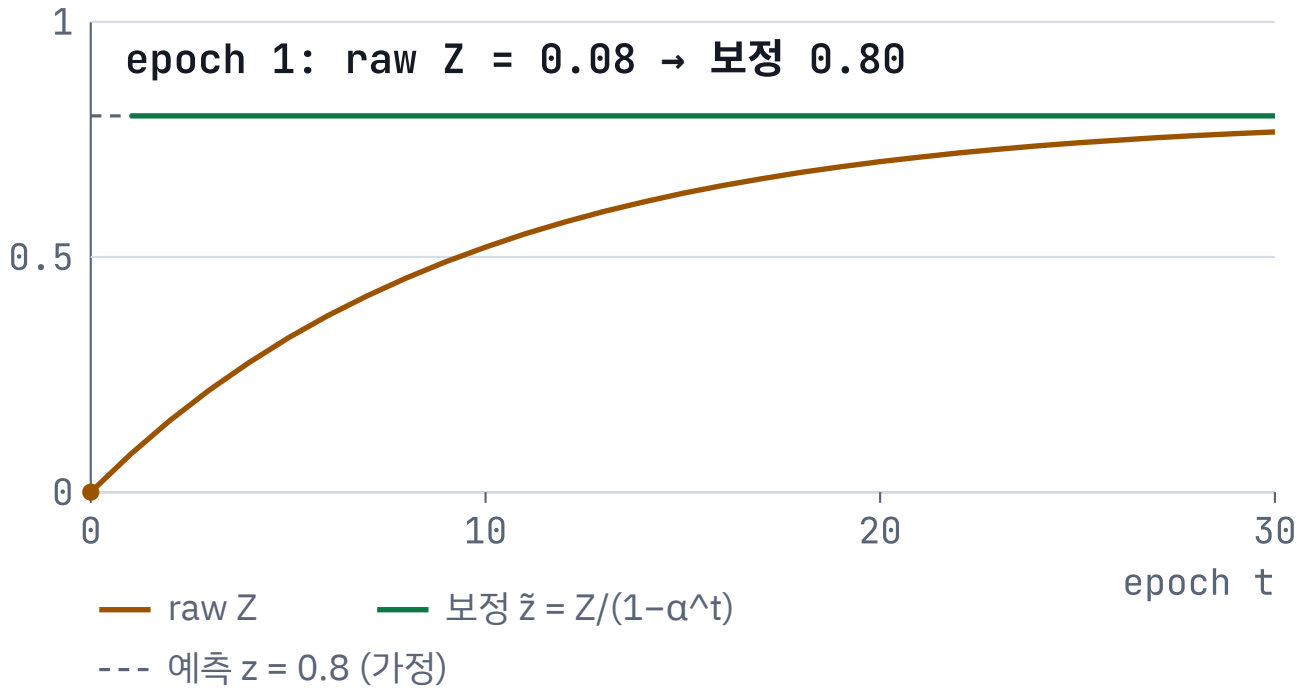


gradient로 학습 EMA로만 갱신 loss

// sg = stop-gradient

세 방법 모두 loss는 `CE(labeled) + μ(t)·MSE(z, z̃)` 야 (Π 코드에선 μ(t)를 w(t)라고 써). z는 지금 학습하는 모델의 출력이고, z̃는 "이거랑 같아져라"는 target이야. 세 방법은 **z̃의 출처** 하나만 달라. Π는 같은 네트워크(같은 θ)를 noise만 바꿔 한 번 더 forward해 (2회). TE는 forward 1회 + 샘플별 **prediction** 표 Z(N×C)를 epoch 끝에 EMA해 (슬라이드 caption: Z·z̃ 갱신은 minibatch 루프 안에서 해도 돼). MT는 **weight**를 매 iteration EMA한 teacher의 예측을 써. 실선 박스 = gradient로 학습, 점선 박스 = gradient 없이 EMA로만 갱신. `sg` (stop-gradient) 표시는 그쪽으로 gradient를 안 흘린다는 뜻이야. (L06 p.17–25)

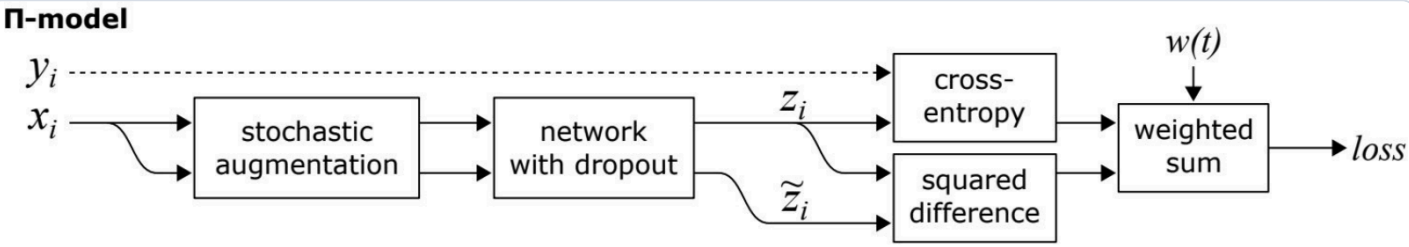
인터랙티브 **Temporal Ensembling의 bias correction: 왜 1−α^t로 나누나** illustrative



α (ensembling momentum) 0.90

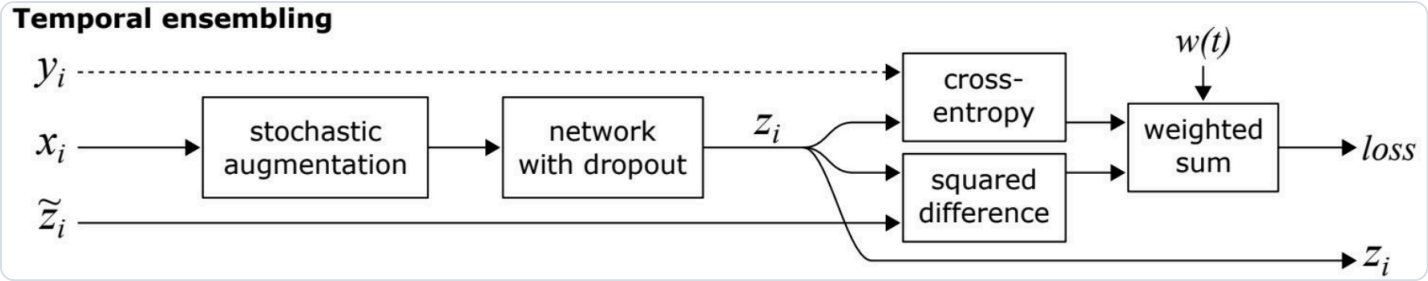
한 샘플의 예측이 매 epoch 0.8로 똑같다고 가정한 **개념 그림**이야. 누적 평균 Z는 0에서 시작하니까 `Z_t = (1-α^t)·0.8`이 돼서 초반엔 진짜 값보다 한참 낮아 (갈색 선). `1-α^t`로 나누면(bias correction) epoch 1부터 정확히 0.8이야 (초록 선). 슬라이더로 α(ensembling momentum)를 바꿔 봐. α가 클수록 과거를 오래 붙잡아서 0 쪽 bias도 오래 가고, 그래서 보정이 더 중요해. (L06 p.20)

▼ 슬라이드 그림 더 보기 (2장)



$$\mathcal{L}_u^{\Pi} = \sum_{\mathbf{x}\in\mathcal{D}} \mathrm{MSE}(f_\theta(\mathbf{x}), f'_\theta(\mathbf{x}))$$

Π-model. 같은 입력 x_i가 두 갈래로 나뉘어 **stochastic augmentation**(랜덤 변형)과 **network with dropout**을 각각 통과해. 같은 network인데 랜덤성 때문에 출력 z_i, $\tilde{z}_i$가 조금 달라. z_i는 label y_i와 cross-entropy(label 있을 때만이라 점선), 두 출력끼리는 squared difference로 비교하고, w(t)로 가중합해 loss를 만들어. 한 step에 forward가 **2번**이야. L06 p.17



Temporal Ensembling. Π-model 그림과 비교해 봐. network를 지나는 갈래가 **하나**뿐이야(forward 1번). target $\tilde{z}_i$는 network 출력이 아니라 **왼쪽 바깥에서 들어오는 입력**이야. 저장해 둔 과거 epoch 예측들의 평균(EMA)이지. 오른쪽 아래로 나가는 z_i는 그 평균을 갱신하는 데 쓰여. 그래서 target은 epoch당 한 번만 바뀌어. `L06 p.19`

`Mean Teacher 실전 팁` (p.23–29, 주로 p.28)

- **input augmentation은 필수**야 (random flip, Gaussian noise). dropout은 **student에만** 걸고, teacher엔 필요 없어.
- **β는 초반 0.99 → 후반 0.999**. 대략 최근 `1/(1-β)` step을 평균 낸다고 보면 돼 (0.99면 약 100 step, 0.999면 약 1000 step. 슬라이드 밖 어림셈). 초반엔 student가 빨리 좋아지니 teacher도 빨리 따라가야 해. 후반엔 student가 수렴하니 teacher를 더 안정적으로 둬 ("Increase the teacher stability as the student converges").
- **consistency loss는 MSE가 KL보다 나아**. 그래서 이후 많은 방법이 MSE를 기본으로 썼어.

perturbation을 어떻게 주나

앞의 세 방법은 target 쪽을 고쳤어. 이번엔 반대쪽, **입력을 어떻게 흔드나**를 고쳐. 흐름은 "network 안 noise(dropout) → 입력에 센 augmentation → 가장 나쁜 방향(adversarial) → 두 입력 섞기(MixUp) → 질 좋은 의미적 augmentation"이야.

- **Noisy samples as targets** `p.30–31`
  같은 모델로, 원본과 strong augmentation 버전의 예측을 같게.
  `쉽게` unlabeled 사진에 **센 augmentation**(strong augmentation)을 줘. 사진이면 mixup, RandAugment, 글이면 back-translation(다른 언어로 번역했다 되돌리기) 같은 거야.
  Π-model과 차이 (p.31): 슬라이드는 Π의 perturbation을 network 안의 stochastic noise(dropout)로 보고, 여기선 입력에 주는 **센** augmentation을 써. Π도 g(x)로 가벼운 augmentation은 하지만 주인공이 다르다는 뜻이야. Mean Teacher와 차이: EMA teacher를 두지 않고, 같은 모델 하나로 원본과 변형본을 맞춰 (self-consistency).

- **Adversarial Training** `Goodfellow, ICLR 2015 · p.32–33`
  모델이 제일 헷갈리는 방향(worst-case)으로 noise를 넣고도 **정답(GT)**을 맞히게. FGSM(Fast Gradient Sign Method) `r ≈ ε·sign(g)`.
  `쉽게` adversarial perturbation은 "모델 예측을 **가장 크게** 바꾸는 방향으로 준 아주 작은 noise"야. 사람 눈엔 안 보이는 noise만 더해도 panda 사진을 99.3% 확신으로 gibbon이라고 틀려 (p.32).
  `식` 크기 제한 $\|r\| \le \varepsilon$ 안에서 정답과 예측의 차이 D를 가장 키우는 r을 찾아 ($r_{adv} = \text{argmax } D$). 실제론 gradient로 근사해. $g = \nabla_r D$는 D가 가장 빨리 커지는 방향이고, $\varepsilon \cdot \text{sign}(g)$는 입력의 각 원소를 그 방향(+/−)으로 ε씩 미는 거야 (FGSM).
  target은 GT label이라 supervised야. 목적은 robustness (공격에 버티기).

- **VAT** `Miyato, TPAMI 2018 · p.34–35`
  GT 대신 **모델 자신의 예측**을 target으로 ("virtual"). unlabeled에도 쓸 수 있어.
  `쉽게` AT는 정답 label이 있어야 해서 unlabeled엔 못 써. VAT는 정답 자리에 "지금 모델이 x에 내는 예측"을 넣어. 가짜(virtual) label인 셈이야. 그러면 "x에서의 예측과, 최악의 방향으로 살짝 민 x + r에서의 예측이 같아야 한다"가 돼. label이 필요 없으니 labeled와 unlabeled 둘 다에 걸어.

식 $L_U^{VAT} = D[p_\theta(y|x), p_\theta(y|x + r_{vadv})]$. $p_\theta(y|x)$는 현재 모델의 예측(target)이야. $\hat{\theta}$(theta hat)은 값은 지금 θ와 같지만 target 쪽이라서 고정된 상수로 취급한다는 표시야. 그래서 gradient는 x + r 쪽 예측(θ)으로만 흘러 (슬라이드엔 "use current model prediction"만 있고, 상수 취급은 노트 기반 설명이야). 그리고 r_{vadv}는 이 D를 가장 키우는 방향이야.

목적은 local smoothness (x 근처에서 예측이 매끄럽게). random 방향으로 흔드는 RPT(Random Perturbation Training, 비교용 방법)가 아니라 가장 adversarial한 방향만 눌러서 regularization이 훨씬 효과적이야 (p.35). (슬라이드 밖 직관) 랜덤 방향은 대부분 모델이 이미 잘 버티는 방향이라 그래.

- **ICT** `Verma · p.36-38`

  unlabeled 두 장을 MixUp. student의 섞인 입력 예측 = Mean Teacher 예측을 섞은 것. "mixed inputs → mixed predictions".

  쉽게 MixUp은 두 사진을 비율 λ로 겹쳐 한 장을 만드는 augmentation이야: $mixup_\lambda(x_i, x_j) = \lambda x_i + (1-\lambda)x_j$, $\lambda \sim Beta(\alpha, \alpha)$ (Beta 분포는 0–1 사이 값을 뽑는 분포라, λ가 섞는 비율이 돼). 여기 α는 MixUp 파라미터라 TE의 α와 다른 거야. 원래 MixUp은 label도 같은 비율로 섞어서 labeled에만 쓸 수 있어. ICT는 label 대신 **teacher 예측을 섞어서** unlabeled에도 써. teacher(θ′)는 섞기 **전** 원본 두 장을 보고, student(θ)는 섞은 사진을 봐.

  예시 teacher가 고양이 사진에 [고양이 0.9, 개 0.1], 개 사진에 [0.2, 0.8]을 냈다고 하자. λ = 0.7로 섞은 사진의 target은 0.7×[0.9, 0.1] + 0.3×[0.2, 0.8] = [0.69, 0.31]이야. student는 섞인 사진에 이 값을 내야 해.

  MixUp 샘플은 boundary 근처라 low-density separation과 맞아. cluster 사이 빈 곳에서 예측이 부드럽게 바뀌도록 강제하니까, boundary가 cluster를 가르지 않고 그 사이로 가게 돼.

- **UDA** `Xie, NeurIPS 2020 · p.39-42`

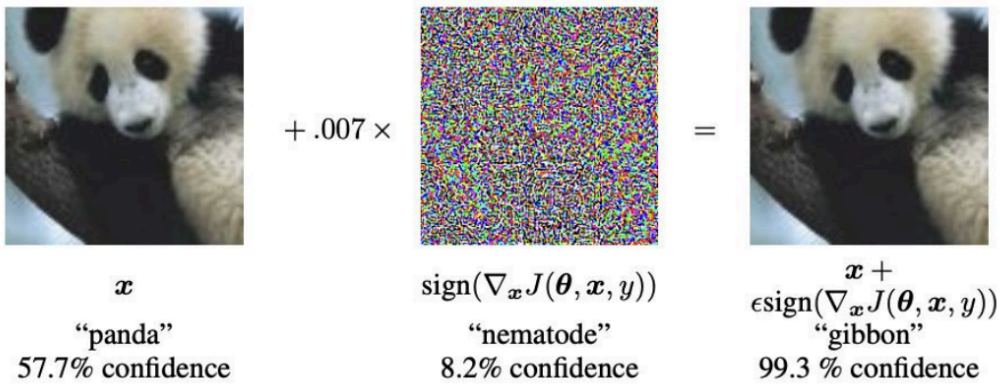
  작은 noise 대신 label을 보존하는 센 의미적 augmentation(RandAugment). 이것만으로 VAT·ICT를 이겨.

  쉽게 VAT의 noise는 눈에 안 보일 만큼 작아서 입력 바로 근처만 매끄럽게 만들어. UDA는 회전, 이동, 색, 밝기, 대비처럼 **눈에 보이게 크지만 class는 그대로인** 변형(RandAugment)을 주고 $p(y|x) \approx p(y|Aug(x))$를 요구해.

  RandAugment는 변형 목록에서 몇 개를 랜덤으로 골라 적용하는 방식이야. (L04 augmentation 표의 그 UDA야. 두 예측의 거리 D로 KL을 써, L04 p.18.)

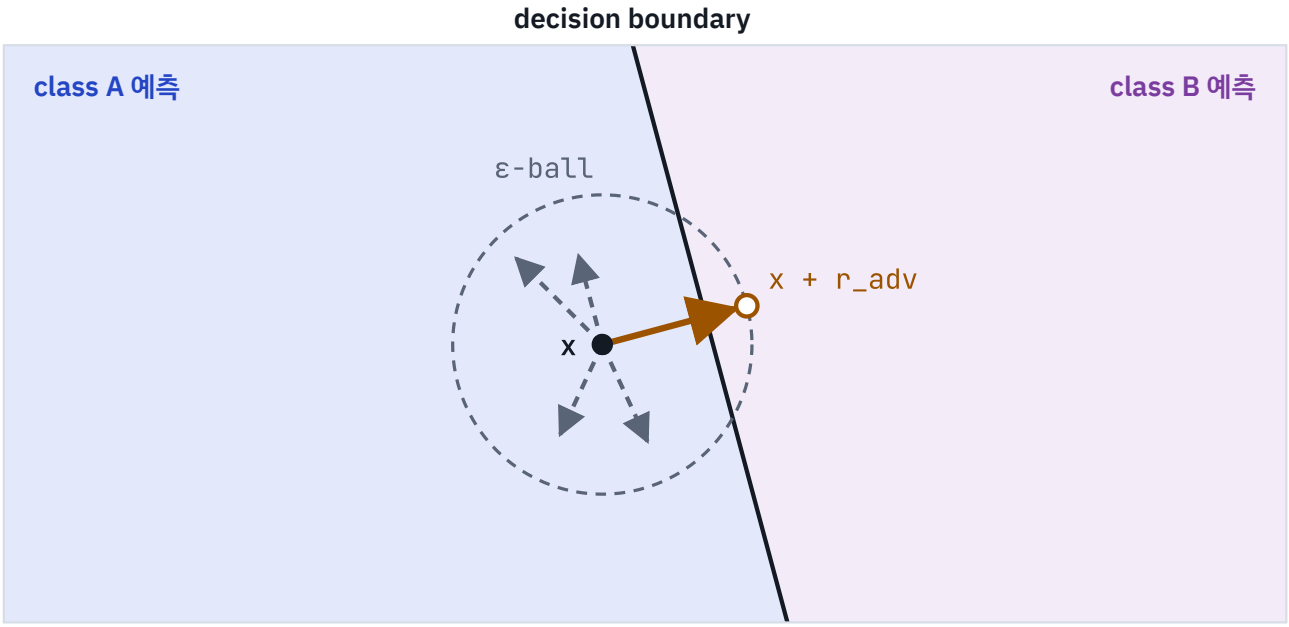
  좋은 augmentation (p.40) label-preserving(변형해도 class 유지), diverse(다양함), realistic(실제로 있을 법한 사진), task-relevant(task에 맞는 변형. 예: 숫자 6을 180도 돌리면 9라서 숫자 인식엔 회전을 조심).

  "How we perturb the input matters more than simply adding perturbations."



Adversarial example. 왼쪽 panda 사진(모델 확신 57.7%)에 가운데 noise를 .007배만 더하면, 사람 눈엔 똑같은데 모델은 99.3% 확신으로 gibbon이라고 해. noise는 $\varepsilon \cdot sign(\nabla_x J)$, 즉 loss를 가장 빨리 키우는 방향의 부호야. 이런 최악의(worst-case) noise에서도 정답을 맞히게 학습하는 게 Adversarial Training이고, 정답 label 대신 모델 자신의 예측을 target으로 쓰면 VAT가 돼. `L06 p.32`

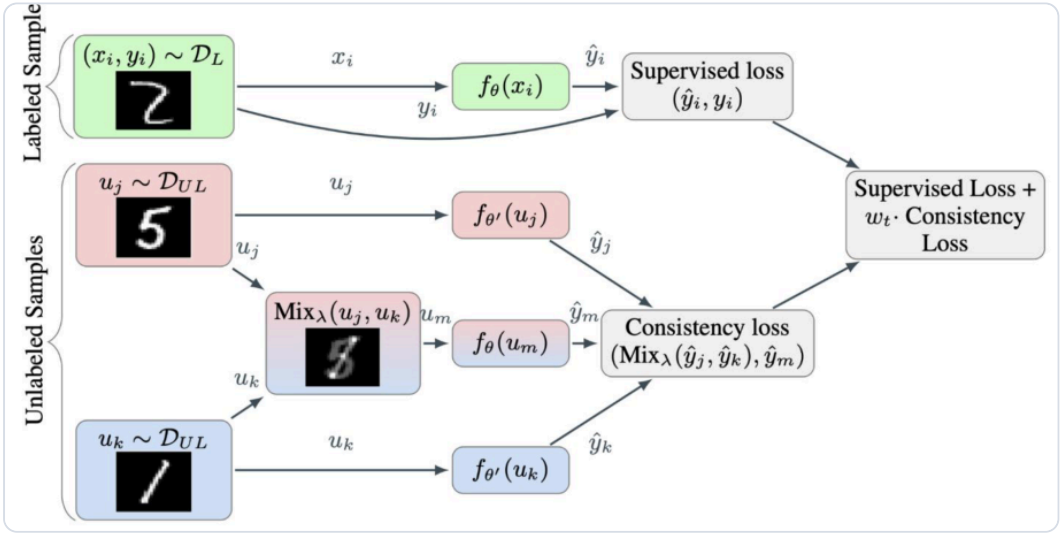
인터랙티브 **Perturbation 방향: random(RPT) vs adversarial(AT/VAT)** `illustrative`



AT	target = 정답 `y (GT)` · CE(f(x+r_adv), y)
VAT	target = 모델 자신의 `f(x)` · D(f(x), f(x+r_adv))
RPT	random r (회색) · 방향이 낭비돼서 효과 약함

ε (perturbation 크기) 70 `ε > 50: r_adv가 boundary를 넘어서 예측이 바뀜`

x는 decision boundary 근처의 점이고, 점선 원(ε-ball)은 "크기 ε 이하의 noise로 갈 수 있는 범위"야. 회색 점선 화살표 = random 방향 r(RPT): 대부분 boundary와 상관없는 쪽이라 예측이 거의 안 바뀌어서 낭비야. 굵은 갈색 화살표 r_adv = ε-ball 안에서 **예측이 가장 크게 바뀌는 방향**(여기선 boundary 쪽)으로 끝까지 민 최악의 noise야. 슬라이더로 ε를 키우면 r_adv가 boundary를 넘어가. AT는 이 점에서도 **정답 y**를 맞히게 학습하고, VAT는 y 대신 **모델 자신의 예측 f(x)**를 target으로 써서 label 없이도 돼. (개념 그림, L06 p.32–35)



ICT. 맨 위 초록 줄은 평범한 supervised loss고, 아래에선 unlabeled u_j(숫자 5), u_k(숫자 1)를 비율 λ로 섞어 u_m을 만들어. teacher $f_{\theta'}$는 섞기 **전** 원본을 각각 예측($\hat{y}_j$, $\hat{y}_k$)하고, student f_θ는 섞은 u_m을 예측($\hat{y}_m$)해. consistency loss는 **teacher 예측을 같은 λ로 섞은 $\text{Mix}_\lambda(\hat{y}_j, \hat{y}_k)$**와 $\hat{y}_m$이 같아지게 해. `L06 p.37`

teacher 고치기

마지막 세 논문은 모두 **semi-supervised semantic segmentation**이야. segmentation은 사진의 **pixel마다** class를 맞히는 task라서 label 달기가 특히 비싸. 셋 다 Mean Teacher(EMA teacher + student) 뼈대는 그대로 두고 약점을 하나씩 고쳐 (p.75).

- **PS-MT**: teacher target을 어떻게 더 믿을 만하게 만들까?
- **Dual Teacher**: teacher가 student를 너무 닮아가는 걸 어떻게 막을까?
- **S⁴Former**: Transformer의 어느 부분에 consistency를 걸까?

공통 구도는 weak-to-strong이야. teacher에겐 약하게 변형한(weak aug) 사진, student에겐 세게 변형한(strong aug) 사진을 주고, student가 teacher 예측을 맞히게 해.

PS-MT

Liu, CVPR 2022 · p.43-54

`한 줄` Perturbed and Strict Mean Teachers. teacher를 먼저 믿을 만하게 만들고, 그 위에서 student를 더 세게 흔들고 더 엄격한 loss로 가르쳐.

`문제` perturbation을 세게 주고 싶은데, teacher가 틀리면 student가 그 오류를 그대로 배워 (p.45). 센 perturbation은 generalization에 좋지만, 틀린 target을 센 변형 아래서까지 따라하면 오류가 굳어. "Before applying stronger perturbations, we need more reliable teacher predictions." 이런 현상을 흔히 confirmation bias라고 불러 (L06 슬라이드 용어는 아니고, L07에서 본격적으로 나와).

`핵심` mean teacher 2개의 ensemble로 target을 만들고 (epoch마다 하나만 갱신), student엔 센 perturbation (input CutMix, feature T-VAT), loss는 MSE 대신 더 엄격한 Conf-CE. 4요소: ① Two Mean Teachers ② Teacher Ensemble ③ Strongly Perturbed Student ④ Strict Consistency Loss (p.46).

`①② teacher 2개` (p.47) 각 teacher는 평범한 Mean Teacher처럼 student의 EMA야: $\theta^k \leftarrow \gamma\theta^k + (1-\gamma)\theta^s$ (γ = EMA decay). 단, **epoch마다 둘 중 하나만** 갱신해서 두 teacher가 student의 서로 다른 시기를 기억해. target은 두 출력을 평균 내고 softmax(σ)를 씌운 $\hat{y} = \sigma(0.5\times(f_{\theta^{t1}}(x) + f_{\theta^{t2}}(x)))$야. 평균 내면 흔들림(variance)이 줄어. "Each teacher averages the student over time. The method then averages the two teachers."

`③ 3단 perturbation` (p.49)

- **input**: teacher엔 weak aug, student엔 strong aug (CutMix, Zoom In/Out). CutMix는 한 사진의 사각형 영역을 다른 사진 조각으로 바꿔 붙이는 augmentation이야.
- **feature**: T-VAT. student feature z^s(network 중간 표현)에 adversarial perturbation을 넣어.
- **network**: 시간 평균으로 만든 teacher 2개 vs gradient로 학습하는 student 1개라 예측이 서로 달라.

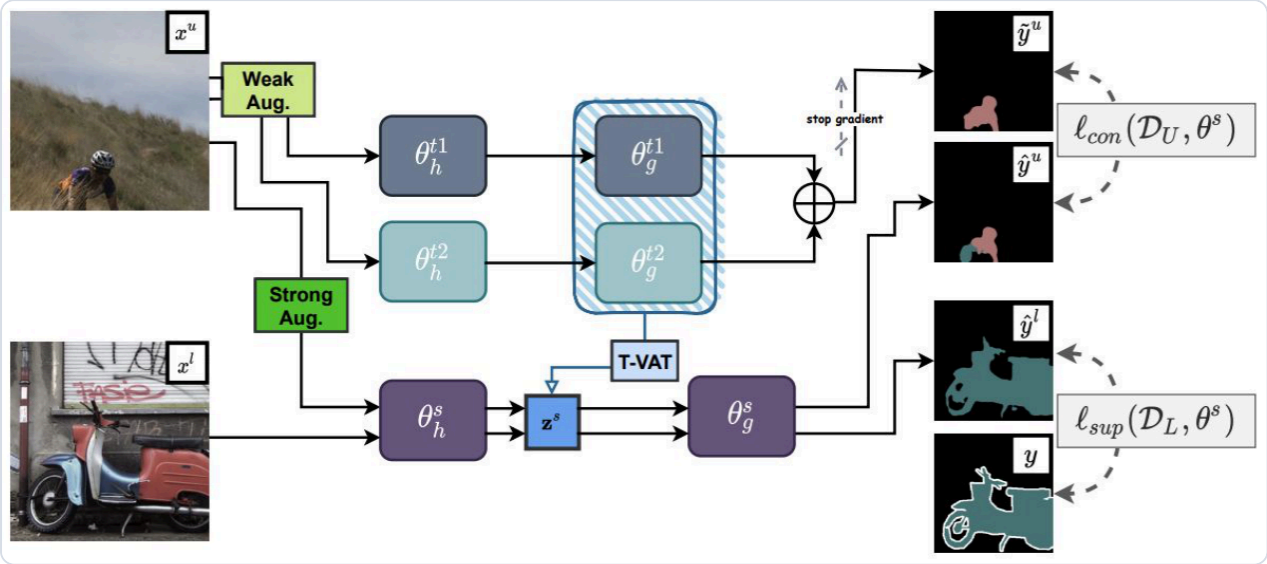
`T-VAT` (p.50) 보통 VAT는 student가 자기 예측으로 방향을 찾는데, 초반 student는 부정확해서 방향도 엉터리일 수 있어. T-VAT는 이렇게 해.

1. student가 feature z^s를 만들어.
2. 두 teacher가 이 feature로 예측하고, 그 예측을 가장 크게 바꾸는 방향 r_{adv}를 찾아 ($\|r_{adv}\|_2 \le \varepsilon$).

3. 그 perturbation을 **student feature**에 더하고, student는 그래도 teacher target을 맞히도록 학습. "Teachers find the perturbation the student learns from it."

CutMix-aware prediction ③ CutMix-aware prediction (p.51) 섞은 사진을 teacher가 보면 잘린 경계 때문에 예측에 흔적(mixing artifacts)이 생겨. 그래서 teacher는 원본 두 장을 **각각 먼저 예측**하고 그 예측을 같은 mask로 섞어 target을 만들어. 섞인 사진은 student만 봐 (ICT와 같은 구조).

④ Conf-CE (p.48) MSE는 teacher가 틀려도 덜 끌려가서 안전하지만, 수렴할 즈음 gradient가 작아서 가르치는 힘이 약해 (weak supervision). teacher를 정확하게 만들었으니 pixel마다 가중치 $c(\omega)$(슬라이드엔 정의가 없고, 이름 Conf-CE로 보아 teacher confidence)를 곱한 더 센 CE를 써. "More accurate teachers allow stronger cross-entropy supervision." Mean Teacher(p.28)는 MSE를 골랐는데 여기선 CE로 간다는 게 포인트야. ablation(구성요소를 하나씩 더하거나 빼 보며 각자의 효과를 재는 실험)에서 제일 큰 단일 향상도 Conf-CE였어 (p.52).



위: unlabeled x^u를 weak aug해서 teacher 2개(θ^{t1}, θ^{t2})에 넣고, 두 출력을 ⊕로 합쳐 pseudo label $\tilde{y}^u$를 만들어. 여기엔 **stop gradient**(이쪽으론 gradient를 안 흘려 target을 상수로 취급)가 걸려 있어. 같은 x^u를 strong aug한 버전은 student(보라)로 가서 $\hat{y}^u$를 내고, ℓ_{con}으로 $\tilde{y}^u$를 맞춰. **T-VAT** 박스: perturbation은 **student feature z^s**에 더하지만, 방향은 student 자신이 아니라 빗금 친 **두 teacher decoder**가 예측이 가장 크게 바뀌는 쪽으로 찾아줘. L06 p.46

Dual Teacher

Na, NeurIPS 2023 · p.55-65

한 줄 EMA teacher 2개를 epoch마다 **번갈아** 써서, teacher가 student를 똑 닮아 버리는 걸 막아.

문제 EMA teacher가 student를 점점 닮아가서(coupling) 새 정보가 없고 오류도 물려받아 (p.57). teacher는 student weight의 평균이라 학습 내내 student를 따라가. 둘의 예측이 거의 같아지면 target이 student 자기 예측과 별 차이가 없어서 배울 게 없어.

세 구조 (p.58)

- **Single Teacher** (= Mean Teacher): student 1 + 영구(permanent) EMA teacher 1. teacher가 같은 student를 계속 따라가.
- **Dual Student**: 따로 학습하는 student 2개. 다양성은 있지만 학습·메모리 비용이 커.
- **Dual Teacher**: student 1 + **임시(temporary)** teacher 2. 한 번에 한 teacher만 가르쳐. teacher는 EMA라 따로 학습할 필요가 없어서, 두 번째 student 없이 다양성을 얻어 ("Dual

Teacher introduces diversity without training a second student").

핵심 임시 teacher 2개가 epoch마다 **교대**. active teacher만 target을 만들고 갱신돼.

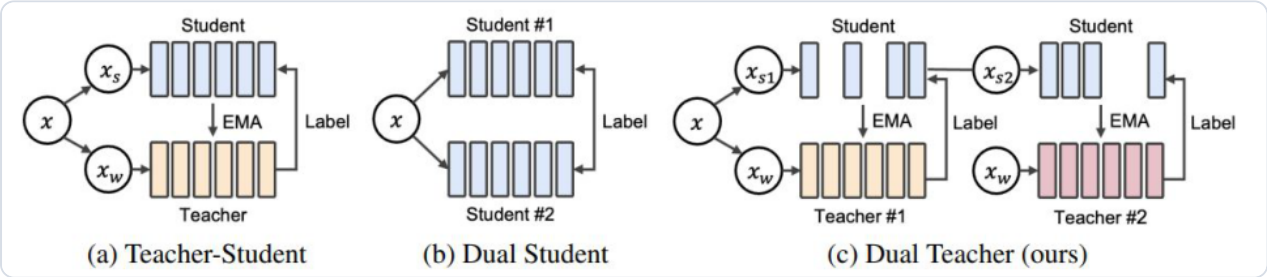
어떻게 (p.59–61)

1. epoch 1은 Teacher 1, epoch 2는 Teacher 2가 active. active teacher는 weak aug 사진으로 pixel 단위 target을 만들고, student는 strong aug 사진으로 그걸 맞혀 (pixel별 CE).
2. teacher가 바뀔 때 student의 strong aug도 바꿔 (ClassMix ↔ CutMix). ClassMix는 한 사진에서 특정 class 영역을 오려 다른 사진에 붙이는 augmentation이야.
3. EMA 식은 그대로고, 갱신 대상만 active teacher야: $\theta_{t_{k_e}} \leftarrow \alpha\theta_{t_{k_e}} + (1-\alpha)\theta_s$, $k_e \in \{1, 2\}$ (epoch e의 active teacher 번호). "The EMA rule remains the same. The teacher schedule changes."

왜 돼? 쉬는 teacher는 그 epoch 동안 멈춰 있어서, 두 teacher가 student의 서로 다른 시기를 간직해 ("Each teacher preserves the student's characteristics from different training stages"). 그래서 student와의 prediction distance가 더 크게 유지되고, 두 teacher는 전체 성능(mIoU, segmentation 정확도 지표)은 비슷해도 class별로 잘하는 게 달라 (p.62).

차이 PS-MT는 teacher 2개를 **ensemble**(동시에 합침), Dual Teacher는 **switching**(한 번에 하나)이야. 합쳐 쓰면 두 teacher가 점점 특색을 잃어서 switching이 더 나았어 (p.63). teacher는 2개가 최선이고 3번째는 이득이 없었어 (p.64).

문장 "A teacher should be stable, but it should not become identical to the student."



(a) Teacher-Student = Mean Teacher: x를 weak(x_w)는 teacher, strong(x_s)은 student에 넣고, student의 EMA인 teacher 하나가 계속 label을 줘. 그래서 teacher가 student를 점점 닮아가(coupling). (b) Dual Student: student 2개가 서로 label을 주고받아 (학습·메모리 비용이 커). (c) Dual Teacher: student 하나 + EMA teacher 둘. 서로 다른 strong aug(x_{s1}, x_{s2})에 맞춰 teacher가 **번갈아** label을 줘서, 두 번째 student 없이 다양성을 얻어. L06 p.58

S⁴Former

Hu, CVPR 2024 · p.66–74

한 줄 ViT(Vision Transformer) 구조에 맞춰 input, feature, output 세 군데에 regularization을 걸어.

문제 (p.68) 대부분의 consistency 방법은 **입력 사진**만 흔들어. 그런데 ViT는 사진을 작은 조각(patch)으로 잘라 token처럼 나열해서 처리해. 그래서 self-attention(각 patch가 다른 patch를 얼마나 참고할지 정해 정보를 섞는 연산)과 출력 분포도 regularization을 걸 자리야.

핵심 ViT 구조에 맞춘 perturbation 3단: Input = PatchShuffle, Feature = PASA, Output = NCR. 단일 효과는 PatchShuffle이 제일 커 (p.73).

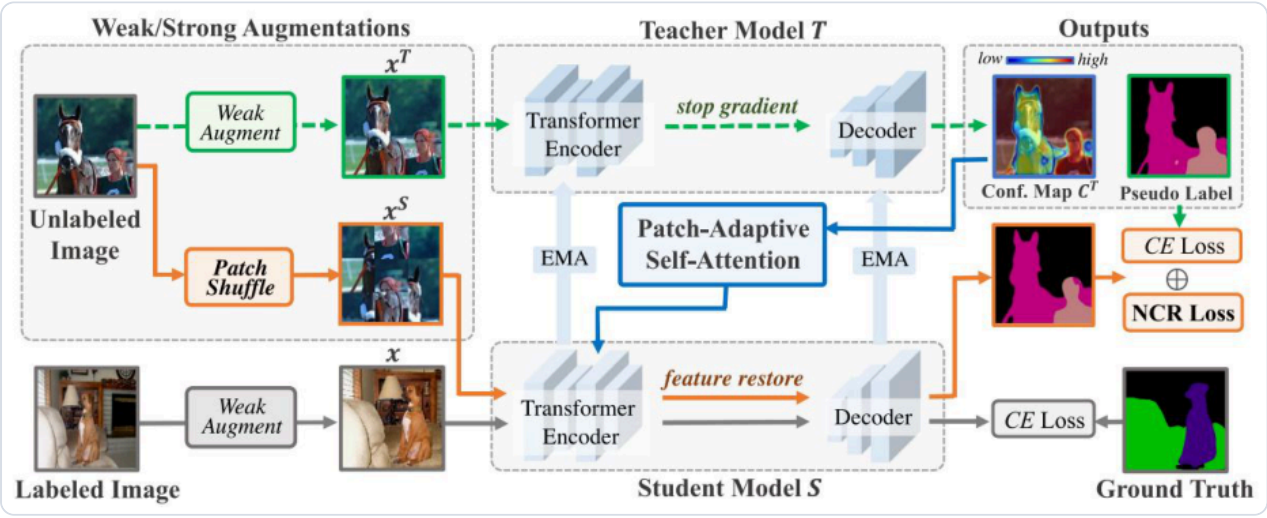
- **Input: PatchShuffle** (p.70). 사진을 L×L patch로 나누고, 이웃 patch를 M×M **block**으로 묶어. encoder에 넣기 전에 block 순서를 섞고, decoder 전에 feature 순서를 원래대로

되돌려 (segmentation은 pixel 위치가 맞아야 pseudo label과 비교할 수 있어서). block 안의 가까운 patch끼리는 그대로고 전체 배치만 바뀌어. "The student must preserve the same semantics even when the global patch layout changes."

- **Feature: PASA** (Patch-Adaptive Self-Attention, p.71). teacher가 patch마다 confidence(얼마나 확신하나)를 매겨. 확신 높은 patch는 원래 attention 그대로 두고, 확신 낮은 patch에만 attention을 추가로 조절해. 그러면 불확실한 영역끼리 더 강하게 주고받아. "PASA perturbs attention where the teacher is uncertain."
- **Output: NCR** (Negative Class Ranking, p.72). CE는 가장 그럴듯한 class 하나(pseudo label)만 맞춰. NCR은 pseudo-label class를 **빼고** 나머지 class끼리 softmax를 다시 계산해서, 그 상대적 분포를 teacher와 맞춰: $L_{NCR} = \Sigma_{k \neq \tilde{y}} (\hat{p}_k^T - \hat{p}_k^S)^2$. $\tilde{y}$는 teacher의 pseudo label, T/S는 teacher/student야.

> 예시로 보면
> teacher가 어떤 pixel을 "고양이"라고 했다고 하자. 고양이를 빼고 나머지 class로 softmax를 다시 하면 [개 0.7, 소 0.25, 자동차 0.05] 같은 순위가 나와. student도 고양이를 뺀 자기 분포가 [0.7, 0.25, 0.05]에 가까워지도록 (class마다 차이의 제곱을 줄이도록) 학습해. 그러면 "개 > 소 > 자동차"라는 순서도 같이 따라와. 이게 NCR이야. 정답 하나보다 많은 정보, 즉 class끼리 얼마나 닮았나를 배워 ("The teacher provides a class relationship, not only a single target class").

차이 (p.74) Mean Teacher의 뼈대(teacher–student, EMA teacher update, weak-to-strong consistency)는 그대로야. 바뀌는 건 patch-aware input perturbation, confidence-aware attention perturbation, class-distribution regularization이야. "Effective consistency regularization should reflect the structure of the model."



unlabeled 사진은 teacher엔 weak aug(초록 점선), student엔 **Patch Shuffle**(주황, input perturbation)로 들어가. teacher는 stop gradient + EMA로만 갱신되고, pseudo label과 confidence map C^T(얼마나 확신하나)를 내. 주황 **feature restore** 화살표는 decoder 앞에서 섞였던 patch 순서를 되돌리는 거야. 파란 **Patch-Adaptive Self-Attention**은 teacher confidence로 student의 attention을 조절하는 feature-level perturbation이야. output 쪽은 CE + NCR(pseudo-label class를 뺀 나머지 class 분포 맞추기). `L06 p.69`

RL로 번역

Mean Teacher의 $\theta' \leftarrow \beta\theta' + (1-\beta)\theta$는 DDPG·SAC의 target network soft update와 같은 식이야. β를 0.99 → 0.999로 키우는 것도 "초반엔 빨리 따라가고 나중엔 안정적으로"라는 같은 직관이야. teacher 쪽에 gradient를 안 흘리는 건 TD target을 상수로 두는 semi-gradient와 닮았어.

consistency regularization은 DrQ·RAD가 pixel RL에서 하는 것(augmentation을 줘도 Q·policy가 같게)과 같은 발상이고, sim2real의 domain randomization도 "겉모습이 바뀌어도 행동은 같아야 한다"는 점에서 친척이야. PS-MT가 걱정한 "틀린 teacher를 따라 해서 오류가 굳는 것"은 bootstrapping error가 쌓이는 것과 닮은 문제야. Dual Teacher의 coupling은 "자기 자신으로 자기 target을 만들 때 생기는 bias"라서, Double DQN이 estimator를 분리하는 이유랑 통해.

시험 함정

- TE = **prediction** EMA, epoch 끝. Mean Teacher = **weight** EMA, 매 iteration.
- μ(t)는 **증가**. H3 boundary는 **low**-density, H4는 **low**-dimensional manifold.
- AT = GT label, VAT = 모델 예측 ("virtual").
- Mean Teacher: teacher는 EMA로만 갱신 (backprop 없음), MSE > KL. 그래도 "teacher는 학습 안 한다"는 틀려. Mean Teacher의 teacher는 EMA로 **매 iteration** 갱신돼. (PS-MT·Dual Teacher는 예외: epoch마다 teacher 하나만 갱신)
- Π-model은 **같은 θ**로 매 step forward **2번** (다른 dropout·augmentation). 파라미터가 다른 두 모델이 아니야. TE는 forward 1번.
- T-VAT: perturbation은 **student feature**에 넣고, 방향은 **teacher**가 찾아. 보통 VAT는 student 자신이 찾아.
- PS-MT = 두 teacher **ensemble** (epoch마다 하나만 갱신). Dual Teacher = **switching** (active 하나만 target을 만들고 갱신).
- PS-MT의 CutMix는 teacher 예측 **이후** (teacher는 자연스러운 원본을 봐). loss는 MSE → **Conf-CE**.
- EMA 기호가 논문마다 달라: Mean Teacher β, TE α (ensembling momentum), PS-MT γ, Dual Teacher α. ICT의 `Beta(α, α)`의 α는 MixUp 파라미터야.
- S^4Former: Input = PatchShuffle, Feature = PASA, Output = NCR. NCR은 pseudo-label class를 **뺀** softmax.

다음으로

consistency는 "두 예측을 같게"야. 정답이 뭔지는 정하지 않아. 더 직접적으로 "confident한 예측을 아예 정답으로 쓰자"는 게 pseudo labeling이야. → L07

셀프 체크

빈칸 Temporal Ensembling keeps an EMA of _____, while Mean Teacher keeps an EMA of _____.

predictions / weights (p.19–25). TE는 샘플마다 예측을 평균 내 ($Z \leftarrow \alpha Z + (1-\alpha)z$, N×C 표). Mean Teacher는 student weight를 평균 내서 teacher를 만들어 ($\theta' \leftarrow \beta\theta' + (1-\beta)\theta$).

T/F Under the low-density separation assumption, decision boundaries should pass through high-density regions.

False. low-density regions (p.11–14). boundary는 점이 드문 곳, 즉 cluster 사이를 지나야 해. 빽빽한 곳을 지나면 한 cluster가 두 label로 쪼개져.

MCQ "Virtual" in VAT means: (A) synthetic images (B) the model's own predictions replace ground-truth labels (C) a virtual teacher network (D) random perturbations

(B) (p.34–35). 정답 label 자리에 모델 자신의 현재 예측을 넣어 "가상의" label로 쓴다는 뜻이야. 그래서 unlabeled에도 쓸 수 있어. (D) random perturbation은 VAT가 아니라 비교 대상 RPT 쪽이야.

L07 Semi-Supervised Learning II 106p · pseudo labeling

질문 ② confident한 예측을 정답으로 쓰자. 근데 틀리면?

상황 복습 semi-supervised learning(L06–L07에선 이걸 SSL로 줄여 써. L02–L05의 self-supervised SSL과 약어만 같아)은 label 달린 데이터는 조금, label 없는(unlabeled) 데이터는 많을 때 둘을 같이 학습해. loss는 $L = L_s + \mu(t) \cdot L_u$야. L_s는 labeled에 거는 보통 supervised loss, L_u는 unlabeled에 거는 loss, $\mu(t)$는 L_u 가중치(초반엔 작게). unlabeled엔 정답이 없으니 L_u의 target을 어디선가 만들어야 해. L06은 consistency(입력을 살짝 바꿔도 예측은 같아야 함)로 만들었고, L07은 **pseudo label**로 만들어.

쉽게 말하면

pseudo label(가짜 정답) = 모델이 unlabeled 샘플에 낸 예측을 **정답인 척** 쓰는 것. 예: label 없는 강아지 사진에 모델이 [cat 0.03, dog 0.96, car 0.01]을 냈어. 충분히 확신(confident)하니까 "dog"를 one-hot [0, 1, 0]으로 바꿔서 labeled 샘플처럼 cross-entropy로 학습해. [0.40, 0.35, 0.25]처럼 애매한 예측은 안 써 (high-confidence만, p.4).

왜 도움이 돼? 자기 예측을 다시 배우면 새로 배울 게 없을 것 같지만, 모델이 **애매한 예측을 덜 하게** 돼. 그 결과 decision boundary가 데이터가 몰린 곳을 피해 **빈 곳(low-density region)**으로 밀려나 (L06의 H3). 왜 그런지는 첫 방법 Pseudo-Label의 "왜 돼?"에서 풀게.

왜 위험해? 모델이 확신을 갖고 **틀릴** 수도 있어. 강아지 사진에 "cat 0.97"을 내면 그걸 정답으로 배우고, 다음엔 더 확신하며 "cat"이라고 해. 틀린 답이 스스로를 강화하는 이 현상이 **confirmation bias**(확증 편향)야 (p.20). 이 강의의 방법들은 이 위험을 각자 다르게 줄여.

- **다른 근거로 label 만들기**: Label Propagation (이웃 관계 graph).
- **배우는 쪽을 흔들기 / teacher 고치기**: Noisy Student, Arazo (noise·MixUp) / Meta Pseudo Labels (student 성적으로 teacher 학습).
- **예측을 다듬고 걸러내기**: MixMatch (평균 + sharpening), FixMatch (weak view + threshold).
- **다른 network가 걸러주기**: DivideMix.
- **FixMatch 이후**: 어떤 걸 고르고(FreeMatch), 버린 건 어떻게 쓰고(AllMatch), 고른 건 얼마나 흔들까(IFMatch).

기본기

- **Pseudo-Label & Entropy Minimization** Lee, ICML 2013 · p.4-9

  confident한 예측을 hard label로 바꿔서 labeled와 같이 학습. pseudo labeling = entropy regularization의 **hard-label 근사**.

  entropy $H(p) = -\Sigma_c p_c \log p_c$ (p_c = class c의 예측 확률). 확률이 고르게 퍼질수록(애매) 크고, 한 class에 몰릴수록(확신) 0에 가까워. 예: 3-class 균등이면 H = ln 3 ≈ 1.09로 최대, Dog 하나에 거의 몰리면 0.09 (p.5). 식에 정답 y가 없어서 unlabeled에도 계산돼.

  왜 돼? entropy regularization은 unlabeled 예측의 entropy를 줄이는 항이야. boundary 근처 점은 두 class 확률이 비슷해서 entropy가 커. 모든 점의 entropy를 낮추려면 boundary 근처에 점이 없어야 하니까 boundary가 low-density로 밀려나 (p.6). 움직이는 건 데이터가 아니라 boundary야.

연결 충분히 confident하면 [0.96, 0.03, 0.01](p.7 예시, 첫 칸이 1등 class)을 `y* = argmax p(y|x)`로 바꿔 (p.7). one-hot target으로 `-log` p_{y*}를 줄이면 $p_{y*} \to 1$이 돼서 entropy도 같이 줄어. entropy regularization은 **soft** 분포 전체에, pseudo labeling은 argmax로 굳힌 **hard** target에 거는 거라 "hard-label 근사"야.

entropy를 줄이면 boundary가 low-density로 이동. MNIST에서 pseudo label을 쓰면 class cluster가 더 잘 갈라져 (p.8–9).

- **Label Propagation** `Iscen, CVPR 2019 · p.10–14 · code p.11`

  feature 공간에 kNN graph를 만들고 labeled에서 unlabeled로 label을 퍼뜨려(diffusion). 자기 softmax가 아니라 **이웃 관계**에서 label이 나와. 쉽게 말해 비슷한 샘플끼리 선으로 잇고 label을 물감 번지듯 퍼뜨리는 거야. Pseudo-Label은 샘플을 **하나씩 따로** 보지만, 이건 주변 샘플을 봐 (p.13).

  어떻게 (1) labeled만으로 학습해서 쓸 만한 feature extractor φ_θ를 만들어.

  (2) 모든 샘플의 feature를 뽑고, 각 점을 가장 가까운 k개 이웃(kNN)과만 이어 affinity(유사도) 행렬 A를 만들어. 가까울수록 값이 크고, `W = A + A`$^\top$로 대칭화한 뒤 정규화해서 $\mathscr{M}$를 얻어.

  (3) diffusion `(I - α`$\mathscr{M}$`)Z = Y`를 CG(conjugate gradient)로 풀어. Y = labeled 행만 one-hot, unlabeled 행은 0인 시작 label 표. Z = 퍼진 뒤 각 샘플의 class별 점수. α가 클수록 멀리 퍼져.

  (4) Z의 argmax를 pseudo label로, certainty ω_i를 weight로 써서 확신 낮은 label은 약하게 반영. 전체 데이터로 1 epoch 학습 → 좋아진 feature로 graph를 다시 만들어 반복 (p.10–11).

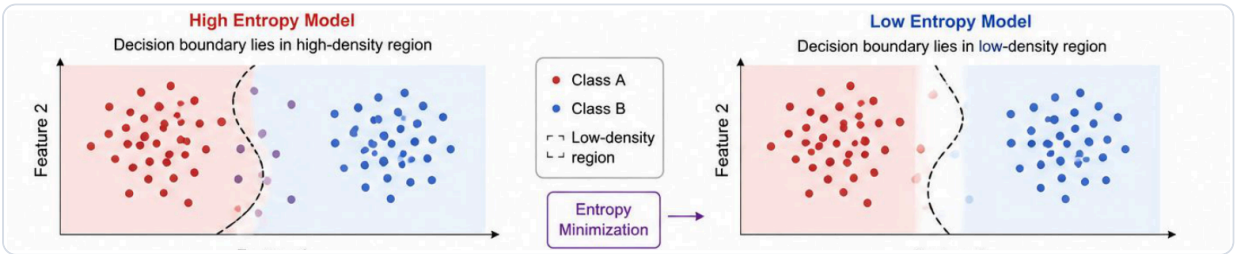
  confidence-based → similarity-based. 대신 graph 비용이 크고 feature 질에 의존 (feature가 나쁘면 틀린 label도 graph 타고 퍼져, p.14).

- **Self-training · Noisy Student** `Xie, CVPR 2020 · p.15–19`

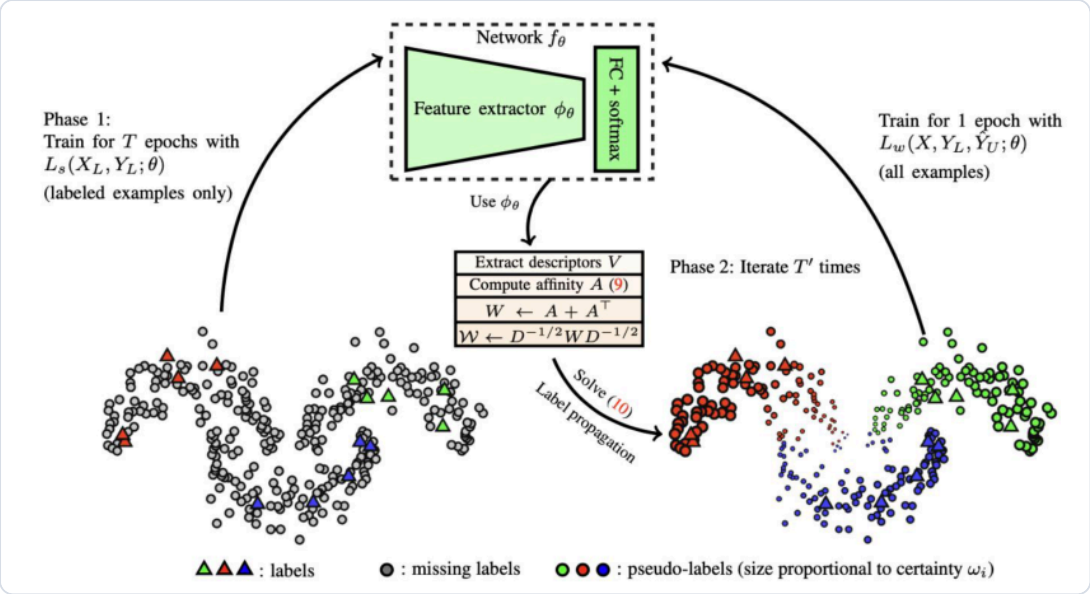
  용어 teacher = pseudo label을 만들어 주는 모델, student = 그걸 보고 배우는 모델. teacher는 이전 모델이나 EMA 모델인 경우가 많아 (p.15). self-training은 labeled로 학습 → unlabeled 예측 → high-confidence 예측을 labeled set에 추가 → 반복이고, student가 다음 round의 teacher가 돼 (p.16).

  `Noisy Student` teacher가 pseudo label → student 학습 → student가 다음 teacher, 반복. student는 teacher와 같거나 더 크고, **student에만** noise (RandAugment = 입력을 무작위로 세게 변형, Dropout = 뉴런 일부 끄기, Stochastic Depth = layer 일부 건너뛰기). teacher는 noise 없이 깨끗하게 예측해.

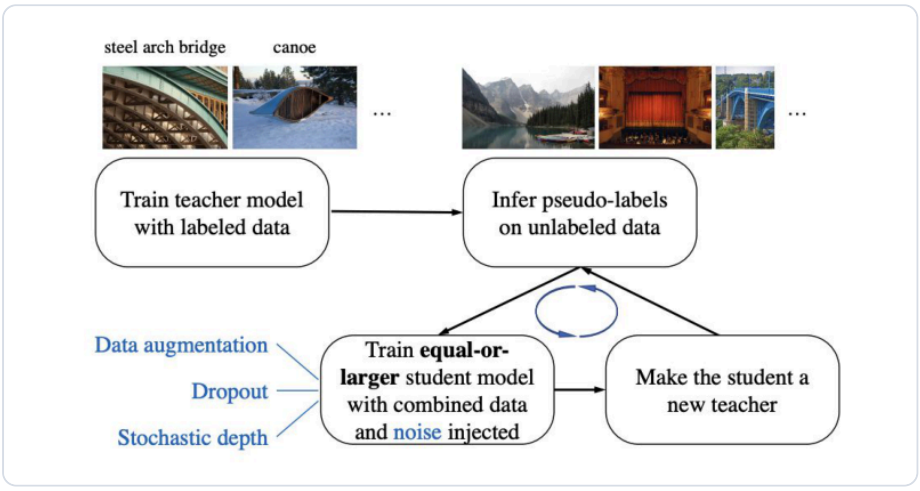
  왜 돼? noise가 없으면 student는 teacher를 그대로 베끼고 끝나. 흔들린 조건에서도 같은 답을 내야 하니까 더 robust한 boundary를 배워서 teacher보다 나아져 (p.17). 보통의 knowledge distillation(큰 teacher → 작은 student로 압축)과 크기 방향이 반대야. p.19: larger student > teacher, **soft** pseudo label > hard, strong augmentation이 도움.



빨강·파랑 점은 두 class, 배경색은 모델이 그 영역을 어느 class로 판단하는지야. 왼쪽(High Entropy Model)은 decision boundary가 데이터가 몰린 곳을 지나서, 경계 근처에 두 class 확률이 비슷한 애매한 점(보라)이 많아. Entropy minimization으로 모든 점을 확신 있게 만들면 오른쪽처럼 **boundary가 두 덩어리 사이 빈 곳(low-density region)으로 밀려나** (low-density separation). `L07 p.6`



삼각형 = label 있는 샘플, 회색 점 = label 없는 샘플, 색칠된 원 = propagation으로 받은 pseudo label이고 **원 크기가 certainty ω_i**야 (작을수록 확신 낮아서 약하게 반영). Phase 1은 labeled로만 학습, Phase 2는 feature로 affinity graph를 만들어 label을 **diffusion**하고 그 pseudo label로 1 epoch 학습하는 걸 T'번 반복해. 같은 초승달 위 점들이 같은 색을 받는 게 "이웃 관계로 label이 퍼진다"는 뜻이야. `L07 p.10`



teacher를 labeled로 학습 → unlabeled에 pseudo label → **equal-or-larger** student를 labeled + pseudo-labeled 데이터로 학습 → student를 새 teacher로 삼아 반복 (가운데 원형 화살표). 왼쪽 파란 글씨 noise(data augmentation, dropout, stochastic depth)가 **student 학습에만** 붙어 있는 점을 봐. `L07 p.18`

틀린 pseudo label 다루기

Confirmation bias

`Arazo, IJCNN 2020 · p.20-23`

문제 불완전한 teacher의 틀린 label을 student가 따라 하면서 오류가 쌓여. 그 student가 다시 teacher가 되면 같은 실수를 더 확신하며 퍼뜨려 (p.20).

핵심 틀린 label을 외우기 어렵게 regularize: soft label MixUp, labeled data oversampling (mini-batch마다 최소 개수), strong augmentation, dropout.

- **MixUp**: 두 샘플과 그 label(진짜든 pseudo든)을 같은 비율로 섞어. $\bar{x} = \lambda x_i + (1-\lambda) x_j$, $\bar{y} = \lambda y_i + (1-\lambda) y_j$ (p.21). 예: $\lambda = 0.7$이면 강아지 70% + 고양이 30% 이미지에 target [dog 0.7, cat 0.3]. soft target이라 틀린 label에 100% 확신하지 않게 돼.
- **Oversampling**: 진짜 정답 신호가 수많은 pseudo label에 묻히지 않게 labeled를 batch마다 꼭 넣어.

결과 two moons에서 MixUp + 최소 labeled 수를 **둘 다** 써야 boundary가 빈 곳으로 가 (p.22). p.23 표(error라 낮을수록 좋음)에선 confirmation bias를 다룬 이 pseudo labeling이 Mean Teacher·ICT 같은 consistency 방법보다 5개 열 모두 error가 낮아.

Meta Pseudo Labels

Pham, CVPR 2021 · p.24–27

한 줄 teacher의 pseudo label이 좋았는지를 "그걸 배운 student가 **labeled data(진짜 정답)**에서 받는 점수"로 채점해서 teacher도 학습시켜. 보통 self-training은 teacher가 좋다고 가정하고 고정하잖아 (p.24).

핵심 teacher를 고정하지 말고 **teacher를 학습**. bi-level(최적화 안에 최적화): student는 teacher의 pseudo label로 학습하고, teacher는 그 student의 **labeled data loss**가 낮아지는 방향으로 갱신.

1. **inner (student)**: $\theta_S^{PL}(\theta_T) = \text{argmin}_{\theta_S}\ E_U\ CE[f_{\theta_T}(u),\ f_{\theta_S}(u)]$. teacher($\theta_T$) 예측을 target으로 student($\theta_S$)를 학습.
2. **outer (teacher)**: $\min_{\theta_T}\ L_s(\theta_S^{PL}(\theta_T))$. 그 student를 **진짜 label**로 채점하고, 그 loss가 줄도록 teacher를 고쳐 (p.25).

왜 돼? θ_S^{PL}이 θ_T의 함수라 student의 labeled loss에서 teacher까지 gradient가 흘러. 채점 기준이 자기 pseudo label이 아니라 ground truth라서 "자기 label로 자기 평가" 순환이 끊겨.

차이 Arazo는 student를 regularize, Meta PL은 student의 feedback으로 teacher를 고쳐.

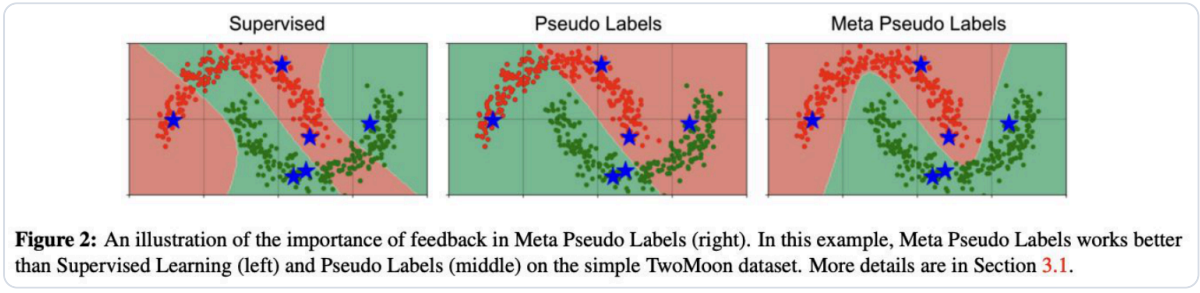


Figure 2: An illustration of the importance of feedback in Meta Pseudo Labels (right). In this example, Meta Pseudo Labels works better than Supervised Learning (left) and Pseudo Labels (middle) on the simple TwoMoon dataset. More details are in Section 3.1.

파란 별 = label 있는 몇 개의 샘플, 빨강·초록 점 = 두 class의 unlabeled 데이터, 배경색 = 모델이 그 영역을 어느 class로 판단하는지. Supervised(왼쪽)와 Pseudo Labels(가운데)는 boundary가 초승달을 가로질러 잘못 갈라. Meta Pseudo Labels(오른쪽)는 student의 feedback으로 teacher가 고쳐져서 두 moon을 따라 제대로 갈라 (importance of feedback). L07 p.26

둘 다 쓰기: MixMatch → ReMixMatch → FixMatch

pseudo labeling + consistency를 합치는 거야 (p.28–29). 한 view로 target을 만들고 다른 view가 그걸 맞히게. view = 같은 이미지에 augmentation(자르기, 뒤집기, 색 바꾸기)을 다르게 준 버전. 예측을 다듬어 target을 만드는 부분이 pseudo labeling(H3 low-density: boundary는 빈 곳에), 다른 view가 같은 답을 내게 하는 부분이 consistency(H1 smoothness: 살짝 바뀐 입력엔 같은 예측)야.

MixMatch

Berthelot, NeurIPS 2019 · p.30–35

한 줄 새 부품이 아니라 잘 되던 SSL(여기 L06–L07에선 semi-supervised learning) 아이디어(consistency, entropy minimization, pseudo labeling, sharpening, MixUp)를 하나로 **통합**한 첫 holistic framework (p.30).

1. **Label guessing**: unlabeled 하나에 augmentation K번 → 예측 K개를 **평균**. $\bar{q}_b = (1/K)\ \Sigma_k\ p_{model}(y \mid \hat{u}_{b,k};\ \theta)$. 여러 view 의견을 모으니까 consistency 역할 (p.32).

2. **Sharpening**: 평균은 평평해지기 쉬워서 temperature T로 뾰족하게 (softmax temperature와 같은 원리). $\text{Sharpen}(p, T)_i = p_i^{1/T} / \Sigma_j p_j^{1/T}$. entropy minimization 역할.
3. **MixUp**: labeled와 unlabeled를 섞어. $\lambda' = \max(\lambda, 1-\lambda) \geq 0.5$라서 섞인 결과는 첫 번째 샘플 쪽에 더 가까워 (p.31).
4. **Loss**: labeled엔 label과 예측 사이 loss D(슬라이드엔 기호만, 논문에선 cross-entropy), unlabeled엔 guessed label과 예측의 **L2**(squared error) (p.33).

예시로 보면 p = (0.6, 0.4), T = 0.5면 1/T = 2니까 각 값을 제곱해서 (0.36, 0.16), 합으로 나누면 (0.69, 0.31). T = 1이면 그대로, T → 0이면 one-hot에 가까워져. 뾰족해져도 여전히 soft label이고 버리는 샘플은 없어.

시험 ablation(부품을 하나씩 빼 보는 실험)에서 가장 크게 무너지는 건 MixUp 제거야 (p.35).

ReMixMatch

2020 · p.36–41

한 줄 MixMatch + 부품 3개 (p.36, 40).

- **Distribution Alignment**: pseudo label이 몇 class에 쏠리는 걸 고쳐. $\tilde{q} = \text{Normalize}(p_\theta(y|u) \cdot p(y) / \tilde{p}(\hat{y}))$. p(y) = labeled의 class 분포(목표), $\tilde{p}(\hat{y})$ = 모든 pseudo label의 추정 class 분포(현실). 너무 잦은 class는 깎고 드문 class는 키워 (p.37).
- **Augmentation Anchoring**: strong view로 만든 pseudo label은 불안정해. **weak** view(anchor)로 pseudo label을 **한 번** 만들고 여러 **strong** view가 맞히게 해 (p.38–39). FixMatch의 원형.
- **Rotation loss**: 회전 각도 맞히기(self-supervised)를 추가 regularization으로.

FixMatch

p.42–48

문제 MixMatch·ReMixMatch는 부품(label guessing, sharpening, MixUp, DA, 여러 augmentation)이 너무 많아. 훨씬 단순하게 해도 될까? (p.42)

핵심 "Predict on an easy example, learn from a difficult one." 아이디어 3개만 써 (weak · strong).

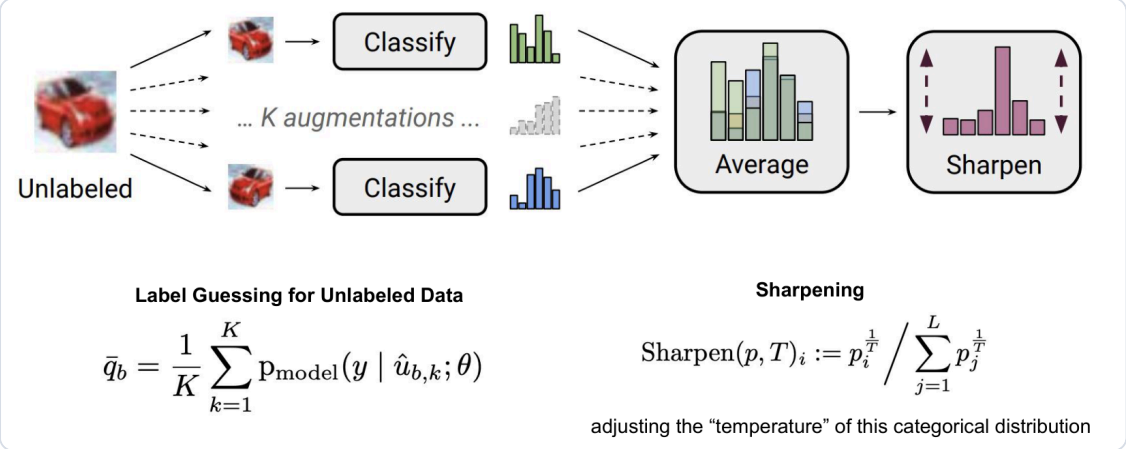
1. **Weak augmentation**(flip-and-shift: 뒤집기 + 살짝 이동)한 이미지로 예측. 거의 원본이라 안정적이야.
2. **Confidence threshold τ**: weak 예측 확률 q_b의 최댓값이 τ 이상이면 $\hat{y}_b$ = argmax q_b(**hard** one-hot)를 pseudo label로 써. 못 넘으면 버려.
3. **Strong augmentation**(RandAugment, CTAugment, Cutout 등: 크게 변형하고 일부를 가림)한 같은 이미지가 $\hat{y}_b$를 맞히도록 CE.

수식 $L_u = (1/\mu B) \Sigma_b 1[\max(\hat{y}_b) \geq \tau] \cdot \text{CE}(\hat{y}_b, p_\theta(y \mid A_{\text{strong}}(u_b)))$ (p.46). $1[\cdot]$는 조건이 맞으면 1, 아니면 0인 **confidence mask**. 슬라이드 식의 $\max(\hat{y}_b)$는 one-hot으로 바꾸기 전 weak 예측 q_b의 max, 즉 confidence야 (용어집의 q와 같은 것). τ는 confidence threshold고 temperature가 아니야 (학습 내내 0.95 고정, p.67). μB는 unlabeled batch 크기야 (B = labeled batch 크기, μ = unlabeled가 그 몇 배인지. 맨 위 목표식의 가중치 μ(t)와는 **다른 기호**). 그래서 통과 못 한 샘플도 분모에 들어가.

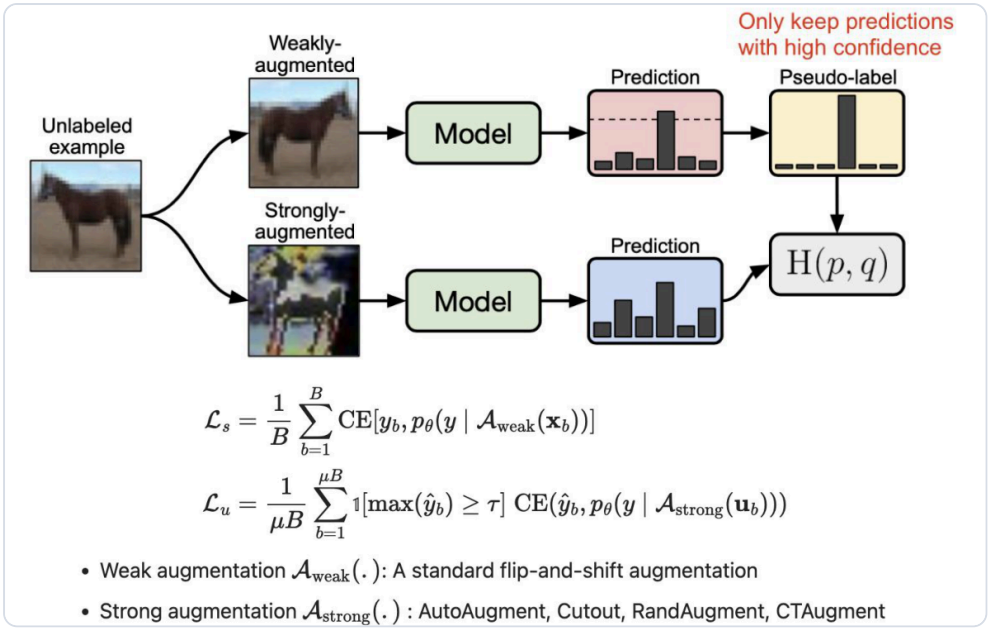
예시로 보면 unlabeled 4장의 weak confidence가 0.98, 0.70, 0.96, 0.40이고 τ = 0.95면 1번, 3번만 strong view에서 CE를 받아. 2번, 4번은 loss 0이라 $L_U = (CE_1 + CE_3) / 4$야. 분모는 통과한 2가 아니라 μB = 4.

왜 돼? weak 예측은 정확해서 믿을 만한 target, strong view는 어려워서 맞히려면 robust해져야 해. threshold가 애매한 예측을 걸러서 틀린 label을 줄여. 별도 teacher나 EMA 없이 **같은 모델**이 weak에서 label을 만들고 strong에서 배워.

	MIXMATCH NEURIPS 2019 · P.30-35	REMIXMATCH 2020 · P.36-41	FIXMATCH P.42-48
pseudo label	K개 aug 예측 **평균 → sharpen** (soft)	weak anchor 예측 + distribution alignment → sharpen	weak aug 예측, max ≥ τ면 one-hot (hard)
학습하는 입력	MixUp한 샘플	여러 strong aug (augmentation anchoring)	strong aug
unlabeled loss	**L2**	(슬라이드 강조 X)	CE
새로 넣은 것	여러 아이디어를 하나로 통합	DA, AA, rotation loss	뺌. sharpening·MixUp 없음



MixMatch의 핵심 두 단계. unlabeled 이미지 하나를 **K번 augmentation**해서 각각 분류하고(막대 = 예측 분포), 그 예측을 **평균(Average, label guessing)**한 뒤 **Sharpen**으로 뾰족하게 만들어 target으로 써. 순서가 평균 → sharpen인 걸 봐. Sharpen 상자의 위아래 화살표는 큰 값은 키우고 작은 값은 줄인다는 뜻이고, T는 sharpening temperature야. `L07 p.32`



$$\mathcal{L}_s = \frac{1}{B}\sum_{b=1}^{B} \mathrm{CE}[y_b, p_\theta(y \mid \mathcal{A}_{\text{weak}}(\mathbf{x}_b))]$$

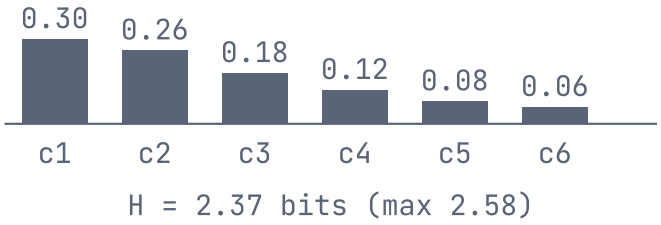
$$\mathcal{L}_u = \frac{1}{\mu B}\sum_{b=1}^{\mu B} \mathbb{1}[\max(\hat{y}_b) \geq \tau]\ \mathrm{CE}(\hat{y}_b, p_\theta(y \mid \mathcal{A}_{\text{strong}}(\mathbf{u}_b)))$$

- Weak augmentation $\mathcal{A}_{\text{weak}}(.)$: A standard flip-and-shift augmentation
- Strong augmentation $\mathcal{A}_{\text{strong}}(.)$: AutoAugment, Cutout, RandAugment, CTAugment

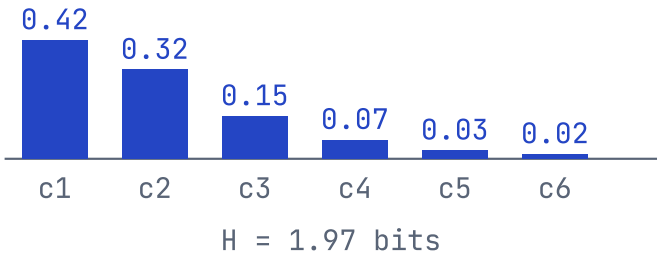
위 경로: weak augmentation(뒤집기·이동 정도) 이미지의 예측에서 점선(threshold τ)을 넘는 막대가 있을 때만 **one-hot pseudo label**을 만들어. 아래 경로: strong augmentation(크게 변형한) 이미지의 예측. 둘을 H(p, q) = cross-entropy로 이어서 strong 예측이 pseudo label을 맞히게 학습해. 식의 indicator $\mathbb{1}[\max(\hat{y}_b) \geq \tau]$가 confidence mask야. `L07 p.46`

인터랙티브 **Sharpening(MixMatch) vs confidence threshold(FixMatch)** illustrative

입력 prediction p



Sharpen(p, T) T = 0.50



방법 Sharpen (MixMatch) | Threshold (FixMatch)

예측 예시 A 애매 | B 꽤 확신 | C 거의 확신

temperature T T = 0.50

막대 c1–c6는 unlabeled 샘플 하나에 대한 6개 class 예측 확률이고, H는 그 entropy야 (여기선 bit 단위 $\log_2$라서 본문의 ln 기준 값 1.09 같은 숫자와 단위가 달라). **Sharpen**(MixMatch)은 `p_i^{1/T} / Σ_j p_j^{1/T}`라서 T가 작을수록 뾰족해지고 entropy가 줄어 (T=1이면 그대로, T→0이면 one-hot에 가까워져). 그래도 결과는 **soft한 분포**고, 애매한 샘플도 버리지 않고 모두 loss에 들어가. **Threshold**(FixMatch)는 max가 τ 이상인 샘플만 **hard one-hot**으로 쓰고 나머지는 버려 (mask = 0). "Threshold" 버튼을 누르고 "B 꽤 확신"(max 0.52)을 고른 뒤 τ를 0.52 위아래로 움직이면 one-hot ↔ 버림이 바뀌는 걸 볼 수 있어. FixMatch는 sharpening 없이 threshold만으로 더 좋았어. 예측값은 개념용 가짜 값. (L07 p.32, p.42–48)

FixMatch 한 줄: "Predict on an easy example, learn from a difficult one." threshold가 sharpening보다 효과적이야. 애매한 예측을 걸러내고 나면 sharpening의 추가 이득은 거의 없어 (p.48).

DivideMix

Li · p.49–55 · code p.54

문제 labeled data에 noisy label이 섞여 있어. 지금까지 SSL(semi-supervised learning)은 labeled는 깨끗하다고 가정했지만, 실제론 잘못 단 label(고양이 사진에 "dog")이 섞여 있어 (p.49–50).

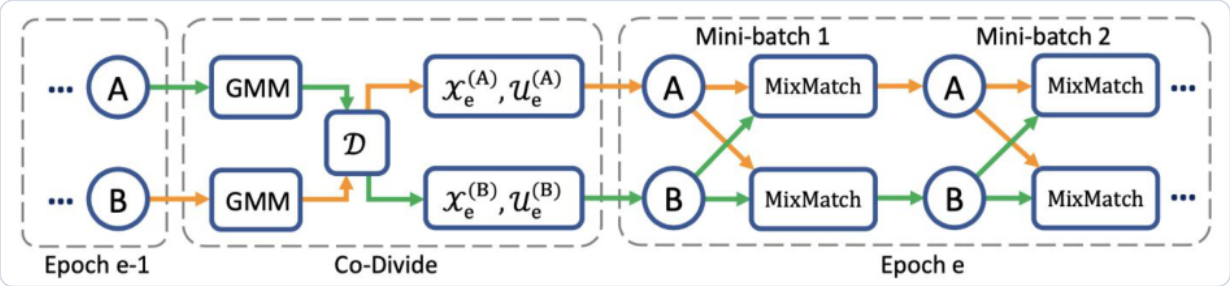
핵심 샘플별 training loss에 2-component **GMM** → loss 낮으면 clean, 높으면 noisy. noisy는 **label만** 버리고 unlabeled로 써서 MixMatch. 네트워크 2개가 **서로의** 분할을 써 (co-

divide).

1. **왜 loss?** 학습 초반엔 clean 샘플의 loss는 빨리 떨어지고 noisy는 높게 남아서 loss 분포가 두 덩어리가 돼 (p.53).
2. **GMM**(Gaussian Mixture Model: 분포를 Gaussian 여러 개의 합으로 봄)을 2개짜리로 맞춰. 평균이 낮은 Gaussian에 속할 확률 = clean probability w_i. $w_i \geq \tau$면 labeled, $w_i < \tau$면 unlabeled (이 τ는 clean probability threshold).
3. **Co-divide**: A의 loss로 만든 분할은 **B**가, B의 분할은 A가 써 (p.51, code line 4–5).
4. **MixMatch**: clean 샘플은 **co-refinement** $\bar{y}_b = w_b y_b + (1-w_b)p_b$ (원래 label을 상대 network가 준 clean 확률 w_b만큼만 믿고 나머지는 자기 예측 p_b로), noisy 샘플은 **co-guessing** (두 network 예측 평균). 둘 다 sharpen (p.54, code line 17–18 / line 20).

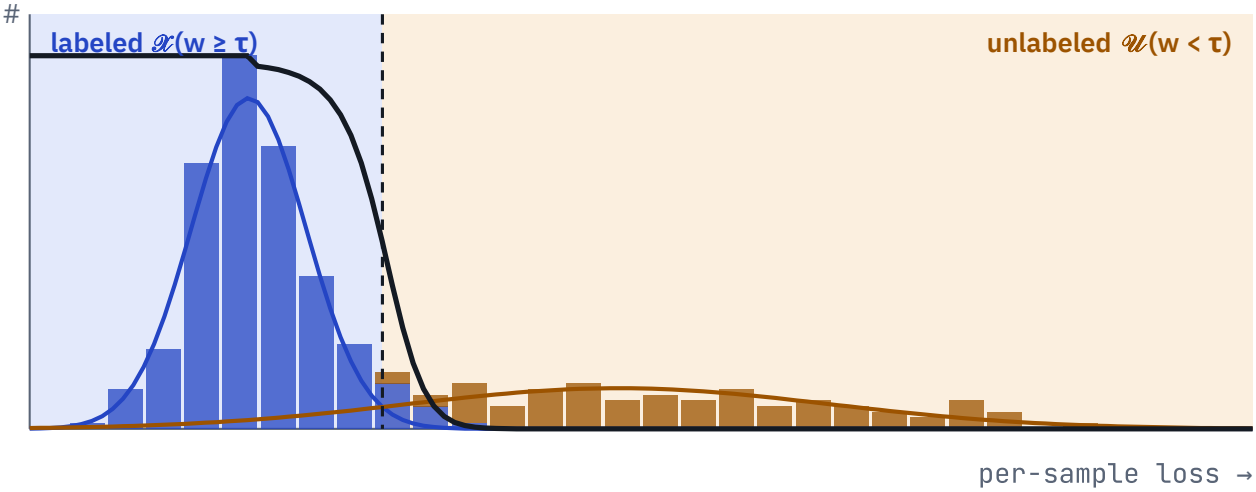
왜 둘? 자기 판단으로 자기를 학습하면 초기 오판이 강화돼. 두 network가 다른 관점에서 판단하니까 서로의 실수를 걸러줘 (p.51). A, B는 대등한 동료지 EMA teacher–student(Mean Teacher)가 아니야.

차이 새 학습 알고리즘이 아니라 MixMatch용 labeled/unlabeled를 만드는 방법 + co-refinement + co-guessing.



epoch마다 network A, B가 각자 샘플별 loss에 GMM을 맞추고(Co-Divide), 그 결과로 만든 labeled $\mathcal{X}$/ unlabeled $\mathcal{U}$ 분할을 **상대에게** 넘겨줘. 화살표 색을 따라가면 A의 GMM 결과가 B의 학습 세트 $\mathcal{X}_e^{(B)}$, $\mathcal{U}_e^{(B)}$로, B의 결과가 A 쪽으로 가. mini-batch에서도 화살표가 교차하는 건 MixMatch 학습 때 두 network가 함께 쓰이기 때문이야 (co-refinement, co-guessing). L07 p.52

인터랙티브 **DivideMix co-divide: loss의 2-GMM으로 clean / noisy 나누기** illustrative



진짜 clean (합성) 진짜 noisy (합성) GMM clean 성분 GMM noisy 성분 w = clean 확률 (0~1, 차트 높이에 맞춰 그림)

clean probability threshold τ 0.50 | 𝒳 | = 228 (그중 진짜 clean 227) · | 𝒰 | = 92

가로축은 샘플별 loss, 막대는 그 loss를 가진 샘플 수야 (범례대로 실제 clean / 실제 noisy를 색으로 구분한 합성 데이터라 모양만 봐). 학습 초반엔 clean은 loss가 낮고 noisy는 높아서 분포가 두 덩어리가 되고, 2-component GMM(Gaussian 두 개를 겹친 모델, 두 곡선)이 이걸 갈라 줘. 굵은 곡선 `w` = 낮은 loss 성분에 속할 확률(clean probability)이야. 슬라이더 τ를 움직이면 `w ≥ τ`인 쪽은 labeled 𝒳, 나머지는 **label만 버리고** unlabeled 𝒰로 MixMatch에 들어가. 실제 DivideMix에선 net A의 GMM 분할로 net B를 학습하고 반대도 마찬가지야 (co-divide, p.54 line 4–5). 단, 그림에서 w는 clean 성분 평균보다 loss가 낮은 구간을 1로 고정해서 그렸어. (L07 p.49–54)

Pre-training + SSL

Zoph · SimCLRv2, NeurIPS 2020 · p.56-63

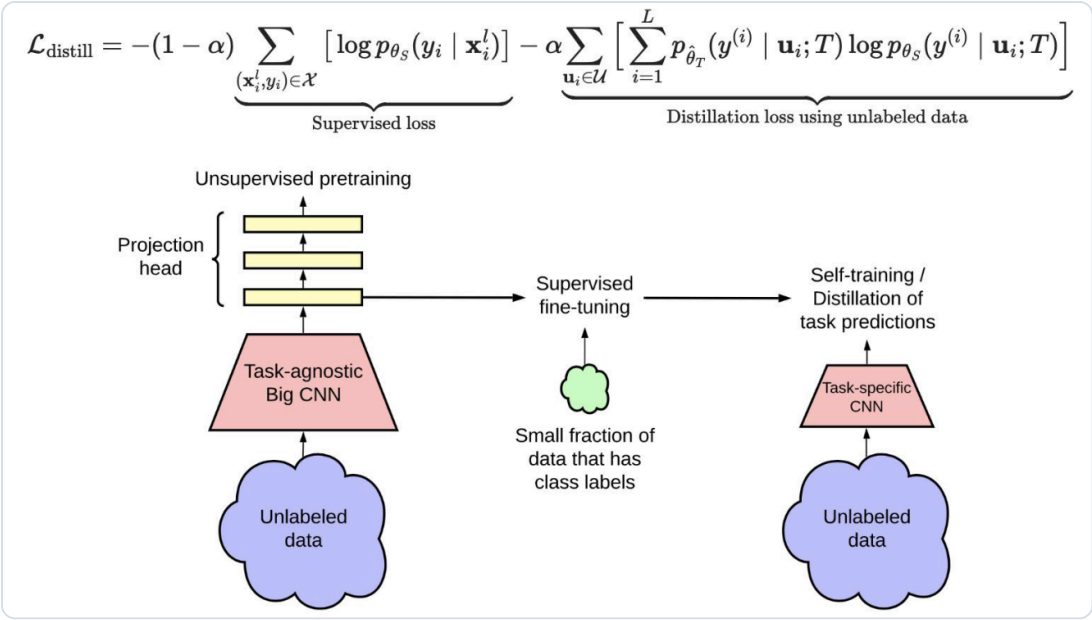
용어 pre-training = 본 학습 전에 큰 데이터(label 없이도)로 먼저 학습해 두기. fine-tuning = 그걸 적은 labeled로 목표 task에 맞게 더 학습하기. pre-training과 SSL(여기 SSL = semi-supervised learning: labeled 조금 + unlabeled 많이를 같이 학습)은 서로 대신하는 관계가 아니라 같이 써 (p.56: "SSL is not a replacement for pre-training"). p.56 정리: self-supervised pre-training + semi-supervised fine-tuning.

Zoph pre-training의 이득은 augmentation이 세지고 label이 많아질수록 줄어 (label이 **부족할 때** 가장 도움). self-training은 그래도 효과가 있어. 둘은 **보완 관계**. 대신 self-training은 계산이 비싸 (p.57–58).

SimCLRv2 (1) self-supervised pre-training (큰 모델, task와 무관한 범용 feature) → (2) supervised fine-tuning (적은 label로 task에 맞추기) → (3) unlabeled로 self-training / distillation (soft label): fine-tuned 모델이 teacher, task-specific student(더 작아도 돼)가 그 예측 분포를 따라 해 (p.59–61).

수식 $L_{distill} = -(1-\alpha) \Sigma \log p_{\theta_S}(y_i|x_i) - \alpha \Sigma_u \Sigma_c p_{\theta_T}(c|u;\ T) \log p_{\theta_S}(c|u;\ T)$ (p.61). 앞은 labeled CE, 뒤는 unlabeled에서 teacher(θ_T, 슬라이드 기호 $\hat{\theta}_T$)의 **soft** 분포를 target으로 한 student(θ_S)의 CE. α = 두 항 비중, T = temperature (아래첨자 T = teacher, 세미콜론 뒤 T = temperature. 같은 글자라 헷갈리기 쉬워).

차이 FixMatch는 hard label + threshold, SimCLRv2 self-training은 soft label + temperature (threshold 없음).



SimCLRv2 3단계. (1) 왼쪽: unlabeled 데이터로 task와 무관한 큰 CNN을 projection head와 함께 self-supervised pre-training → (2) 가운데: label 있는 적은 데이터로 supervised fine-tuning → (3) 오른쪽: 이 모델을 teacher로 삼아 unlabeled 데이터에서 task-specific CNN(student)으로 **distillation (soft label)**. 위 식 $L_{distill}$의 앞 항이 supervised loss, 뒤 항이 distillation loss야. `L07 p.61`

threshold 3부작 (p.105)

FixMatch 이후 남은 질문 셋: threshold는 고정이어야 하나? 버린 샘플은 쓸모없나? 통과한 샘플은 다 같은 perturbation을 받아야 하나? 셋 다 FixMatch 구조(weak로 pseudo label, strong에 학습) 위에서 답해.

- **FreeMatch** `Wang · p.64-75`

  FixMatch의 고정 τ 대신 **self-adaptive threshold**(SAT, Self-Adaptive Thresholding): 모델이 자기 학습 상태를 보고 threshold를 정해. 여기에 SAF(Self-Adaptive Fairness)를 더해.

  `문제` 고정 τ는 학습 내내, 모든 class에 같아. 초반엔 confidence가 낮아서 높은 τ가 거의 다 버리고, class마다 배우는 속도도 달라 (p.66–67).

  `global` $\tau_0 = 1/C$, $\tau_t = \lambda\tau_{t-1} + (1-\lambda)\cdot(1/\mu B)\ \Sigma_b\ \max(q_b)$ (p.69). q_b = weak view 예측, C = class 수. batch 평균 confidence를 EMA(이전 값 λ + 새 값 1−λ로 섞어 부드럽게 따라가기)로 누적해. 1/C는 아무것도 모를 때(균등 예측)의 confidence라서 처음엔 거의 다 통과하고, 모델이 자신감이 붙으면 같이 올라가. weight EMA가 아니라 **threshold 값**의 EMA야.

  `class별` $\tau_t(c) = \text{MaxNorm}(\tilde{p}_t(c)) \cdot \tau_t$ (p.70). $\tilde{p}_t(c)$ = class c 예측 확률의 EMA(그 class를 얼마나 배웠나), MaxNorm = 최댓값으로 나눠 최고 class를 1로. 쉬운 class는 τ_t에 가깝게(높게), 어려운 class는 더 낮게 받아. 샘플은 **자기가 예측한 class**의 threshold로 판정해 (p.71).

  `SAF` label이 아주 적으면 초반 예측이 몇 class에 쏠리고, 그 class가 pseudo label을 더 받아 더 강해져 (p.72). Self-Adaptive Fairness가 이 쏠림을 막아.

  어떤 pseudo label을 고를까?

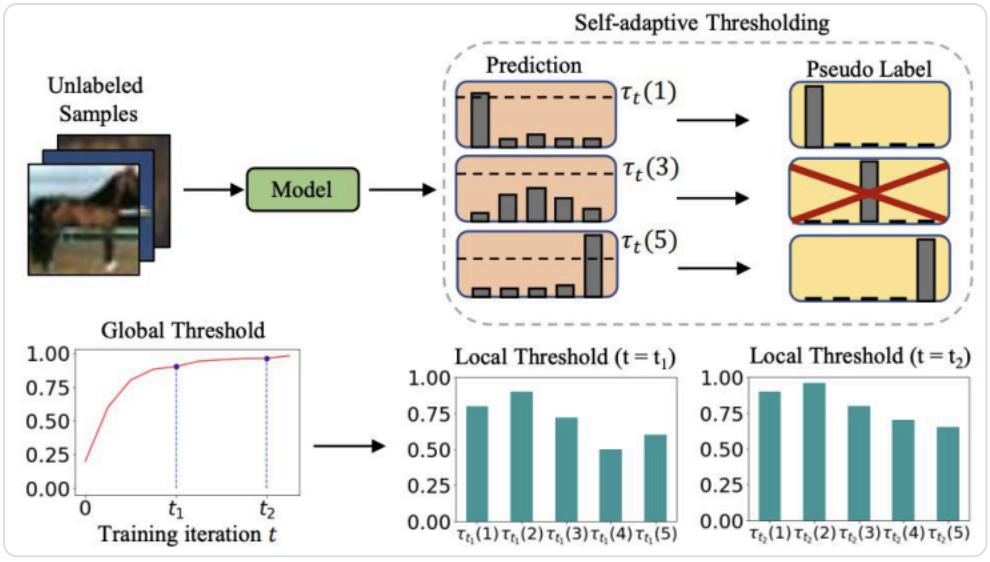
- **AllMatch** `Wu, IJCAI 2024 · p.76-91`

  버리던 low-confidence 샘플까지 **모든** 샘플을 쓰는 방법. class별 threshold는 **classifier weight norm**으로 정하고 (CAT), BCC로 후보 class vs negative class 이진 구분을 strong view에도 강제해.

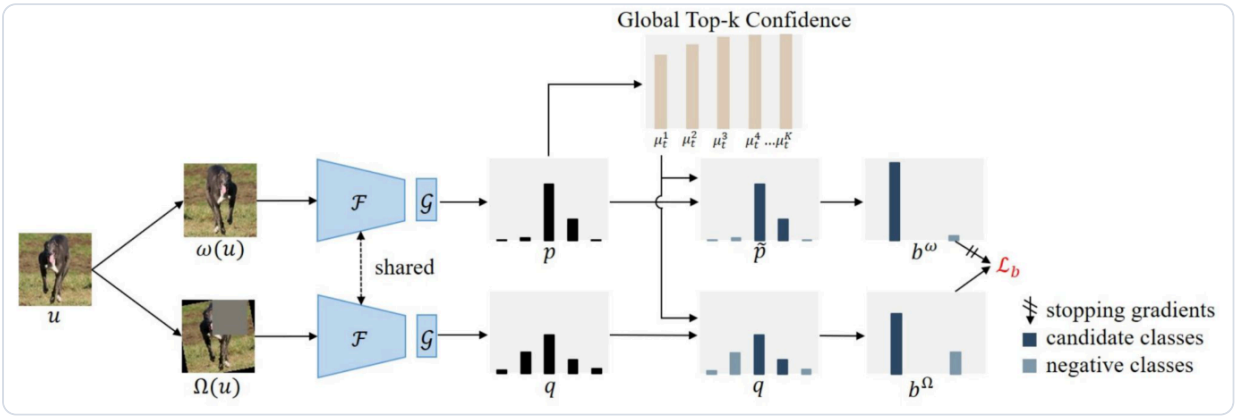
  `문제` threshold 미달 샘플은 학습 신호가 0이야. 그런데 버린 샘플을 모아 보면 1등(top-1) 예측의 정답률은 낮아도, top-5 정확도는 거의 100%야 (p.78).

CAT (Class-Specific Adaptive Threshold) global은 FreeMatch와 같은 EMA. class별 보정은 $\tau_t(c) = \tau_t \cdot \|W_c\| / \max_c \|W_c\|$ (p.82). W_c = 마지막 분류층의 class c weight. weight norm이 크면 logit이 커지고, confident한 샘플이 많은 class일수록 norm이 커서 학습 상태 지표가 돼. 어려운 class는 낮은 threshold.
BCC (Binary Classification Consistency) 상위 k_i개 class를 candidate(후보), 나머지를 negative로 나눠서 b = [candidate 확률 합, negative 확률 합]인 이진 예측으로 바꿔. candidate는 **weak view**에서 정하고 strong view에도 **같은 분할**을 써서 두 b가 같아지게 학습 (p.83, 86). indicator가 없어서 **모든** 샘플에 걸려. k_i는 confident하면 1, 애매할수록 크고, K(candidate 수 상한. MixMatch의 K와 다른 기호)를 넘지 않아 (전부 candidate면 정보 0, p.85).
예시로 보면 weak 예측 [cat 0.40, dog 0.35, fox 0.15, 나머지 0.10]은 top-1이 미달이야. candidate가 {cat, dog, fox}면 b = [0.90, 0.10]이고, strong view도 이 셋의 합이 높아야 해. "정답은 몰라도 car나 ship은 아니다"를 배우는 거야. 애매한 샘플은 coarse but reliable supervision (p.86–87).
안 고른 샘플은 어떻게 할까?

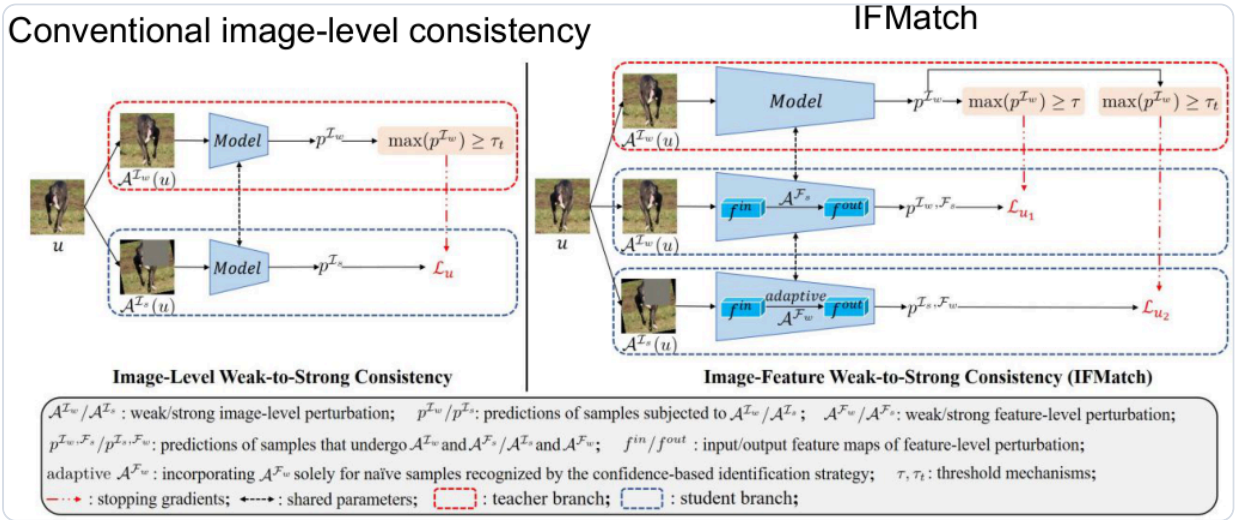
- **IFMatch** Wu & Cui, ECCV 2024 · p.92-104
  image perturbation에 **feature perturbation**을 더하고, strong aug 뒤에도 confident한 쉬운 샘플엔 추가 perturbation (CBI, Confidence-Based Identification). plug-in.
  문제 threshold를 통과한 샘플은 다 같은 strong aug를 받는데, 난이도는 같지 않아. naive(쉬운) 샘플은 strong aug 뒤에도 잘 맞혀서 loss가 거의 0이라 배울 게 없고, challenging 샘플은 이미 어려워서 더 흔들면 쓸모 있는 정보가 망가질 수 있어. naive가 너무 많은 게 문제 (p.96).
  핵심 feature perturbation = 이미지 대신 network 중간 feature map을 흔드는 것. image perturbation을 **대체하지 않고 추가** (p.97). residual block **뒤**에 넣으면 strong, residual branch **안**에 넣으면 identity 연결이 완화해서 weak (p.98).
  구조 teacher branch(별도 network가 아니라 같은 모델·같은 parameter에 weak image를 넣은 갈래. FixMatch의 weak 쪽과 같아)가 pseudo label을 만들고(stop-gradient), 두 student branch가 같은 pseudo label을 맞혀 (p.100).
  Branch I = **weak image + strong feature**. 틀린 label이 특히 해로워서 높은 고정 threshold (p.103).
  Branch II = **strong image + 조건부 weak feature**. 기존 image-level consistency, base 방법(FlexMatch, SoftMatch, FreeMatch)의 dynamic threshold.
  CBI strong image 뒤에도 pseudo label class 확률이 높으면 아직 "쉬움" → weak feature perturbation **추가**, 낮아졌으면 "어려움" → **안 줌** (p.102). strong image + strong feature를 한 branch에 다 넣으면 오히려 떨어져 (p.101). 새 pseudo label 생성법이 아니라 다른 thresholding 방법에 얹는 plug-in (p.104).
  모든 pseudo label이 같은 perturbation을 받아야 할까?



위: 샘플마다 **예측한 class의 threshold** $\tau_t(c)$(점선)와 비교해서, 넘으면 pseudo label(노란 상자)이 되고 못 넘으면 버려 (빨간 X, class 3). 아래 왼쪽: **global threshold**가 학습하면서 올라가. 아래 오른쪽: class별 **local threshold**. 같은 시점에도 class마다 높이가 달라서(쉬운 class는 높게, 어려운 class는 낮게) 어려운 class 샘플을 덜 버리고, $t_1 \rightarrow t_2$로 가면서 전반적으로 올라가. `L07 p.68`

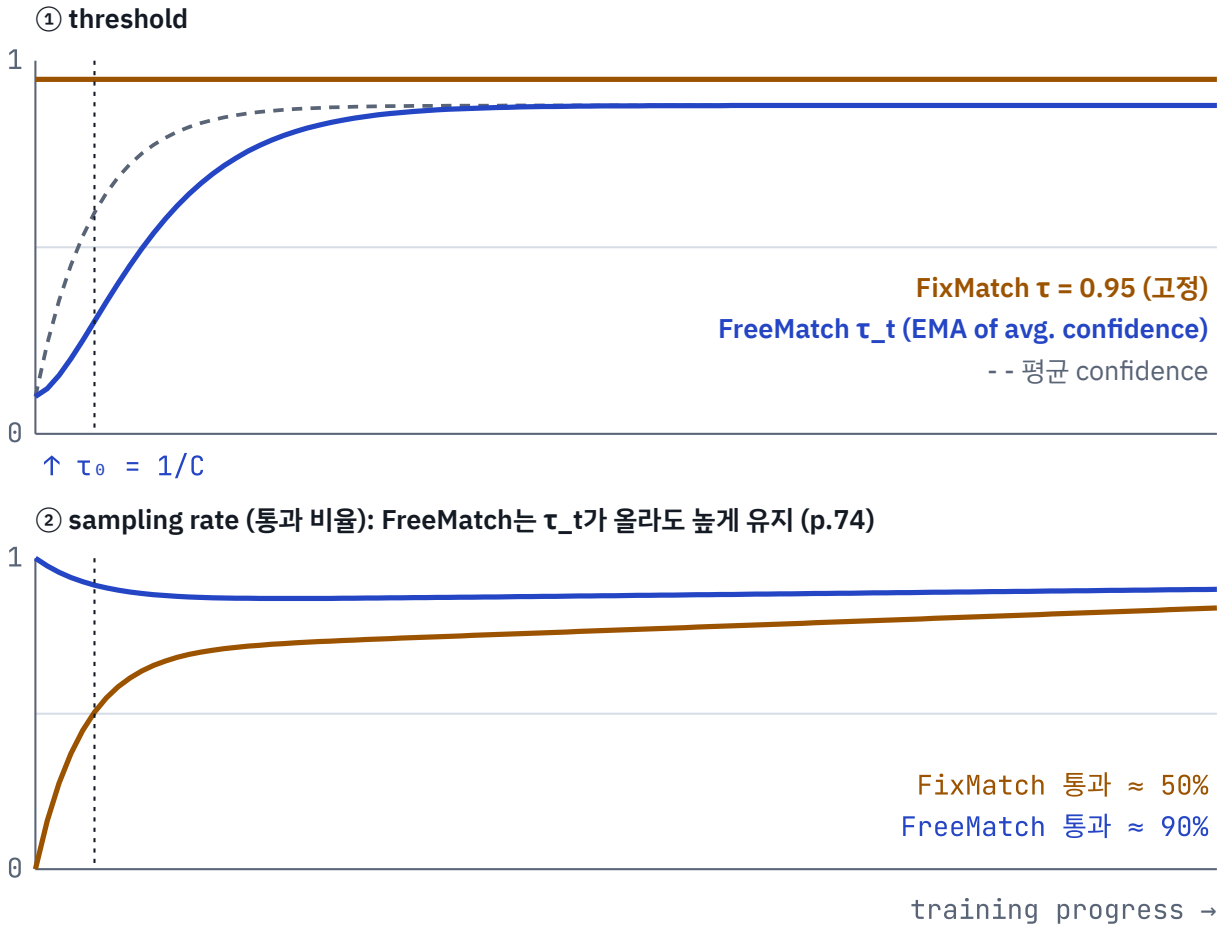


AllMatch BCC. 같은 u를 weak ω(u)·strong Ω(u)로 바꿔 shared network에 넣어. weak 예측으로 상위 class(진한 막대 = **candidate**)를 정하고 나머지(연한 막대 = **negative**)와 나눠서, 두 칸짜리 b^ω = [candidate 합, negative 합]을 만들어. strong 예측도 **같은 분할**로 b^Ω를 만들어 둘이 같아지게 L_b로 학습해. 위의 Global Top-k Confidence가 candidate 수 k를 정하는 기준이고, weak 쪽엔 stop-gradient야. `L07 p.83`



왼쪽은 기존 image-level weak→strong. 오른쪽 IFMatch에서 빨간 점선 상자는 teacher branch(별도 network가 아니라 같은 모델에 weak image를 넣은 갈래로 pseudo label을 만들어. 상자 사이 양방향 화살표가 shared parameters야), 파란 점선 상자 둘은 student branch야. **Branch I** = weak image + **strong feature** perturbation(새로 추가된 경로, 고정 τ로 거름), **Branch II** = strong image + **조건부(adaptive) weak feature** perturbation(τ_t로 거름)이고, weak feature는 strong image aug 뒤에도 confidence가 높은 "쉬운" 샘플(CBI)에만 붙어. image perturbation을 **대체하지 않고 feature perturbation을 추가**한 구조야. `L07 p.97`

개념 그래프 **고정 τ(FixMatch) vs self-adaptive τ_t(FreeMatch)** illustrative



training progress 5%

① 위는 threshold 값(0~1)이 학습하면서 어떻게 변하는지, ② 아래는 그 threshold를 넘어 pseudo label로 쓰이는 unlabeled 샘플 비율(sampling rate)이야. 슬라이더로 학습 시점을 옮겨 봐. FixMatch는 τ가 처음부터 끝까지 0.95라서 학습 맨 처음(confidence가 낮을 때)엔 거의 다 버리고, 모델이 confident해지면서 통과 비율이 빠르게 올라와. FreeMatch는 $\tau_0 = 1/C$ (여기선 C = 10이라 0.1)에서 시작해 평균 confidence(점선)의 EMA를 따라 올라가. 그래서 처음부터 거의 다 통과시키고, τ_t가 올라가 틀린 pseudo label을 거르는 후반에도 sampling rate가 **비교적 높게 유지돼** (p.74: remains relatively high). 즉 초반엔 양, 후반엔 질을 챙기면서 통과 비율을 크게 잃지 않아. (직관: 대부분 샘플의 confidence가 아주 높고 일부만 낮으면, 평균 confidence는 τ보다 낮아도 통과 비율은 높을 수 있어.) ② 곡선은 p.67 (d)·p.74 (b) 그래프의 대략적인 모양만 따라 그린 개념 그림이라 수치는 실제 값이 아냐. class별 보정(local threshold)은 생략. (L07 p.64–75)

RL로 번역

pseudo labeling은 자기 예측으로 자기 target을 만드는 **bootstrapping**이야.

- confirmation bias ↔ Q-learning에서 자기 추정치로 target을 만들 때 오차가 쌓이는 문제랑 같은 모양이고, 해법도 닮았어.
- DivideMix가 두 network의 분할을 서로 바꿔 쓰는 것 ↔ Double DQN·TD3 twin critic처럼 "다른 estimator로 내 target을 견제"하는 발상.
- Meta Pseudo Labels ↔ student가 labeled data에서 얼마나 좋아졌는지가 teacher한테 reward처럼 돌아가는 구조 (실제 전달은 gradient로).
- FixMatch의 weak → strong ↔ "target은 안정적인 입력에서, 학습은 어려운 입력에서". target을 덜 흔들리게 두려는 동기는 target network와 같지만, FixMatch엔 **target network(EMA teacher)가 없어**. 같은 network에 weak 입력을 넣어 target을 만들어. EMA teacher를 쓰는 건 Mean Teacher야.
- FreeMatch의 올라가는 threshold ↔ curriculum. 처음엔 기준을 낮춰 많이 배우고, 실력이 늘면 기준을 올려.

시험 함정

- Noisy Student: noise는 **student에만**, student ≥ teacher.
- MixMatch: **평균 먼저, 그다음 sharpen**. unlabeled는 **L2**.
- FixMatch: sharpening·MixUp·별도 teacher(EMA) 모두 **없음**. 같은 모델이 weak에서 label, strong에서 학습 (p.42–48). EMA teacher는 L06 Mean Teacher.
- DivideMix: noisy 샘플은 **label만** 버림. 분할은 **상대** network가 해줌. loss 낮으면 clean. (slide엔 ICML 2020인데 실제론 ICLR 2020 → 시험은 slide 표기로)
- SimCLRv2 순서: pre-train → fine-tune → distill.
- Pseudo labeling = **hard**-label 근사, boundary는 **low**-density로. Label Propagation = **similarity**-based. Meta PL teacher는 student의 **labeled loss**로 평가.
- τ: FixMatch = **confidence threshold**, DivideMix = clean probability threshold. sharpening(soft, 다 씀) ↔ threshold(hard, 버림).
- FreeMatch τ_t는 1/C에서 **상승**, 어려운 class는 **낮게**. local은 FreeMatch = 예측 EMA, AllMatch = **weight norm**. BCC candidate는 **weak** view에서, **모든** 샘플에.
- IFMatch: Branch I = weak image + **strong feature**, Branch II = **strong image** + 조건부 weak feature. 대체가 아니라 **추가**, CBI는 **쉬운** 샘플에만.

셀프 체크

빈칸 FixMatch: the pseudo label comes from a _____-augmented image, and the model is trained to predict it on a _____-augmented image.

weakly / strongly (p.42–48). weak view는 예측이 안정적이라 target을 만들고, strong view는 어려워서 맞히면서 robust해져.

T/F In DivideMix, each network divides its own training data into clean and noisy sets for itself.

False. $\theta^{(1)}$의 loss로 만든 분할은 $\theta^{(2)}$가 써 (co-divide, p.51–54). 자기 분할을 자기가 쓰면 자기 실수를 강화하니까 상대 판단을 빌려.

빈칸 FreeMatch's global threshold starts at _____ and rises as the model becomes more confident.

1/C (C = class 수, p.64–75). 균등 예측(아무것도 모를 때)의 confidence라서 처음엔 거의 다 통과하고, 평균 confidence의 EMA를 따라 올라가.

공통 패턴: 7강에 계속 나오는 부품

논문은 30개가 넘지만 부품은 몇 개 안 돼. 이 표 두 개가 잡히면 "이건 어느 방법?" 문제가 거의 풀려.

왜 같은 부품이 계속 나오나

7강 내내 사람이 붙인 label(정답)이 없거나, 적거나, 믿기 어려워(noisy, L07 DivideMix). 그래서 **학습 target(모델이 맞혀야 할 값)을 직접 만들어야** 해. 같은 사진의 두 변형본(augmentation)을 짝으로 정하거나, 모델 자기 출력을 target으로 써. 자기 출력을 쓰면 문제가 두 개 따라와.

1. **target이 같이 흔들려.** 모델이 매 step 바뀌니까 target도 매 step 바뀌어. 움직이는 과녁이야. → 부품: **EMA**(target 쪽을 천천히 움직이기) + **stop-gradient**(target 쪽엔 gradient 안 흘리기).
2. **꼼수 답이 생겨.** negative 없이 "두 출력을 같게"만 시키면 모든 입력에 같은 벡터를 내도 loss가 0이야. 이게 collapse. → 부품: negative(멀어져야 할 다른 샘플)로 밀어내기, 또는 predictor(online 쪽에만 붙인 작은 MLP) + EMA (BYOL), centering(teacher 출력에서 평균 빼기) + sharpening(낮은 temperature로 분포를 뾰족하게) (DINO), 상관행렬 → identity (Barlow Twins) 같은 장치.

그리고 target의 **모양**(정답 번호 하나 / 확률 분포 / 벡터 / 행렬)이 loss 종류를 거의 정해. 그래서 자주 나오는 loss도 몇 개 안 돼.

EMA: 뭘 천천히 따라가게 하나

EMA(exponential moving average, 지수 이동 평균)는 새 값을 조금씩만 섞는 평균이야. 기호만 다르고 식은 전부 이 모양이야: `slow ← m · slow + (1 - m) · fast`.

- `fast`: 매 step gradient로 바뀌는 쪽 (student · online · query), 또는 새로 잰 값 (이번 batch 평균, TE는 이번 epoch 예측).
- `slow`: 그걸 천천히 따라가며 target을 만드는 쪽 (teacher · target · key).
- `m`: 0과 1 사이 계수. 1에 가까울수록 **느리고 안정적**이야.

예시로 보면

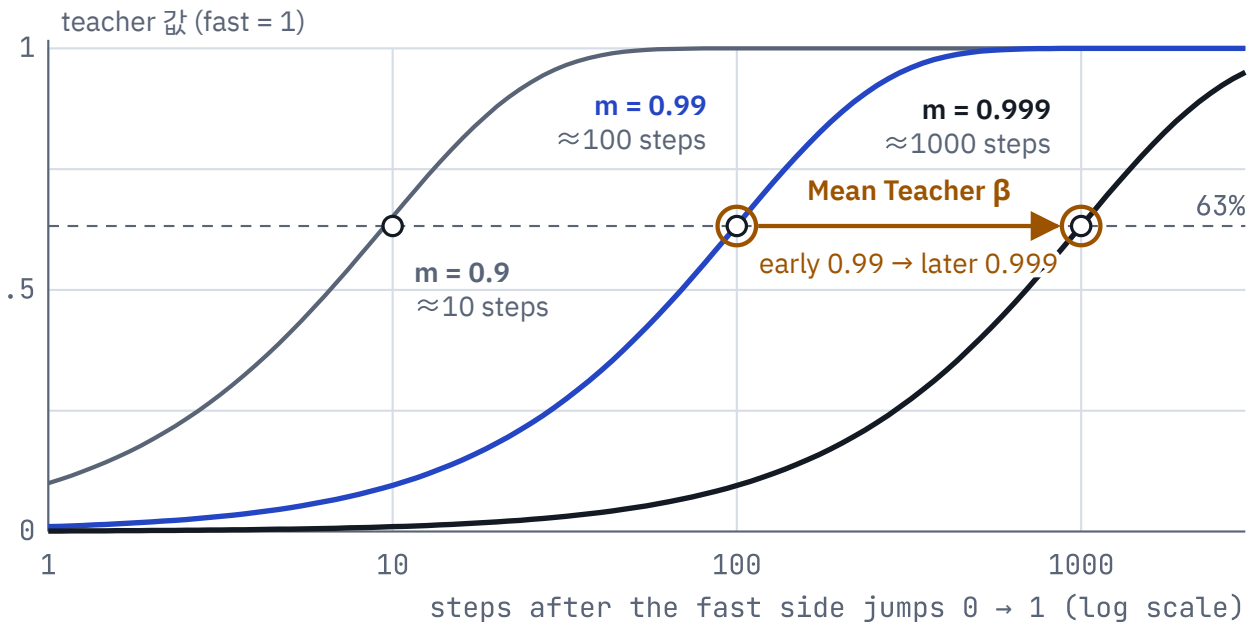
m = 0.99면 매 step 새 값을 1%만 섞어. 그래서 slow는 대략 최근 100 step(= 1/(1−m))의 평균처럼 움직여. m = 0.999면 약 1000 step이야. (설명용 숫자야.)

표에서 각 칸 두 번째 줄은 기호 풀이와 "왜 EMA를 쓰나"야. step(= iteration)은 mini-batch 하나로 update 한 번, epoch은 데이터 전체 한 바퀴야.

방법	EMA 대상	식 (슬라이드 기호)	언제
MoCo L05 p.12	key encoder weights queue에 쌓을 key를 만드는 encoder. 왜: key들이 일관(consistent)되게	`f_k = m*f_k + (1-m)*f_q` f_q: query encoder (gradient로 학습)	매 step

방법	EMA 대상	식 (슬라이드 기호)	언제
BYOL L04 p.30	target weights ξ 왜: 천천히 변하는 안정적 target	$\xi \leftarrow \tau\xi + (1-\tau)\theta$ θ: online, τ: EMA 계수	매 step
DINO L04 p.40	teacher weights **+ center C** EMA 2개. C = teacher 출력 평균, teacher 출력에서 빼(centering)	`gt = l*gt + (1-l)*gs` / `C = m*C + (1-m)*mean` gt / gs: teacher / student, l: 소문자 L	매 step
Mean Teacher L06 p.23-25	teacher weights teacher 예측 = consistency target	$\theta' \leftarrow \beta\theta' + (1-\beta)\theta$ θ: student, θ′: teacher, β: 0.99 → 0.999 (p.28)	매 iteration
Temporal Ensembling (TE) L06 p.20	**predictions** (weight 아님) sample마다 예측을 저장 → 메모리 많이 듦	$Z \leftarrow \alpha Z + (1-\alpha)z$, $\tilde{z} = Z/(1-\alpha^t)$ t: epoch 번호. Z가 0에서 시작해 생긴 치우침을 보정 (bias correction)	epoch 끝
FreeMatch L07 p.64-75	평균 confidence → threshold weight가 아니라 pseudo label 합격선 숫자 하나	$\tau_t = \lambda\tau_{t-1} + (1-\lambda)\cdot(1/\mu B)\Sigma \max(q_b)$, $\tau_0 = 1/C$ λ: EMA 계수, μB: batch 속 unlabeled 수, $\max(q_b)$: weak 예측의 최대 확률(confidence), C: class 수. 1/C(아무것도 모를 때의 confidence)에서 시작해서 모델이 확신할수록 올라가 (p.69)	매 step

개념 그래프 **EMA 계수 m이 클수록 teacher가 얼마나 느리게 따라오나** illustrative



TE만 예외

식 모양은 같지만 network **weight가 아니라 sample별 prediction**(예측값)의 EMA야 ($z \leftarrow \alpha z + (1-\alpha)z$). 매 step이 아니라 **epoch 끝**에 갱신해. Z가 0에서 출발해서 처음 몇 epoch은 값이 0쪽으로 치우치는데, 이걸 $\tilde{z} = z/(1-\alpha^t)$로 나눠 보정해 (t = epoch 번호). 위 곡선은 weight EMA 쪽 얘기야 (L06 p.20).

읽는 법: 빠른 쪽(student) 값이 어느 순간 0 → 1로 확 바뀌었다고 치고, 느린 쪽(teacher) 값이 1을 향해 따라오는 모양을 그린 거야. 가로축은 그 뒤로 지난 step 수(1 → 10 → 100 → 1000, 10배씩 늘어나는 log 눈금), 세로축은 teacher 값이야. 곡선 세 개가 m = 0.9 / 0.99 / 0.999이고, 식으로는 $1 - m^t$ (t step 뒤까지 따라온 비율)야. 점선 63%(= 1 − 1/e)에 닿는 데 대략 $1/(1-m)$ step이 걸려서 m = 0.9면 약 10, 0.99면 약 100, 0.999면 약 1000 step이야. 즉 m이 1에 가까울수록 teacher는 느리지만 안정적이야. "Mean Teacher β" 화살표는 처음엔 β = 0.99로 빨리 따라가다 student가 수렴하면 β = 0.999로 teacher를 더 안정적으로 만드는 schedule이야 (L06 p.28). MoCo의 key encoder도 같은 weight EMA인데 m 값은 슬라이드에 안 나와서 표시 안 했어 (L05 p.12). 개념 그림.

Stop-gradient: gradient는 빠른 쪽으로만

방향은 항상 **느린 쪽 ← 빠른 쪽**. gradient는 빠른 쪽(student · online · query)만 받고, 느린 쪽엔 stop-gradient를 걸어. stop-gradient는 "이 값은 상수 취급, backprop 금지"라는 뜻이고 PyTorch로는 `.detach()`야. 그래서 느린 쪽은 **gradient가 아니라 EMA로만** 바뀌어 (안 바뀌는 게 아니야).

- MoCo `k = k.detach()` (L05 p.12) · BYOL target에 sg (L04 p.30) · DINO `t = t.detach()` (L04 p.40) · Mean Teacher teacher "Parameters updated only by EMA", "No backpropagation" (L06 p.24).
- X-CLR도 target을 만드는 text encoder 출력을 `.detach()` 해. EMA가 아니라 고정(frozen) encoder지만 "target 쪽엔 gradient 없음"은 같아 (L03 p.67).
- SimCLR · Barlow Twins · CLIP엔 EMA도 stop-gradient도 없어. 두 branch 모두 backprop으로 학습해.

RL로 번역

RL의 target network랑 똑같아. TD target $r + \gamma Q_{\text{target}}(s', a')$은 천천히 따라오는 target network로 계산하고 (SAC · TD3의 soft update = EMA), 거기엔 gradient를 안 흘려(semi-gradient). 자기 자신으로 만든 target이 같이 움직이면 불안정해지니까. teacher = target network, student = online network야.

Loss: 누가 뭘 쓰나

loss는 "출력이 target과 얼마나 다른가"를 숫자 하나로 잰 거야. 여기선 자주 나오는 6종만 target 모양별로 묶었어 (L03의 margin 계열 contrastive · triplet · lifted loss와 NCE는 이 묶음 밖이야).

- **InfoNCE**: 후보 N개 중 positive 하나 맞히기. similarity를 τ(temperature)로 나눠 softmax에 넣고 positive 칸 확률을 키워: $-\log \left[\exp(\mathrm{sim}(q, k^+)/\tau) / \Sigma_i \exp(\mathrm{sim}(q, k_i)/\tau) \right]$. 분모에 negative가 들어가. 정답이 index 하나인 N-way cross-entropy(CE)야.
- **분포 간 CE (soft target)**: $H(t, s) = -\Sigma_k t_k \log s_k$. target t가 one-hot(정답 칸만 1)이 아니라 퍼진 확률 분포야. t = X-CLR은 text similarity 분포, DINO는 teacher softmax, SwAV는 다른 view의 code(feature를 prototype = 학습되는 cluster 중심 벡터들에 배정한 확률).
- **MSE / L2**: 두 벡터 차이 제곱 $\|a - b\|^2$. BYOL은 정규화한 벡터라 $2 - 2\cos$. L06 consistency 계열은 두 예측끼리.
- **KL**: 두 확률 분포의 차이 $\Sigma\ p \log(p/q)$. UDA가 $p(y|x)$와 $p(y|\mathrm{Aug}(x))$를 맞출 때 써.
- **hard pseudo label CE**: argmax(가장 큰 확률의 class)나 cluster 번호를 one-hot 정답으로 놓고 보통 CE.
- **상관행렬 → identity**: feature 차원끼리 비교. 대각 → 1 (invariance), 비대각 → 0 (redundancy reduction).

negative(밀어낼 다른 샘플)를 softmax 분모에 넣는 건 이 6종 중 InfoNCE 계열(Soft-NN · SupCon 포함)과 X-CLR뿐이야. L03의 contrastive · triplet loss(p.11–13)도 negative를 쓰지만 분모가 아니라 margin 식으로 쓰고, NCE(p.21–22)는 sigmoid 이진 분류로 써.

그리고 soft CE와 KL은 target 쪽에 gradient가 안 흐르면(stop-gradient) 학습 방향이 같아. $H(t, s) = H(t) + KL(t\|s)$에서 $H(t) = -\Sigma_k t_k \log t_k$는 target 자신의 entropy라서 student 입장에선 상수거든. 그래도 시험에선 슬라이드 이름 그대로 써.

LOSS	쓰는 방법
InfoNCE (positive 하나 맞히기)	N-pair · CPC (L03), SimCLR (L04), MoCo · CLIP · SupCon (L05) CPC = InfoNCE를 처음 낸 논문, SupCon = label로 같은 class를 전부 positive로
분포 간 CE (soft target)	X-CLR (L03), DINO (L04), SwAV code 예측 (L05)
MSE / L2	BYOL (normalized, L04), Π-model · TE · Mean Teacher (L06), MixMatch unlabeled (L07) TE = Temporal Ensembling
KL	UDA (L04 p.18, L06 p.39–42) UDA = Unsupervised Data Augmentation
hard pseudo label CE	DeepCluster (L05), Pseudo-Label · FixMatch (L07)
상관행렬 → identity	Barlow Twins (L04), All4One의 feature contrast (L05)

구조 비교 **loss 지도: target의 모양 × negative를 직접 쓰나**

	hard index one-hot CE	**soft distribution** CE / KL	**regression vector** MSE / L2	**matrix** correlation → I
negative 사용 O	● N-pair ● CPC (InfoNCE) ● SimCLR ● MoCo ● CLIP ● SupCon*	● X-CLR	—	—
negative 사용 X	● DeepCluster ○ Pseudo-Label ○ FixMatch	● DINO ● SwAV ● UDA	● BYOL ● Pi-model ● Temp. Ens. ● Mean Teacher ○ MixMatch	● Barlow Twins

lecture ● L03 ● L04 ● L05 ● L06 ○ L07

읽는 법: 점 하나가 방법 하나, 점 색은 강의 번호야 (맨 아래 범례, L07은 속이 빈 점). 가로 4칸은 loss가 맞히려는 target의 모양이야: **hard index**(정답 번호 하나 → one-hot CE) / **soft distribution**(확률 분포 → CE · KL) / **regression vector**(벡터 자체를 맞히기 → MSE · L2) / **matrix**(상관행렬을 identity로). 세로 2줄은 negative(밀어낼 다른 샘플)를 loss에 직접 넣는지야. negative를 쓰는 방법은 거의 전부 **InfoNCE(N개 후보 중 positive index 하나 맞히기)** 칸에 몰려 있고, X-CLR만 그 target을 soft 분포로 바꿨어 (L03 p.62–72). negative 없는 쪽은 **soft 분포(DINO · SwAV · UDA)**, **회귀(BYOL · L06 consistency 계열 · MixMatch)**, **행렬(Barlow Twins)**, **hard pseudo label(DeepCluster · Pseudo-Label (Lee 2013) · FixMatch)**로 갈려. InfoNCE와 hard pseudo label이 같은 열에 있는 건 둘 다 "정답 번호 하나"를 맞히는 CE라서야. 차이는 후보가 다른 샘플(batch, MoCo는 queue)이냐(InfoNCE), class · cluster냐(pseudo label)야. 이 칸 하나만 기억해도 "이건 어느 방법?"이 좁혀져. 위 loss 표를 한 장으로 다시 그린 거야.

* SupCon은 positive가 여러 개(multi-positive)라 "index 하나"는 대략적인 표현이야. All4One의 feature contrast(행렬)는 neighbour · centroid contrast랑 섞인 방법이라 이 지도에선 뺐어. L03의 margin 계열(contrastive · triplet · lifted)과 NCE도 negative를 쓰지만 target 모양 축에 안 맞아서 이 지도엔 없어.

양방향(symmetric)은 "view 1로 view 2 맞히기"와 "view 2로 view 1 맞히기"를 둘 다 계산해서 더하는(평균 내는) 거야 (CLIP은 image → text, text → image): SimCLR, CLIP, DINO (`H(t1,s2)/2 + H(t2,s1)/2`), BYOL, SwAV ($\ell(z_t,\ q_s)$ + $\ell(z_s,\ q_t)$, L05 p.27: 두 view가 서로의 code를 예측). 이 다섯과 비교하면 **MoCo만 한 방향** (query → key)이야.

시험 함정 · 패턴끼리 헷갈리기

- TE = **prediction**의 EMA (epoch 끝), Mean Teacher = **weight**의 EMA (매 iteration), FreeMatch = **threshold**의 EMA.
- BYOL의 τ = EMA 계수. DINO 코드의 `l` = teacher momentum (숫자 1 아님). DINO centering은 **teacher에만**.
- stop-gradient 위치: key · target · teacher (gradient 받는 쪽 = query · online · student).
- BYOL = MSE, DINO = CE, UDA = KL, Mean Teacher = MSE (KL보다 나음, L06 p.28), MixMatch unlabeled = L2, FixMatch = hard label CE.

Pseudo code 9개: 핵심 줄만

교수님이 깊게 안 다뤘으니까 구현 디테일보다 **core idea가 박힌 줄**이 나올 거야. 방법마다 이 줄만 알아도 "이 코드는 어느 방법?"이랑 "이 줄의 역할은?"은 풀려. 전체 정리는 repo의 `exam/pseudocode.md`에 있어.

빈칸 유형도 나와 (`k = k._____()`). 코드 블록은 슬라이드에서 핵심 줄만 골라 옮긴 거고 (한국어 주석은 설명용), 색이 들어간 줄이 core idea가 박힌 줄이야. 블록마다 **한 줄 → 입력·출력 → 변수 → (예시) → 핵심 줄 → core idea → 알아보는 법** 순서로 풀었어.

먼저: pseudo code 읽는 법

슬라이드 코드는 두 종류야. MoCo · DINO · CLIP · X-CLR은 **PyTorch/numpy 스타일**(`#` 뒤가 주석), SimCLR · Π-model · Temporal Ensembling · Label Propagation · DivideMix는 **논문식 Algorithm 박스**(`←`나 `=`가 대입, `▷` · `//` · `#` 뒤가 주석). 9개 모두 결국 세 질문에 답해: **정답(target)을 어디서 만드나, gradient를 어디서 끊나, 무엇을 천천히(EMA) 바꾸나.** 답이 "없음"인 것도 시험 포인트야 (SimCLR · CLIP엔 detach도 EMA도 없어). 아래 줄 모양만 알면 답이 코드에서 보여.

`x.detach()`
MoCo · DINO · X-CLR

텐서를 계산 그래프에서 떼어내. `loss.backward()`를 해도 이 텐서를 거쳐서는 gradient가 안 흘러. 이게 stop-gradient(도식의 "sg")야. 거의 항상 **정답(target) 쪽**에 붙어: "정답은 고정된 숫자로 보고 예측하는 쪽만 고쳐라".

RL TD target $r + \gamma \max Q_{target}(s', a')$를 상수로 보는 semi-gradient, `with torch.no_grad()`와 같은 역할.

`mm` · `np.dot` · `@`
MoCo · CLIP · X-CLR

`mm(A, B)`, `np.dot(A, B)`, `A @ B` 셋 다 행렬 곱. `(N×C)·(C×K) → N×K`처럼 안쪽 차원이 같아야 하고 곱하면 사라져. (i, j) 칸 = A의 i번째 행 · B의 j번째 열의 dot product라서, 행마다 샘플 벡터를 넣으면 **모든 쌍의 similarity 표**가 한 줄로 나와. 벡터 길이를 1로 맞춰 두면 (`l2_normalize`) dot product = cosine similarity. `.T` = transpose, `.view(N,C)` = 모양만 바꾸는 reshape, `bmm` = 샘플마다 따로 하는 행렬 곱.

`logits` · `labels`
MoCo · CLIP

`logits`는 softmax 전 점수표(logit). 한 행 = 기준 샘플(anchor/query), 한 칸 = 후보 하나, 값 = similarity. `labels`는 각 행에서 **정답 칸의 번호**야. 사람이 단 class label이 아니라 "positive가 몇 번 칸에 있나"를 적은 것. MoCo `zeros(N)` = 모든 행의 정답이 0번 칸, CLIP `np.arange(n)` = `[0, 1, …, n-1]`이라 i번째 행의 정답이 i번 칸(대각선).

`cross_entropy`
MoCo · CLIP

`cross_entropy(logits, labels)`, `CrossEntropyLoss`는 평범한 분류 loss(cross-entropy). 행마다 softmax 후 정답 칸 확률에 `-log`를 씌워 평균. 줄이려면 정답 칸 점수는 올리고 나머지는 내려야 해. 정답 칸 = positive, 나머지 = negative로 두면 이게 바로 InfoNCE야. 즉 **InfoNCE = "positive가 몇 번 칸인지 맞히는 분류"**.

RL behavior cloning의 $-\log \pi(a^*|s)$와 같은 모양. 정답 action 대신 정답 칸을 맞혀.

`softmax(x / τ)`
SimCLR · MoCo · DINO · X-CLR

각 행(`dim=1`)을 합이 1인 확률분포로. temperature τ가 작으면 분포가 뾰족(sharp), 크면 평평(soft)해져. 이름은 SimCLR `τ`, MoCo `t`, DINO `tps`·`tpt`, X-CLR `tao`·`tao_s`이고 다 **나누기**야. **CLIP만 `* np.exp(t)`로 곱해** (t를 학습).

`CE(p, q)` · `H(t, s)` X-CLR · DINO	정답이 칸 번호가 아니라 **확률분포**일 때의 cross-entropy $-\Sigma_k\ p_k\ \log\ q_k$. 앞이 정답 분포, 뒤가 예측 분포야. 한 칸만 1인 one-hot 대신 여러 칸에 퍼진 **soft target**을 쓸 때 이 모양이 나와.
EMA 줄 MoCo · DINO · Temporal Ensembling	`slow = m*slow + (1-m)*fast` 모양이면 EMA(exponential moving average)야. 왼쪽(대입받는 쪽)이 매번 오른쪽 값과의 차이의 (1−m)배만큼만 다가가서, m이 1에 가까울수록 천천히 따라가. 대입받는 쪽이 느린 쪽(teacher, key encoder, 누적 예측)이야. 시험 포인트는 **무엇을 EMA하나**: MoCo = key encoder weight, DINO = teacher weight + center, Temporal Ensembling = 샘플별 prediction. RL target network soft update $\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-$와 같은 식. RL은 작은 τ를 빠른 쪽에 곱하고 여기선 큰 m을 느린 쪽에 곱해서, m이 $1 - \tau_{RL}$ 자리야.
`.mean(dim=0)` · `cat` · Algorithm 기호 DINO · 수식 박스	`dim=0` = 샘플(batch) 축이라 `.mean(dim=0)`은 샘플끼리 평균 내서 벡터 하나로. `cat` = 이어 붙이기. Algorithm 박스의 $\Sigma_{i\in B}$ = minibatch B 안에서 합, $\|B\|$ = 샘플 수, $\|v\|^2$ = 원소 제곱의 합. `▷` 주석("▷ construct target vectors by bias correction", "▷ diffusion")은 빈칸으로 내기 좋아.

같은 글자, 다른 뜻

- **N / n**: 보통 batch 크기. 단 Temporal Ensembling의 N과 Label Propagation의 n은 **전체 샘플 수**.
- **C**: MoCo = embedding 차원 (`q`가 N×C), Π-model · TE = class 수, DINO = center.
- **K / k**: MoCo `K` = queue 속 key 개수, MoCo `k` = key 벡터 (N×C, `k.detach()`의 그 k), DINO K = 출력 차원, LP k = kNN 이웃 수. SimCLR k는 두 뜻이야: x_k와 L의 Σ_k에선 이미지 번호, ℓ의 분모 $\Sigma_{k\neq i}$에선 2N개 view를 훑는 번호.
- **τ / T / t** (temperature 이름은 softmax 항목 참고, 여기선 temperature가 **아닌** 뜻만): SimCLR `t, t'` = augmentation 함수, T = augmentation 모음. DivideMix τ = clean probability **threshold** (DivideMix의 sharpening temperature는 대문자 T). LP T = epoch 수. Π · TE t = epoch 번호.
- **m · α**: MoCo m = momentum, DINO m = center momentum (network는 `l`), DivideMix m = augmentation 번호. TE α = ensembling momentum, LP α = diffusion 식의 parameter.

InfoNCE 가족

셋 다 loss 뼈대는 InfoNCE야: positive 1개와 negative 여러 개를 한 행에 놓고 softmax CE로 "positive가 어느 거?"를 맞혀. 차이는 (1) **negative를 어디서 가져오나**(같은 batch · queue), (2) **labels를 어떻게 만드나**, (3) **gradient가 어디로 흐르나**(MoCo만 key 쪽 detach + EMA), (4) **양방향이냐**(MoCo만 한 방향)야.

SimCLR Alg.1 · L04 p.25

```
draw two augmentation functions t~T, t'~T
ℓ(i,j) = -log exp(s_ij/τ) / Σ_{k≠i} exp(s_ik/τ)
L = 1/(2N) Σ_k [ℓ(2k-1, 2k) + ℓ(2k, 2k-1)]
return encoder f(·), and throw away g(·)
```

한 줄 같은 이미지 두 view가 positive, 나머지 2N−2개가 negative. 양방향으로 평균.

입력 → 출력 이미지 N장 minibatch, temperature τ, encoder f, projection head g(encoder 뒤의 작은 MLP), augmentation 모음 T를 받아서, 학습이 끝나면 **encoder f**를 돌려줘.

변수

- `t, t' ~ T`: 랜덤 augmentation(crop, 색 변경 같은 변형) 두 개. 이미지 x_k에서 view 두 개를 만들어 번호 2k-1, 2k를 붙여 → 총 **2N개 view**.
- s_{ij}: view i, j의 projection `z = g(f(x̃))`끼리의 cosine similarity (2N×2N).
- `ℓ(i,j)`: view i가 anchor(기준), view j가 positive일 때의 loss. 분모 $\Sigma_{k\neq i}$는 자기 자신만 뺀 2N−1개 (positive 1 + negative 2N−2).

예시 N = 4면 view 8개. view 1이 anchor면 positive는 view 2, negative는 view 3–8의 6개(= 2N−2), softmax는 7칸짜리야. (숫자는 설명용)

핵심 줄

1. `L = 1/(2N) Σ_k [ℓ(2k-1, 2k) + ℓ(2k, 2k-1)]`: 이미지마다 두 view가 번갈아 anchor가 돼서 loss가 2N개, 그 평균이야 ("use each of the 2N sample as reference") → **symmetric**.
2. `throw away g(·)`: 학습 후 g는 버리고 f의 출력 h만 downstream에 써. z는 augmentation에 invariant하게 학습돼서 색 · 방향 같은 정보가 빠지는데, 그 손실을 g가 떠안으니 h에 정보가 더 남아 (p.28).

core idea supervision이 사람 label이 아니라 **augmentation**에서 나와 ("maximizing agreement between differently augmented views of the same sample", p.23). negative는 memory bank나 queue 없이 **같은 batch**에서만 오니까 large batch가 중요해 (p.29).

알아보는 법 수식형 박스, view 번호 `2k-1, 2k`, `1/(2N)`, `throw away g(·)`. `labels` 변수가 없어 (분자에 positive s_{ij}를 직접 넣음). modality가 이미지 하나면 SimCLR, image · text 둘이면 CLIP.

▼ 슬라이드 원본 보기 (L04 p.25)

SimCLR

Algorithm 1 SimCLR's main learning algorithm.

input: batch size N, constant τ, structure of $f, g, \mathcal{T}$.
for sampled minibatch $\{\boldsymbol{x}_k\}_{k=1}^{N}$ **do**
  for all $k \in \{1, \ldots, N\}$ **do**
    draw two augmentation functions $t \sim \mathcal{T}, t' \sim \mathcal{T}$
    # the first augmentation
    $\tilde{\boldsymbol{x}}_{2k-1} = t(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k-1} = f(\tilde{\boldsymbol{x}}_{2k-1})$ # representation
    $\boldsymbol{z}_{2k-1} = g(\boldsymbol{h}_{2k-1})$ # projection
    # the second augmentation
    $\tilde{\boldsymbol{x}}_{2k} = t'(\boldsymbol{x}_k)$
    $\boldsymbol{h}_{2k} = f(\tilde{\boldsymbol{x}}_{2k})$ # representation
    $\boldsymbol{z}_{2k} = g(\boldsymbol{h}_{2k})$ # projection
  end for
  for all $i \in \{1, \ldots, 2N\}$ and $j \in \{1, \ldots, 2N\}$ **do**
    $s_{i,j} = \boldsymbol{z}_i^\top \boldsymbol{z}_j / (\|\boldsymbol{z}_i\| \|\boldsymbol{z}_j\|)$ # pairwise similarity
  end for
  define $\ell(i, j)$ **as** $\ell(i,j) = -\log \frac{\exp(s_{i,j}/\tau)}{\sum_{k=1}^{2N} \mathbb{1}_{[k \neq i]} \exp(s_{i,k}/\tau)}$
  $\mathcal{L} = \frac{1}{2N} \sum_{k=1}^{N} [\ell(2k-1, 2k) + \ell(2k, 2k-1)]$
  update networks f and g to minimize $\mathcal{L}$
end for
return encoder network $f(\cdot)$, and throw away $g(\cdot)$

Generate a **positive pair** by sampling data augmentation functions

Iterate through and use each of the 2N sample as reference, compute average loss

InfoNCE loss:
Use all non-positive samples in the batch as x -

Chen et al., ICML 2020, A Simple Framework for Contrastive Learning of Visual Representations

MoCo Alg.1 · L05 p.12

```
k = k.detach()                      # no gradient to keys
l_neg = mm(q.view(N,C), queue.view(C,K))
labels = zeros(N)                   # positive = 0번째 칸
f_k.params = m*f_k.params + (1-m)*f_q.params
enqueue(queue, k); dequeue(queue)
```

한 줄 momentum encoder + FIFO queue로, batch 크기와 상관없이 크고 일관된 negative 모음(dictionary)을 쓰는 InfoNCE.

입력 → 출력 minibatch x(N장)를 두 번 augment해서 `x_q`, `x_k`를 만들고, 매 step contrastive loss로 query encoder `f_q`를 SGD 학습해. key encoder `f_k`와 `queue`는 학습 내내 들고 다니는 상태야.

변수

- `q`(query), `k`(key): 각각 N×C. **C는 embedding 차원**(class 수 아님). 같은 이미지의 q와 k가 positive 짝.
- `queue`: 과거 batch들의 key K개를 쌓은 dictionary(negative 모음), C×K. K는 batch 크기와 따로 정해.

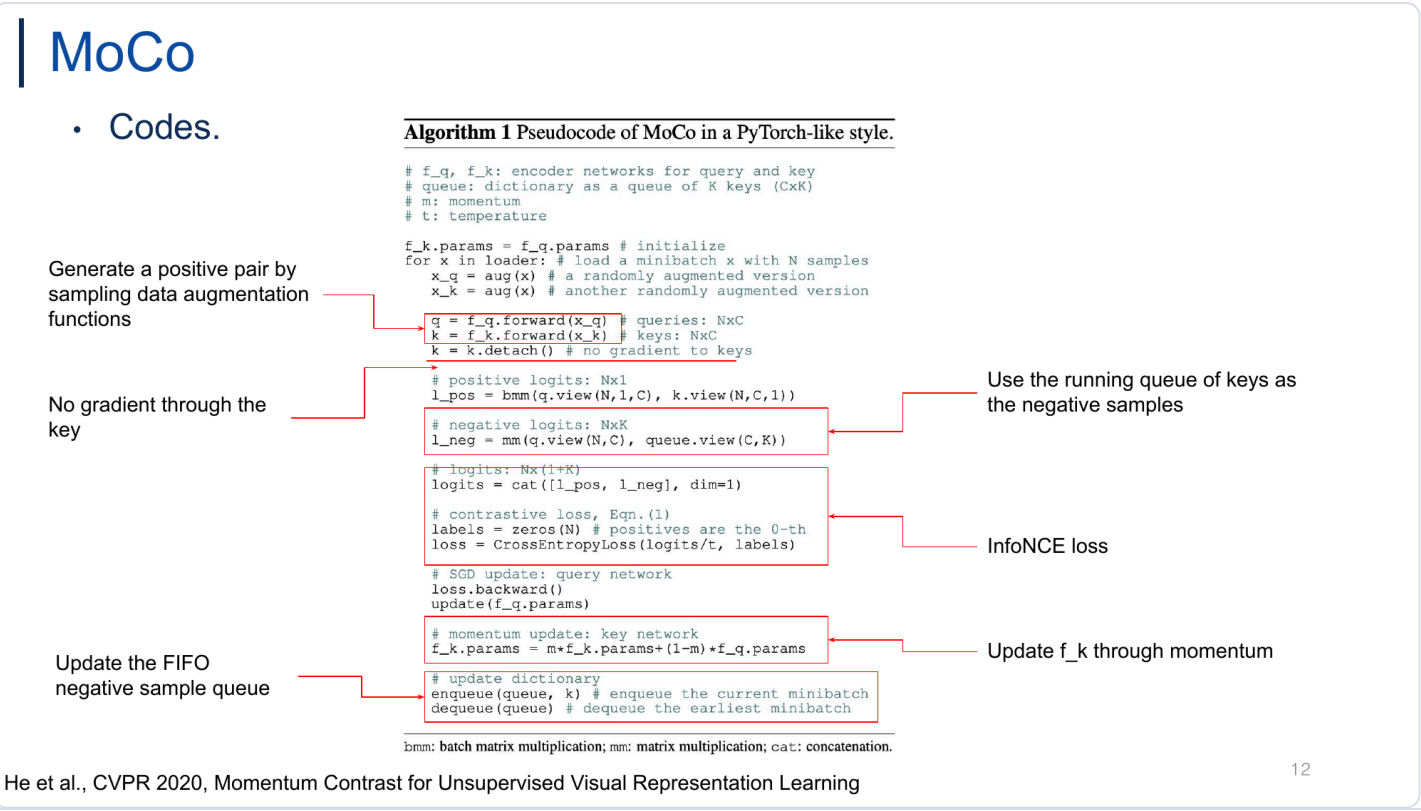
핵심 줄

1. `k = k.detach()`: key 쪽 gradient를 끊어서, gradient는 `f_q`로만 가 ("No gradient through the key"). key encoder는 backprop으로 안 바뀌어.
2. `l_neg = mm(…)`: `(N×C)·(C×K) = N×K`, 각 query와 queue 속 key 전부의 점수 = **negative logits**. 슬라이드엔 바로 위에 각 query와 자기 key의 `l_pos`(N×1)도 있고, 둘을 `cat`해서 `logits`(N×(1+K))를 만들어.
3. `labels = zeros(N)`: positive를 0번 칸에 붙였으니까 모든 query의 정답 = 0번 칸. `CrossEntropyLoss(logits/t, labels)`가 곧 (1+K)칸 InfoNCE.
4. `f_k.params = m*f_k.params + (1-m)*f_q.params`: key encoder = query encoder weight의 EMA. key encoder가 아주 천천히 변해 ("slowly progressing").
5. `enqueue` / `dequeue`: 이번 key를 넣고 가장 오래된(earliest) batch를 빼 (FIFO). update **다음에** 하니까 이번 key는 다음 step부터 negative가 돼.

core idea 좋은 dictionary는 **크고**(large) key들이 **비슷한 encoder로 만들어져야**(consistent) 해. queue가 크기를 batch와 분리(decouple)하고, momentum update가 오래된 key와 새 key를 비교 가능하게 유지해: "Replace the memory bank with a momentum-updated queue" (p.10–11).

알아보는 법 `queue`, `f_q` / `f_k`, `enqueue` / `dequeue`, **`labels = zeros(N)`**. `zeros`면 MoCo, `arange`면 CLIP. EMA 줄 주인공이 `f_k`면 MoCo, `gt`면 DINO. loss가 한 방향(query → keys)뿐이라 양방향 평균이 없어.

▼ 슬라이드 원본 보기 (L05 p.12)



CLIP L05 p.35

```
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
logits = np.dot(I_e, T_e.T) * np.exp(t)
labels = np.arange(n)
loss = (loss_i + loss_t)/2
```

한 줄 n×n 행렬의 대각선이 positive, 행 방향·열 방향 CE를 평균 (symmetric). temperature는 곱해.

입력 → 출력 짝지어진 이미지 n장 `I`와 caption n개 `T` (i번째끼리 한 쌍)를 받아 loss 하나. image encoder, text encoder, projection `W_i`·`W_t`, temperature `t`가 전부 같이 학습돼.

변수

- `I_f` [n, d_i], `T_f` [n, d_t]: 서로 다른 두 encoder의 출력이라 차원도 달라.
- `W_i` [d_i, d_e], `W_t` [d_t, d_e]: 둘 다 같은 차원 d_e로 보내서 이미지와 text를 **shared embedding space**에 놓아. 그다음 `l2_normalize` (행마다 길이 1).

핵심 줄

1. `logits = np.dot(I_e, T_e.T) * np.exp(t)` : [n, n] 표, (i, j) = 이미지 i와 text j의 cosine similarity. `np.exp(t)`를 **곱해**: "t로 나누기"와 같은 역할인데 t를 학습하고, exp라 항상 양수야.
2. `labels = np.arange(n)` : 이미지 i의 정답은 text i → **대각선이 positive**, 같은 행 · 열의 나머지 n−1개가 negative. n = 3이면 (0,0), (1,1), (2,2)가 정답.
3. `(loss_i + loss_t)/2` : 같은 logits · labels로 `axis=0`, `axis=1` 두 방향 CE를 평균. "이 이미지의 caption은?"과 "이 caption의 이미지는?"을 둘 다 푸니까 **symmetric** (p.33).

core idea positive를 augmentation이 아니라 **자연어 caption**에서 얻어 (p.32). 두 modality를 정렬해 두면 추론 때 class 이름을 넣은 text prompt와 비교해서 zero-shot 분류(그 class로 따로 학습 안 하고 분류)가 돼 (p.33).

알아보는 법 `np.`, `I_` / `T_`, `W_i` / `W_t`, `* np.exp(t)`, `labels = np.arange(n)`, `(loss_i + loss_t)/2`.

▼ 슬라이드 원본 보기 (L05 p.35)

CLIP

- CLIP (Contrastive Language-Image Pre-training)

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t             - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

Radford et al., PMLR 2021, Learning Transferable Visual Models From Natural Language Supervision

35

Soft target · self-distillation

정답이 one-hot이 아니라 **확률분포(soft target)**인 두 방법이야. 둘 다 정답 쪽에 `detach`가 붙어. 정답 분포를 X-CLR은 frozen text encoder가, DINO는 student의 EMA인 teacher가 만들어.

X-CLR L03 p.67

```
txt_emb  = txt_enc(captions).detach()     # frozen
tgt_sims = softmax(txt_emb @ txt_emb.T / tao_s, dim=1)
img_sims = softmax(img_emb @ img_emb.T / tao,   dim=1)
return CE(tgt_sims, img_sims, dim=1).mean()
```

한 줄 InfoNCE의 one-hot 정답을 text similarity의 soft graph로 바꿈.

입력 → 출력 `training_step(captions, images)` : caption N개와 이미지 N장으로 loss 하나. 학습되는 건 image encoder `img_enc` 뿐이야.

변수

- `txt_emb` N × D_txt, `img_emb` N × D_img. 서로 직접 비교하지 않으니 차원이 달라도 돼.
- `txt_emb @ txt_emb.T` , `img_emb @ img_emb.T` : 각각 N×N similarity 표. `tao_s` = τ_s(target 쪽), `tao` = τ (철자는 슬라이드 그대로).

핵심 줄

1. `txt_enc(captions).detach()` : caption을 **frozen**(학습 안 하는) text encoder로 encode하고 gradient도 끊어. 그래서 정답이 학습 내내 안 변해.
2. `tgt_sims = softmax(… / tao_s, dim=1)` : 행 i = "샘플 i의 caption이 batch 안 caption들과 각각 얼마나 비슷한가"의 확률분포 = **target graph**.

같은 계산을 이미지로 한 `img_sims` 가 **learned graph**이고, `CE(tgt_sims, img_sims)` 로 learned graph가 target graph를 따라가 ("Match the learned graph to the target graph").

예시 batch가 {강아지, 고양이, 피아노}면 강아지 행의 정답이 InfoNCE에선 `[1, 0, 0]` , X-CLR에선 `[0.7, 0.2, 0.1]` 처럼 "고양이와는 좀 비슷, 피아노와는 거의 무관"이 들어가 (숫자는 설명용). τ_s가 작으면 가장 강한 positive에 몰려 보통 contrastive와 비슷해지고, 크면 관련 샘플에 weight가 더 퍼져 (p.68).

core idea InfoNCE 정답(binary graph)은 다른 강아지, 고양이, 피아노를 똑같이 밀어내 (p.64). X-CLR은 정답을 text가 주는 **soft similarity graph**로 바꿔서 "개는 피아노보다 고양이에 가깝다"까지 배워 (p.65).

알아보는 법 `captions` , `txt_enc(…).detach()` , `tgt_sims` / `img_sims` , `tao_s` . CLIP은 image–text **교차** similarity 하나 + hard label `arange(n)` + text encoder도 학습, X-CLR은 text–text와 image–image **따로** + soft target + text 고정.

▼ 슬라이드 원본 보기 (L03 p.67)

X-CLR Objective

```
def training_step(captions, images):
    # encode captions a frozen text encoder
    txt_emb = txt_enc(captions).detach() # N x D_txt
    # encode images with the model we are training
    img_emb = img_enc(images)  # N x D_img

    # calculate similarities
    tgt_sims =
        softmax(txt_emb @ txt_emb.T / tao_s, dim=1)
    img_sims =
        softmax(img_emb @ img_emb.T / tao, dim=1)
    # the loss is the cross-entropy between the two
    return CE(tgt_sims, img_sims, dim=1).mean()
```

Target graph
- Similarity from text or metadata

Learned graph
- Similarity between image representations

Training objective
- Match the learned graph to the target graph

$$\mathcal{L}_{\text{X-CLR}} = \frac{1}{2N_b} \sum_{i=1}^{2N_b} H(s_i, p_i)$$

$$p_{i,j} = \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)}$$

$$s_{i,j} = \frac{\exp(\boldsymbol{G}_{i,j}^{(\text{soft})}/\tau_s)}{\sum_{k=1}^{2N_b} \exp(\boldsymbol{G}_{i,k}^{(\text{soft})}/\tau_s)} \qquad \boldsymbol{G}_{i,j}^{(\text{soft})} = \text{sim}(f_{\text{text}}(c_i), f_{\text{text}}(c_j))$$

DINO L04 p.40

```
s1, s2 = gs(x1), gs(x2)              # student
t1, t2 = gt(x1), gt(x2)              # teacher
loss = H(t1, s2)/2 + H(t2, s1)/2      # 교차 view
gt.params = l*gt.params + (1-l)*gs.params
C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)
# H 안: t = t.detach(); t = softmax((t - C) / tpt)
```

한 줄 student가 EMA teacher의 centered + sharpened 분포를 다른 view에서 CE로 따라가. centering은 teacher 쪽에만.

입력 → 출력 label 없는 이미지 n장에서 random view `x1`, `x2`를 만들어 student와 teacher 둘 다에 넣어. 매 step student `gs`는 SGD로, teacher `gt`와 center `C`는 EMA로 갱신돼.

변수

- `gs`, `gt`: student, teacher. 구조가 같고 처음에 `gt.params = gs.params`로 복사해. 출력 `s1, s2, t1, t2`는 각각 n × K (K = 출력 차원).
- `C`: center (K차원 벡터). `tps` / `tpt`: student/teacher temperature. `l` / `m`: network/center momentum.
- `H(t, s)` = $-\Sigma_k\ t_k\ \log\ s_k$. 안에서 student는 `softmax(s / tps)`만, teacher는 detach + centering + sharpening.

핵심 줄

1. `H(t1, s2)/2 + H(t2, s1)/2`: view 1의 teacher ↔ view 2의 student, 그리고 반대. **같은 view끼리는 비교 안 해.** "다른 view를 본 teacher의 답"을 맞혀야 하니 augmentation이 바뀌어도 같은 출력을 배워.
2. `gt.params = l*gt.params + (1-l)*gs.params`: teacher = student weight의 EMA. teacher는 SGD를 안 받고 (`update(gs)`만 있음) 이 줄로만 바뀌어. `l`은 숫자 1이 아니라 소문자 L.
3. `C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)`: 두 view의 **teacher 출력** batch 평균(K차원)을 EMA. 그래서 DINO엔 EMA가 **두 개** (teacher weight, center).
4. `t = t.detach()`: teacher 출력 쪽 stop-gradient. gradient는 student로만 가.
5. `(t - C) / tpt`: center를 빼고(centering) teacher temperature `tpt` (sharpen하려면 작은 값)로 나눠 뾰족하게 (sharpening) 한 뒤 softmax. 주석 그대로 "center + sharpen", **teacher 쪽에만**.

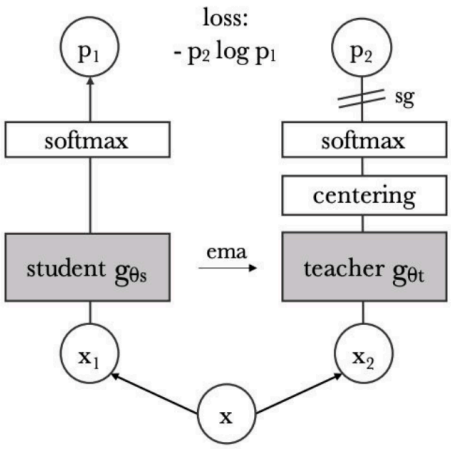
core idea **Self-distillation without labels** (p.39). distillation(teacher의 출력 분포를 student가 따라 배우기)인데, teacher가 미리 학습된 모델이 아니라 **student 자신의 EMA**야. negative가 없어서 출력이 다 같아지는 collapse 위험이 있고, teacher 쪽 centering + sharpening으로 막아. (노트 설명: centering은 한 차원이 늘 이기는 쏠림을 막지만 혼자 쓰면 uniform 쪽으로 가고, sharpening은 그 반대라 둘이 균형을 잡아. 슬라이드엔 주석 "center + sharpen"만 있어.)

알아보는 법 `gs` / `gt`, 교차된 `H(t1, s2)`, `C`와 `tpt`, EMA 줄 두 개. queue도 `labels`도 없어 (negative 없음).

▼ 슬라이드 원본 보기 (L04 p.40)

DINO: Self-Distillation with No Labels

- DINO (2021)



Algorithm 1 DINO PyTorch pseudocode w/o multi-crop.

```
# gs, gt: student and teacher networks
# C: center (K)
# tps, tpt: student and teacher temperatures
# l, m: network and center momentum rates
gt.params = gs.params
for x in loader: # load a minibatch x with n samples
    x1, x2 = augment(x), augment(x) # random views

    s1, s2 = gs(x1), gs(x2) # student output n-by-K
    t1, t2 = gt(x1), gt(x2) # teacher output n-by-K

    loss = H(t1, s2)/2 + H(t2, s1)/2
    loss.backward() # back-propagate

    # student, teacher and center updates
    update(gs) # SGD
    gt.params = l*gt.params + (1-l)*gs.params
    C = m*C + (1-m)*cat([t1, t2]).mean(dim=0)

def H(t, s):
    t = t.detach() # stop gradient
    s = softmax(s / tps, dim=1)
    t = softmax((t - C) / tpt, dim=1) # center + sharpen
    return - (t * log(s)).sum(dim=1).mean()
```

$$c \leftarrow mc + (1-m)\frac{1}{B}\sum_{i=1}^{B} g_{\theta_t}(x_i),$$

Caron et al., ICCV 2021, Emerging Properties in Self-Supervised Vision Transformers

40

Consistency

label이 일부만 있는 semi-supervised 방법이야. 아이디어는 consistency regularization: 작은 perturbation(흔들기)을 줘도 예측은 같아야 해 (L06 p.16). perturbation은 두 군데서 와. 하나는 입력 쪽 data augmentation, 다른 하나는 모델 쪽 stochasticity(dropout)야. 그러면 label 없는 샘플에서도 "두 예측이 같아져라"는 학습 신호가 나와. 두 코드는 비교 대상(target) ž를 어디서 가져오는지만 달라.

Π-model `Alg.1 · L06 p.18`

```
z_i ← f_θ(g(x_i))
ž_i ← f_θ(g(x_i))    ▷ again, with different dropout and augmentation
loss ← -(1/|B|) Σ_{i∈B∩L} log z_i[y_i]
       + w(t) (1/(C|B|)) Σ_{i∈B} ‖z_i - ž_i‖²
```

한 줄 같은 network로 같은 입력을 두 번 forward해서 두 출력이 같아지게 하는 consistency 방법.

입력 → 출력 학습 데이터 x_i(일부만 label y_i), dropout이 있는 stochastic network f_θ, 랜덤 augmentation g, ramp-up 함수 w(t)를 받아 parameter θ를 돌려줘.

변수

- B: minibatch, |B|: 그 크기, L: label 있는 샘플 집합 → `B∩L` = batch 중 label 있는 것.
- z_i, $\tilde{z}_i$: 같은 x_i의 두 출력 (class 확률 벡터). $z_i[y_i]$ = 정답 class 칸의 확률. C = class 수 (Alg.1엔 정의가 없고 Alg.2의 N×C로 보면 class 수).

핵심 줄

1. `ž_i ← f_θ(g(x_i))` : 바로 위 줄과 글자가 똑같지만 오타가 아니야. g(augmentation)와 dropout이 랜덤이라 같은 입력도 다른 출력이 나와 (▷ again, with different dropout and augmentation). **같은 θ로 forward 2번**.
2. `w(t) (1/(C|B|)) Σ_{i∈B} ‖z_i - ž_i‖²` : 두 출력 차이의 제곱을 C와 |B|로 나눈 평균 = MSE. 합이 `i∈B` **전체**(labeled + unlabeled)에 걸려. label이 필요 없는 항이니까. w(t)는 작게 시작해서 키워 (ramp-up, 초반 예측은 믿기 어려우니까).

첫 항 $-(1/|B|) \sum_{i\in B\cap L} \log z_i[y_i]$은 label 있는 샘플에만 거는 보통 CE야.

core idea "Predictions should remain consistent under small perturbations" (p.16). label 대신 **자기 자신의 두 번째 예측**을 target으로 써. 대가는 forward 2번이고, 이게 Temporal Ensembling의 동기야 (p.19).

알아보는 법 글자가 똑같은 두 줄 + "again, with different dropout and augmentation", $\|z_i - \tilde{z}_i\|^2$. `Z` , α, bias correction이 있으면 Temporal Ensembling.

▼ 슬라이드 원본 보기 (L06 p.18)

Π-model

Algorithm 1 Π-model pseudocode.

Require: x_i = training stimuli
Require: L = set of training input indices with known labels
Require: y_i = labels for labeled inputs $i \in L$
Require: $w(t)$ = unsupervised weight ramp-up function
Require: $f_\theta(x)$ = stochastic neural network with trainable parameters θ
Require: $g(x)$ = stochastic input augmentation function
for t in $[1, num_epochs]$ **do**
  for each minibatch B **do**
    $z_{i\in B} \leftarrow f_\theta(g(x_{i\in B}))$ ▷ evaluate network outputs for augmented inputs
    $\tilde{z}_{i\in B} \leftarrow f_\theta(g(x_{i\in B}))$ ▷ again, with different dropout and augmentation
    $loss \leftarrow -\frac{1}{|B|}\sum_{i\in(B\cap L)} \log z_i[y_i]$ ▷ supervised loss component
      $+ w(t)\frac{1}{C|B|}\sum_{i\in B}||z_i - \tilde{z}_i||^2$ ▷ unsupervised loss component
    update θ using, e.g., ADAM ▷ update network parameters
  end for
end for
return θ

Sajjadi et al., ICLR 2016, Regularization With Stochastic Transformations and Perturbations for Deep Semi-Supervised Learning

Temporal Ensembling `Alg.2 · L06 p.20`

```
(epoch 끝)
Z ← αZ + (1 - α)z        ▷ accumulate ensemble predictions
z̃ ← Z / (1 - α^t)        ▷ bias correction
```

한 줄 target = 샘플별 과거 예측의 EMA. forward 1회, 대신 N×C 메모리와 epoch당 1번 갱신.

입력 → 출력 Π-model 입력에 ensembling momentum α (0 ≤ α < 1)가 추가돼. minibatch 안의 loss 식은 Π-model과 글자 하나 안 달라. z̃를 어디서 가져오나만 달라.

변수

- Z: 샘플별 누적 예측, N×C (N = **전체** training 샘플 수). 처음엔 0.
- z: 이번 epoch에 모은 전체 예측. z̃: 다음 epoch의 target. t: epoch 번호.

핵심 줄 둘 다 epoch 끝, minibatch loop 밖이야.

1. `Z ← αZ + (1 - α)z`: 샘플별 **예측**의 EMA (▷ accumulate ensemble predictions). epoch마다 augmentation · dropout · weight가 달랐던 예측의 평균이라 "시간축 ensemble"이고, 한 번 forward한 값보다 안정적인 target이 돼 (p.19, p.22).
2. `z̃ ← Z / (1 - α^t)`: Z가 0에서 시작해서 초반엔 0 쪽으로 작게 치우쳐. $1 - \alpha^t$로 나누면 원래 크기로 돌아와 (▷ construct target vectors by bias correction).

예시 α = 0.5, 어떤 샘플의 예측이 매 epoch 1이라면 (숫자는 설명용) epoch 1: `Z = 0.5` → `0.5 / (1 - 0.5) = 1`. epoch 2: `Z = 0.5·0.5 + 0.5·1 = 0.75` → `0.75 / (1 - 0.25) = 1`. 나누기 덕분에 진짜 값 1이 나와. Adam의 bias correction과 같은 원리.

core idea target을 "지금 한 번 더 forward"(Π-model) 대신 "**같은 샘플의 과거 예측 평균**"으로 바꿔 (p.19, p.22). 슬라이드 caption: 이 갱신은 minibatch loop 안에서 해도 되는데(could equally well be done inside the minibatch loop), 이해하기 쉽게 epoch 사이에 둔 거래.

알아보는 법 `Z`, α, $1 - \alpha^t$. EMA 대상이 **prediction**이야. weight를 EMA하는 건 Mean Teacher (teacher weight, 매 iteration). Mean Teacher는 pseudo code 슬라이드가 없어서 이 둘 구별 문제로 나오기 좋아.

▼ 슬라이드 원본 보기 (L06 p.20)

Temporal Ensembling

Algorithm 2 Temporal ensembling pseudocode. Note that the updates of Z and $\tilde{z}$ could equally well be done inside the minibatch loop; in this pseudocode they occur between epochs for clarity.

Require: x_i = training stimuli
Require: L = set of training input indices with known labels
Require: y_i = labels for labeled inputs $i \in L$
Require: α = ensembling momentum, $0 \leq \alpha < 1$
Require: $w(t)$ = unsupervised weight ramp-up function
Require: $f_\theta(x)$ = stochastic neural network with trainable parameters θ
Require: $g(x)$ = stochastic input augmentation function

$Z \leftarrow \mathbf{0}_{[N \times C]}$ ▷ initialize ensemble predictions
$\tilde{z} \leftarrow \mathbf{0}_{[N \times C]}$ ▷ initialize target vectors
for t in $[1, num_epochs]$ **do**
  for each minibatch B **do**
    $z_{i \in B} \leftarrow f_\theta(g(x_{i \in B}, t))$ ▷ evaluate network outputs for augmented inputs
    $loss \leftarrow -\frac{1}{|B|} \sum_{i \in (B \cap L)} \log z_i[y_i]$ ▷ supervised loss component
      $+ w(t) \frac{1}{C|B|} \sum_{i \in B} ||z_i - \tilde{z}_i||^2$ ▷ unsupervised loss component
    update θ using, e.g., ADAM ▷ update network parameters
  end for
  $Z \leftarrow \alpha Z + (1 - \alpha) z$ ▷ accumulate ensemble predictions
  $\tilde{z} \leftarrow Z / (1 - \alpha^t)$ ▷ construct target vectors by bias correction
end for
return θ

Laine et al., ICLR 2017, Temporal Ensembling for Semi-Supervised Learning

20

Pseudo label

label 없는 샘플에 모델이 만든 가짜 정답(pseudo label)을 붙여 학습해. 위험은 틀린 pseudo label을 자기가 계속 믿고 강화하는 confirmation bias야. 두 코드는 pseudo label을 "자기 softmax 확신 하나"에만 기대지 않아서 이걸 줄여. LP는 feature graph의 이웃 관계를 쓰고, DivideMix는 두 network의 교차 판단을 써: 상대가 데이터를 나눠 주고, unlabeled 정답 추측은 두 network(자기 포함)의 평균이야.

Label Propagation Alg.1 · L07 p.11

```
v_i ← ψ_θ(x_i)                          ▷ extract descriptors
a_ij ← affinity (kNN)  ;  W ← A + Aᵀ
𝒲 ← D^{-1/2} W D^{-1/2}
Z ← solve (I - α𝒲)Z = Y with CG  ▷ diffusion
ŷ_i ← argmax_j ẑ_ij  ;  ω_i ← certainty of ŷ_i
```

한 줄 pseudo label이 자기 softmax가 아니라 graph diffusion에서 나와. 불확실한 건 버리지 않고 ω_i로 가중.

입력 → 출력 전체 샘플 X(labeled + unlabeled)와 일부 label Y_L. 먼저 labeled만으로 T epoch 학습하고 (line 3–5), 그다음 T′ epoch 동안 **매 epoch** graph를 새로 만들고 pseudo label을 붙여 전체로 학습해 (line 6–17). 출력은 θ.

변수

- $v_i = \psi_\theta(x_i)$: 샘플 i의 descriptor(feature 벡터). n = 전체 샘플 수.
- A: n×n affinity 행렬. v_i가 v_j의 k-nearest neighbor(kNN)일 때만 값이 있어서 sparse해. kNN은 한쪽 방향일 수 있으니 $W = A + A^T$로 대칭을 맞추고 $D^{-1/2}WD^{-1/2}$로 정규화. D는 슬라이드에 정의가 없어. 논문 기준으로는 대각 행렬이고, i번째 대각 원소 = W의 i번째 행 합 = 샘플 i의 연결 가중치 합(degree)이야 (`exam/pseudocode.md` 보충).
- Y: 샘플 × class label 행렬. labeled 샘플 행은 one-hot, unlabeled 행은 전부 0이야 (슬라이드엔 정의 없음, notes 07 기준). Z: diffusion 결과 점수 (샘플 × class). 그래서 Z의 unlabeled 행 값은 전부 이웃을 타고 번져 온 거야. $\hat{y}_i$: unlabeled 샘플의 pseudo label, ω_i: 그 label의 certainty weight.

핵심 줄

1. `Z ← solve (I - α𝒲)Z = Y with CG` : **diffusion**. labeled 샘플의 label이 이웃, 이웃의 이웃으로 graph를 타고 번져서 unlabeled에 닿아 ("propagate labels from labeled samples to nearby unlabeled samples", p.10). 역행렬 $(I - \alpha\mathcal{W})^{-1}$은 sparse하지 않아 n이 크면 비현실적이라, conjugate gradient(CG)로 연립방정식을 풀어 (p.11).
2. `ω_i ← certainty of ŷ_i` : argmax로 뽑은 pseudo label이 얼마나 확실한지를 weight로 줘. threshold로 **버리지 않고** 불확실한 건 loss에서 작게 반영해. 그다음 줄은 class 불균형을 맞추는 ζ_j (▷ class weight/balancing).

`core idea` pseudo label을 자기 softmax의 확신(confidence)이 아니라 **feature 공간의 이웃 관계(similarity)**에서 만들어: "Construct a similarity graph in the feature space" (p.10). feature가 좋아지면 graph도 pseudo label도 좋아져. 대신 graph 비용이 크고 feature 품질에 의존해 (p.14).

알아보는 법 ψ_θ, affinity / kNN, $A + A^T$, $D^{-1/2}WD^{-1/2}$, CG, ▷ diffusion, ω_i. network 하나에 GMM이 없으면 LP, θ가 두 개고 GMM이 있으면 DivideMix.

▼ 슬라이드 원본 보기 (L07 p.11)

Algorithm 1 Label propagation for deep SSL

procedure LPDSSL(Training examples X, labels Y_L)
  $\theta \leftarrow$ initialize randomly
  for epoch $\in [1, \dots, T]$ **do**
    $\theta \leftarrow$ OPTIMIZE$(L_s(X_L, Y_L; \theta))$ ▷ mini-batch optimization
  end for
  for epoch $\in [1, \dots, T']$ **do**
    for $i \in \{1, \dots, n\}$ **do** $\mathbf{v}_i \leftarrow \phi_\theta(x_i)$ ▷ extract descriptors
    for $(i, j) \in \{1, \dots, n\}^2$ **do** $a_{ij} \leftarrow$ affinity values (9)
    $W \leftarrow A + A^\top$ ▷ symmetric affinity
    $\mathcal{W} \leftarrow D^{-1/2} W D^{-1/2}$ ▷ symmetrically normalized affinity
    $Z \leftarrow$ solve (10) with CG ▷ diffusion
    for $(i, j) \in U \times C$ **do** $\hat{z}_{ij} \leftarrow z_{ij} / \sum_k z_{ik}$ ▷ normalize Z
    for $i \in U$ **do** $\hat{y}_i \leftarrow \arg\max_j \hat{z}_{ij}$ ▷ pseudo-label
    for $i \in U$ **do** $\omega_i \leftarrow$ certainty of $\hat{y}_i$ (11) ▷ pseudo-label weight
    for $j \in C$ **do** $\zeta_j \leftarrow (|L_j| + |U_j|)^{-1}$ ▷ class weight/balancing
    $\theta \leftarrow$ OPTIMIZE$(L_w(X, Y_L, \hat{Y}_U; \theta))$ ▷ mini-batch optimization
  end for
end procedure

Nearest neighbor graph. Given a network with parameters θ, we construct the descriptor set $V = (\mathbf{v}_1, \dots, \mathbf{v}_l, \mathbf{v}_{l+1}, \dots, \mathbf{v}_n)$, where $\mathbf{v}_i := \phi_\theta(x_i)$. A sparse *affinity matrix* $A \in \mathbb{R}^{n \times n}$ with elements

$$a_{ij} := \begin{cases} [\mathbf{v}_i^\top \mathbf{v}_j]_+^\gamma, & \text{if } i \neq j \wedge \mathbf{v}_i \in \mathrm{NN}_k(\mathbf{v}_j) \\ 0, & \text{otherwise} \end{cases} \tag{9}$$

Label propagation. Estimating matrix Z by (6) is impractical for large n because the inverse matrix $(I - \alpha\mathcal{W})^{-1}$ is not sparse. We rather use the the conjugate gradient (CG) method to solve linear system

$$(I - \alpha\mathcal{W})Z = Y, \tag{10}$$

DivideMix Alg.1 · L07 p.54

```
𝒲^(2) = GMM(𝒳, 𝒴, θ^(1))     // θ^(1)의 loss로 θ^(2)용 clean 확률   (co-divide)
labeled ← w_i ≥ τ  ;  unlabeled ← w_i < τ
ȳ_b = w_b y_b + (1 - w_b) p_b          (co-refinement, clean 쪽)
q̄_b = 1/(2M) Σ_m (p(û; θ^(1)) + p(û; θ^(2)))   (co-guessing, noisy 쪽)
𝓛_𝒳, 𝓛_𝒰 = MixMatch(𝒳, 𝒰)
```

한 줄 noisy label 문제를 SSL(semi-supervised learning: label 있는 데이터 조금 + label 없는 데이터를 같이 학습. 여기선 self-supervised 뜻이 아니야)로 바꿔 풀어. 두 network가 서로에게 데이터를 나눠줘서 confirmation bias를 줄여.

입력 → 출력 두 network $\theta^{(1)}$, $\theta^{(2)}$, label이 **일부 틀렸을 수 있는**(noisy) 데이터셋 ($\mathcal{X}$, $\mathcal{Y}$), threshold τ, augmentation 횟수 M, sharpening temperature T 등. warm-up(처음에 나누기 없이 전체 데이터로 보통 학습, "standard training") 뒤 매 epoch "데이터 나누기 → 두 network를 하나씩 학습"을 반복해서 두 network를 돌려줘.

변수

- GMM(Gaussian Mixture Model): 샘플별 loss 값에 Gaussian 두 개를 맞춰 "loss 작은 쪽 = clean, 큰 쪽 = noisy"로 나눠. w_i = 샘플 i가 clean일 확률.
- **τ는 temperature가 아니라 clean probability threshold**. sharpening temperature는 T.
- y_b: 주어진(틀렸을 수도 있는) one-hot label. p_b: 지금 학습하는 network $\theta^{(k)}$가 x_b의 M개 augmentation에 낸 예측 평균.

핵심 줄

1. `𝒲^(2) = GMM(𝒳, 𝒴, θ^(1))` (**co-divide**): $\theta^{(1)}$의 loss로 구한 clean 확률을 **$\theta^{(2)}$가 쓴다**. 위첨자가 엇갈린 게 핵심이야. 자기 판단으로 자기 학습 데이터를 고르면 자기 실수를 계속 믿게 되니까 상대의 판단을 써 (p.51).
2. `labeled ← w_i ≥ τ ; unlabeled ← w_i < τ`: clean 확률이 높으면 label을 유지해서 labeled set, 낮으면 **label만 버리고** unlabeled set으로. noisy label 문제가 semi-supervised 문제로 바뀌어 (p.50).
3. `ȳ_b = w_b y_b + (1 - w_b) p_b` (**co-refinement**, labeled 쪽): 주어진 label을 상대가 준 w_b만큼만 믿고 나머지는 자기 예측 p_b로 채워. 그다음 Sharpen(T로 분포를 뾰족하게).
4. `q̄_b = 1/(2M) Σ_m (…θ^(1) + …θ^(2))` (**co-guessing**, unlabeled 쪽): 정답을 **두 network × M개 augmentation = 2M개 예측의 평균**으로 추측해. 그다음 Sharpen.
5. `MixMatch(𝒳, 𝒰)`: 이렇게 만든 두 set으로 MixMatch(L07: unlabeled에 추측 label을 붙이고 MixUp으로 섞어 학습하는 semi-supervised 방법)를 그대로 돌려. "DivideMix does not create a new learning algorithm" (p.52).

예시 label y_b = 고양이 `[1, 0, 0]`, w_b = 0.8, p_b = `[0.5, 0.5, 0]` 이면 $\bar{y}_b$ = `0.8·[1, 0, 0] + 0.2·[0.5, 0.5, 0] = [0.9, 0.1, 0]`. label을 거의 믿되 조금 의심하는 soft label이야 (숫자는 설명용).

core idea label이 틀린 데이터를 버리지 않고 "label 없는 데이터"로 재활용해서 noisy label 학습을 semi-supervised learning(MixMatch)으로 풀어. 두 network가 서로 데이터를 나눠 주고, 상대 판단으로 label을 고치고, 같이 추측해서 한 network의 실수가 자기 강화되는 걸 막아.

RL Double Q-learning에서 한 Q가 고른 action을 다른 Q로 평가해 자기 과대평가를 줄이는 것과 비슷한 발상이야: 자기 평가를 자기가 안 해. (비유일 뿐, DivideMix가 Q-learning을 쓰는 건 아님)

알아보는 법 $\theta^{(1)}$, $\theta^{(2)}$, `GMM`, $w_i \geq \tau$, `1/(2M)`, `MixMatch`. "co-" 셋: co-**divide** = 상대가 데이터를 나눠 줌, co-**refinement** = labeled 쪽 label 고치기 (상대의 w_b + 자기 p_b), co-**guessing** = unlabeled 쪽 정답 추측 (두 network 평균). 두 network는 대등한 peer라서 EMA teacher–student(Mean Teacher)와 달라.

▼ 슬라이드 원본 보기 (L07 p.54)

Algorithm 1: DivideMix. Line 4-8: co-divide; Line 17-18: label co-refinement; Line 20: label co-guessing.

Input: $\theta^{(1)}$ and $\theta^{(2)}$, training dataset $(\mathcal{X}, \mathcal{Y})$, clean probability threshold τ, number of augmentations M, sharpening temperature T, unsupervised loss weight λ_u, Beta distribution parameter α for MixMatch.

$\theta^{(1)}, \theta^{(2)} = \text{WarmUp}(\mathcal{X}, \mathcal{Y}, \theta^{(1)}, \theta^{(2)})$ *// standard training (with confidence penalty)*

while $e < \text{MaxEpoch}$ **do**

Co-divide

$\mathcal{W}^{(2)} = \text{GMM}(\mathcal{X}, \mathcal{Y}, \theta^{(1)})$ *// model per-sample loss with $\theta^{(1)}$ to obtain clean proabability for $\theta^{(2)}$*

$\mathcal{W}^{(1)} = \text{GMM}(\mathcal{X}, \mathcal{Y}, \theta^{(2)})$ *// model per-sample loss with $\theta^{(2)}$ to obtain clean proabability for $\theta^{(1)}$*

for $k = 1, 2$ **do** *// train the two networks one by one*

$\mathcal{X}_e^{(k)} = \{(x_i, y_i, w_i) | w_i \geq \tau, \forall (x_i, y_i, w_i) \in (\mathcal{X}, \mathcal{Y}, \mathcal{W}^{(k)})\}$ *// labeled training set for $\theta^{(k)}$*

$\mathcal{U}_e^{(k)} = \{x_i | w_i < \tau, \forall (x_i, w_i) \in (\mathcal{X}, \mathcal{W}^{(k)})\}$ *// unlabeled training set for $\theta^{(k)}$*

for iter $= 1$ **to** num_iters **do**

From $\mathcal{X}_e^{(k)}$, draw a mini-batch $\{(x_b, y_b, w_b); b \in (1, ..., B)\}$

From $\mathcal{U}_e^{(k)}$, draw a mini-batch $\{u_b; b \in (1, ..., B)\}$

for $b = 1$ **to** B **do**

for $m = 1$ **to** M **do**

$\hat{x}_{b,m} = \text{Augment}(x_b)$ *// apply m^{th} round of augmentation to x_b*

$\hat{u}_{b,m} = \text{Augment}(u_b)$ *// apply m^{th} round of augmentation to u_b*

end

Label co-refine

$p_b = \frac{1}{M} \sum_m \text{p}_{\text{model}}(\hat{x}_{b,m}; \theta^{(k)})$ *// average the predictions across augmentations of x_b*

$\bar{y}_b = w_b y_b + (1 - w_b) p_b$

// refine ground-truth label guided by the clean probability produced by the other network

$\hat{y}_b = \text{Sharpen}(\bar{y}_b, T)$ *// apply temperature sharpening to the refined label*

$\bar{q}_b = \frac{1}{2M} \sum_m \left(\text{p}_{\text{model}}(\hat{u}_{b,m}; \theta^{(1)}) + \text{p}_{\text{model}}(\hat{u}_{b,m}; \theta^{(2)})\right)$ Label co-guessing

// co-guessing: average the predictions from both networks across augmentations of u_b

$q_b = \text{Sharpen}(\bar{q}_b, T)$ *// apply temperature sharpening to the guessed label*

end

$\hat{\mathcal{X}} = \{(\hat{x}_{b,m}, \hat{y}_b); b \in (1, ..., B), m \in (1, ..., M)\}$ *// augmented labeled mini-batch*

$\hat{\mathcal{U}} = \{(\hat{u}_{b,m}, q_b); b \in (1, ..., B), m \in (1, ..., M)\}$ *// augmented unlabeled mini-batch*

$\mathcal{L}_{\mathcal{X}}, \mathcal{L}_{\mathcal{U}} = \text{MixMatch}(\hat{\mathcal{X}}, \hat{\mathcal{U}})$ *// apply MixMatch*

$\mathcal{L} = \mathcal{L}_{\mathcal{X}} + \lambda_u \mathcal{L}_{\mathcal{U}} + \lambda_r \mathcal{L}_{\text{reg}}$ *// total loss*

$\theta^{(k)} = \text{SGD}(\mathcal{L}, \theta^{(k)})$ *// update model parameters*

end

end

end

준-알고리즘도 기억

BYOL 단계 (L04 p.32: predictor + EMA target + normalized MSE), DeepCluster process (L05 p.24: k-means ↔ classification 교대). 코드 박스는 아니지만 "어느 방법?"으로 나올 수 있어.

셀프 체크 · 코드

MCQ Which method's pseudo code contains `labels = zeros(N)`? (A) CLIP (B) SimCLR (C) MoCo (D) DINO

(C) MoCo. positive logit을 0번째 column에 붙여서 (L05 p.12). 그래서 모든 query의 정답 칸 번호가 0이야. CLIP은 `np.arange(n)`(대각선), SimCLR은 `labels` 없이 수식, DINO는 정답이 teacher 분포라 labels가 없어.

빈칸 In DINO, the teacher output is centered and sharpened: `softmax((t - ____) / tpt)`.

C (center, teacher output 평균의 EMA, L04 p.40). `C`를 빼는 centering은 teacher 쪽에만 있어. temperature 나누기는 student(`s / tps`)와 teacher(`/ tpt`) 둘 다 하지만, 주석 "center + sharpen"이 붙은 건 teacher 줄이야.

T/F In Temporal Ensembling, Z is an exponential moving average of the network weights.

False. network **predictions**의 EMA. weight EMA는 Mean Teacher (L06 p.20). Z가 N×C(샘플 수 × class 수)인 것만 봐도 weight가 아니라 샘플별 예측표야.

이게 Physical AI로 어떻게 이어지나

슬라이드 밖 · 시험 범위 아님 · 동기 부여용

"이걸 왜 공부하나"에 대한 답이야. 네 분야(VLA action head, pixel RL, world model)에서 이 수업 개념이 어디 쓰이는지 이어 볼게. 여기 나오는 모델 이름과 구조(OpenVLA, π0, SigLIP, R3M ...)는 슬라이드에 없으니 시험 답으로 쓰지 마. 괄호 속 L0N은 연결되는 강의 번호야. 단 CURL 블록의 "한 줄"(L05 p.20–21)과 teacher–student 블록의 "이 수업" · "예외" 줄은 슬라이드 내용이라 시험 범위야.

VLA의 눈

L04 · L05

한 줄 VLA가 카메라 이미지를 읽는 앞단(vision encoder)이 바로 이 수업에서 배운 모델들이야.

배경 VLA(Vision-Language-Action model)는 카메라 이미지 + 언어 지시("컵을 집어")를 받아서 로봇 action을 내는 모델이야. 맨 앞엔 이미지를 벡터로 바꾸는 vision encoder가 있어. L01의 `z = f(x)`에서 f가 이거야. 맨 끝엔 네가 다루는 action head가 있어.

- **OpenVLA**: DINOv2(L04) feature와 SigLIP(CLIP 계열, L05) feature를 합쳐서 vision encoder로 써.
- **π0**: PaliGemma(SigLIP vision encoder를 쓰는 vision-language model) 위에 flow matching action expert를 얹은 구조야.

수업과 연결 DINOv2는 label도 caption도 없이 이미지만으로 DINO식 self-distillation(student가 EMA teacher의 출력 분포를 따라 하기)을 키운 모델이고, 이미지 전체(CLS token)와 patch 단위 feature를 둘 다 배워 (L04 p.55–67). SigLIP은 CLIP처럼 image–text 짝을 맞춘 모델이라 feature가 언어 의미와 정렬돼 있어 (L05). 성격이 다른 둘을 같이 쓰는 거야.

SigLIP = NCE vs InfoNCE

L03 p.21-23

한 줄 CLIP과 SigLIP의 차이는 L03의 "NCE = binary, InfoNCE = categorical" 구분과 같은 축이야.

CLIP batch에 (image, text) 짝 N개 → N×N similarity 행렬을 만들어. 대각선 칸이 진짜 짝(positive)이야. 행 하나를 softmax에 넣어서 "이 이미지의 짝 text는 N개 중 몇 번?"을 맞혀. 열 방향도 똑같이 해. 이게 InfoNCE = categorical(여러 개 중 하나 고르기) cross-entropy야.

SigLIP 칸마다 독립 sigmoid를 써서 "이 쌍이 짝이야, 아니야?"를 yes/no로 맞혀 (이진 분류). L03 NCE와 같은 이진 분류(sigmoid + binary CE) 방식이야. NCE 식을 그대로 쓴다는 뜻은 아니야. L03의 NCE는 data(진짜)와 noise(가짜)를 가르는 logistic regression이었어.

> 예시로 보면
> N = 4면 행렬은 4×4 = 16칸. CLIP은 "4개 중 하나 고르기"를 행 4번 + 열 4번 풀어. SigLIP은 16칸 각각에 yes/no로 답해. yes가 정답인 칸은 대각선 4개, no가 정답인 칸은 12개야. (설명용 숫자야.)

로봇용 representation

L01–L03

한 줄 로봇 제어용 encoder를 사람 영상 같은 데이터로 미리 학습해 두는 연구들이야.

- **R3M**: Ego4D(사람 1인칭 영상 데이터) 영상으로 time-contrastive + language alignment. time-contrastive = "같은 영상에서 시간이 가까운 frame은 가깝게, 먼 frame은 멀게". L03 contrastive의 positive를 '시간'으로 정한 셈이야. language alignment는 영상을 설명 문장과 연결하는 거고.
- **MVP**: MAE식 masked pretraining → motor control. 이미지 patch 일부를 가리고 복원하게 학습한 encoder(L01 p.20의 "Missing Data" 예측)를 로봇 제어에 붙여.
- **VIP**: 사람 영상으로 goal-conditioned value를 배워. 즉 "지금 관측 o에서 목표 g까지 얼마나 남았나"를 재는 `V(o; g)`야. 그런데 학습 식의 형태가 time-contrastive야.

L01의 prediction, L02의 pretext, L03의 contrastive가 섞여 있어.

World model

L01 p.20

한 줄 L01에서 "미래 예측(future prediction)" 예시로 나온 V-JEPA가 로봇 planning까지 이어져.

V-JEPA 영상에서 보이는 부분(context)의 표현으로 target 영역의 **표현(feature)**을 예측해. pixel을 그대로 복원하는 MAE와 달리 feature space에서 예측하는 게 포인트야 (L01 p.8 그림의 context encoder → target encoder 구조. "feature space 예측"이라는 풀이는 슬라이드 밖).

V-JEPA 2 후속 V-JEPA 2는 action-conditioned 버전으로 로봇 planning까지 가. action을 조건으로 받아서 다음 표현을 예측하니까, RL로 치면 latent space에서 dynamics를 배우는 world model이야.

Pixel RL

L05 p.20–21

한 줄 CURL은 이 수업(L05 p.20–21)에 직접 나온 RL 논문이야. MoCo식 contrastive learning을 pixel RL의 보조(auxiliary) loss로 붙였어. 목적은 sample efficiency야.

CURL 구조(replay buffer → random crop 두 번 → query / momentum key encoder → RL loss와 동시 학습)는 L05 CURL 블록 그대로야.

그 후 (슬라이드 밖) CURL 이후 RAD · DrQ가 "contrastive 없이 augmentation만 잘 줘도 된다"를 보여줬어. L04의 augmentation, L06의 consistency(입력을 바꿔도 예측은 같아야 한다) 얘기 그대로야.

용어 충돌 주의

teacher – student

한 줄 같은 "teacher–student"라도 로봇 쪽과 이 수업은 뜻이 달라.

- **로봇 쪽**: teacher는 privileged 정보(시뮬레이터에서만 알 수 있는 정확한 정보)를 가지고 먼저 학습된 policy야. student는 실제로 쓸 수 있는 입력만 보고 teacher를 따라 하도록 distill돼 (RMA, Learning by Cheating). teacher가 더 많이 아는 쪽이야.
- **이 수업**: teacher는 대부분 **student의 EMA 복사본**이야 (BYOL, DINO, Mean Teacher). student와 같은 구조의 network이고 (BYOL target은 predictor만 없음), 따로 학습된 더 똑똑한 모델이 아니야. DINO 이름의 "distillation"도 큰 teacher를 작은 student로 옮기는 게 아니라, 이 EMA teacher를 따라 하는 self-distillation이야 (L04 p.39).
- **예외**: L07 self-training · Noisy Student의 teacher는 보통 이전 단계에서 학습된 모델이야 (student가 다음 teacher가 돼). Meta Pseudo Labels는 teacher 자체를 student의 feedback으로 학습해. DINOv2가 큰 ViT-g를 작은 모델로 옮기는 단계는 보통 의미의 distillation이야 (L04).

시험장에서 네 직관("teacher = 따로 학습된 더 똑똑한 모델")이 오히려 헷갈리게 만들 수 있으니 조심해. teacher가 나오면 먼저 "EMA인가?"부터 확인해.

마지막 체크: 이 12줄은 영어로 바로 나와야 해

시험 전날 이것만 다시 봐. 읽는 법은 이래.

- 영어 문장을 소리 내서 말해 보고, 막히면 밑의 한국어 풀이를 봐.
- 굵은 단어는 빈칸(fill-in-the-blank)이나 T/F 함정으로 나오기 좋은 자리야. 그 단어가 짝꿍 단어로 바뀐 문장(query ↔ key, predictions ↔ weights, MSE ↔ cross-entropy 등)이 나오면 False일 가능성이 커.

1. MoCo: the key encoder is a **momentum** encoder; gradients flow only through the **query** encoder.
   key encoder(queue에 쌓을 key = 비교 대상을 만드는 encoder)는 backprop이 아니라 query encoder의 EMA(momentum update)로만 바뀌어. gradient는 query encoder로만 흘러 (코드의 `k.detach()`). L05 p.11–12
2. SimCLR: **2N–2** in-batch negatives; after training, **throw away g** and use f.
   사진 N장 → 각각 augmentation 두 번 → view 2N개. view 하나 기준으로 짝 view 1개가 positive, 자기 자신과 짝을 뺀 나머지 2N–2개가 negative야 (N = 4면 view 8개, negative 6개). loss는 projection head g(encoder 뒤에 붙은 작은 MLP)의 출력에서 계산하지만, 학습이 끝나면 g는 버리고 encoder f의 출력을 downstream에 써. L04 p.23–29
3. BYOL: **no negatives**; online network with a **predictor**, EMA target, **MSE**.
   negative 없이 학습해. online network(gradient로 학습, 끝에 predictor MLP가 하나 더 붙음)가 target network(online의 EMA, stop-gradient)의 출력을 예측해. 두 벡터를 길이 1로 정규화한 뒤 MSE를 줄여. predictor는 online 쪽에만 있어. L04 p.30–34
4. DINO: student matches the EMA teacher's **centered + sharpened** softmax with **cross-entropy**.
   student가 teacher의 softmax 분포를 cross-entropy로 따라 해. teacher는 student의 EMA야. teacher 출력에만 centering(평균 C를 빼기)과 sharpening(낮은 temperature로 분포를 뾰족하게)을 해서 collapse를 막아. L04 p.39–41
5. Barlow Twins: cross-correlation matrix → **identity** (invariance + redundancy reduction).
   두 view의 feature로 D×D 상관행렬을 batch 방향으로 계산해서 identity로 보내. 대각 → 1은 "같은 feature는 두 view에서 같이 움직여라"(invariance), 비대각 → 0은 "feature끼리 정보가 겹치지 마라"(redundancy reduction). 샘플끼리가 아니라 feature 차원끼리 비교해. L04 p.35–38
6. CLIP: symmetric loss over an N×N matrix, `labels = np.arange(n)`, **zero-shot** via text prompts.
   image N개와 text N개로 N×N similarity 행렬을 만들어. i번째 이미지의 정답은 i번째 text라서 정답 번호가 0, 1, …, n−1 = `np.arange(n)`야. 행 방향(image → text)과 열 방향(text → image) CE를 평균 내니까 symmetric. 추론은 "a photo of a {class}" 같은 text prompt들과 비교해서, 그 task용 학습 없이 바로 분류해 (zero-shot). L05 p.32–37
7. SwAV: **swapped prediction** of codes; Feature ↔ **Prototype**.
   prototype은 학습되는 cluster 중심 벡터 K개, code는 한 view의 feature를 prototype들에 배정한 결과야. swapped prediction은 view 1의 feature로 view 2의 code를 맞히고, 반대로도 맞히는 것. feature끼리(Feature ↔ Feature) 직접 비교하는 대신 feature ↔ prototype을 비교해서 explicit negative가 필요 없어. L05 p.26–30
8. Temporal Ensembling = EMA of **predictions**; Mean Teacher = EMA of **weights**.
   TE는 sample마다 과거 **예측**을 EMA로 저장해서 target으로 써 (epoch 끝에 갱신, sample 수만큼 메모리). Mean Teacher는 student **weight**의 EMA로 teacher 모델을 만들고 그 예측을 target으로 써 (매 iteration 갱신). L06 p.19–25
9. VAT: "virtual" = the model's own prediction instead of the ground-truth label.
   adversarial training은 모델이 가장 헷갈리는 방향의 작은 noise를 넣고도 진짜 label(ground truth)을 맞히게 해. VAT는 진짜 label 대신 모델 자신의 현재 예측을 target으로 써. 그래서 label 없는 데이터에도 쓸 수 있어. "virtual"이 이 뜻이야. L06 p.32–35
0. MixMatch: average over K augmentations → **sharpen** → MixUp; **L2** loss for unlabeled data.
   unlabeled 이미지 하나를 K번 augmentation해서 예측을 평균 → sharpen(temperature T로 확신을 높임) → 그걸 추측 label로 삼고, labeled · unlabeled를 MixUp(두 샘플을 비율 λ로 섞기)해. loss는 labeled엔 CE, unlabeled엔 L2(제곱 오차). 순서는 평균 먼저,

sharpen 나중. L07 p.30–35

1. FixMatch: **weak** → confidence threshold → **strong**, cross-entropy; no sharpening, no MixUp.
   weak augmentation(flip-and-shift 정도) 버전의 예측에서 최대 확률이 threshold τ를 넘으면, 그 argmax를 one-hot pseudo label로 삼아. 그다음 strong augmentation 버전이 그 label을 맞히게 CE로 학습해. MixMatch에 있던 sharpening · MixUp은 없어. L07 p.42–48
2. DivideMix: GMM on per-sample loss; each network uses the **other** network's clean/noisy division.
   sample마다 training loss를 재서 2-component GMM(가우시안 두 개를 섞은 분포)을 맞춰. loss 작은 쪽 = clean, 큰 쪽 = noisy label로 봐. noisy sample은 label만 버리고 unlabeled로 써서 MixMatch로 학습해. network 두 개가 서로의 분할을 바꿔 써서(co-divide) 자기 실수를 자기가 다시 배우는 confirmation bias를 줄여. L07 p.49–55

다음 할 일

이 페이지로 흐름을 잡았으면 repo에서 이 순서로 가면 돼.

1. `exam/pseudocode.md`의 "한눈에 보기" + 함정 Top 12
   pseudo code 9개에서 핵심 줄(detach, EMA update, loss 식, labels)만 모아 둔 곳이야.
2. `exam/mock_quiz_4.md`, `exam/mock_quiz_5.md` 풀기 (빈칸 · pseudo code 포함 새 형식)
   실제 퀴즈 형식(T/F · 객관식 · 빈칸)에 가장 가까운 모의고사야.
3. 틀린 건 답 옆의 `← pool:` 표시를 따라 `exam/_parts/` pool에서 비슷한 문제 더 풀기
   pool = 문제 은행 (강의별 01–07 + pseudo code별 pc_1–4). 표시된 파일과 문제 번호로 가면 같은 개념의 다른 문제가 있어.
4. 개념이 흔들리면 `exam/core_ideas.md`에서 그 방법만 다시 보기
   방법마다 Problem → Core idea → vs. previous 순서로 영어 정리가 돼 있어.